

Lab2_FFNN_RNN_GNN

June 19, 2026

1 Laboratory 2 - Model Engineering

This notebook implements the second lab of the *AI and Cybersecurity* course.

It follows the official brief and develops a full **Machine Learning and Deep Learning pipeline** using PyTorch and PyTorch Geometric to explore, train, and evaluate models for malware classification from API-call sequences.

This lab is organized into tasks: - Task 1: Frequency-based baseline with sparse API-call count vectors and Logistic Regression - Task 2: Feed Forward Neural Networks with fixed-length sequences, sequential identifiers, learnable embeddings, and embedding-size analysis - Task 3: Recurrent models with dynamic padding, Simple RNN, Bidirectional RNN, and Bidirectional LSTM - Task 4: Graph-based sequence representation and GNN architectures, including GCN, GraphSAGE, and GAT

1.1 Setup

```
[1]: # --- Check Python and pip versions ---  
!python --version  
!pip install --upgrade pip
```

Python 3.12.13

Requirement already satisfied: pip in /usr/local/lib/python3.12/dist-packages (24.1.2)

Collecting pip

Downloading pip-26.1.2-py3-none-any.whl.metadata (4.6 kB)

Downloading pip-26.1.2-py3-none-any.whl (1.8 MB)

1.8/1.8 MB

20.8 MB/s eta 0:00:00

Installing collected packages: pip

Attempting uninstall: pip

Found existing installation: pip 24.1.2

Uninstalling pip-24.1.2:

Successfully uninstalled pip-24.1.2

Successfully installed pip-26.1.2

```
[2]: # --- Install required libraries ---  
!pip install torch  
!pip install numpy pandas scikit-learn matplotlib seaborn
```

```
!pip install tqdm
!pip install torch_geometric
# torch_sparse torch_scatter # TODO: check
```

```
Requirement already satisfied: torch in /usr/local/lib/python3.12/dist-packages
(2.11.0+cu128)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-
packages (from torch) (3.29.2)
Requirement already satisfied: typing-extensions>=4.10.0 in
/usr/local/lib/python3.12/dist-packages (from torch) (4.15.0)
Requirement already satisfied: setuptools<82 in /usr/local/lib/python3.12/dist-
packages (from torch) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-
packages (from torch) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in
/usr/local/lib/python3.12/dist-packages (from torch) (3.6.1)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages
(from torch) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-
packages (from torch) (2025.3.0)
Requirement already satisfied: cuda-toolkit==12.8.1 in
/usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cudart,cufft,c
ufile,cupti,curand,cusolver,cusparsenvjitlink,nvrtc,nvtx]==12.8.1;
platform_system == "Linux"->torch) (12.8.1)
Requirement already satisfied: cuda-bindings<13,>=12.9.4 in
/usr/local/lib/python3.12/dist-packages (from torch) (12.9.7)
Requirement already satisfied: nvidia-cudnn-cu12==9.19.0.56 in
/usr/local/lib/python3.12/dist-packages (from torch) (9.19.0.56)
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in
/usr/local/lib/python3.12/dist-packages (from torch) (0.7.1)
Requirement already satisfied: nvidia-nccl-cu12==2.28.9 in
/usr/local/lib/python3.12/dist-packages (from torch) (2.28.9)
Requirement already satisfied: nvidia-nvshmem-cu12==3.4.5 in
/usr/local/lib/python3.12/dist-packages (from torch) (3.4.5)
Requirement already satisfied: triton==3.6.0 in /usr/local/lib/python3.12/dist-
packages (from torch) (3.6.0)
Requirement already satisfied: nvidia-cublas-cu12==12.8.4.1.* in
/usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cudart,cufft,c
ufile,cupti,curand,cusolver,cusparsenvjitlink,nvrtc,nvtx]==12.8.1;
platform_system == "Linux"->torch) (12.8.4.1)
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.8.90.* in
/usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cudart,cufft,c
ufile,cupti,curand,cusolver,cusparsenvjitlink,nvrtc,nvtx]==12.8.1;
platform_system == "Linux"->torch) (12.8.90)
Requirement already satisfied: nvidia-cufft-cu12==11.3.3.83.* in
/usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cudart,cufft,c
ufile,cupti,curand,cusolver,cusparsenvjitlink,nvrtc,nvtx]==12.8.1;
platform_system == "Linux"->torch) (11.3.3.83)
```

Requirement already satisfied: nvidia-cufile-cu12==1.13.1.3.* in
 /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cudart,cufft,c
 ufile,cupti,curand,cusolver,cusparsenvjitlink,nvrtc,nvtx]==12.8.1;
 platform_system == "Linux"->torch) (1.13.1.3)

Requirement already satisfied: nvidia-cuda-cupti-cu12==12.8.90.* in
 /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cudart,cufft,c
 ufile,cupti,curand,cusolver,cusparsenvjitlink,nvrtc,nvtx]==12.8.1;
 platform_system == "Linux"->torch) (12.8.90)

Requirement already satisfied: nvidia-curand-cu12==10.3.9.90.* in
 /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cudart,cufft,c
 ufile,cupti,curand,cusolver,cusparsenvjitlink,nvrtc,nvtx]==12.8.1;
 platform_system == "Linux"->torch) (10.3.9.90)

Requirement already satisfied: nvidia-cusolver-cu12==11.7.3.90.* in
 /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cudart,cufft,c
 ufile,cupti,curand,cusolver,cusparsenvjitlink,nvrtc,nvtx]==12.8.1;
 platform_system == "Linux"->torch) (11.7.3.90)

Requirement already satisfied: nvidia-cusparsenvjitlink-cu12==12.5.8.93.* in
 /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cudart,cufft,c
 ufile,cupti,curand,cusolver,cusparsenvjitlink,nvrtc,nvtx]==12.8.1;
 platform_system == "Linux"->torch) (12.5.8.93)

Requirement already satisfied: nvidia-nvjitlink-cu12==12.8.93.* in
 /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cudart,cufft,c
 ufile,cupti,curand,cusolver,cusparsenvjitlink,nvrtc,nvtx]==12.8.1;
 platform_system == "Linux"->torch) (12.8.93)

Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.8.93.* in
 /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cudart,cufft,c
 ufile,cupti,curand,cusolver,cusparsenvjitlink,nvrtc,nvtx]==12.8.1;
 platform_system == "Linux"->torch) (12.8.93)

Requirement already satisfied: nvidia-nvtx-cu12==12.8.90.* in
 /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cudart,cufft,c
 ufile,cupti,curand,cusolver,cusparsenvjitlink,nvrtc,nvtx]==12.8.1;
 platform_system == "Linux"->torch) (12.8.90)

Requirement already satisfied: cuda-pathfinder~=1.1 in
 /usr/local/lib/python3.12/dist-packages (from cuda-bindings<13,>=12.9.4->torch)
 (1.5.5)

Requirement already satisfied: mpmath<1.4,>=1.1.0 in
 /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch) (1.3.0)

Requirement already satisfied: MarkupSafe>=2.0 in
 /usr/local/lib/python3.12/dist-packages (from jinja2->torch) (3.0.3)

Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages
 (2.0.2)

Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages
 (2.2.2)

Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-
 packages (1.6.1)

Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-
 packages (3.10.0)

Requirement already satisfied: seaborn in /usr/local/lib/python3.12/dist-

packages (0.13.2)
 Requirement already satisfied: python-dateutil>=2.8.2 in
 /usr/local/lib/python3.12/dist-packages (from pandas) (2.9.0.post0)
 Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-
 packages (from pandas) (2025.2)
 Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-
 packages (from pandas) (2026.2)
 Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-
 packages (from scikit-learn) (1.16.3)
 Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-
 packages (from scikit-learn) (1.5.3)
 Requirement already satisfied: threadpoolctl>=3.1.0 in
 /usr/local/lib/python3.12/dist-packages (from scikit-learn) (3.6.0)
 Requirement already satisfied: contourpy>=1.0.1 in
 /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
 Requirement already satisfied: cyclor>=0.10 in /usr/local/lib/python3.12/dist-
 packages (from matplotlib) (0.12.1)
 Requirement already satisfied: fonttools>=4.22.0 in
 /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.63.0)
 Requirement already satisfied: kiwisolver>=1.3.1 in
 /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.5.0)
 Requirement already satisfied: packaging>=20.0 in
 /usr/local/lib/python3.12/dist-packages (from matplotlib) (26.2)
 Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-
 packages (from matplotlib) (11.3.0)
 Requirement already satisfied: pyparsing>=2.3.1 in
 /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.2)
 Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-
 packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
 Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages
 (4.67.3)
 Collecting torch_geometric
 Downloading torch_geometric-2.8.0-py3-none-any.whl.metadata (64 kB)
 Requirement already satisfied: aiohttp in /usr/local/lib/python3.12/dist-
 packages (from torch_geometric) (3.14.1)
 Requirement already satisfied: fsspec in /usr/local/lib/python3.12/dist-packages
 (from torch_geometric) (2025.3.0)
 Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages
 (from torch_geometric) (3.1.6)
 Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages
 (from torch_geometric) (2.0.2)
 Requirement already satisfied: psutil>=5.8.0 in /usr/local/lib/python3.12/dist-
 packages (from torch_geometric) (5.9.5)
 Requirement already satisfied: pyparsing in /usr/local/lib/python3.12/dist-
 packages (from torch_geometric) (3.3.2)
 Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-
 packages (from torch_geometric) (2.32.4)
 Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages

```

(from torch_geometric) (4.67.3)
Requirement already satisfied: xxhash in /usr/local/lib/python3.12/dist-packages
(from torch_geometric) (3.7.0)
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in
/usr/local/lib/python3.12/dist-packages (from aiohttp->torch_geometric) (2.6.2)
Requirement already satisfied: aiosignal>=1.4.0 in
/usr/local/lib/python3.12/dist-packages (from aiohttp->torch_geometric) (1.4.0)
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.12/dist-
packages (from aiohttp->torch_geometric) (26.1.0)
Requirement already satisfied: frozenlist>=1.1.1 in
/usr/local/lib/python3.12/dist-packages (from aiohttp->torch_geometric) (1.8.0)
Requirement already satisfied: multidict<7.0,>=4.5 in
/usr/local/lib/python3.12/dist-packages (from aiohttp->torch_geometric) (6.7.1)
Requirement already satisfied: propcache>=0.2.0 in
/usr/local/lib/python3.12/dist-packages (from aiohttp->torch_geometric) (0.5.2)
Requirement already satisfied: typing_extensions>=4.4 in
/usr/local/lib/python3.12/dist-packages (from aiohttp->torch_geometric) (4.15.0)
Requirement already satisfied: yarl<2.0,>=1.17.0 in
/usr/local/lib/python3.12/dist-packages (from aiohttp->torch_geometric) (1.24.2)
Requirement already satisfied: idna>=2.0 in /usr/local/lib/python3.12/dist-
packages (from yarl<2.0,>=1.17.0->aiohttp->torch_geometric) (3.18)
Requirement already satisfied: MarkupSafe>=2.0 in
/usr/local/lib/python3.12/dist-packages (from jinja2->torch_geometric) (3.0.3)
Requirement already satisfied: charset_normalizer<4,>=2 in
/usr/local/lib/python3.12/dist-packages (from requests->torch_geometric) (3.4.7)
Requirement already satisfied: urllib3<3,>=1.21.1 in
/usr/local/lib/python3.12/dist-packages (from requests->torch_geometric) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in
/usr/local/lib/python3.12/dist-packages (from requests->torch_geometric)
(2026.5.20)
Downloading torch_geometric-2.8.0-py3-none-any.whl (1.3 MB)
1.3/1.3 MB
8.8 MB/s 0:00:00
Installing collected packages: torch_geometric
Successfully installed torch_geometric-2.8.0

```

```

[3]: # --- Import libraries ---
import os
import time
import random
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import json
import math

```

```

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from sklearn.preprocessing import StandardScaler, MinMaxScaler, RobustScaler,
    ↳LabelEncoder
from sklearn.utils.class_weight import compute_class_weight
from sklearn.metrics import classification_report, confusion_matrix,
    ↳accuracy_score, f1_score, log_loss
from sklearn.decomposition import PCA
from sklearn.impute import SimpleImputer

import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader, TensorDataset
from torch.nn.utils.rnn import pad_sequence, pack_padded_sequence,
    ↳pad_packed_sequence
import torch_geometric
from torch_geometric.nn import GCNConv, SAGEConv, GATConv, global_mean_pool,
    ↳global_max_pool
from torch_geometric.loader import DataLoader as PyGDataLoader
from torch_geometric.utils import to_networkx
from torch_geometric.data import Data

import networkx as nx

from tqdm import tqdm

```

1.1.1 Runtime Check

```

[4]: # --- Check GPU availability ---
gpu_info = !nvidia-smi
gpu_info = '\n'.join(gpu_info)
if gpu_info.find('failed') >= 0:
    print('Not connected to a GPU')
else:
    print(gpu_info)

```

Wed Jun 17 08:07:06 2026

```

+-----+
+-----+
| NVIDIA-SMI 580.82.07                  Driver Version: 580.82.07          CUDA Version:
13.0      |
+-----+-----+-----+-----+
+-----+
| GPU   Name                               Persistence-M | Bus-Id        Disp.A | Volatile

```

```

Uncorr. ECC |
| Fan Temp Perf Pwr:Usage/Cap | Memory-Usage | GPU-Util
Compute M. |
|
MIG M. |
|=====+=====+=====
=====|
| 0 Tesla T4 Off | 00000000:00:04.0 Off |
0 |
| N/A 32C P8 9W / 70W | 3MiB / 15360MiB | 0%
Default |
|
N/A |
+-----+-----+-----+
-----+

+-----+
-----+
| Processes:
|
| GPU GI CI PID Type Process name
GPU Memory |
| ID ID
Usage |
|=====+=====+=====
=====|
| No running processes found
|
+-----+
-----+

```

```

[5]: # --- Check RAM availability ---
from psutil import virtual_memory
ram_gb = virtual_memory().total / 1e9
print('Your runtime has {:.1f} gigabytes of available RAM\n'.format(ram_gb))

if ram_gb < 20:
    print('Not using a high-RAM runtime')
else:
    print('You are using a high-RAM runtime!')

```

Your runtime has 13.6 gigabytes of available RAM

Not using a high-RAM runtime

1.1.2 Paths setup

```
[6]: # --- Local notebook: Google Drive is not required ---  
# Paths are configured in the next cell relative to the Laboratory2 project_  
↳ folder.  
print("Running local version: Google Drive mount skipped.")
```

Mounted at /content/drive

```
[7]: # --- Define local paths ---  
from pathlib import Path  
  
def find_laboratory2_root(start=None):  
    """Find the Laboratory2 project root from the current notebook/session path.  
    ↳ """  
    candidates = []  
  
    if start is not None:  
        candidates.append(Path(start).resolve())  
  
    # In a normal Jupyter session, the current working directory is enough.  
    candidates.append(Path.cwd().resolve())  
  
    # If the notebook is launched from another folder, also inspect common_  
    ↳ relative roots.  
    candidates.extend([  
        Path.cwd().resolve() / "Laboratory2",  
        Path.cwd().resolve().parent,  
        Path.cwd().resolve().parent.parent,  
    ])  
  
    for candidate in candidates:  
        for parent in [candidate, *candidate.parents]:  
            data_dir = parent / "lab" / "data"  
            if (data_dir / "train.json").exists() and (data_dir / "test.json").  
↳ exists():  
                return parent  
  
        raise FileNotFoundError(  
            "Could not locate Laboratory2/lab/data/train.json and test.json. "  
            "Run the notebook from inside the Laboratory2 project or keep the lab/  
↳ data folder structure."  
        )  
  
project_root = find_laboratory2_root()  
project_path = str(project_root) + os.sep  
data_path = str(project_root / "lab" / "data") + os.sep  
results_path = str(project_root / "results") + os.sep
```



```

# Ensure output directories exist.
os.makedirs(results_path, exist_ok=True)

print(f"Project path: {project_path}")
print(f>Data path: {data_path}")
print(f"Results path: {results_path}")

```

```

Project path: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/
Data path: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/data/
Results path: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/

```

```

[8]: # --- Set visual style ---
# Plot style tuned for report figures: the figure size stays compact,
# while axes, tick labels, legends, and titles remain readable after LaTeX
↪ scaling.
REPORT_PLOT_TITLE_SIZE = 16
REPORT_PLOT_LABEL_SIZE = 14
REPORT_PLOT_TICK_SIZE = 12
REPORT_PLOT_LEGEND_SIZE = 12
REPORT_PLOT_LINEWIDTH = 2.3

sns.set_theme(
    style="whitegrid",
    palette="muted",
    font_scale=1.15,
    rc={
        "axes.titlesize": REPORT_PLOT_TITLE_SIZE,
        "axes.labelsize": REPORT_PLOT_LABEL_SIZE,
        "xtick.labelsize": REPORT_PLOT_TICK_SIZE,
        "ytick.labelsize": REPORT_PLOT_TICK_SIZE,
        "legend.fontsize": REPORT_PLOT_LEGEND_SIZE,
        "figure.dpi": 120,
        "savefig.dpi": 300,
    },
)

plt.rcParams.update({
    "axes.titlesize": REPORT_PLOT_TITLE_SIZE,
    "axes.labelsize": REPORT_PLOT_LABEL_SIZE,
    "xtick.labelsize": REPORT_PLOT_TICK_SIZE,
    "ytick.labelsize": REPORT_PLOT_TICK_SIZE,
    "legend.fontsize": REPORT_PLOT_LEGEND_SIZE,
})

def format_report_axes(ax, legend=True, legend_loc="best"):
    """Make plot text readable after the figure is scaled inside the report."""

```

```

ax.tick_params(axis="both", labelsz=REPORT_PLOT_TICK_SIZE)
ax.xaxis.label.set_size(REPORT_PLOT_LABEL_SIZE)
ax.yaxis.label.set_size(REPORT_PLOT_LABEL_SIZE)
ax.title.set_size(REPORT_PLOT_TITLE_SIZE)

handles, labels = ax.get_legend_handles_labels()
current_legend = ax.get_legend()
if legend and handles and current_legend is None:
    current_legend = ax.legend(fontsize=REPORT_PLOT_LEGEND_SIZE,
loc=legend_loc, frameon=True)
elif current_legend is not None:
    for text in current_legend.get_texts():
        text.set_fontsize(REPORT_PLOT_LEGEND_SIZE)
    current_legend.set_frame_on(True)
return ax

def save_plot(fig: plt.Figure, filename: str, path: str = "./plots/", fmt: str
    => "png", dpi: int = 300, close_fig: bool = False) -> None:
    """
    Save a Matplotlib figure in a specific to a specified directory.

    Args:
        fig (plt.Figure): Matplotlib figure object to save.
        filename (str): Name of the file to save (e.g., 'plot.png').
        path (str, optional): Directory path to save the figure. Defaults to './
    => plots/'.
        fmt (str, optional): File format for the saved figure. Defaults to
    => 'png'.
        dpi (int, optional): Dots per inch for the saved figure. Defaults to
    => 300.

    Returns:
        None
    """
    # Ensure the directory exists
    os.makedirs(path, exist_ok=True)
    save_path = os.path.join(path, f"{filename}.{fmt}")

    # Save the figure
    fig.savefig(save_path, bbox_inches='tight', pad_inches=0.18, dpi=dpi,
    => format=fmt)
    # plt.close(fig) # Removed to display plots in notebook

    if close_fig:
        plt.close(fig)

```

```
print(f"Saved plot: {save_path}")
```

```
[9]: def _internal_plot_cm(y_true, y_pred, model_name, save_dir):  
    """Internal helper to draw the confusion matrix plot."""  
    cm = confusion_matrix(y_true, y_pred)  
  
    # Create figure explicitly  
    fig, ax = plt.subplots(figsize=(5, 4))  
  
    sns.heatmap(  
        cm,  
        annot=True,  
        fmt='d',  
        cmap='Blues',  
        cbar=False,  
        xticklabels=['Benign (0)', 'Malware (1)'],  
        yticklabels=['Benign (0)', 'Malware (1)'],  
        ax=ax  
    )  
  
    ax.set_xlabel('Predicted Label', fontsize=12)  
    ax.set_ylabel('True Label', fontsize=12)  
    ax.set_title(f'Confusion Matrix: {model_name}', fontsize=14)  
  
    if save_dir:  
        clean_name = model_name.lower().replace(" ", "_").replace("-", "_")  
        save_plot(  
            fig=fig,  
            filename=f"cm_{clean_name}",  
            path=save_dir,  
            fmt="png",  
            dpi=300,  
            close_fig=False  
        )  
  
    plt.show()
```

1.2 Task 1 — Frequency-based baseline

We implement a simple frequency-based baseline.

- Transform sequences into feature vectors counting API call occurrences.
- Helps evaluate whether simple approaches already perform well before using complex models.

```
[10]: # Create directory for plots  
save_dir = results_path + 'images/' + 'task1_plots/'  
os.makedirs(save_dir, exist_ok=True)
```

1.2.1 Frequency-Based Approach

- Extract vocabulary from train and test datasets.
- Use vocabulary to create feature vectors (frequency counts per API call).
- Output dataframe: one row per sequence, one column per API call.

```
[11]: # --- Load dataset and perform initial inspection ---

# Train set
file_path_train = data_path + 'train.json'
df_train = pd.read_json(file_path_train)

# Basic info
print("Train set:")
print("  Shape (raw):", df_train.shape)
print("  Columns:", list(df_train.columns))

# Test set
file_path_test = data_path + 'test.json'
df_test = pd.read_json(file_path_test)

# Basic info
print("\nTest set:")
print("  Shape (raw):", df_test.shape)
print("  Columns:", list(df_test.columns))
```

```
Train set:
  Shape (raw): (16325, 2)
  Columns: ['api_call_sequence', 'is_malware']
```

```
Test set:
  Shape (raw): (6505, 2)
  Columns: ['api_call_sequence', 'is_malware']
```

```
[12]: df_train
```

```
[12]:
```

	api_call_sequence	is_malware
0	[LdrGetDllHandle, LdrGetProcedureAddress, LdrL...	1
1	[NtAllocateVirtualMemory, LdrLoadDll, LdrGetPr...	1
2	[FindResourceExW, LoadResource, FindResourceEx...	1
3	[FindResourceExW, LoadResource, FindResourceEx...	1
4	[LdrGetProcedureAddress, SetErrorMode, LdrLoad...	1
...	...	...
16320	[LdrGetProcedureAddress, LdrLoadDll, LdrGetPro...	1
16321	[NtClose, LdrGetProcedureAddress, CryptCreateH...	1
16322	[LdrGetProcedureAddress, LdrGetDllHandle, LdrG...	1
16323	[LdrGetProcedureAddress, LdrGetDllHandle, LdrG...	1
16324	[SetFilePointer, NtReadFile, SetFilePointer, N...	1

[16325 rows x 2 columns]

```
[13]: malware_counts = df_train['is_malware'].value_counts()
num_total_samples = len(df_train)

print("Malware vs. Benign Samples in Training Set:")
print(malware_counts)

print("\nPercentages:")
for is_malware, count in malware_counts.items():
    percentage = (count / num_total_samples) * 100
    if is_malware == 1:
        print(f"  Malware (1): {count} samples ({percentage:.2f}%) - {(count / num_total_samples)*100:.2f}% of total")
    else:
        print(f"  Benign (0): {count} samples ({percentage:.2f}%) - {(count / num_total_samples)*100:.2f}% of total")
```

Malware vs. Benign Samples in Training Set:

is_malware

1 15714

0 611

Name: count, dtype: int64

Percentages:

Malware (1): 15714 samples (96.26%) - 96.26% of total

Benign (0): 611 samples (3.74%) - 3.74% of total

```
[14]: df_test
```

```
[14]:
```

	api_call_sequence	is_malware
0	[NtQueryValueKey, NtClose, NtOpenKey, NtQueryV...	1
1	[LdrGetProcedureAddress, NtClose, NtOpenKey, N...	1
2	[NtOpenKey, NtQueryValueKey, NtClose, NtOpenKe...	1
3	[NtAllocateVirtualMemory, LdrLoadDll, LdrGetPr...	1
4	[NtOpenKey, NtQueryValueKey, NtClose, LdrGetPr...	1
...	...	...
6500	[SetErrorMode, NtOpenFile, NtClose, SHGetFolde...	0
6501	[NtProtectVirtualMemory, RegOpenKeyExW, RegQue...	1
6502	[RegOpenKeyExW, RegQueryValueExW, RegCloseKey,...	1
6503	[NtQueryValueKey, NtClose, RegOpenKeyExA, RegQ...	1
6504	[NtClose, LdrGetProcedureAddress, CryptCreateH...	1

[6505 rows x 2 columns]

```
[15]: malware_counts = df_test['is_malware'].value_counts()
num_total_samples = len(df_test)

print("Malware vs. Benign Samples in Test Set:")
print(malware_counts)

print("\nPercentages:")
for is_malware, count in malware_counts.items():
    percentage = (count / num_total_samples) * 100
    if is_malware == 1:
        print(f"  Malware (1): {count} samples ({percentage:.2f}%) - {(count / num_total_samples)*100:.2f}% of total")
    else:
        print(f"  Benign (0): {count} samples ({percentage:.2f}%) - {(count / num_total_samples)*100:.2f}% of total")
```

Malware vs. Benign Samples in Test Set:

is_malware

1 6262

0 243

Name: count, dtype: int64

Percentages:

Malware (1): 6262 samples (96.26%) - 96.26% of total

Benign (0): 243 samples (3.74%) - 3.74% of total

1.2.2 Extract the vocabularies

```
[16]: # Extract vocabulary from training set
train_vocab = set(api_call for seq in df_train['api_call_sequence'] for api_call in seq)

# Count unique API calls
print(f"Train: {len(train_vocab)}")
```

Train: 258

```
[17]: # Extract vocabulary from test set
test_vocab = set(api_call for seq in df_test['api_call_sequence'] for api_call in seq)

# Count unique API calls
print(f"Test: {len(test_vocab)}")
```

Test: 232

Q: How many unique API calls does the training set contain? How many in the test set?

- Training set: there are **258** unique API calls.
- Test set: there are **232** unique API calls.

[18]: # --- Find API calls only in the training set ---

```
only_in_train = train_vocab - test_vocab

# Count and print them
print(f"Number of API calls only in training set: {len(only_in_train)}")
print("API calls only in training set:", only_in_train)
```

```
Number of API calls only in training set: 29
API calls only in training set: {'InternetSetStatusCallback',
'GetFileInformationByHandleEx', 'IWbemServices_ExecMethod',
'CreateRemoteThread', 'CryptProtectMemory', 'CertOpenSystemStoreW',
'CryptDecodeObjectEx', 'NtReadVirtualMemory',
'RtlRemoveVectoredExceptionHandler', 'FindFirstFileExA', 'InternetReadFile',
'NetUserGetInfo', 'GetUserNameExA', 'WSASocketW', 'listen', 'GetKeyboardState',
'NtDeleteFile', 'NetUserGetLocalGroups', 'InternetOpenUrlW', 'GetUserNameExW',
'CertOpenStore', 'recvfrom', 'ObtainUserAgentString', 'GetAddrInfoW', 'sendto',
'RegDeleteKeyA', 'NtLoadKey', 'NtWriteVirtualMemory', 'SendNotifyMessageW'}
```

[19]: # --- Find API calls only in the test set ---

```
only_in_test = test_vocab - train_vocab

# Count and print them
print(f"Number of API calls only in test set: {len(only_in_test)}")
print("API calls only in test set:", only_in_test)
```

```
Number of API calls only in test set: 3
API calls only in test set: {'WSASocketA', 'NtDeleteKey', 'ControlService'}
```

[20]: # Create a boolean series to identify samples in df_test that contain any of
↪ these API calls

```
has_test_only_apis = df_test['api_call_sequence'].apply(
    lambda seq: any(api in only_in_test for api in seq)
)
```

Count the samples

```
samples_with_test_only_apis = has_test_only_apis.sum()
```

```
print(f"Number of test set samples with API calls unique to the test set: {samples_with_test_only_apis}")
```

Display the actual rows

```
print("\nTest samples containing API calls unique to the test set:")
display(df_test[has_test_only_apis])
```

Number of test set samples with API calls unique to the test set: 3

Test samples containing API calls unique to the test set:

	api_call_sequence	is_malware
2873	[LdrGetDllHandle, LdrGetProcedureAddress, LdrG...	1
4105	[NtProtectVirtualMemory, CreateThread, NtAlloc...	1
5948	[NtDelayExecution, gethostbyname, NtDelayExecu...	1

Q: Are there any API calls that appear only in the test set (but not in the training set)? If yes, how many? Which ones? Yes — there are API calls that appear only in the test set and not in the training set.

- Number of such API calls: 3
- API calls only in the test set: NtDeleteKey, ControlService, WSASocketA

1.2.3 New frequency-based dataframes

- Create dataframe using training vocabulary as features.
- Count occurrences of each API call per sequence.

```
[21]: # Create frequency-based dataframe for training data
train_df_freq = pd.DataFrame([api: seq.count(api) for api in train_vocab} for
↪seq in df_train['api_call_sequence']])
```

```
[22]: # Dataframe characteristics
print("Shape (row):", train_df_freq.shape)
print("Columns:", list(train_df_freq.columns))
```

Shape (row): (16325, 258)

Columns: ['InternetSetStatusCallback', 'GetFileInformationByHandleEx', 'DrawTextExA', 'GetVolumePathNamesForVolumeNameW', 'getaddrinfo', 'RemoveDirectoryW', 'RegisterHotKey', 'DeleteUrlCacheEntryW', 'NtCreateSection', 'CryptHashData', 'LookupPrivilegeValueW', 'RegCreateKeyExW', 'GetFileVersionInfoExW', 'GetAdaptersInfo', 'GetFileVersionInfoW', 'NtCreateFile', 'LoadStringA', 'CryptDecodeObjectEx', 'GetFileAttributesExW', 'RemoveDirectoryA', 'SetFilePointer', 'FindFirstFileExA', 'closesocket', 'RegQueryInfoKeyA', 'NtEnumerateValueKey', 'CryptCreateHash', 'NtOpenThread', 'GetSystemWindowsDirectoryW', 'OleInitialize', 'OpenServiceA', 'HttpQueryInfoA', 'NtQueueApcThread', 'GetUserNameExW', 'NtTerminateProcess', 'FindWindowA', 'RegEnumValueA', 'LdrGetProcedureAddress', 'SHGetSpecialFolderLocation', 'CertOpenStore', 'GetSystemDirectoryA', 'recvfrom', 'GetSystemMetrics', 'ObtainUserAgentString', 'CoGetClassObject', 'GetAdaptersAddresses', 'CopyFileA', 'RegEnumKeyW', 'NtQueryKey', 'RegDeleteKeyW', 'GetVolumeNameForVolumeMountPointW', 'FindResourceExA', 'GetSystemDirectoryW', 'NtOpenKey', 'NtGetContextThread', 'IwbemServices_ExecMethod', 'Module32FirstW', 'CreateRemoteThread', 'CopyFileExW', 'GetSystemTimeAsFileTime', 'NtCreateMutant', 'gethostbyname', 'SetFileTime', 'getsockname', 'NtTerminateThread', 'RegOpenKeyExW', 'RegCreateKeyExA', 'OutputDebugStringA',

'InternetOpenA', 'socket', 'DeviceIoControl', 'CertOpenSystemStoreW',
 'CreateToolhelp32Snapshot', 'ioctlsocket', 'SetEndOfFile',
 'MoveFileWithProgressW', 'WSAStartup', 'GetSystemInfo', 'CreateActCtxW',
 'OpenSCManagerA', 'SearchPathW', 'SendNotifyMessageW', 'GetVolumePathNameW',
 'CopyFileW', 'NtSetInformationFile', 'GetTempPathW', 'NtDeleteFile',
 'StartServiceW', 'WSARecv', 'NtOpenDirectoryObject', 'NtOpenFile',
 'IsDebuggerPresent', 'GlobalMemoryStatusEx', 'GetFileSizeEx', 'CreateThread',
 'UuidCreate', 'ReadProcessMemory', 'InternetCrackUrlA', 'RegCloseKey',
 'SizeofResource', 'GetUserNameA', 'DeleteUrlCacheEntryA', 'CreateJobObjectW',
 'CryptAcquireContextW', 'CoCreateInstanceEx', 'bind', 'GetAddrInfoW',
 'CoCreateInstance', 'RtlAddVectoredExceptionHandler', 'LdrLoadDll',
 'WriteConsoleW', 'sendto', 'RegOpenKeyExA', 'EnumWindows', 'GetAsyncKeyState',
 'InternetCloseHandle', 'NtLoadKey', 'InternetQueryOptionA', 'LoadResource',
 'CoUninitialize', 'HttpOpenRequestW', 'DrawTextExW', 'LdrUnloadDll',
 'Thread32Next', 'LookupAccountSidW', 'InternetConnectA',
 'GetFileInformationByHandle', 'NtProtectVirtualMemory', 'shutdown',
 'RegQueryValueExW', 'NtQueryAttributesFile', 'LoadStringW',
 'UnhookWindowsHookEx', 'SetStdHandle', 'timeGetTime', 'EnumServicesStatusA',
 'GetUserNameW', 'CryptEncrypt', 'NtClose', 'RegEnumKeyExW', 'OpenSCManagerW',
 'RtlRemoveVectoredExceptionHandler', 'OpenServiceW', 'CryptDecrypt',
 'NetUserGetInfo', 'StartServiceA', 'GetBestInterfaceEx', 'InternetOpenUrlA',
 'GetUserNameExA', 'DnsQuery_A', 'CreateServiceW', 'listen', 'FindResourceW',
 'RegQueryValueExA', 'CoInitializeSecurity', 'SetFilePointerEx',
 'CryptProtectData', 'SetFileAttributesW', 'NetUserGetLocalGroups',
 'NtCreateThreadEx', 'CreateServiceA', 'GetComputerNameA', 'Process32FirstW',
 'NtOpenMutant', 'InternetSetOptionA', 'GetFileSize',
 'SetUnhandledExceptionFilter', 'GetForegroundWindow', 'HttpSendRequestA',
 'IWbemServices_ExecQuery', 'RegDeleteKeyA', 'NtDelayExecution',
 'NtWriteVirtualMemory', 'RegEnumKeyExA', 'FindWindowExA', 'GetDiskFreeSpaceW',
 'FindWindowExW', 'GetFileType', 'SetWindowsHookExA', 'ShellExecuteExW',
 'InternetGetConnectedState', 'recv', 'NtQueryDirectoryFile', 'CoInitializeEx',
 'NtOpenSection', 'SetWindowsHookExW', 'GetKeyState', 'NtQueryValueKey',
 'CryptProtectMemory', 'LdrGetDllHandle', 'NtEnumerateKey', 'WriteProcessMemory',
 'GetSystemWindowsDirectoryA', 'NtDeviceIoControlFile', 'NtResumeThread',
 'Process32NextW', 'MessageBoxTimeoutA', 'NtSetContextThread', 'SetErrorMode',
 'RegSetValueExA', 'GetShortPathNameW', 'FindWindowW', 'RegEnumValueW',
 'connect', 'InternetOpenW', 'Module32NextW', 'RegSetValueExW',
 'RegQueryInfoKeyW', 'GetComputerNameW', 'WriteConsoleA', 'Thread32First',
 'CryptGenKey', 'RtlDecompressBuffer', 'NtQuerySystemInformation',
 'NtReadVirtualMemory', 'FindResourceExW', 'InternetReadFile',
 'MessageBoxTimeoutW', 'GetFileAttributesW', 'NtSetValueKey', 'SHGetFolderPathW',
 'NtSuspendThread', 'WSASocketW', 'HttpOpenRequestA', 'send', 'NtCreateKey',
 'GetKeyboardState', 'NtFreeVirtualMemory', 'NtAllocateVirtualMemory',
 'RegDeleteValueA', 'InternetCrackUrlW', 'InternetOpenUrlW', 'FindResourceA',
 'FindFirstFileExW', 'GetNativeSystemInfo', 'GlobalMemoryStatus',
 'NtMapViewOfSection', 'NtQueryInformationFile', 'GetFileVersionInfoSizeW',
 'NtUnmapViewOfSection', 'RegDeleteValueW', 'CryptExportKey', 'select',
 'GetDiskFreeSpaceExW', 'GetCursorPos', 'CryptAcquireContextA',

```
'NtDuplicateObject', 'InternetConnectW', 'GetFileVersionInfoSizeExW',
'CreateProcessInternalW', 'NtWriteFile', 'NtOpenKeyEx', 'DeleteFileW',
'GetTimeZoneInformation', '__exception__', 'CreateDirectoryW', 'NtOpenProcess',
'setsockopt', 'NtReadFile']
```

```
[23]: train_df_freq.head(10)
```

```
[23]:  InternetSetStatusCallback  GetFileInformationByHandleEx  DrawTextExA  \
0          0          0          0          0
1          0          0          0          0
2          0          0          0          0
3          0          0          0          0
4          0          0          0          0
5          0          0          0          0
6          0          0          0          0
7          0          0          0          0
8          0          0          0          0
9          0          0          0          0

      GetVolumePathNamesForVolumeNameW  getaddrinfo  RemoveDirectoryW  \
0          0          0          0
1          0          0          0
2          0          0          0
3          0          0          0
4          0          0          0
5          0          0          0
6          0          0          0
7          0          0          0
8          0          0          0
9          0          0          0

      RegisterHotKey  DeleteUrlCacheEntryW  NtCreateSection  CryptHashData  ...  \
0          0          0          0          0          0  ...
1          0          0          0          0          0  ...
2          0          0          0          0          0  ...
3          0          0          1          0          0  ...
4          0          0          0          0          0  ...
5          0          0          1          0          0  ...
6          0          0          0          0          0  ...
7          0          0          1          0          0  ...
8          0          0          1          0          0  ...
9          0          0          0          0          0  ...

      CreateProcessInternalW  NtWriteFile  NtOpenKeyEx  DeleteFileW  \
0          0          0          0          0
1          0          0          5          0
2          0          0          0          0
```

3	0	0	0	0
4	0	0	0	0
5	0	0	0	0
6	0	0	0	0
7	0	0	0	0
8	0	0	0	0
9	0	0	0	0

	GetTimeZoneInformation	__exception__	CreateDirectoryW	NtOpenProcess	\
0	1	0	0	0	0
1	0	0	0	0	0
2	0	0	0	0	0
3	0	0	0	0	0
4	0	0	0	0	0
5	0	0	0	0	0
6	0	0	0	0	1
7	0	0	0	0	0
8	0	0	0	0	0
9	0	0	0	0	0

	setsockopt	NtReadFile
0	0	0
1	0	0
2	0	0
3	0	0
4	0	0
5	0	0
6	0	0
7	0	0
8	0	0
9	0	0

[10 rows x 258 columns]

```
[24]: # Create frequency-based dataframe for test data
test_df_freq = pd.DataFrame([api: seq.count(api) for api in test_vocab] for
    seq in df_test['api_call_sequence']])
```

```
[25]: # Dataframe characteristics
print("Shape (row):", test_df_freq.shape)
print("Columns:", list(test_df_freq.columns))
```

```
Shape (row): (6505, 232)
Columns: ['DrawTextExA', 'GetVolumePathNamesForVolumeNameW', 'getaddrinfo',
'RemoveDirectoryW', 'RegisterHotKey', 'DeleteUrlCacheEntryW', 'NtCreateSection',
'CryptHashData', 'LookupPrivilegeValueW', 'RegCreateKeyExW',
'GetFileVersionInfoExW', 'GetAdaptersInfo', 'GetFileVersionInfoW',
'NtCreateFile', 'LoadStringA', 'GetFileAttributesExW', 'RemoveDirectoryA',
```

'SetFilePointer', 'closesocket', 'RegQueryInfoKeyA', 'NtEnumerateValueKey',
 'CryptCreateHash', 'NtOpenThread', 'GetSystemWindowsDirectoryW',
 'OleInitialize', 'OpenServiceA', 'HttpQueryInfoA', 'NtQueueApcThread',
 'FindWindowA', 'NtTerminateProcess', 'RegEnumValueA', 'LdrGetProcedureAddress',
 'SHGetSpecialFolderLocation', 'GetSystemDirectoryA', 'GetSystemMetrics',
 'GetAdaptersAddresses', 'CoGetClassObject', 'CopyFileA', 'RegEnumKeyW',
 'NtQueryKey', 'RegDeleteKeyW', 'GetVolumeNameForVolumeMountPointW',
 'FindResourceExA', 'GetSystemDirectoryW', 'NtOpenKey', 'NtGetContextThread',
 'Module32FirstW', 'CopyFileExW', 'GetSystemTimeAsFileTime', 'NtCreateMutant',
 'SetFileTime', 'gethostbyname', 'getsockname', 'NtTerminateThread',
 'RegOpenKeyExW', 'RegCreateKeyExA', 'OutputDebugStringA', 'InternetOpenA',
 'socket', 'DeviceIoControl', 'WSASocketA', 'CreateToolhelp32Snapshot',
 'ioctlsocket', 'SetEndOfFile', 'MoveFileWithProgressW', 'WSAStartup',
 'CreateActCtxW', 'GetSystemInfo', 'OpenSCManagerA', 'SearchPathW',
 'NtDeleteKey', 'GetVolumePathNameW', 'CopyFileW', 'NtSetInformationFile',
 'GetTempPathW', 'WSARecv', 'StartServiceW', 'NtOpenDirectoryObject',
 'NtOpenFile', 'IsDebuggerPresent', 'GlobalMemoryStatusEx', 'GetFileSizeEx',
 'CreateThread', 'UuidCreate', 'ReadProcessMemory', 'InternetCrackUrlA',
 'RegCloseKey', 'SizeofResource', 'GetUserNameA', 'DeleteUrlCacheEntryA',
 'ControlService', 'CreateJobObjectW', 'CryptAcquireContextW',
 'CoCreateInstanceEx', 'bind', 'CoCreateInstance',
 'RtlAddVectoredExceptionHandler', 'LdrLoadDll', 'WriteConsoleW',
 'RegOpenKeyExA', 'EnumWindows', 'InternetCloseHandle', 'GetAsyncKeyState',
 'InternetQueryOptionA', 'LoadResource', 'CoUninitialize', 'HttpOpenRequestW',
 'DrawTextExW', 'LdrUnloadDll', 'Thread32Next', 'LookupAccountSidW',
 'InternetConnectA', 'GetFileInformationByHandle', 'NtProtectVirtualMemory',
 'shutdown', 'NtQueryAttributesFile', 'RegQueryValueExW', 'LoadStringW',
 'UnhookWindowsHookEx', 'SetStdHandle', 'timeGetTime', 'EnumServicesStatusA',
 'GetUserNameW', 'CryptEncrypt', 'NtClose', 'RegEnumKeyExW', 'OpenSCManagerW',
 'OpenServiceW', 'CryptDecrypt', 'StartServiceA', 'GetBestInterfaceEx',
 'InternetOpenUrlA', 'DnsQuery_A', 'CreateServiceW', 'FindResourceW',
 'RegQueryValueExA', 'CoInitializeSecurity', 'SetFilePointerEx',
 'SetFileAttributesW', 'CryptProtectData', 'NtCreateThreadEx', 'CreateServiceA',
 'GetComputerNameA', 'Process32FirstW', 'NtOpenMutant', 'InternetSetOptionA',
 'GetFileSize', 'SetUnhandledExceptionFilter', 'GetForegroundWindow',
 'HttpSendRequestA', 'IWbemServices_ExecQuery', 'NtDelayExecution',
 'RegEnumKeyExA', 'FindWindowExA', 'GetDiskFreeSpaceW', 'FindWindowExW',
 'GetFileType', 'SetWindowsHookExA', 'ShellExecuteExW',
 'InternetGetConnectedState', 'recv', 'NtQueryDirectoryFile', 'CoInitializeEx',
 'NtOpenSection', 'SetWindowsHookExW', 'GetKeyState', 'NtQueryValueKey',
 'LdrGetDllHandle', 'NtEnumerateKey', 'WriteProcessMemory',
 'GetSystemWindowsDirectoryA', 'NtDeviceIoControlFile', 'NtResumeThread',
 'Process32NextW', 'MessageBoxTimeoutA', 'NtSetContextThread', 'SetErrorMode',
 'FindWindowW', 'RegSetValueExA', 'GetShortPathNameW', 'RegEnumValueW',
 'InternetOpenW', 'CryptGenKey', 'Module32NextW', 'RegSetValueExW',
 'RegQueryInfoKeyW', 'GetComputerNameW', 'connect', 'Thread32First',
 'RtlDecompressBuffer', 'WriteConsoleA', 'NtQuerySystemInformation',
 'FindResourceExW', 'MessageBoxTimeoutW', 'SHGetFolderPathW',

```
'GetFileAttributesW', 'NtSetValueKey', 'NtSuspendThread', 'HttpOpenRequestA',
'send', 'NtCreateKey', 'NtFreeVirtualMemory', 'NtAllocateVirtualMemory',
'RegDeleteValueA', 'InternetCrackUrlW', 'FindResourceA', 'FindFirstFileExW',
'GetNativeSystemInfo', 'GlobalMemoryStatus', 'NtMapViewOfSection',
'NtQueryInformationFile', 'GetFileVersionInfoSizeW', 'NtUnmapViewOfSection',
'RegDeleteValueW', 'CryptExportKey', 'select', 'GetDiskFreeSpaceExW',
'GetCursorPos', 'CryptAcquireContextA', 'NtDuplicateObject', 'InternetConnectW',
'GetFileVersionInfoSizeExW', 'CreateProcessInternalW', 'NtWriteFile',
'NtOpenKeyEx', 'DeleteFileW', 'GetTimeZoneInformation', '__exception__',
'CreateDirectoryW', 'NtOpenProcess', 'setsockopt', 'NtReadFile']
```

```
[26]: test_df_freq.head(10)
```

```
[26]: DrawTextExA  GetVolumePathNamesForVolumeNameW  getaddrinfo  \
0                0                0                0
1                0                0                0
2                0                0                0
3                0                0                0
4                0                0                0
5                0                0                0
6                0                0                0
7                0                0                0
8                0                0                0
9                0                0                0

RemoveDirectoryW  RegisterHotKey  DeleteUrlCacheEntryW  NtCreateSection  \
0                0                0                0                1
1                0                0                0                1
2                0                0                0                0
3                0                0                0                1
4                0                0                0                0
5                0                0                0                0
6                0                0                0                0
7                0                0                0                0
8                0                0                0                0
9                0                0                0                0

CryptHashData  LookupPrivilegeValueW  RegCreateKeyExW  ...  \
0                0                0                0  ...
1                0                0                0  ...
2                0                0                0  ...
3                0                0                0  ...
4                1                0                1  ...
5                0                0                0  ...
6                1                0                1  ...
7                0                0                0  ...
8                0                0                0  ...
```

9	1	0	1	...
---	---	---	---	-----

	CreateProcessInternalW	NtWriteFile	NtOpenKeyEx	DeleteFileW	\
0	0	0	0	0	
1	0	0	0	0	
2	0	0	0	0	
3	0	0	0	0	
4	0	0	0	0	
5	0	0	0	0	
6	0	0	0	0	
7	0	0	0	0	
8	0	0	0	0	
9	0	0	0	0	

	GetTimeZoneInformation	__exception__	CreateDirectoryW	NtOpenProcess	\
0	0	0	0	0	
1	0	0	0	0	
2	1	0	0	0	
3	0	0	0	0	
4	0	0	0	0	
5	0	0	0	0	
6	0	0	0	0	
7	0	0	0	0	
8	0	0	0	0	
9	0	0	0	0	

	setsockopt	NtReadFile
0	0	0
1	0	13
2	0	0
3	0	0
4	0	1
5	0	0
6	0	1
7	0	0
8	0	0
9	0	1

[10 rows x 232 columns]

Q: Can you use the test vocabulary to build the new test dataframe? If not, how do you handle API calls in the test set that do not exist in the training vocabulary? No, you cannot use the test vocabulary to build the test dataframe.

Machine learning models require the feature space (columns) during inference (testing) to be identical to the feature space used during training.

Common strategies for handling API calls in the test set that did not exist in the training vocabulary

include:

1. **Add an “Unknown” Column:** Map all unseen API calls to a single generic token (e.g., <UNK>) or a specific “is_unknown” feature. This preserves the information that *an* action occurred, even if the specific action is unrecognized.
2. **Ignore Them:** Simply discard or zero-out any API calls that were not seen during training.

Decision: In this specific case, since there are **very few unseen API calls** (i.e. *only 3* samples) in the test set, we decided to **ignore them**. This approach simplifies the pipeline without significantly impacting the model’s performance.

Sparse Vectors Analysis

```
[27]: # --- Non-zero elements analysis for training data ---

# Calculate average non-zero elements per row
avg_nonzero_train = (train_df_freq != 0).sum(axis=1).mean()
print(f"Average non-zero elements per row in train_df_freq: {avg_nonzero_train:.
↪2f}")

# Calculate ratio of non-zero elements
ratio_nonzero_train = avg_nonzero_train / train_df_freq.shape[1]
print(f"Ratio of non-zero elements per row in train_df_freq:
↪{ratio_nonzero_train:.4f}")
```

Average non-zero elements per row in train_df_freq: 21.95

Ratio of non-zero elements per row in train_df_freq: 0.0851

```
[28]: # --- Non-zero elements analysis for test data ---

# Calculate average non-zero elements per row
avg_nonzero_test = (test_df_freq != 0).sum(axis=1).mean()
print(f"Average non-zero elements per row in test_df_freq: {avg_nonzero_test:.
↪2f}")

# Calculate ratio of non-zero elements
ratio_nonzero_test = avg_nonzero_test / test_df_freq.shape[1]
print(f"Ratio of non-zero elements per row in test_df_freq: {ratio_nonzero_test:
↪.4f}")
```

Average non-zero elements per row in test_df_freq: 24.28

Ratio of non-zero elements per row in test_df_freq: 0.1046

Q: How many non-zero elements per row do you have on average in the training set? How many in the test set? What is the ratio with respect to the number of elements per row? On average:

- **Training set:** ~22 non-zero elements per row, which is **8.5%** of all elements.
- **Test set:** ~24 non-zero elements per row, which is **10.5%** of all elements.

Q: The original API sequences were ordered. Is it still the case now in the frequency-based dataframe? Why? No, the original order of API calls is **not preserved** in the frequency-based dataframe.

This is because the frequency-based representation **counts how many times each API call occurs** in a sequence, without keeping track of their positions. Each row now simply contains **one column per API call in the vocabulary**, with the value representing its count, so the sequential order is lost.

This approach converts sequences into fixed-dimensional vectors, simplifying the computation but discarding temporal information. As a result, complex behavioral patterns based on the chronology of calls cannot be captured by this representation.

1.2.4 Feed the frequency-based datasets to a classifier

- Any classifier can be used (shallow/deep neural or non-neural).
- Goal: evaluate baseline performance on sparse vectors without sequence information.

Training

```
[29]: # --- Training with a classifier ---

# Split data into features and target
X = train_df_freq
y = df_train['is_malware']

# Create a validation set from training set
X_train, X_val, y_train, y_val = train_test_split(
    X, y, test_size=0.4, random_state=42, stratify=y
)

# Construct test set with features aligned to train_vocab
# TODO: justify this strategy in the report (too few samples)
X_test = pd.DataFrame([{api: seq.count(api) for api in train_vocab} for seq in
    df_test['api_call_sequence']])
y_test = df_test['is_malware']

# Train a simple classifier (using default hyperparameters for a simple
    baseline)
clf = LogisticRegression(random_state=42, solver='liblinear') # 'liblinear'
    solver is good for small datasets and sparse data
clf.fit(X_train, y_train)

# Compute training and validation loss
y_train_proba = clf.predict_proba(X_train)
y_val_proba = clf.predict_proba(X_val)

print(f"Training log-loss: {log_loss(y_train, y_train_proba):.4f}")
print(f"Validation log-loss: {log_loss(y_val, y_val_proba):.4f}")
```


Training log-loss: 0.0806
Validation log-loss: 0.1225

Evaluation

```
[30]: def evaluate_model(model, X, y_true, model_name: str = "Unnamed model"):
    """
    Evaluate a trained scikit-learn model on a given dataset and return the
    ↪classification report.
    Handles missing predicted classes gracefully (zero_division=0).
    """
    # Get predictions
    y_pred = model.predict(X)

    # Identify missing classes (not predicted at all)
    missing_classes = set(np.unique(y_true)) - set(np.unique(y_pred))
    if missing_classes:
        missing_classes = [int(x) for x in sorted(missing_classes)]
        print(f"Warning: {model_name} made no predictions for classes:
    ↪{missing_classes}")

    # Generate classification report
    report = classification_report(y_true, y_pred, digits=4, zero_division=0)

    return report

[31]: def plot_cm_task1(model, X_test, y_test, model_name, save_dir=None):
    """
    Specific for Task 1 (Scikit-Learn models).
    Generates predictions using .predict() and then plots.
    """
    # 1. Get predictions (Scikit-learn models work directly on X_test)
    y_pred = model.predict(X_test)

    # 2. Use the core helper to plot
    # Note: y_test might need to be converted to a simple list/array if it's a
    ↪pandas Series
    _internal_plot_cm(y_test, y_pred, model_name, save_dir)

[32]: # --- Evaluate validation and test set and print classification reports ---

print(f"\nValidation classification report:\n")
report = evaluate_model(clf, X_val, y_val, "Logistic Regression (Validation)")
print(report)

print(f"\nTest classification report:\n")
report = evaluate_model(clf, X_test, y_test, "Logistic Regression (Test)")
print(report)
```

Validation classification report:

	precision	recall	f1-score	support
0	0.6750	0.3320	0.4451	244
1	0.9746	0.9938	0.9841	6286
accuracy			0.9691	6530
macro avg	0.8248	0.6629	0.7146	6530
weighted avg	0.9634	0.9691	0.9639	6530

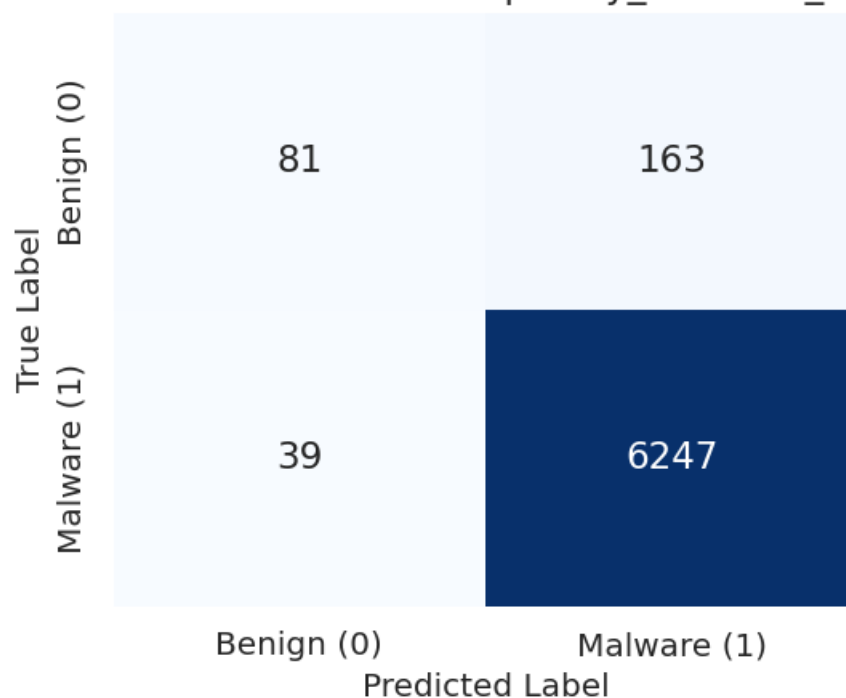
Test classification report:

	precision	recall	f1-score	support
0	0.6885	0.3457	0.4603	243
1	0.9751	0.9939	0.9844	6262
accuracy			0.9697	6505
macro avg	0.8318	0.6698	0.7223	6505
weighted avg	0.9644	0.9697	0.9648	6505

```
[33]: plot_cm_task1(  
    model=clf,  
    X_test=X_val,  
    y_test=y_val,  
    model_name="Frequency_Baseline_val",  
    save_dir=results_path + 'images/task1_plots/'  
)
```

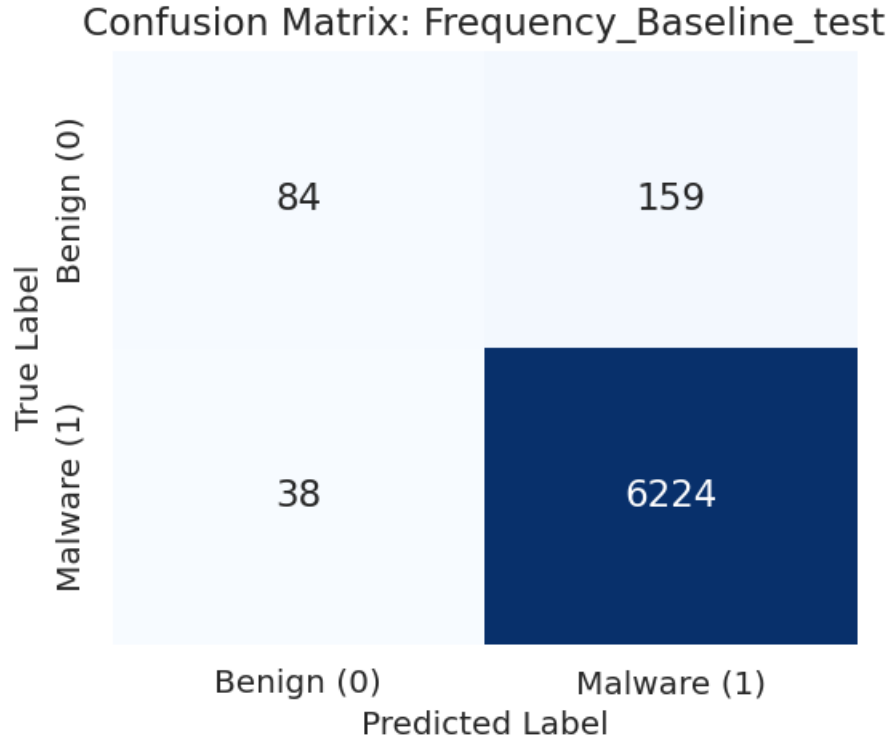
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task1_plots/cm_frequency_baseline_val.png

Confusion Matrix: Frequency_Baseline_val



```
[34]: plot_cm_task1(  
    model=clf,  
    X_test=X_test,  
    y_test=y_test,  
    model_name="Frequency_Baseline_test",  
    save_dir=results_path + 'images/task1_plots/'  
)
```

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task1_plots/cm_frequency_baseline_test.png



Q: Report how you chose the hyperparameters of your classifier, and the final performance on the test set. We used Logistic Regression as a deliberately simple frequency-based baseline. This choice is useful because the goal of Task 1 is not to maximize complexity, but to understand how much discriminative information is already contained in API-call counts before introducing sequence-aware models.

The model was trained with the `liblinear` solver, default L2 regularization (`C=1.0`), no class weighting, and `random_state=42` for reproducibility. We did not perform an extensive hyperparameter search at this stage, because the classifier is meant to be a stable reference point for the later FFNN, RNN, and GNN experiments.

The final test performance is strong in terms of global accuracy: the model reaches **96.97%** accuracy and macro F1 **0.7223**. However, the class-wise report shows a clear imbalance effect. Malware is detected very well, with precision **0.9751** and recall **0.9939**, while the benign class has good precision (**0.6885**) but low recall (**0.3457**). Therefore, the baseline is effective at identifying malware, but it tends to classify many benign samples as malware.

Q: Is the final performance good, even ignoring the order of API calls and handling very sparse vectors? Yes, the final performance is good for a simple frequency-based baseline. Even though the representation is sparse and completely discards the order of API calls, the model still reaches **96.97%** accuracy and macro F1 **0.7223** on the test set. This indicates that the presence and frequency of API calls already contain substantial information for distinguishing malware from benign software.

At the same time, the per-class metrics show the limitation of this approach. The model obtains excellent malware recall (0.9939), but benign recall is only 0.3457. This means that the high accuracy is partly driven by the majority malware class. The baseline is therefore useful and competitive, but not fully balanced: later models should try to improve minority-class recall and macro F1, even if this slightly reduces overall accuracy.

1.3 Task 2 - Feed Forward Neural Network (FFNN)

```
[35]: # Create directory for plots
save_dir = results_path + 'images/' + 'task2_plots/'
os.makedirs(save_dir, exist_ok=True)
```

1.3.1 API Call Statistics

```
[36]: # --- Compute the length (number of API calls) per sequence ---
df_train['seq_length'] = df_train['api_call_sequence'].apply(len)
df_test['seq_length'] = df_test['api_call_sequence'].apply(len)

# --- Basic descriptive statistics ---
print("Train set sequence length stats:")
print(df_train['seq_length'].describe(), "\n")

print("Test set sequence length stats:")
print(df_test['seq_length'].describe(), "\n")

# --- Check if all sequences have the same length ---
same_length_train = df_train['seq_length'].nunique() == 1
same_length_test = df_test['seq_length'].nunique() == 1
print(f"All train sequences same length? {same_length_train}")
print(f"All test sequences same length? {same_length_test}\n")
```

Train set sequence length stats:

count	16325.000000
mean	75.030812
std	8.951411
min	60.000000
25%	67.000000
50%	75.000000
75%	83.000000
max	90.000000

Name: seq_length, dtype: float64

Test set sequence length stats:

count	6505.000000
mean	86.331130
std	9.113511
min	70.000000
25%	79.000000

```

50%      87.000000
75%      94.000000
max      100.000000
Name: seq_length, dtype: float64

```

```

All train sequences same length? False
All test sequences same length? False

```

```
[37]: df_train
```

```

[37]:
          api_call_sequence  is_malware  \
0      [LdrGetDllHandle, LdrGetProcedureAddress, LdrL...      1
1      [NtAllocateVirtualMemory, LdrLoadDll, LdrGetPr...      1
2      [FindResourceExW, LoadResource, FindResourceEx...      1
3      [FindResourceExW, LoadResource, FindResourceEx...      1
4      [LdrGetProcedureAddress, SetErrorMode, LdrLoad...      1
...
16320  [LdrGetProcedureAddress, LdrLoadDll, LdrGetPro...      1
16321  [NtClose, LdrGetProcedureAddress, CryptCreateH...      1
16322  [LdrGetProcedureAddress, LdrGetDllHandle, LdrG...      1
16323  [LdrGetProcedureAddress, LdrGetDllHandle, LdrG...      1
16324  [SetFilePointer, NtReadFile, SetFilePointer, N...      1

      seq_length
0              73
1              88
2              79
3              71
4              63
...
16320          64
16321          78
16322          84
16323          62
16324          77

[16325 rows x 3 columns]

```

```

[38]: # --- Compare distributions visually ---

fig, ax = plt.subplots(figsize=(6, 4))
ax.hist(df_train['seq_length'], bins=50, alpha=0.6, label='Train', density=True)
ax.hist(df_test['seq_length'], bins=50, alpha=0.6, label='Test', density=True)
ax.set_xlabel("Number of API calls per sequence",
              ↪fontsize=REPORT_PLOT_LABEL_SIZE)
ax.set_ylabel("Density", fontsize=REPORT_PLOT_LABEL_SIZE)

```

```

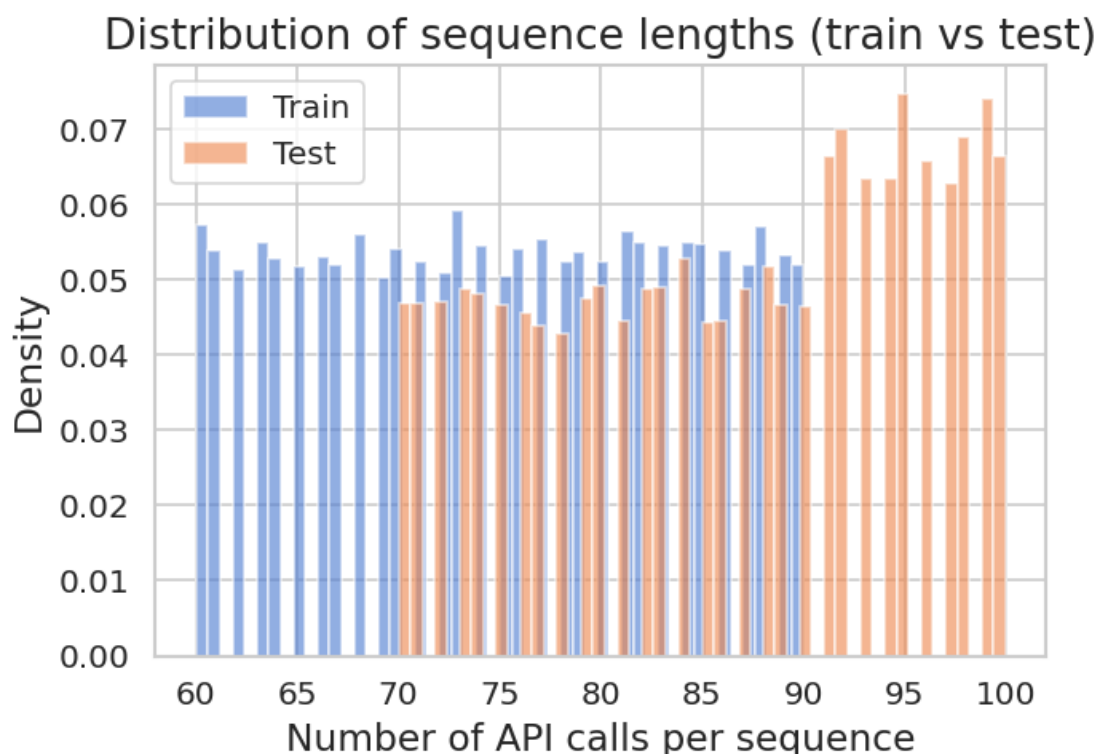
ax.set_title("Distribution of sequence lengths (train vs test)",
            ↪fontsize=REPORT_PLOT_TITLE_SIZE)
format_report_axes(ax)

# Save the plot to the specified path
save_plot(fig, f"distribution_sequence_lengths_train_test", save_dir)

plt.show()

```

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task2_plots/distribution_sequence_lengths_train_test.png



Q: Do you have the same number of API calls per sequence? If not, is the distribution of API calls per sequence the same for training and test sets? No, the sequences **do not** all have the same length in either the training or test sets.

- **Training set:** sequence lengths range from 60 to 90 API calls (mean = 75).
- **Test set:** sequence lengths range from 70 to 100 API calls (mean = 86).

The **sequences in the test set** are not only longer on average, but also fall within a range (90–100) that is completely absent from the training set. This discrepancy means that the model will have to generalize to sequence lengths it has never encountered during training, posing an additional challenge for sequential architectures.

Q: Can a FFNN handle a variable number of elements? If not, why? No, a feedforward neural network (FFNN) **cannot directly handle a variable number** of elements per input.

- FFNNs require a **fixed-size input vector**, because each input neuron corresponds to a specific feature, and the weight matrix has a fixed shape.
- If the number of elements (features) changes between samples, the network **cannot align inputs with the learned weights**, so it won't work properly.

Due to this architectural constraint, in order to process variable-length API sequences using a FFNN, they must be converted into fixed-length vectors using padding (to extend short sequences) or truncation (to shorten longer ones).

1.3.2 Fixed-Size Sequences

Q: How to estimate a fixed-size candidate? Which partition do you use to estimate it? To estimate a fixed-size input for an FFNN, it's possible to choose a representative sequence length from the training set only, since the model should not use information from the test set.

Common strategies include: 1. **Maximum length**: ensures all training sequences fit, but can be too large and waste memory. 2. **Median length**: gives a typical sequence size, but some longer sequences will need truncation. 3. **75th percentile length**: a compromise that covers most sequences without being overly large.

Among these strategies, we opted for the maximum training set length (90). This choice is justified by the fact that the distribution of lengths (as seen earlier) is fairly uniform and does not contain any extreme outliers; therefore, the computational “cost” of using 90 as a fixed size is offset by the advantage of not having to truncate (and thus lose) information in any training sequence.

```
[39]: # Compute sequence lengths in training set
seq_lengths = df_train['api_call_sequence'].apply(len)

# Option 1: maximum length
max_len = seq_lengths.max()
print(f"Max length: {max_len}")

# Option 2: median
median_len = int(seq_lengths.median())
print(f"Median length: {median_len}")

# Option 3: 75th percentile
p75_len = int(np.percentile(seq_lengths, 75))
print(f"75th percentile length: {p75_len}")
```

Max length: 90

Median length: 75

75th percentile length: 83

Q: Given the estimate, what technique could you use to obtain the same number of API calls per sequence? To obtain the same number of API calls per sequence, the main strategies are:

1. **Padding:** Add “dummy” or special tokens to sequences **shorter than the fixed length** so they reach the target size.
2. **Truncation:** Cut sequences **longer than the fixed length** to fit the target size.

By combining padding and truncation based on the estimated fixed length, you can ensure that **all sequences have the same number of elements**, which is required for an FFNN.

```
[40]: def pad_truncate_sequence(seq, fixed_len, pad_value=0):
    """
    Convert a variable-length sequence into a fixed-length sequence.

    Args:
        seq (list): original sequence (list of API calls)
        fixed_len (int): target length L
        pad_value: value to use for padding (0 is common for integers)

    Returns:
        list of length fixed_len
    """
    if len(seq) < fixed_len:
        # Pad at the end
        seq_padded = seq + [pad_value] * (fixed_len - len(seq))
    else:
        # Truncate at the end
        seq_padded = seq[:fixed_len]
    return seq_padded
```

```
[41]: # --- Perform padding or truncating on the training data ---

L = max_len # chosen fixed size

# Apply the padding or truncating to all the dataframe
df_train['seq_fixed'] = df_train['api_call_sequence'].apply(lambda x:
    ↪pad_truncate_sequence(x, L))
```

```
[42]: # --- Inspect the train set (e.g. the first row) ---

print("First row of the train set before padding:")
print(df_train['api_call_sequence'].iloc[0])
print(f"Lenght of the sequence: {len(df_train['api_call_sequence'].iloc[0])}")
    ↪# should be less than L

print("\nFirst row of the train set after padding:")
print(df_train['seq_fixed'].iloc[0])
print(f"Lenght of the sequence: {len(df_train['seq_fixed'].iloc[0])}") # should
    ↪be exactly L
```

First row of the train set before padding:
 ['LdrGetDllHandle', 'LdrGetProcedureAddress', 'LdrLoadDll',

```

'LdrGetProcedureAddress', 'LdrGetDllHandle', 'LdrGetProcedureAddress',
'GetTimeZoneInformation', 'LoadStringW', 'RegOpenKeyExW', 'RegQueryValueExW',
'RegCloseKey', 'LdrGetDllHandle', 'LdrGetProcedureAddress', 'LdrGetDllHandle',
'LdrGetProcedureAddress', 'SetErrorMode', 'LdrLoadDll', 'SetErrorMode',
'LdrLoadDll', 'SetErrorMode', 'GetSystemMetrics', 'FindResourceExW',
'OleInitialize', 'FindResourceExW', 'LoadResource', 'FindResourceExW',
'LoadResource', 'NtClose', 'NtAllocateVirtualMemory', 'GetSystemMetrics',
'NtAllocateVirtualMemory', 'LdrLoadDll', 'LdrGetProcedureAddress', 'LdrLoadDll',
'LdrGetProcedureAddress', 'LdrLoadDll', 'LdrGetProcedureAddress', 'LdrLoadDll',
'LdrGetProcedureAddress', 'LdrLoadDll', 'LdrGetProcedureAddress',
'LookupAccountSidW', 'LdrGetProcedureAddress', 'LookupAccountSidW',
'LdrLoadDll', 'LdrGetProcedureAddress', 'LdrLoadDll', 'LdrGetProcedureAddress',
'LdrLoadDll', 'LdrGetProcedureAddress', 'LdrGetDllHandle', 'FindResourceExW',
'LoadResource', 'FindResourceExW', 'LoadResource', 'DrawTextExW',
'GetSystemMetrics', 'FindResourceExW', 'LoadResource', 'GetSystemMetrics',
'DrawTextExW', 'LdrGetDllHandle', 'FindResourceExW', 'LoadResource',
'FindResourceExW', 'LoadResource', 'FindResourceExW', 'LoadResource',
'LdrGetDllHandle', 'LdrGetProcedureAddress', 'LoadStringW', 'LdrGetDllHandle',
'LdrGetProcedureAddress']

```

Lenght of the sequence: 73

First row of the train set after padding:

```

['LdrGetDllHandle', 'LdrGetProcedureAddress', 'LdrLoadDll',
'LdrGetProcedureAddress', 'LdrGetDllHandle', 'LdrGetProcedureAddress',
'GetTimeZoneInformation', 'LoadStringW', 'RegOpenKeyExW', 'RegQueryValueExW',
'RegCloseKey', 'LdrGetDllHandle', 'LdrGetProcedureAddress', 'LdrGetDllHandle',
'LdrGetProcedureAddress', 'SetErrorMode', 'LdrLoadDll', 'SetErrorMode',
'LdrLoadDll', 'SetErrorMode', 'GetSystemMetrics', 'FindResourceExW',
'OleInitialize', 'FindResourceExW', 'LoadResource', 'FindResourceExW',
'LoadResource', 'NtClose', 'NtAllocateVirtualMemory', 'GetSystemMetrics',
'NtAllocateVirtualMemory', 'LdrLoadDll', 'LdrGetProcedureAddress', 'LdrLoadDll',
'LdrGetProcedureAddress', 'LdrLoadDll', 'LdrGetProcedureAddress', 'LdrLoadDll',
'LdrGetProcedureAddress', 'LdrLoadDll', 'LdrGetProcedureAddress',
'LookupAccountSidW', 'LdrGetProcedureAddress', 'LookupAccountSidW',
'LdrLoadDll', 'LdrGetProcedureAddress', 'LdrLoadDll', 'LdrGetProcedureAddress',
'LdrLoadDll', 'LdrGetProcedureAddress', 'LdrGetDllHandle', 'FindResourceExW',
'LoadResource', 'FindResourceExW', 'LoadResource', 'DrawTextExW',
'GetSystemMetrics', 'FindResourceExW', 'LoadResource', 'GetSystemMetrics',
'DrawTextExW', 'LdrGetDllHandle', 'FindResourceExW', 'LoadResource',
'FindResourceExW', 'LoadResource', 'FindResourceExW', 'LoadResource',
'LdrGetDllHandle', 'LdrGetProcedureAddress', 'LoadStringW', 'LdrGetDllHandle',
'LdrGetProcedureAddress', 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]

```

Lenght of the sequence: 90

Q: If at test time you have more API calls than the fixed-size, what do you do with the exceeding API calls? At test time, if a sequence has **more API calls than the fixed size**, the exceeding API calls should be **truncated** to fit the fixed length. Sequences shorter

than the fixed size are **padded** with dummy tokens.

We have decided to perform the same pad or truncate action on the test set, so that all sequences have a consistent length compatible with the FFNN.

```
[43]: # To ensure consistency, we apply the padding or truncating to also on the test
      ↪df
      df_test['seq_fixed'] = df_test['api_call_sequence'].apply(lambda x:
      ↪pad_truncate_sequence(x, L))
```

```
[44]: df_train
```

```
[44]:
```

		api_call_sequence	is_malware	\
0		[LdrGetDllHandle, LdrGetProcedureAddress, LdrL...	1	
1		[NtAllocateVirtualMemory, LdrLoadDll, LdrGetPr...	1	
2		[FindResourceExW, LoadResource, FindResourceEx...	1	
3		[FindResourceExW, LoadResource, FindResourceEx...	1	
4		[LdrGetProcedureAddress, SetErrorMode, LdrLoad...	1	
...		...	...	
16320		[LdrGetProcedureAddress, LdrLoadDll, LdrGetPro...	1	
16321		[NtClose, LdrGetProcedureAddress, CryptCreateH...	1	
16322		[LdrGetProcedureAddress, LdrGetDllHandle, LdrG...	1	
16323		[LdrGetProcedureAddress, LdrGetDllHandle, LdrG...	1	
16324		[SetFilePointer, NtReadFile, SetFilePointer, N...	1	

	seq_length		seq_fixed
0	73	[LdrGetDllHandle, LdrGetProcedureAddress, LdrL...	
1	88	[NtAllocateVirtualMemory, LdrLoadDll, LdrGetPr...	
2	79	[FindResourceExW, LoadResource, FindResourceEx...	
3	71	[FindResourceExW, LoadResource, FindResourceEx...	
4	63	[LdrGetProcedureAddress, SetErrorMode, LdrLoad...	
...	...	...	...
16320	64	[LdrGetProcedureAddress, LdrLoadDll, LdrGetPro...	
16321	78	[NtClose, LdrGetProcedureAddress, CryptCreateH...	
16322	84	[LdrGetProcedureAddress, LdrGetDllHandle, LdrG...	
16323	62	[LdrGetProcedureAddress, LdrGetDllHandle, LdrG...	
16324	77	[SetFilePointer, NtReadFile, SetFilePointer, N...	

[16325 rows x 4 columns]

```
[45]: df_test
```

```
[45]:
```

		api_call_sequence	is_malware	\
0		[NtQueryValueKey, NtClose, NtOpenKey, NtQueryV...	1	
1		[LdrGetProcedureAddress, NtClose, NtOpenKey, N...	1	
2		[NtOpenKey, NtQueryValueKey, NtClose, NtOpenKe...	1	
3		[NtAllocateVirtualMemory, LdrLoadDll, LdrGetPr...	1	
4		[NtOpenKey, NtQueryValueKey, NtClose, LdrGetPr...	1	

```

...
6500 [SetErrorMode, NtOpenFile, NtClose, SHGetFolde... 0
6501 [NtProtectVirtualMemory, RegOpenKeyExW, RegQue... 1
6502 [RegOpenKeyExW, RegQueryValueExW, RegCloseKey,... 1
6503 [NtQueryValueKey, NtClose, RegOpenKeyExA, RegQ... 1
6504 [NtClose, LdrGetProcedureAddress, CryptCreateH... 1

      seq_length      seq_fixed
0          98 [NtQueryValueKey, NtClose, NtOpenKey, NtQueryV...
1          98 [LdrGetProcedureAddress, NtClose, NtOpenKey, N...
2          99 [NtOpenKey, NtQueryValueKey, NtClose, NtOpenKe...
3          85 [NtAllocateVirtualMemory, LdrLoadDll, LdrGetPr...
4          80 [NtOpenKey, NtQueryValueKey, NtClose, LdrGetPr...
...
6500      75 [SetErrorMode, NtOpenFile, NtClose, SHGetFolde...
6501      83 [NtProtectVirtualMemory, RegOpenKeyExW, RegQue...
6502      70 [RegOpenKeyExW, RegQueryValueExW, RegCloseKey,...
6503      96 [NtQueryValueKey, NtClose, RegOpenKeyExA, RegQ...
6504      78 [NtClose, LdrGetProcedureAddress, CryptCreateH...

```

[6505 rows x 4 columns]

1.3.3 Handling Categorical Features

```
[46]: # --- Build a unified vocabulary ---
```

```

all_sequences = df_train['seq_fixed']

# Flatten all sequences into a single list of API calls
all_api_calls = [api for seq in all_sequences for api in seq]

```

```
[47]: # --- Build vocabulary with padding handling ---
```

```

# 1. Filter strings only. We ignore 0 here because we will manually assign it
↳ later.
unique_api_calls = sorted(list(set(api for api in all_api_calls if
↳ isinstance(api, str))))

# 2. Define Special Tokens
PAD_IDX = 0 # <PAD>
UNK_IDX = 1 # <UNK>

# 3. Map API call -> sequential ID (Start from 2 to save space for PAD and UNK)
api_to_id = {api: i + 2 for i, api in enumerate(unique_api_calls)}

# 4. Add the special tokens to the dictionary
# (Optional: helpful if you need to reverse look up '<PAD>' later)

```

```

api_to_id['<PAD>'] = PAD_IDX
api_to_id['<UNK>'] = UNK_IDX

# Map sequential ID -> API call (for verification/debugging)
id_to_api = {i: api for api, i in api_to_id.items()}

print(f"Number of unique API calls (including special tokens): {len(api_to_id)}")
print("\nExample mapping (API -> ID):", list(api_to_id.items())[:5])
print("\nExample mapping (ID -> API):", list(id_to_api.items())[:5])

```

Number of unique API calls (including special tokens): 260

Example mapping (API -> ID): [('CertOpenStore', 2), ('CertOpenSystemStoreW', 3), ('CoCreateInstance', 4), ('CoCreateInstanceEx', 5), ('CoGetClassObject', 6)]

Example mapping (ID -> API): [(2, 'CertOpenStore'), (3, 'CertOpenSystemStoreW'), (4, 'CoCreateInstance'), (5, 'CoCreateInstanceEx'), (6, 'CoGetClassObject')]

Sequential Identifiers Approach

```

[48]: def sequence_to_ids(seq, mapping, pad_idx, unk_idx):
    """
    Convert a sequence of API calls to their corresponding IDs.
    - If item is 0 (integer padding), map to pad_idx.
    - If item is string but not in mapping, map to unk_idx.
    """
    ids = []
    for item in seq:
        # Case A: It is existing padding (integer 0)
        if item == 0:
            ids.append(pad_idx)
        # Case B: It is a known string API
        elif item in mapping:
            ids.append(mapping[item])
        # Case C: It is an unknown string API (Test set OOV)
        else:
            ids.append(unk_idx)
    return ids

```

```

[49]: # --- Convert sequences to sequential IDs ---

# Apply conversion to train and test sets
# Note: We pass UNK_IDX (1) so unknown calls in Test set become 1.
df_train['seq_sequential_ids'] = df_train['seq_fixed'].apply(
    lambda seq: sequence_to_ids(seq, api_to_id, PAD_IDX, UNK_IDX)
)

```

```

df_test['seq_sequential_ids'] = df_test['seq_fixed'].apply(
    lambda seq: sequence_to_ids(seq, api_to_id, PAD_IDX, UNK_IDX)
)

# Quick check
print("\nSequential IDs - first row of train set:")
print(df_train['seq_sequential_ids'].iloc[0])
print("Length of the sequence:", len(df_train['seq_sequential_ids'].iloc[0]))

print("\nSequential IDs - first row of test set:")
print(df_test['seq_sequential_ids'].iloc[0])
print("Length of the sequence:", len(df_test['seq_sequential_ids'].iloc[0]))

# Verify that the logic works (check for any None values)
# If this prints nothing, your data is clean!
for seq in df_test['seq_sequential_ids']:
    if None in seq:
        print("Error: Found None in sequence! Check mapping.")
        break

```

Sequential IDs - first row of train set:

```

[117, 118, 119, 118, 117, 118, 86, 123, 201, 205, 188, 117, 118, 117, 118, 219,
119, 219, 119, 219, 81, 46, 179, 46, 121, 46, 121, 134, 133, 81, 133, 119, 118,
119, 118, 119, 118, 119, 118, 119, 118, 124, 118, 124, 119, 118, 119, 118, 119,
118, 117, 46, 121, 46, 121, 39, 81, 46, 121, 81, 39, 117, 46, 121, 46, 121, 46,
121, 117, 118, 123, 117, 118, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
Length of the sequence: 90

```

Sequential IDs - first row of test set:

```

[164, 134, 152, 164, 134, 159, 122, 133, 122, 117, 118, 117, 118, 117, 122, 118,
81, 46, 121, 46, 121, 46, 121, 46, 121, 46, 121, 46, 121, 46, 121, 46, 121, 46,
121, 46, 121, 46, 121, 46, 121, 46, 121, 46, 121, 118, 46, 121, 46, 121, 133,
81, 134, 81, 133, 119, 118, 117, 46, 121, 46, 121, 39, 81, 46, 121, 81, 39, 117,
46, 121, 46, 121, 46, 121, 117, 118, 122, 117, 118, 117, 118, 81, 44, 121, 229,
133, 119, 118, 84]
Length of the sequence: 90

```

[50]: df_train

```

[50]:
      api_call_sequence  is_malware  \
0  [LdrGetDllHandle, LdrGetProcedureAddress, LdrL...      1
1  [NtAllocateVirtualMemory, LdrLoadDll, LdrGetPr...      1
2  [FindResourceExW, LoadResource, FindResourceEx...      1
3  [FindResourceExW, LoadResource, FindResourceEx...      1
4  [LdrGetProcedureAddress, SetErrorMode, LdrLoad...      1
...  ...  ...
16320 [LdrGetProcedureAddress, LdrLoadDll, LdrGetPro...      1

```

16321	[NtClose, LdrGetProcedureAddress, CryptCreateH...	1
16322	[LdrGetProcedureAddress, LdrGetDllHandle, LdrG...	1
16323	[LdrGetProcedureAddress, LdrGetDllHandle, LdrG...	1
16324	[SetFilePointer, NtReadFile, SetFilePointer, N...	1

	seq_length		seq_fixed \
0	73	[LdrGetDllHandle, LdrGetProcedureAddress, LdrL...	
1	88	[NtAllocateVirtualMemory, LdrLoadDll, LdrGetPr...	
2	79	[FindResourceExW, LoadResource, FindResourceEx...	
3	71	[FindResourceExW, LoadResource, FindResourceEx...	
4	63	[LdrGetProcedureAddress, SetErrorMode, LdrLoad...	
...	...	...	
16320	64	[LdrGetProcedureAddress, LdrLoadDll, LdrGetPro...	
16321	78	[NtClose, LdrGetProcedureAddress, CryptCreateH...	
16322	84	[LdrGetProcedureAddress, LdrGetDllHandle, LdrG...	
16323	62	[LdrGetProcedureAddress, LdrGetDllHandle, LdrG...	
16324	77	[SetFilePointer, NtReadFile, SetFilePointer, N...	

	seq_sequential_ids
0	[117, 118, 119, 118, 117, 118, 86, 123, 201, 2...
1	[133, 119, 118, 82, 80, 137, 82, 152, 153, 163...
2	[46, 121, 46, 121, 46, 121, 46, 121, 46, 121, ...
3	[46, 121, 46, 121, 46, 121, 46, 121, 46, 121, ...
4	[118, 219, 119, 219, 81, 46, 121, 46, 121, 46,...
...	...
16320	[118, 119, 118, 119, 118, 119, 118, 119, 118, ...
16321	[134, 118, 24, 118, 30, 118, 135, 66, 221, 166...
16322	[118, 117, 118, 81, 216, 119, 118, 46, 121, 46...
16323	[118, 117, 118, 117, 118, 117, 118, 117, 118, ...
16324	[221, 166, 221, 166, 221, 45, 121, 133, 146, 1...

[16325 rows x 5 columns]

```
[51]: df_test
```

```
[51]:
```

	api_call_sequence	is_malware \
0	[NtQueryValueKey, NtClose, NtOpenKey, NtQueryV...	1
1	[LdrGetProcedureAddress, NtClose, NtOpenKey, N...	1
2	[NtOpenKey, NtQueryValueKey, NtClose, NtOpenKe...	1
3	[NtAllocateVirtualMemory, LdrLoadDll, LdrGetPr...	1
4	[NtOpenKey, NtQueryValueKey, NtClose, LdrGetPr...	1
...	...	...
6500	[SetErrorMode, NtOpenFile, NtClose, SHGetFolde...	0
6501	[NtProtectVirtualMemory, RegOpenKeyExW, RegQue...	1
6502	[RegOpenKeyExW, RegQueryValueExW, RegCloseKey,...	1
6503	[NtQueryValueKey, NtClose, RegOpenKeyExA, RegQ...	1
6504	[NtClose, LdrGetProcedureAddress, CryptCreateH...	1

```

seq_length                                seq_fixed \
0      98 [NtQueryValueKey, NtClose, NtOpenKey, NtQueryV...
1      98 [LdrGetProcedureAddress, NtClose, NtOpenKey, N...
2      99 [NtOpenKey, NtQueryValueKey, NtClose, NtOpenKe...
3      85 [NtAllocateVirtualMemory, LdrLoadDll, LdrGetPr...
4      80 [NtOpenKey, NtQueryValueKey, NtClose, LdrGetPr...
...      ...
6500    75 [SetErrorMode, NtOpenFile, NtClose, SHGetFolde...
6501    83 [NtProtectVirtualMemory, RegOpenKeyExW, RegQue...
6502    70 [RegOpenKeyExW, RegQueryValueExW, RegCloseKey,...
6503    96 [NtQueryValueKey, NtClose, RegOpenKeyExA, RegQ...
6504    78 [NtClose, LdrGetProcedureAddress, CryptCreateH...

                                seq_sequential_ids
0      [164, 134, 152, 164, 134, 159, 122, 133, 122, ...
1      [118, 134, 152, 164, 134, 152, 164, 134, 119, ...
2      [152, 164, 134, 152, 164, 134, 117, 118, 117, ...
3      [133, 119, 118, 119, 118, 82, 200, 204, 188, 1...
4      [152, 164, 134, 118, 24, 118, 30, 118, 135, 66...
...      ...
6500    [219, 151, 134, 214, 219, 63, 219, 151, 134, 1...
6501    [158, 201, 205, 188, 140, 133, 158, 133, 146, ...
6502    [201, 205, 188, 201, 205, 188, 80, 117, 118, 2...
6503    [164, 134, 200, 204, 188, 153, 164, 134, 119, ...
6504    [134, 118, 24, 118, 30, 118, 135, 66, 221, 166...

[6505 rows x 5 columns]

```

Learnable Embeddings Approach

```

[52]: # The learnable embeddings approach is implemented in the first layer of the
      ↪neural network model.
      # The nn.Embedding layer in the MalwareFFNN class handles this process.

```

1.3.4 FFNN Training

```

[53]: class MalwareFFNN(nn.Module):
      def __init__(self, seq_len, hidden_dims=[128,64,32],
                    dropout_rates=None, batchnorm_flags=None,
                    use_embedding=True, vocab_size=None, embedding_dim=16,
                    dropout_embedding_rate=0.1):
          """
          Flexible FFNN for malware classification.

          Parameters:
          - seq_len: length of each API call sequence

```



```

        - vocab_size: number of unique API calls (required if
↪use_embedding=True)
        - hidden_dims: list of hidden layer sizes
        - embedding_dim: size of embedding vectors (only used if
↪use_embedding=True)
        - use_embedding: whether to use an embedding layer (True) or raw
↪sequential IDs / one-hot (False)
    """
    super(MalwareFFNN, self).__init__()
    self.use_embedding = use_embedding

    if self.use_embedding:
        if vocab_size is None:
            raise ValueError("vocab_size must be provided if
↪use_embedding=True")

        # Embedding layer for sequential IDs
        self.embedding = nn.Embedding(num_embeddings=vocab_size,
↪embedding_dim=embedding_dim, padding_idx=0)
        input_dim = seq_len * embedding_dim

        self.dropout_embed = nn.Dropout(dropout_embedding_rate)
    else:
        # Input will be one-hot encoded or numeric vector
        input_dim = seq_len # assumes IDs are already numeric

    # Ensure defaults if none are provided
    if dropout_rates is None:
        dropout_rates = [0.0] * len(hidden_dims) # no dropout by default
    if batchnorm_flags is None:
        batchnorm_flags = [False] * len(hidden_dims) # no batchnorm by
↪default

    # Safety check for lengths
    assert len(dropout_rates) == len(hidden_dims), \
        "dropout_rates length must match hidden_dims length"
    assert len(batchnorm_flags) == len(hidden_dims), \
        "batchnorm_flags length must match hidden_dims length"

    # Build hidden layers
    layers = []
    prev_dim = input_dim
    for h_dim, dropout_rate, use_bn in zip(hidden_dims, dropout_rates,
↪batchnorm_flags):
        layers.append(nn.Linear(prev_dim, h_dim))
        if use_bn:

```

```

        layers.append(nn.BatchNorm1d(h_dim))
    layers.append(nn.ReLU())
    if dropout_rate > 0:
        layers.append(nn.Dropout(dropout_rate))
    prev_dim = h_dim
    self.ffnn = nn.Sequential(*layers)

    # Output layer: binary classification
    self.output_layer = nn.Linear(prev_dim, 1)

def forward(self, x):
    if self.use_embedding:
        # x: [batch_size, seq_len] of IDs
        x = self.embedding(x)      # [batch_size, seq_len, embedding_dim]
        x = self.dropout_embed(x)
        x = x.view(x.size(0), -1) # flatten: [batch_size, ↵
↵seq_len*embedding_dim]
    else:
        # x: [batch_size, seq_len] or [batch_size, seq_len*vocab_size] if ↵
↵one-hot flattened
        x = x.float()

    x = self.ffnn(x)
    out = self.output_layer(x) # [batch_size, 1]
    return out

```

```

[54]: # --- Training function with early stopping ---

def train_model(model, train_loader, val_loader, epochs, optimizer, criterion, ↵
↵min_delta=None, patience=None, scheduler=None):
    """
    Train a PyTorch model with early stopping and metric tracking.
    Returns: model, history dictionary (losses + metrics), and elapsed time.
    """

    # --- DEVICE DETECTION ---
    device = next(model.parameters()).device

    # --- HISTORY STORAGE ---
    history = {
        'train_loss': [],
        'val_loss': [],
        'val_f1_macro': [],
        'val_f1_weighted': [],
    }

    best_val_loss = float('inf')

```

```

best_model_state = None
trigger_times = 0

start_time = time.time()

for epoch in range(epochs):
    # --- TRAINING ---
    model.train()
    batch_losses = []
    for X_batch, y_batch in train_loader:
        X_batch, y_batch = X_batch.to(device), y_batch.to(device)
        optimizer.zero_grad()
        outputs = model(X_batch)
        loss = criterion(outputs, y_batch)
        loss.backward()
        optimizer.step()
        batch_losses.append(loss.item())
    train_loss = np.mean(batch_losses) if batch_losses else 0.0

    # --- VALIDATION & METRICS ---
    model.eval()
    val_batch_losses = []
    all_preds = []
    all_targets = []

    with torch.no_grad():
        for X_batch, y_batch in val_loader:
            X_batch, y_batch = X_batch.to(device), y_batch.to(device)
            outputs = model(X_batch)
            loss = criterion(outputs, y_batch)
            val_batch_losses.append(loss.item())

        # Convert logits to probabilities and then binary predictions
        ↪ (0 or 1)

        # Assumes Binary Classification (BCEWithLogitsLoss)
        probs = torch.sigmoid(outputs)
        preds = (probs > 0.5).float()

        # Store for epoch-level metric calculation
        all_preds.extend(preds.cpu().numpy())
        all_targets.extend(y_batch.cpu().numpy())

    val_loss = np.mean(val_batch_losses) if val_batch_losses else 0.0

    # Calculate Metrics for this Epoch
    epoch_f1_macro = f1_score(all_targets, all_preds, average='macro', ↪
    ↪ zero_division=0)

```

```

        epoch_f1_weighted = f1_score(all_targets, all_preds,
↪average='weighted', zero_division=0)

    # Update History
    history['train_loss'].append(train_loss)
    history['val_loss'].append(val_loss)
    history['val_f1_macro'].append(epoch_f1_macro)
    history['val_f1_weighted'].append(epoch_f1_weighted)

    # --- SCHEDULER STEP ---
    if scheduler is not None:
        last_lr = optimizer.param_groups[0]['lr']
        if isinstance(scheduler, torch.optim.lr_scheduler.
↪ReduceLROnPlateau):
            scheduler.step(val_loss)
        else:
            scheduler.step()
        curr_lr = optimizer.param_groups[0]['lr']
        if curr_lr != last_lr:
            print(f"Epoch {epoch}: LR reduced from {last_lr:.6f} to
↪{curr_lr:.6f}")

    # --- EARLY STOPPING ---
    if min_delta is not None:
        if val_loss < best_val_loss - min_delta:
            best_val_loss = val_loss
            # Save best model state
            best_model_state = {k: v.cpu().clone() for k, v in model.
↪state_dict().items()}
            trigger_times = 0
        else:
            trigger_times += 1
            if trigger_times >= patience:
                print(f"Early stopping at epoch {epoch+1} (best val loss:
↪{best_val_loss:.6f})")
                break

    if (epoch+1) % 5 == 0 or epoch == 0 or epoch == epochs:
        print(f"Epoch {epoch+1}/{epochs} - Train Loss: {train_loss:.4f},
↪Val Loss: {val_loss:.4f}, Val F1(W): {epoch_f1_weighted:.4f}")

    # Restore best model after training loop completes
    if best_model_state is not None:
        model.load_state_dict(best_model_state)
        model.to(device)

```

```

elapsed = time.time() - start_time
elapsed_str = time.strftime('%H:%M:%S', time.gmtime(elapsed))
print(f'\nTraining completed in {elapsed_str} (hh:mm:ss) - {elapsed:.2f}␣
↪seconds')

return model, history, elapsed

```

```

[55]: def evaluate_model(model, X_tensor, y_true, model_name: str = "Unnamed model"):
    """
    Evaluate a trained model on a given dataset and return the classification␣
    ↪report.

    Handles missing predicted classes gracefully (zero_division=0) and
    reports which classes were not predicted, along with the model/config name.
    """
    model.eval()
    with torch.no_grad():
        outputs = model(X_tensor)
        probs = torch.sigmoid(outputs)
        y_pred = (probs > 0.5).long().cpu().numpy().flatten() # Binary␣
        ↪prediction (0 or 1)

    # Convert y_true to numpy if it's a tensor
    if isinstance(y_true, torch.Tensor):
        y_true = y_true.cpu().numpy().flatten()

    # Identify missing classes (not predicted at all)
    missing_classes = set(np.unique(y_true)) - set(np.unique(y_pred))
    if missing_classes:
        missing_classes = [int(x) for x in sorted(missing_classes)]
        print(f"Warning: {model_name} made no predictions for classes:␣
        ↪{missing_classes}")

    # Generate classification report without raising warnings
    report = classification_report(y_true, y_pred, digits=4, zero_division=0)

    return report

```

```

[56]: def plot_cm_task2(model, X_tensor, y_tensor, model_name, save_dir=None):
    """
    Specific for FFNN. Takes tensors, moves to device, predicts, and plots.
    """
    # Ensure model is in eval mode
    model.eval()

    # Detect device (cuda or cpu) from the model parameters
    device = next(model.parameters()).device

```

```

# Move inputs to the same device as model
X_input = X_tensor.to(device)

with torch.no_grad():
    # Get logits
    outputs = model(X_input)
    # Apply sigmoid to get probabilities
    probs = torch.sigmoid(outputs)
    # Convert to binary predictions (0 or 1)
    y_pred = (probs > 0.5).long().cpu().numpy().flatten()

# Ensure y_true is on CPU and numpy
y_true = y_tensor.cpu().numpy().flatten()

# Plot
_internal_plot_cm(y_true, y_pred, model_name, save_dir)

```

```

[57]: def plot_training_metrics(histories, model_names=None, save_dir=None,
    ↪ plot_name="validation_f1_macro_comparison"):
    """
    Compare multiple models' validation F1 Macro across epochs.
    Only shows validation F1 macro (no weighted, no markers for cleaner
    ↪ plots).
    """
    # Normalize input
    if isinstance(histories, dict):
        hist_map = histories
        names = [f"lr_{k:.0e}" if isinstance(k, float) else str(k) for k in
    ↪ hist_map.keys()]
        hist_map = dict(zip(names, hist_map.values()))
    else:
        if model_names is None:
            names = [f"Model_{i+1}" for i in range(len(histories))]
        else:
            names = list(model_names)
            hist_map = dict(zip(names, histories))

    # Validate
    for name, h in hist_map.items():
        if 'val_f1_macro' not in h:
            raise KeyError(f"History for '{name}' must contain 'val_f1_macro'."
    ↪)

    colors = ['#E74C3C', '#F39C12', '#27AE60', '#3498DB', '#9B59B6',
    ↪ '#34495E']

```

```

fig, ax = plt.subplots(figsize=(10, 5))

for idx, (name, h) in enumerate(hist_map.items()):
    epochs = range(1, len(h['val_f1_macro']) + 1)
    color = colors[idx % len(colors)]
    ax.plot(epochs, h['val_f1_macro'], label=name,
↳linewidth=REPORT_PLOT_LINEWIDTH, color=color)

    ax.set_title("Validation F1 Macro Comparison",
↳fontsize=REPORT_PLOT_TITLE_SIZE)
    ax.set_xlabel("Epochs", fontsize=REPORT_PLOT_LABEL_SIZE)
    ax.set_ylabel("F1 Macro", fontsize=REPORT_PLOT_LABEL_SIZE)
    ax.grid(True, alpha=0.3)
    format_report_axes(ax)
    plt.tight_layout()

    if save_dir:
        save_plot(fig, filename=plot_name, path=save_dir, fmt="png",
↳dpi=300, close_fig=False)
    plt.show()

```

```

[58]: # --- Plotting function for LR comparison ---
def plot_lr_comparison(histories_dict, model_name, save_dir=None):
    """
    Plot ONLY validation losses for different learning rates.
    (When multiple LRs are present, we show only val curves for readability.)
    """
    plt.style.use('seaborn-v0_8-paper')

    fig, ax = plt.subplots(figsize=(8, 5))

    colors = ['#E74C3C', '#F39C12', '#27AE60', '#3498DB', '#9B59B6',
↳'#34495E']

    for idx, (lr, history) in enumerate(sorted(histories_dict.items(),
↳reverse=True)):
        color = colors[idx % len(colors)]
        label = f'lr_{lr:.0e}'
        ax.plot(history['val_loss'], color=color,
↳linewidth=REPORT_PLOT_LINEWIDTH, label=label, alpha=0.8)

    ax.set_title(f'{model_name} - Validation Loss (LR Comparison)',
↳fontsize=REPORT_PLOT_TITLE_SIZE, pad=10)
    ax.set_ylabel('Validation Loss', fontsize=REPORT_PLOT_LABEL_SIZE)
    ax.set_xlabel('Epochs', fontsize=REPORT_PLOT_LABEL_SIZE)
    ax.grid(True, linestyle='-', alpha=0.3)

```

```

ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
format_report_axes(ax)

plt.tight_layout()

if save_dir:
    save_plot(fig, f'{model_name.lower().replace(" ", "_")}_lr_comparison', save_dir)

plt.show()

```

[59]: # --- Plot Train vs Val ---

```

def plot_train_val_grid(histories_dict, model_name, save_dir=None):
    """
    Plots a grid (1 row, N columns) showing Train vs Val loss for EACH learning
    ↪rate.
    This helps diagnose overfitting/underfitting for each specific setting.
    Works for both FFNN (Task 2) and RNN (Task 3) models.

    Args:
        histories_dict: Dictionary with LR values as keys and history dicts as
        ↪values
        model_name: Name of the model architecture
        save_dir: Directory to save the plot
    """
    lrs = sorted(histories_dict.keys(), reverse=True) # Highest LR first
    num_lrs = len(lrs)

    # Set style
    plt.style.use('seaborn-v0_8-paper')

    # Create figure with 1 row and N columns
    fig, axes = plt.subplots(1, num_lrs, figsize=(5 * num_lrs, 4), sharey=True)

    # Handle case if there is only 1 LR (axes is not a list)
    if num_lrs == 1:
        axes = [axes]

    # Common colors
    col_train = '#2980B9' # Blue
    col_val = '#E74C3C' # Red

    for idx, lr in enumerate(lrs):
        ax = axes[idx]
        history = histories_dict[lr]

```



```

epochs = range(1, len(history['train_loss']) + 1)

# Plot Lines
ax.plot(epochs, history['train_loss'], label='Train Loss',
color=col_train, linewidth=REPORT_PLOT_LINEWIDTH)
ax.plot(epochs, history['val_loss'], label='Val Loss', color=col_val,
linewidth=REPORT_PLOT_LINEWIDTH)

# Find best epoch (min val loss)
min_val_loss = min(history['val_loss'])
best_epoch = history['val_loss'].index(min_val_loss) + 1

# Mark best epoch
ax.axvline(x=best_epoch, color='green', linestyle=':', alpha=0.6,
label=f'Best Ep: {best_epoch}')
ax.scatter(best_epoch, min_val_loss, color='green', zorder=5)

# Styling
ax.set_title(f'LR: {lr:.0e}', fontsize=REPORT_PLOT_TITLE_SIZE,
fontweight='bold')
ax.set_xlabel('Epochs', fontsize=REPORT_PLOT_LABEL_SIZE)
if idx == 0:
    ax.set_ylabel('Loss', fontsize=REPORT_PLOT_LABEL_SIZE)

format_report_axes(ax)
ax.grid(True, linestyle='-', alpha=0.3)
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)

fig.suptitle(f'{model_name} - Overfitting Diagnosis (Train vs Val)',
fontsize=REPORT_PLOT_TITLE_SIZE, y=1.05)
plt.tight_layout()

if save_dir:
    save_plot(fig, f'{model_name.lower().replace(" ", "_")}_diagnosis',
save_dir)

plt.show()

```

Training with Sequential Identifiers Approach

```

[60]: # --- Create a validation set ---

# Convert sequences to a numpy array (shape: [samples, seq_len])
X_train = np.array(df_train['seq_sequential_ids'].tolist())
X_test = np.array(df_test['seq_sequential_ids'].tolist())

```

```

# Labels
y_train = df_train['is_malware'].values
y_test = df_test['is_malware'].values

# Split train into train + val
X_train_split, X_val, y_train_split, y_val = train_test_split(
    X_train, y_train, test_size=0.2, random_state=42, stratify=y_train
)

print("Train set:", X_train_split.shape, y_train_split.shape)
print("Validation set:", X_val.shape, y_val.shape)

```

Train set: (13060, 90) (13060,)
Validation set: (3265, 90) (3265,)

```

[61]: # --- Prepare PyTorch datasets and loaders ---

# Convert to tensors
X_train_tensor = torch.tensor(X_train_split.astype(np.int64), dtype=torch.long)
y_train_tensor = torch.tensor(y_train_split, dtype=torch.float32).unsqueeze(1)

X_val_tensor = torch.tensor(X_val.astype(np.int64), dtype=torch.long)
y_val_tensor = torch.tensor(y_val, dtype=torch.float32).unsqueeze(1)

X_test_tensor = torch.tensor(X_test.astype(np.int64), dtype=torch.long)
y_test_tensor = torch.tensor(y_test, dtype=torch.float32).unsqueeze(1)

# Create datasets and loaders
batch_size = 64

train_dataset = TensorDataset(X_train_tensor, y_train_tensor)
val_dataset = TensorDataset(X_val_tensor, y_val_tensor)
test_dataset = TensorDataset(X_test_tensor, y_test_tensor)

train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=batch_size, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=True)

```

```

[62]: unique, counts = np.unique(y_train_split, return_counts=True)
print(dict(zip(unique, counts)))

```

```
{np.int64(0): np.int64(489), np.int64(1): np.int64(12571)}
```

```

[63]: def compute_pos_weight(y):
    """
    Compute positive class weight for BCEWithLogitsLoss.
    Returns a tensor > 1 if positive class is minority.
    """

```

```

y_np = np.array(y, dtype=int)
class_counts = np.bincount(y_np)

if len(class_counts) < 2:
    raise ValueError("Dataset must contain both classes 0 and 1.")

count_neg, count_pos = class_counts[0], class_counts[1]

# if 1 = malware (majority class)
ratio = count_neg / count_pos

# Optionally stabilize the weight
ratio = max(ratio, 0.2)

print(f"Negatives: {count_neg}, Positives: {count_pos}")
print(f"pos_weight (applied to y=1 samples): {ratio:.4f}")

# return torch.tensor([ratio], dtype=torch.float32)
return torch.tensor([ratio], dtype=torch.float32)

```

```

[64]: # --- Configuration Constants ---
seq_len = X_train.shape[1]
hidden_dims = [256, 128, 64]
dropout_rates = [0.4, 0.35, 0.3]
batchnorm_flags = [True, True, True]
min_delta = 0.00001
patience_early_stop = 15
epochs = 100

# --- Device Setup ---
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Using device: {device}")

# Compute weights and move to device
pos_weight = compute_pos_weight(y_train_split).to(device)

# --- Define LR Candidates ---
learning_rates_seq = [0.0001, 0.00005, 0.00001]
histories_seq = {} # Store results here
models_seq = {} # Store models here
elapsed_seq_times = {} # Store elapsed times here

print(f"\n{'='*80}")
print("Training FFNN (Sequential IDs) with different learning rates")
print(f"{'='*80}")

for lr in learning_rates_seq:

```

```

print(f"\n--- Training FFNN (Sequential IDs) with lr={lr:.0e} ---")

# 1. Re-Initialize Model and move to device
model_seq = MalwareFFNN(
    seq_len,
    hidden_dims,
    dropout_rates,
    batchnorm_flags,
    use_embedding=False,
    dropout_embedding_rate=0.1
).to(device)

# 2. Re-Initialize Criterion
criterion = nn.BCEWithLogitsLoss(pos_weight=pos_weight)

# 3. Initialize Optimizer with current LR
optimizer = optim.AdamW(model_seq.parameters(), lr=lr, weight_decay=1e-5)

# 4. Initialize Scheduler
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
    optimizer,
    mode='min',
    factor=0.7,
    patience=10,
    min_lr=1e-6
)

# 5. Train
model_seq, history, elapsed_seq = train_model(
    model_seq,
    train_loader,
    val_loader,
    epochs,
    optimizer,
    criterion,
    min_delta,
    patience=patience_early_stop,
    scheduler=scheduler
)

# Store model and history with numeric LR key
models_seq[lr] = model_seq
histories_seq[lr] = history
elapsed_seq_times[lr] = elapsed_seq

elapsed_str = time.strftime('%H:%M:%S', time.gmtime(elapsed_seq))

```

```
print(f"Completed. Training time: {elapsed_str} ({elapsed_seq:.2f}s) -  
↳Final val loss: {history['val_loss'][-1]:.4f}")
```

Using device: cuda

Negatives: 489, Positives: 12571

pos_weight (applied to y=1 samples): 0.2000

```
=====
```

Training FFNN (Sequential IDs) with different learning rates

```
=====
```

--- Training FFNN (Sequential IDs) with lr=1e-04 ---

Epoch 1/100 - Train Loss: 0.1472, Val Loss: 0.1403, Val F1(W): 0.9120
Epoch 5/100 - Train Loss: 0.1002, Val Loss: 0.1062, Val F1(W): 0.9438
Epoch 10/100 - Train Loss: 0.0915, Val Loss: 0.0990, Val F1(W): 0.9390
Epoch 15/100 - Train Loss: 0.0854, Val Loss: 0.0962, Val F1(W): 0.9293
Epoch 20/100 - Train Loss: 0.0820, Val Loss: 0.0946, Val F1(W): 0.9270
Epoch 25/100 - Train Loss: 0.0794, Val Loss: 0.0921, Val F1(W): 0.9270
Epoch 30/100 - Train Loss: 0.0756, Val Loss: 0.0931, Val F1(W): 0.9180
Epoch 35/100 - Train Loss: 0.0750, Val Loss: 0.0929, Val F1(W): 0.9261
Epoch 35: LR reduced from 0.000100 to 0.000070
Early stopping at epoch 40 (best val loss: 0.092065)

Training completed in 00:00:32 (hh:mm:ss) - 32.39 seconds

Completed. Training time: 00:00:32 (32.39s) - Final val loss: 0.0960

--- Training FFNN (Sequential IDs) with lr=5e-05 ---

Epoch 1/100 - Train Loss: 0.1532, Val Loss: 0.1546, Val F1(W): 0.7340
Epoch 5/100 - Train Loss: 0.1103, Val Loss: 0.1272, Val F1(W): 0.9411
Epoch 10/100 - Train Loss: 0.0981, Val Loss: 0.1095, Val F1(W): 0.9420
Epoch 15/100 - Train Loss: 0.0932, Val Loss: 0.1035, Val F1(W): 0.9408
Epoch 20/100 - Train Loss: 0.0897, Val Loss: 0.1005, Val F1(W): 0.9391
Epoch 25/100 - Train Loss: 0.0863, Val Loss: 0.0987, Val F1(W): 0.9276
Epoch 30/100 - Train Loss: 0.0890, Val Loss: 0.0976, Val F1(W): 0.9274
Epoch 35/100 - Train Loss: 0.0842, Val Loss: 0.0970, Val F1(W): 0.9273
Epoch 40/100 - Train Loss: 0.0814, Val Loss: 0.0956, Val F1(W): 0.9248
Epoch 45/100 - Train Loss: 0.0805, Val Loss: 0.0960, Val F1(W): 0.9214
Epoch 46: LR reduced from 0.000050 to 0.000035
Epoch 50/100 - Train Loss: 0.0814, Val Loss: 0.0958, Val F1(W): 0.9161
Early stopping at epoch 51 (best val loss: 0.094783)

Training completed in 00:00:44 (hh:mm:ss) - 44.37 seconds

Completed. Training time: 00:00:44 (44.37s) - Final val loss: 0.0966

--- Training FFNN (Sequential IDs) with lr=1e-05 ---

Epoch 1/100 - Train Loss: 0.1672, Val Loss: 0.1605, Val F1(W): 0.6043
Epoch 5/100 - Train Loss: 0.1492, Val Loss: 0.1508, Val F1(W): 0.8074
Epoch 10/100 - Train Loss: 0.1342, Val Loss: 0.1390, Val F1(W): 0.9253

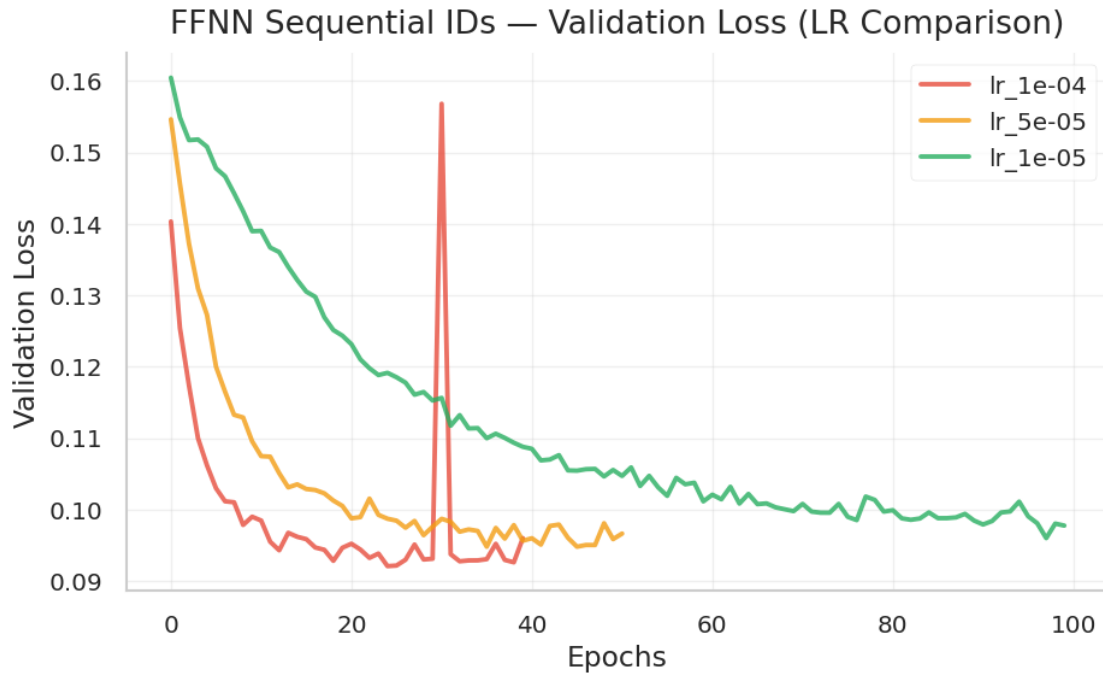
Epoch 15/100 - Train Loss: 0.1224, Val Loss: 0.1321, Val F1(W): 0.9391
Epoch 20/100 - Train Loss: 0.1143, Val Loss: 0.1243, Val F1(W): 0.9426
Epoch 25/100 - Train Loss: 0.1090, Val Loss: 0.1191, Val F1(W): 0.9440
Epoch 30/100 - Train Loss: 0.1037, Val Loss: 0.1153, Val F1(W): 0.9437
Epoch 35/100 - Train Loss: 0.1012, Val Loss: 0.1114, Val F1(W): 0.9434
Epoch 40/100 - Train Loss: 0.0989, Val Loss: 0.1088, Val F1(W): 0.9440
Epoch 45/100 - Train Loss: 0.0981, Val Loss: 0.1055, Val F1(W): 0.9438
Epoch 50/100 - Train Loss: 0.0971, Val Loss: 0.1055, Val F1(W): 0.9441
Epoch 55/100 - Train Loss: 0.0953, Val Loss: 0.1031, Val F1(W): 0.9441
Epoch 60/100 - Train Loss: 0.0948, Val Loss: 0.1011, Val F1(W): 0.9445
Epoch 65/100 - Train Loss: 0.0943, Val Loss: 0.1022, Val F1(W): 0.9451
Epoch 70/100 - Train Loss: 0.0960, Val Loss: 0.0998, Val F1(W): 0.9440
Epoch 75/100 - Train Loss: 0.0916, Val Loss: 0.1008, Val F1(W): 0.9451
Epoch 80/100 - Train Loss: 0.0918, Val Loss: 0.0997, Val F1(W): 0.9451
Epoch 85/100 - Train Loss: 0.0910, Val Loss: 0.0996, Val F1(W): 0.9451
Epoch 87: LR reduced from 0.000010 to 0.000007
Epoch 90/100 - Train Loss: 0.0913, Val Loss: 0.0984, Val F1(W): 0.9441
Epoch 95/100 - Train Loss: 0.0911, Val Loss: 0.1011, Val F1(W): 0.9428
Epoch 100/100 - Train Loss: 0.0902, Val Loss: 0.0977, Val F1(W): 0.9420

Training completed in 00:01:21 (hh:mm:ss) - 81.54 seconds

Completed. Training time: 00:01:21 (81.54s) - Final val loss: 0.0977

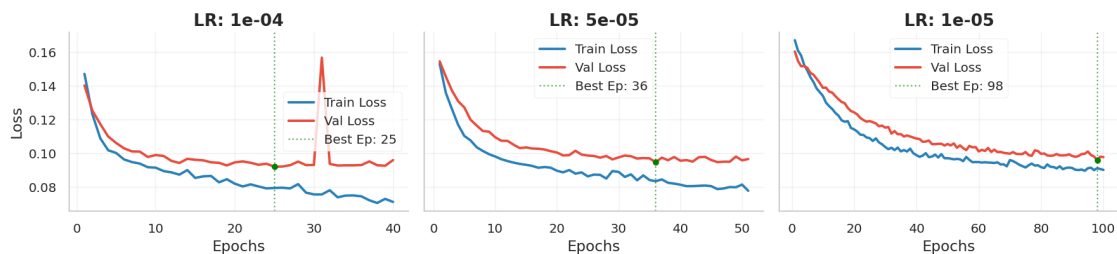
```
[65]: # --- Plot loss curves ---  
  
# Plot comparison  
plot_lr_comparison(histories_seq, "FFNN Sequential IDs", save_dir)  
  
# Plot Train vs Validation  
plot_train_val_grid(histories_seq, "FFNN Sequential IDs", save_dir)
```

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task2_plots/ffnn_sequential_ids_lr_comparison.png



Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task2_plots/ffnn_sequential_ids_diagnosis.png

FFNN Sequential IDs - Overfitting Diagnosis (Train vs Val)



```
[66]: # --- Evaluate validation and test sets for ALL learning rates ---
```

```
print("\n" + "=" * 80)
print("Evaluation for ALL FFNN (Sequential IDs) learning rates")
print("=" * 80)

for lr in learning_rates_seq:
    model = models_seq[lr]

    print(f"\n{'='*60}")
```

```

print(f"FFNN (Sequential IDs) with lr={lr:.0e}")
print(f"{'='*60}")

# Validation evaluation
print(f'\nValidation report (FFNN Sequential IDs - lr={lr:.0e}):')
report = evaluate_model(model, X_val_tensor.to(device), y_val) # Moved
↪X_val_tensor to device
print(report)

# Test evaluation
print(f'Test report (FFNN Sequential IDs - lr={lr:.0e}):')
report_test = evaluate_model(model, X_test_tensor.to(device), y_test) #
↪Moved X_test_tensor to device
print(report_test)

# Find and highlight best model
best_lr_seq = min(histories_seq.items(), key=lambda x: min(x[1]['val_loss']))
print(f"\n{'='*60}")
print(f" Best FFNN (Sequential IDs) - LR: {best_lr_seq[0]:.0e} (Min Val Loss:
↪{min(best_lr_seq[1]['val_loss']):.4f})")
print(f"{'='*60}")

```

```

=====
Evaluation for ALL FFNN (Sequential IDs) learning rates
=====

```

```

=====
FFNN (Sequential IDs) with lr=1e-04
=====

```

Validation report (FFNN Sequential IDs - lr=1e-04):

	precision	recall	f1-score	support
0	0.1034	0.1475	0.1216	122
1	0.9664	0.9504	0.9583	3143
accuracy			0.9204	3265
macro avg	0.5349	0.5490	0.5400	3265
weighted avg	0.9341	0.9204	0.9270	3265

Test report (FFNN Sequential IDs - lr=1e-04):

	precision	recall	f1-score	support
0	0.1225	0.1276	0.1250	243
1	0.9661	0.9645	0.9653	6262

accuracy			0.9333	6505
macro avg	0.5443	0.5461	0.5452	6505
weighted avg	0.9346	0.9333	0.9339	6505

```
=====
FFNN (Sequential IDs) with lr=5e-05
=====
```

Validation report (FFNN Sequential IDs - lr=5e-05):

	precision	recall	f1-score	support
0	0.1124	0.1639	0.1333	122
1	0.9670	0.9497	0.9583	3143

accuracy			0.9204	3265
macro avg	0.5397	0.5568	0.5458	3265
weighted avg	0.9350	0.9204	0.9274	3265

Test report (FFNN Sequential IDs - lr=5e-05):

	precision	recall	f1-score	support
0	0.1301	0.1440	0.1367	243
1	0.9666	0.9626	0.9646	6262

accuracy			0.9321	6505
macro avg	0.5484	0.5533	0.5507	6505
weighted avg	0.9354	0.9321	0.9337	6505

```
=====
FFNN (Sequential IDs) with lr=1e-05
=====
```

Validation report (FFNN Sequential IDs - lr=1e-05):

	precision	recall	f1-score	support
0	0.2157	0.0902	0.1272	122
1	0.9655	0.9873	0.9762	3143

accuracy			0.9538	3265
macro avg	0.5906	0.5387	0.5517	3265
weighted avg	0.9374	0.9538	0.9445	3265

Test report (FFNN Sequential IDs - lr=1e-05):

	precision	recall	f1-score	support
0	0.1600	0.0329	0.0546	243

1	0.9636	0.9933	0.9782	6262
accuracy			0.9574	6505
macro avg	0.5618	0.5131	0.5164	6505
weighted avg	0.9336	0.9574	0.9437	6505

```
=====
Best FFNN (Sequential IDs) - LR: 1e-04 (Min Val Loss: 0.0921)
=====
```

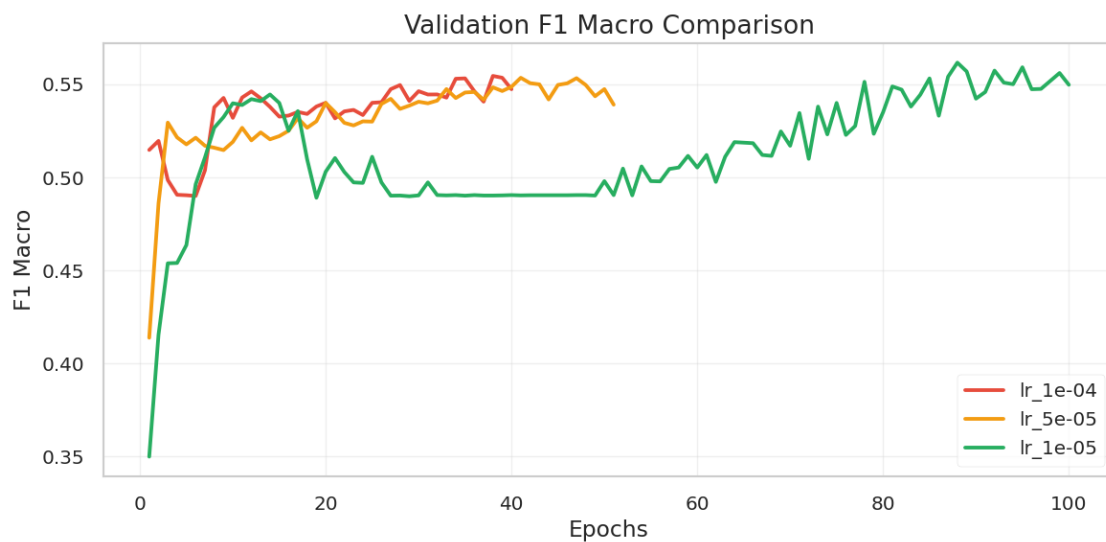
```
[67]: # --- Plot metric curves for ALL learning rates (Sequential IDs) ---
```

```
print("\n" + "=" * 80)
print("Metric Curves for ALL FFNN (Sequential IDs) learning rates")
print("=" * 80)

plot_training_metrics(histories_seq, save_dir=save_dir,
    plot_name="ffnn_sequential_ids_f1_comparison")
```

```
=====
Metric Curves for ALL FFNN (Sequential IDs) learning rates
=====
```

```
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task2_plots/ffnn_sequential_ids_f1_comparison.png
```



```
[68]: # --- Plot confusion matrices for ALL learning rates (Sequential IDs) ---
      VALIDATION ---
```

```

print("\n" + "=" * 80)
print("Confusion matrices for ALL FFNN (Sequential IDs) learning rates -_
↳VALIDATION")
print("=" * 80)

for lr in learning_rates_seq:
    model = models_seq[lr]
    print(f"\nGenerating confusion matrix for FFNN (Sequential IDs) with lr={lr:
↳.0e}...")

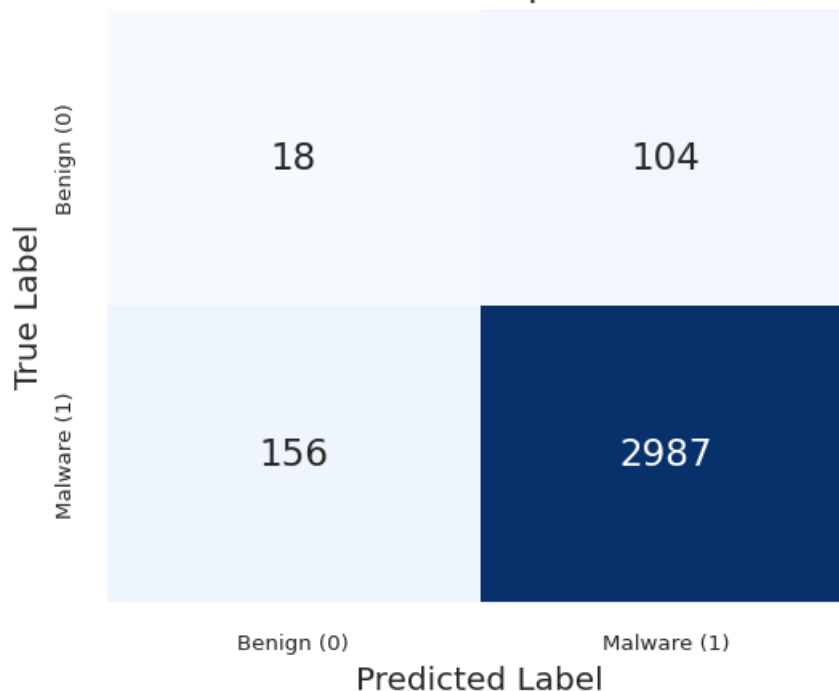
    plot_cm_task2(model=model, X_tensor=X_val_tensor, y_tensor=y_val_tensor,
↳model_name=f"FFNN Sequential IDs (lr={lr:.0e})", save_dir=save_dir)

```

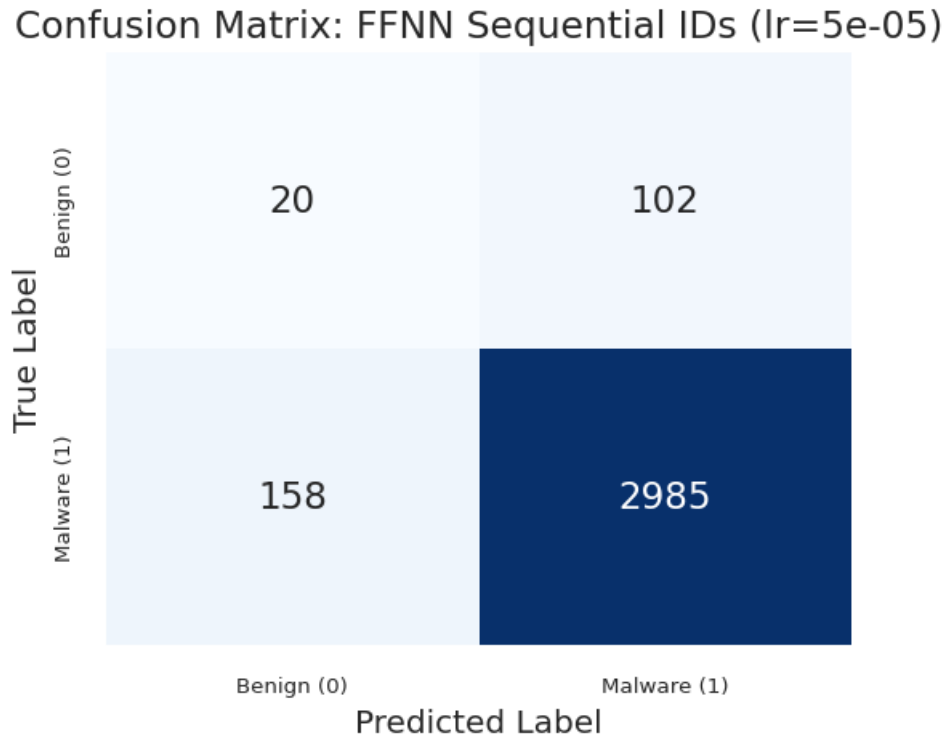
=====
Confusion matrices for ALL FFNN (Sequential IDs) learning rates - VALIDATION
=====

Generating confusion matrix for FFNN (Sequential IDs) with lr=1e-04..
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task2_plots/cm_ffnn_sequential_ids_(lr=1e_04).png

Confusion Matrix: FFNN Sequential IDs (lr=1e-04)

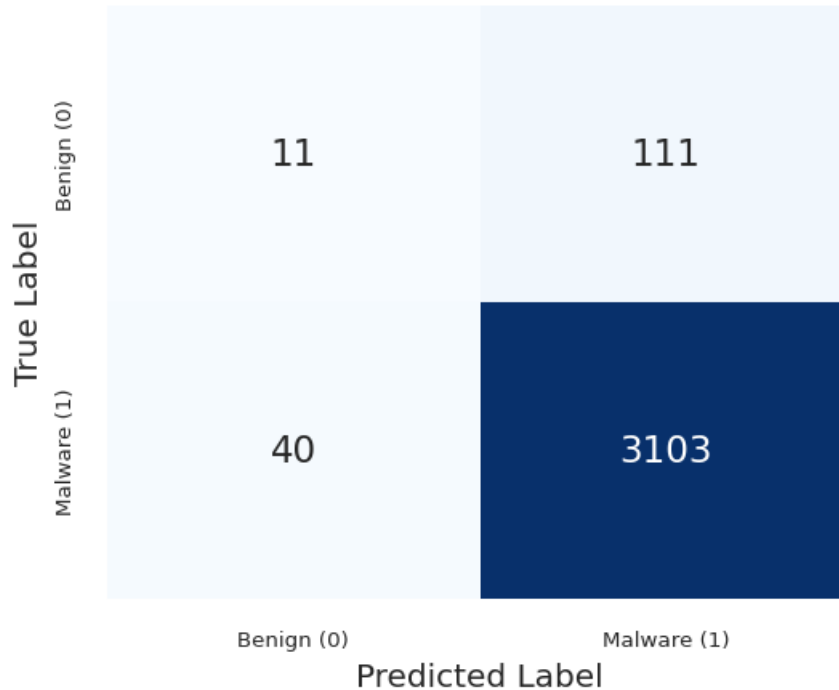


Generating confusion matrix for FFNN (Sequential IDs) with lr=5e-05...
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task2_plots/cm_ffnn_sequential_ids_(lr=5e_05).png



Generating confusion matrix for FFNN (Sequential IDs) with lr=1e-05...
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task2_plots/cm_ffnn_sequential_ids_(lr=1e_05).png

Confusion Matrix: FFNN Sequential IDs (lr=1e-05)



```
[69]: # --- Plot confusion matrices for ALL learning rates (Sequential IDs) - TEST ---
```

```
print("\n" + "=" * 80)
print("Confusion matrices for ALL FFNN (Sequential IDs) learning rates - TEST")
print("=" * 80)

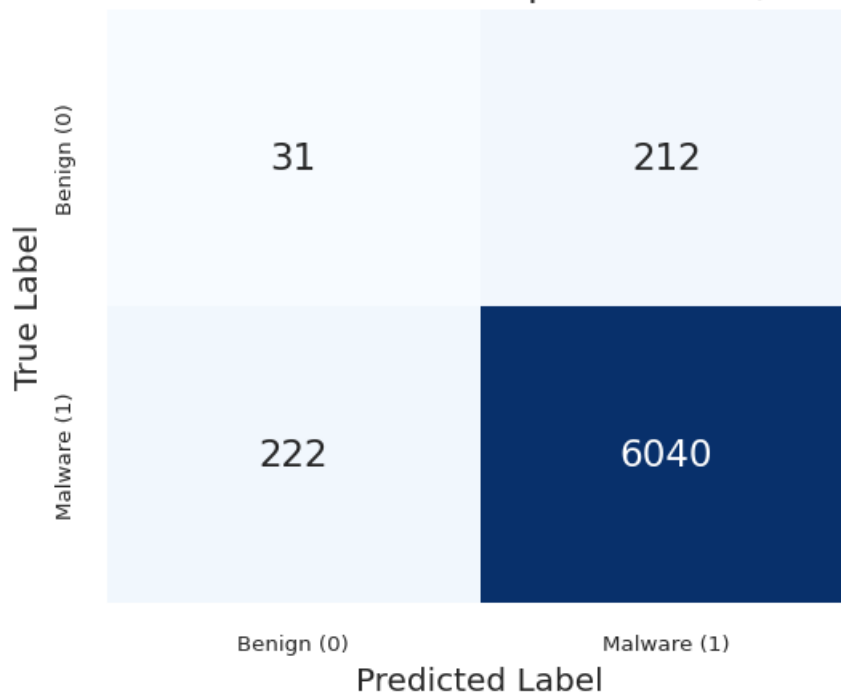
for lr in learning_rates_seq:
    model = models_seq[lr]
    print(f"\nGenerating confusion matrix for FFNN (Sequential IDs) with lr={lr:
↪.0e}...")

    plot_cm_task2(model=model, X_tensor=X_test_tensor, y_tensor=y_test_tensor,
↪model_name=f"FFNN Sequential IDs (lr={lr:.0e})", save_dir=save_dir)
```

```
=====
Confusion matrices for ALL FFNN (Sequential IDs) learning rates - TEST
=====
```

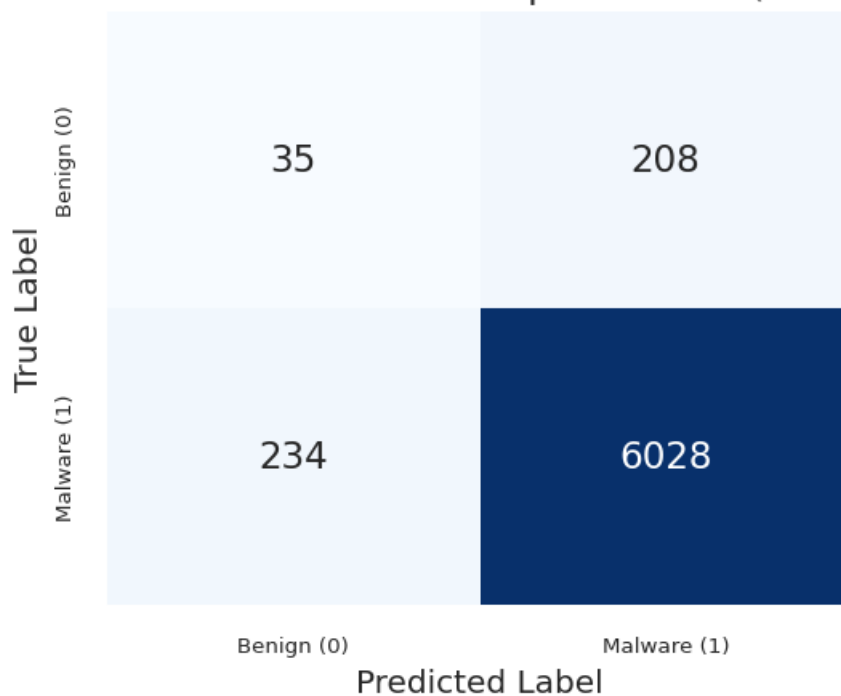
```
Generating confusion matrix for FFNN (Sequential IDs) with lr=1e-04...
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task2_plots/cm_ffnn_sequential_ids_(lr=1e_04).png
```

Confusion Matrix: FFNN Sequential IDs (lr=1e-04)



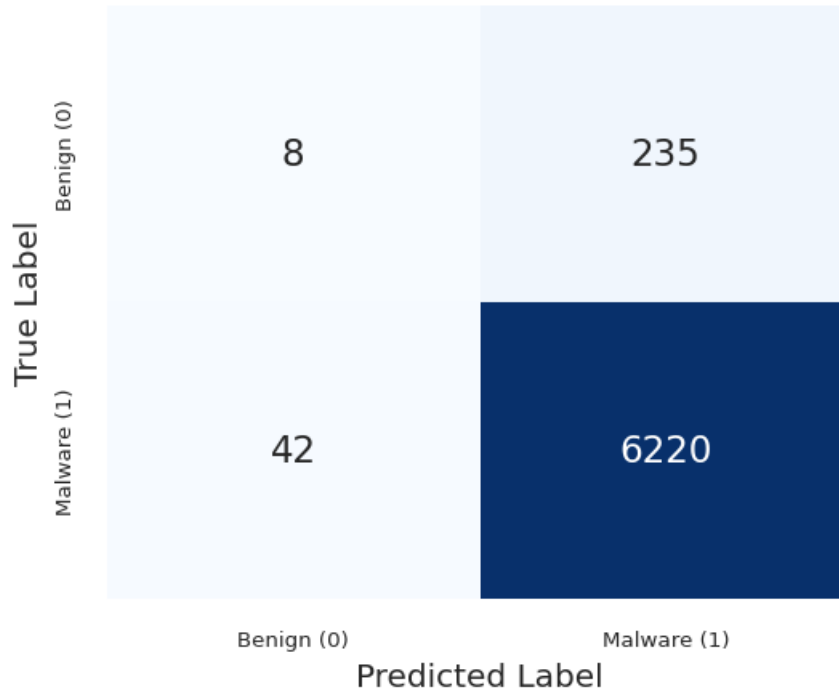
Generating confusion matrix for FFNN (Sequential IDs) with lr=5e-05...
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task2_plots/cm_ffnn_sequential_ids_(lr=5e_05).png

Confusion Matrix: FFNN Sequential IDs (lr=5e-05)



Generating confusion matrix for FFNN (Sequential IDs) with lr=1e-05...
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task2_plots/cm_ffnn_sequential_ids_(lr=1e_05).png

Confusion Matrix: FFNN Sequential IDs (lr=1e-05)



Training with Learnable Embeddings Approach

```
[70]: # --- Configuration Constants ---
seq_len = X_train.shape[1]
hidden_dims = [128, 64, 32]
dropout_rates = [0.4, 0.4, 0.4]
batchnorm_flags = [True, True, True]
min_delta = 0.00001
patience_early_stop = 15
epochs = 100

vocab_size = len(api_to_id)
embedding_dim = 128

# --- Device Setup (reuse from previous section if available) ---
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Using device: {device}")

# Compute weights and move to device
pos_weight = compute_pos_weight(y_train_split).to(device)

# --- Define LR Candidates ---
learning_rates_emb = [0.00005, 0.00001, 0.000005]
```



```

histories_emb = {}      # Store results here
models_emb = {}        # Store models here
elapsed_emb_times = {} # Store elapsed times here

print(f"\n{'='*80}")
print("Training FFNN (Learnable Embeddings) with different learning rates")
print(f"{'='*80}")

for lr in learning_rates_emb:
    print(f"\n--- Training FFNN (Learnable Embeddings) with lr={lr:.0e} ---")

    # Initialize model with embedding layer and move to device
    model_emb = MalwareFFNN(
        seq_len,
        hidden_dims,
        dropout_rates,
        batchnorm_flags,
        use_embedding=True,
        vocab_size=vocab_size,
        embedding_dim=embedding_dim,
    ).to(device)

    # Criterion (pos_weight already on device)
    criterion = nn.BCEWithLogitsLoss(pos_weight=pos_weight)

    # Initialize Optimizer
    optimizer = optim.AdamW(model_emb.parameters(), lr=lr, weight_decay=1e-4)

    # Setup Scheduler
    scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
        optimizer,
        mode='min',
        factor=0.5,
        patience=5,
        min_lr=1e-6
    )

    # Train the model
    model_emb, history, elapsed_emb = train_model(
        model_emb,
        train_loader,
        val_loader,
        epochs,
        optimizer,
        criterion,
        min_delta,
        patience_early_stop,

```

```

        scheduler=None
    )

    # Store model and history with numeric LR key
    models_emb[lr] = model_emb
    histories_emb[lr] = history
    elapsed_emb_times[lr] = elapsed_emb

    elapsed_str = time.strftime('%H:%M:%S', time.gmtime(elapsed_emb))
    print(f"Completed. Training time: {elapsed_str} ({elapsed_emb:.2f}s) -  

    ↪Final val loss: {history['val_loss'][-1]:.4f}")

```

Using device: cuda
 Negatives: 489, Positives: 12571
 pos_weight (applied to y=1 samples): 0.2000

=====

Training FFNN (Learnable Embeddings) with different learning rates

=====

--- Training FFNN (Learnable Embeddings) with lr=5e-05 ---

Epoch 1/100 - Train Loss: 0.1319, Val Loss: 0.1279, Val F1(W): 0.9479
 Epoch 5/100 - Train Loss: 0.1059, Val Loss: 0.1075, Val F1(W): 0.9465
 Epoch 10/100 - Train Loss: 0.0827, Val Loss: 0.0898, Val F1(W): 0.9467
 Epoch 15/100 - Train Loss: 0.0624, Val Loss: 0.0795, Val F1(W): 0.9432
 Epoch 20/100 - Train Loss: 0.0467, Val Loss: 0.0714, Val F1(W): 0.9456
 Epoch 25/100 - Train Loss: 0.0363, Val Loss: 0.0741, Val F1(W): 0.9513
 Epoch 30/100 - Train Loss: 0.0269, Val Loss: 0.0777, Val F1(W): 0.9502
 Epoch 35/100 - Train Loss: 0.0215, Val Loss: 0.0843, Val F1(W): 0.9497
 Early stopping at epoch 36 (best val loss: 0.071219)

Training completed in 00:00:33 (hh:mm:ss) - 33.17 seconds

Completed. Training time: 00:00:33 (33.17s) - Final val loss: 0.0819

--- Training FFNN (Learnable Embeddings) with lr=1e-05 ---

Epoch 1/100 - Train Loss: 0.1894, Val Loss: 0.1666, Val F1(W): 0.4403
 Epoch 5/100 - Train Loss: 0.1728, Val Loss: 0.1571, Val F1(W): 0.6586
 Epoch 10/100 - Train Loss: 0.1565, Val Loss: 0.1519, Val F1(W): 0.7195
 Epoch 15/100 - Train Loss: 0.1435, Val Loss: 0.1463, Val F1(W): 0.8041
 Epoch 20/100 - Train Loss: 0.1311, Val Loss: 0.1350, Val F1(W): 0.8623
 Epoch 25/100 - Train Loss: 0.1200, Val Loss: 0.1266, Val F1(W): 0.8936
 Epoch 30/100 - Train Loss: 0.1108, Val Loss: 0.1183, Val F1(W): 0.9080
 Epoch 35/100 - Train Loss: 0.1015, Val Loss: 0.1151, Val F1(W): 0.9060
 Epoch 40/100 - Train Loss: 0.0926, Val Loss: 0.1062, Val F1(W): 0.9289
 Epoch 45/100 - Train Loss: 0.0858, Val Loss: 0.0993, Val F1(W): 0.9332
 Epoch 50/100 - Train Loss: 0.0779, Val Loss: 0.0929, Val F1(W): 0.9414
 Epoch 55/100 - Train Loss: 0.0713, Val Loss: 0.0927, Val F1(W): 0.9391
 Epoch 60/100 - Train Loss: 0.0672, Val Loss: 0.1140, Val F1(W): 0.9360

Epoch 65/100 - Train Loss: 0.0612, Val Loss: 0.0820, Val F1(W): 0.9470
 Epoch 70/100 - Train Loss: 0.0570, Val Loss: 0.0808, Val F1(W): 0.9443
 Epoch 75/100 - Train Loss: 0.0512, Val Loss: 0.0784, Val F1(W): 0.9444
 Epoch 80/100 - Train Loss: 0.0484, Val Loss: 0.0779, Val F1(W): 0.9449
 Epoch 85/100 - Train Loss: 0.0445, Val Loss: 0.0746, Val F1(W): 0.9466
 Epoch 90/100 - Train Loss: 0.0429, Val Loss: 0.0719, Val F1(W): 0.9522
 Epoch 95/100 - Train Loss: 0.0398, Val Loss: 0.0726, Val F1(W): 0.9498
 Epoch 100/100 - Train Loss: 0.0360, Val Loss: 0.0714, Val F1(W): 0.9500

Training completed in 00:01:32 (hh:mm:ss) - 92.36 seconds
 Completed. Training time: 00:01:32 (92.36s) - Final val loss: 0.0714

--- Training FFNN (Learnable Embeddings) with lr=5e-06 ---

Epoch 1/100 - Train Loss: 0.1725, Val Loss: 0.1702, Val F1(W): 0.2839
 Epoch 5/100 - Train Loss: 0.1648, Val Loss: 0.1648, Val F1(W): 0.4361
 Epoch 10/100 - Train Loss: 0.1565, Val Loss: 0.1593, Val F1(W): 0.5832
 Epoch 15/100 - Train Loss: 0.1502, Val Loss: 0.1517, Val F1(W): 0.7299
 Epoch 20/100 - Train Loss: 0.1421, Val Loss: 0.1469, Val F1(W): 0.7826
 Epoch 25/100 - Train Loss: 0.1371, Val Loss: 0.1431, Val F1(W): 0.8185
 Epoch 30/100 - Train Loss: 0.1306, Val Loss: 0.1393, Val F1(W): 0.8380
 Epoch 35/100 - Train Loss: 0.1236, Val Loss: 0.1364, Val F1(W): 0.8453
 Epoch 40/100 - Train Loss: 0.1181, Val Loss: 0.1303, Val F1(W): 0.8645
 Epoch 45/100 - Train Loss: 0.1135, Val Loss: 0.1258, Val F1(W): 0.8830
 Epoch 50/100 - Train Loss: 0.1081, Val Loss: 0.1215, Val F1(W): 0.8918
 Epoch 55/100 - Train Loss: 0.1033, Val Loss: 0.1210, Val F1(W): 0.8862
 Epoch 60/100 - Train Loss: 0.0981, Val Loss: 0.1171, Val F1(W): 0.8956
 Epoch 65/100 - Train Loss: 0.0948, Val Loss: 0.1102, Val F1(W): 0.9048
 Epoch 70/100 - Train Loss: 0.0902, Val Loss: 0.1077, Val F1(W): 0.9234
 Epoch 75/100 - Train Loss: 0.0870, Val Loss: 0.1041, Val F1(W): 0.9176
 Epoch 80/100 - Train Loss: 0.0815, Val Loss: 0.1021, Val F1(W): 0.9191
 Epoch 85/100 - Train Loss: 0.0790, Val Loss: 0.0996, Val F1(W): 0.9186
 Epoch 90/100 - Train Loss: 0.0740, Val Loss: 0.0938, Val F1(W): 0.9294
 Epoch 95/100 - Train Loss: 0.0720, Val Loss: 0.0939, Val F1(W): 0.9321
 Epoch 100/100 - Train Loss: 0.0674, Val Loss: 0.1002, Val F1(W): 0.9360

Training completed in 00:01:33 (hh:mm:ss) - 93.15 seconds
 Completed. Training time: 00:01:33 (93.15s) - Final val loss: 0.1002

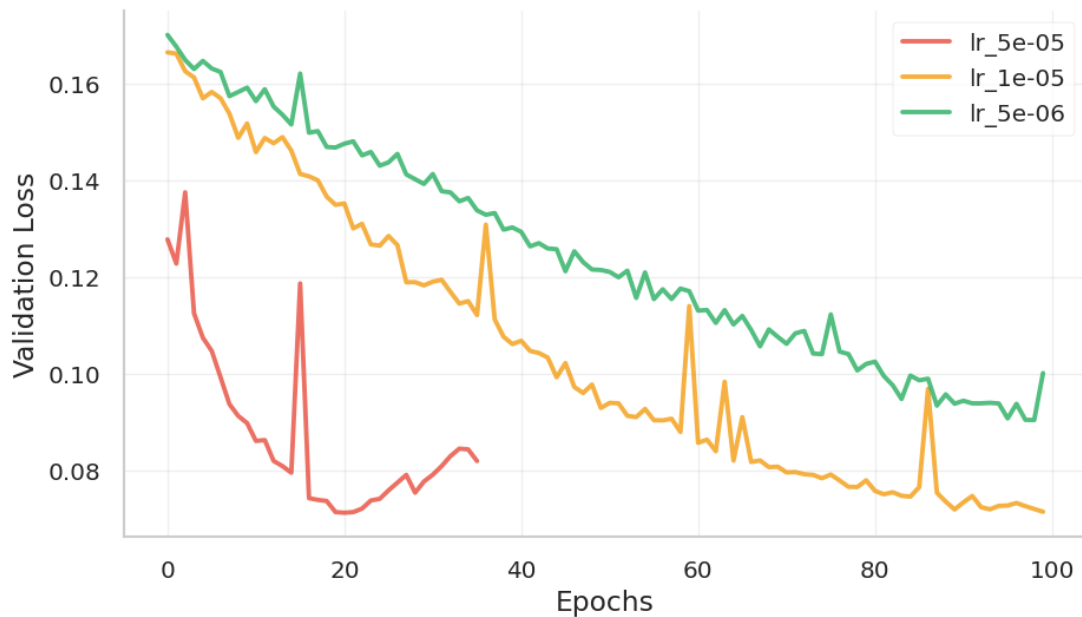
[71]: # --- Plot loss curves ---

```
# Plot comparison
plot_lr_comparison(histories_emb, "FFNN Learnable Embeddings", save_dir)

# Plot Train vs Validation
plot_train_val_grid(histories_emb, "FFNN Learnable Embeddings", save_dir)
```

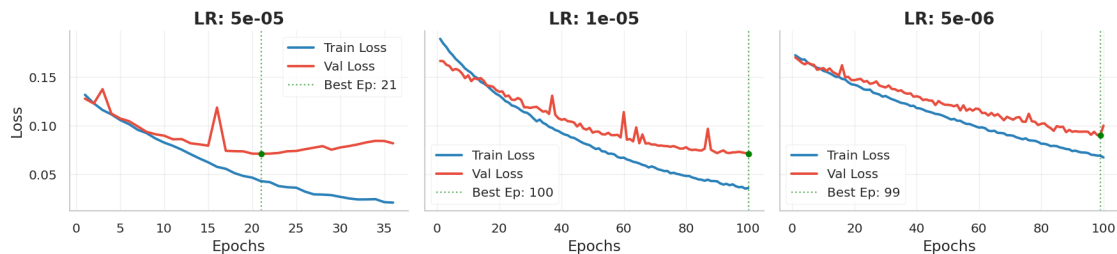
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task2_plots/ffnn_learnable_embeddings_lr_comparison.png

FFNN Learnable Embeddings — Validation Loss (LR Comparison)



Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task2_plots/ffnn_learnable_embeddings_diagnosis.png

FFNN Learnable Embeddings - Overfitting Diagnosis (Train vs Val)



```
[72]: # --- Evaluate validation and test sets for ALL learning rates ---

print("\n" + "=" * 80)
print("Evaluation for ALL FFNN (Learnable Embeddings) learning rates")
print("=" * 80)

for lr in learning_rates_emb:
    model = models_emb[lr]

    print(f"\n{'='*60}")
```

```

print(f"FFNN (Learnable Embeddings) with lr={lr:.0e}")
print(f"{'='*60}")

# Validation evaluation
print(f'\nValidation report (FFNN Learnable Embeddings - lr={lr:.0e}):')
report = evaluate_model(model, X_val_tensor.to(device), y_val) # Moved
↪X_val_tensor to device
print(report)

# Test evaluation
print(f'Test report (FFNN Learnable Embeddings - lr={lr:.0e}):')
report_test = evaluate_model(model, X_test_tensor.to(device), y_test) #
↪Moved X_test_tensor to device
print(report_test)

# Find and highlight best model
best_lr_emb = min(histories_emb.items(), key=lambda x: min(x[1]['val_loss']))
print(f"\n{'='*60}")
print(f" Best FFNN (Learnable Embeddings) - LR: {best_lr_emb[0]:.0e} (Min Val
↪Loss: {min(best_lr_emb[1]['val_loss']):.4f})")
print(f"{'='*60}")

```

```

=====
Evaluation for ALL FFNN (Learnable Embeddings) learning rates
=====

```

```

=====
FFNN (Learnable Embeddings) with lr=5e-05
=====

```

Validation report (FFNN Learnable Embeddings - lr=5e-05):

	precision	recall	f1-score	support
0	0.3312	0.4180	0.3696	122
1	0.9772	0.9672	0.9722	3143
accuracy			0.9467	3265
macro avg	0.6542	0.6926	0.6709	3265
weighted avg	0.9530	0.9467	0.9497	3265

Test report (FFNN Learnable Embeddings - lr=5e-05):

	precision	recall	f1-score	support
0	0.1676	0.2346	0.1955	243
1	0.9698	0.9548	0.9623	6262

accuracy			0.9279	6505
macro avg	0.5687	0.5947	0.5789	6505
weighted avg	0.9399	0.9279	0.9336	6505

```
=====
FFNN (Learnable Embeddings) with lr=1e-05
=====
```

Validation report (FFNN Learnable Embeddings - lr=1e-05):

	precision	recall	f1-score	support
0	0.3355	0.4262	0.3755	122
1	0.9775	0.9672	0.9723	3143

accuracy			0.9470	3265
macro avg	0.6565	0.6967	0.6739	3265
weighted avg	0.9535	0.9470	0.9500	3265

Test report (FFNN Learnable Embeddings - lr=1e-05):

	precision	recall	f1-score	support
0	0.1350	0.2634	0.1785	243
1	0.9703	0.9345	0.9521	6262

accuracy			0.9095	6505
macro avg	0.5527	0.5990	0.5653	6505
weighted avg	0.9391	0.9095	0.9232	6505

```
=====
FFNN (Learnable Embeddings) with lr=5e-06
=====
```

Validation report (FFNN Learnable Embeddings - lr=5e-06):

	precision	recall	f1-score	support
0	0.2510	0.5082	0.3360	122
1	0.9801	0.9411	0.9602	3143

accuracy			0.9250	3265
macro avg	0.6156	0.7247	0.6481	3265
weighted avg	0.9529	0.9250	0.9369	3265

Test report (FFNN Learnable Embeddings - lr=5e-06):

	precision	recall	f1-score	support
0	0.1388	0.3786	0.2031	243

1	0.9742	0.9088	0.9404	6262
accuracy			0.8890	6505
macro avg	0.5565	0.6437	0.5717	6505
weighted avg	0.9429	0.8890	0.9128	6505

```
=====
Best FFNN (Learnable Embeddings) - LR: 5e-05 (Min Val Loss: 0.0712)
=====
```

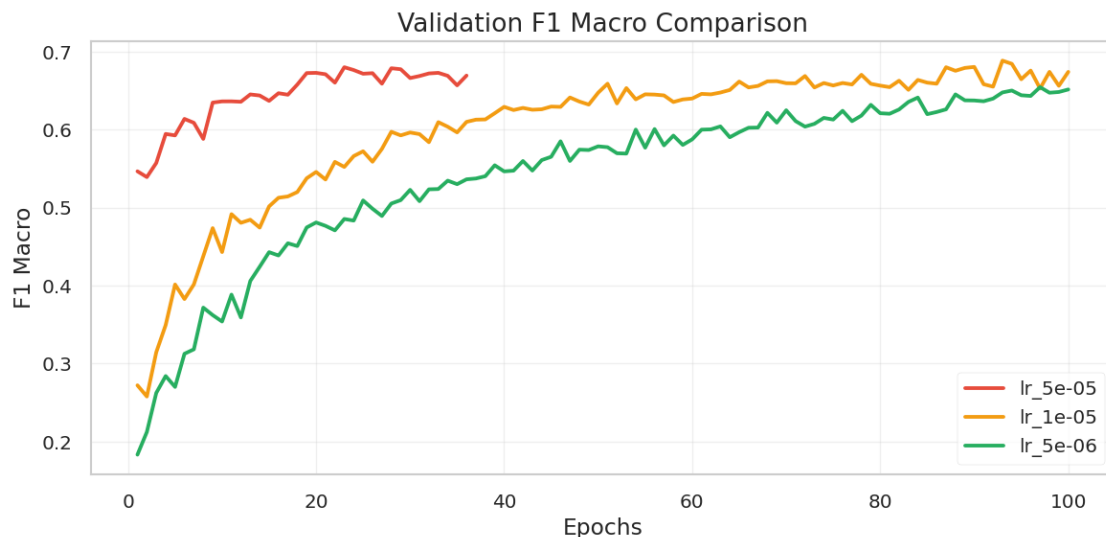
```
[73]: # --- Plot metric curves for ALL learning rates (Learnable Embeddings) ---

print("\n" + "=" * 80)
print("Metric Curves for ALL FFNN (Learnable Embeddings) learning rates")
print("=" * 80)

# The plot_training_metrics function is designed to compare multiple histories.
plot_training_metrics(histories_emb, save_dir=save_dir,
    ↪plot_name="ffnn_learnable_embeddings_f1_comparison")
```

```
=====
Metric Curves for ALL FFNN (Learnable Embeddings) learning rates
=====
```

```
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task2_plots/ffnn_learnable_embeddings_f1_comparison.png
```



```
[74]: # --- Plot confusion matrices for ALL learning rates (Learnable Embeddings) - VALIDATION ---

print("\n" + "=" * 80)
print("Confusion matrices for ALL FFNN (Learnable Embeddings) learning rates - VALIDATION")
print("=" * 80)

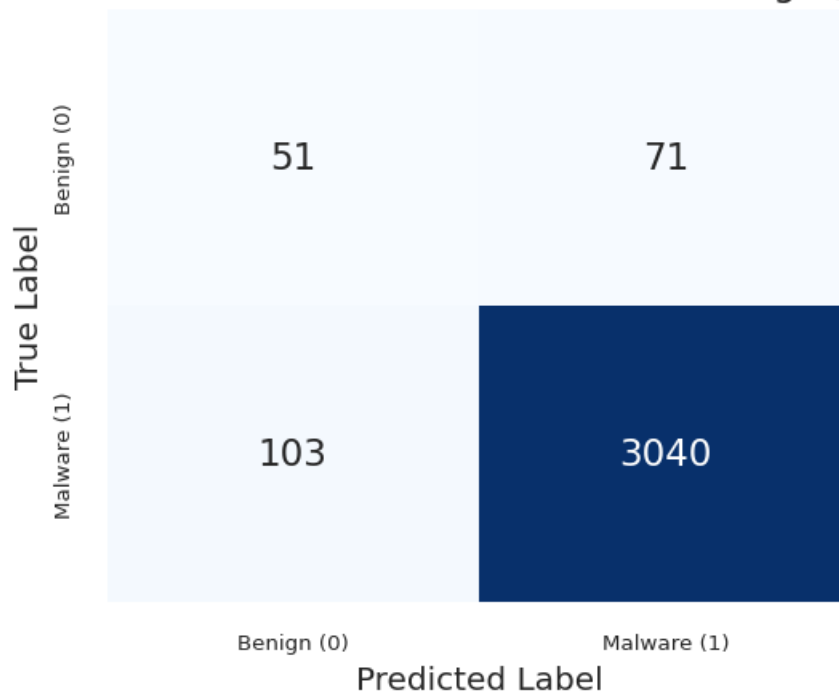
for lr in learning_rates_emb:
    model = models_emb[lr]
    print(f"\nGenerating confusion matrix for FFNN (Learnable Embeddings) with lr={lr:.0e}...")

    plot_cm_task2(model=model, X_tensor=X_val_tensor, y_tensor=y_val_tensor, model_name=f"FFNN Learnable Embeddings (lr={lr:.0e})", save_dir=save_dir)
```

```
=====
Confusion matrices for ALL FFNN (Learnable Embeddings) learning rates -
VALIDATION
=====
```

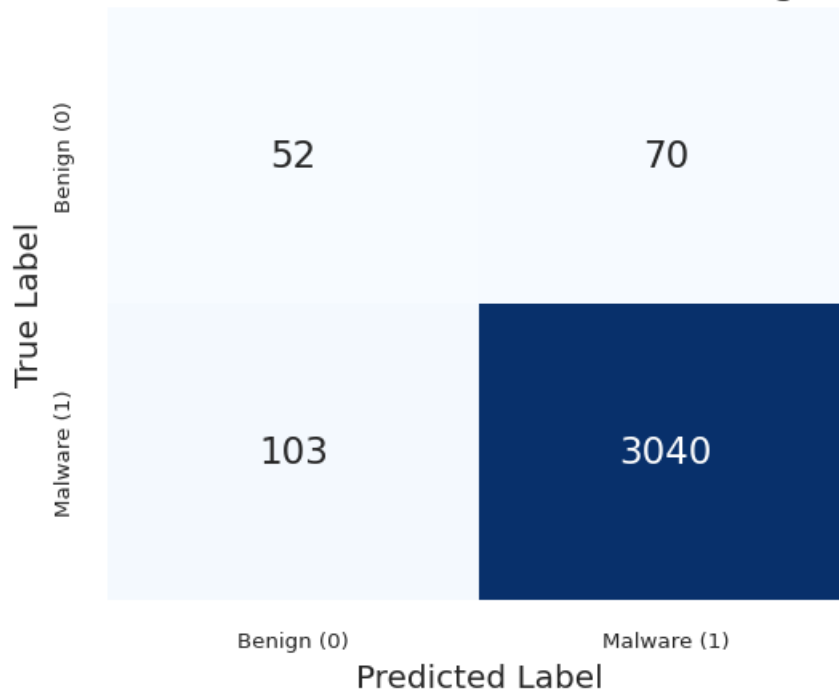
```
Generating confusion matrix for FFNN (Learnable Embeddings) with lr=5e-05...
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task2_plots/cm_ffnn_learnable_embeddings_(lr=5e_05).png
```

Confusion Matrix: FFNN Learnable Embeddings (lr=5e-05)



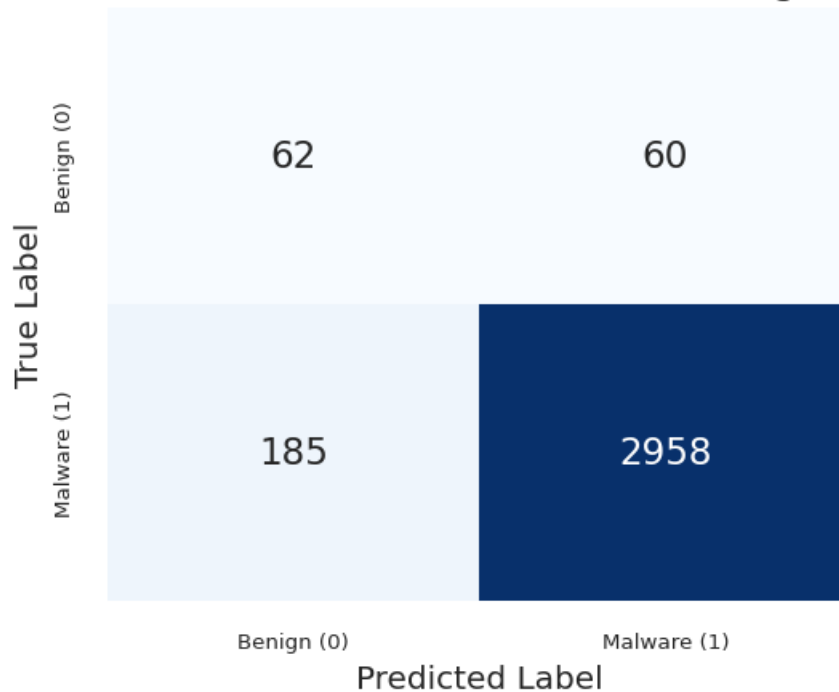
Generating confusion matrix for FFNN (Learnable Embeddings) with $lr=1e-05$..
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task2_plots/cm_ffnn_learnable_embeddings_(lr=1e_05).png

Confusion Matrix: FFNN Learnable Embeddings ($lr=1e-05$)



Generating confusion matrix for FFNN (Learnable Embeddings) with $lr=5e-06$..
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task2_plots/cm_ffnn_learnable_embeddings_(lr=5e_06).png

Confusion Matrix: FFNN Learnable Embeddings (lr=5e-06)



```
[75]: # --- Plot confusion matrices for ALL learning rates (Learnable Embeddings) ->TEST ---

print("\n" + "=" * 80)
print("Confusion matrices for ALL FFNN (Learnable Embeddings) learning rates ->TEST")
print("=" * 80)

for lr in learning_rates_emb:
    model = models_emb[lr]
    print(f"\nGenerating confusion matrix for FFNN (Learnable Embeddings) with lr={lr:.0e}...")

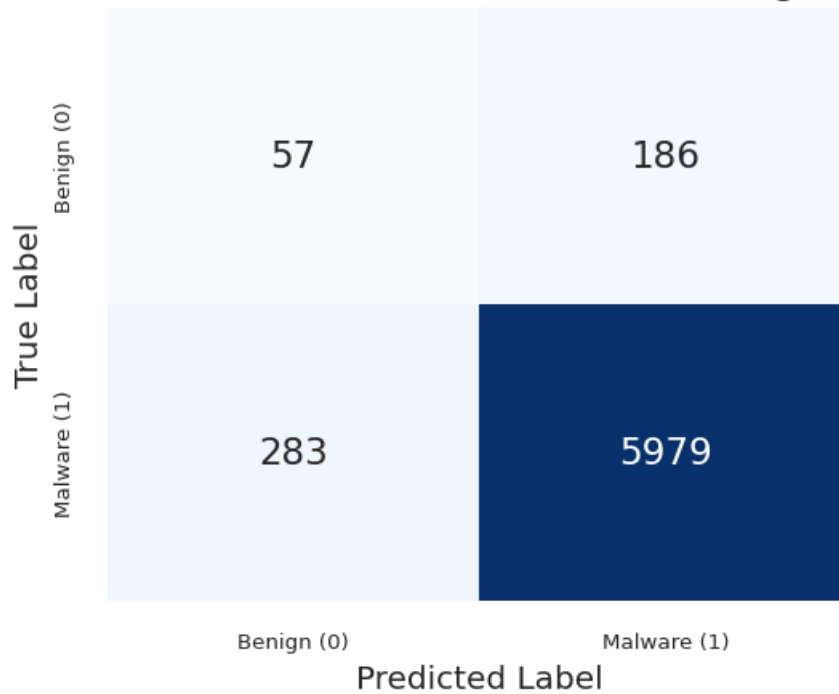
    plot_cm_task2(model=model, X_tensor=X_test_tensor, y_tensor=y_test_tensor,
    model_name=f"FFNN Learnable Embeddings (lr={lr:.0e})", save_dir=save_dir)
```

=====
Confusion matrices for ALL FFNN (Learnable Embeddings) learning rates - TEST
=====

Generating confusion matrix for FFNN (Learnable Embeddings) with lr=5e-05...
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images

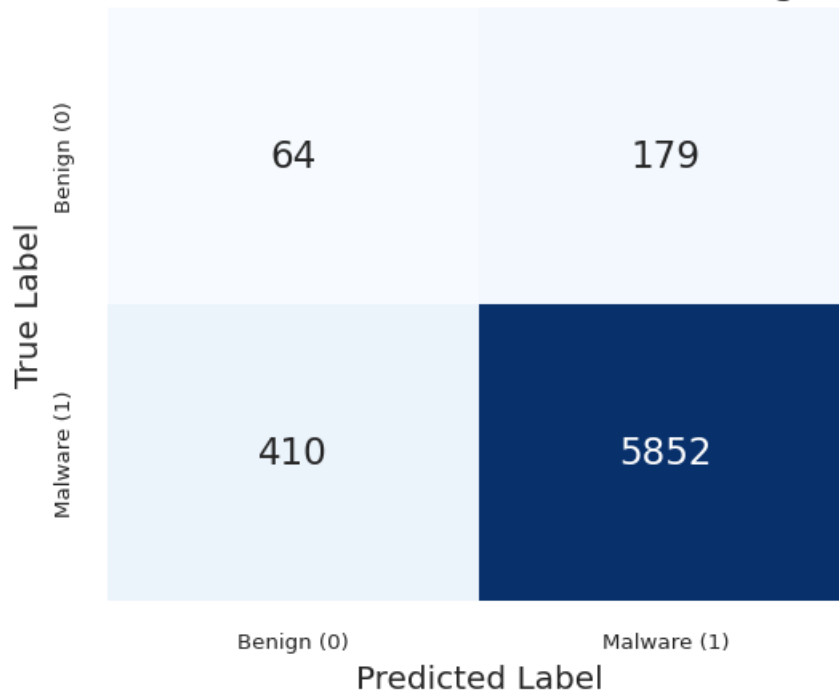
/task2_plots/cm_ffnn_learnable_embeddings_(lr=5e_05).png

Confusion Matrix: FFNN Learnable Embeddings (lr=5e-05)



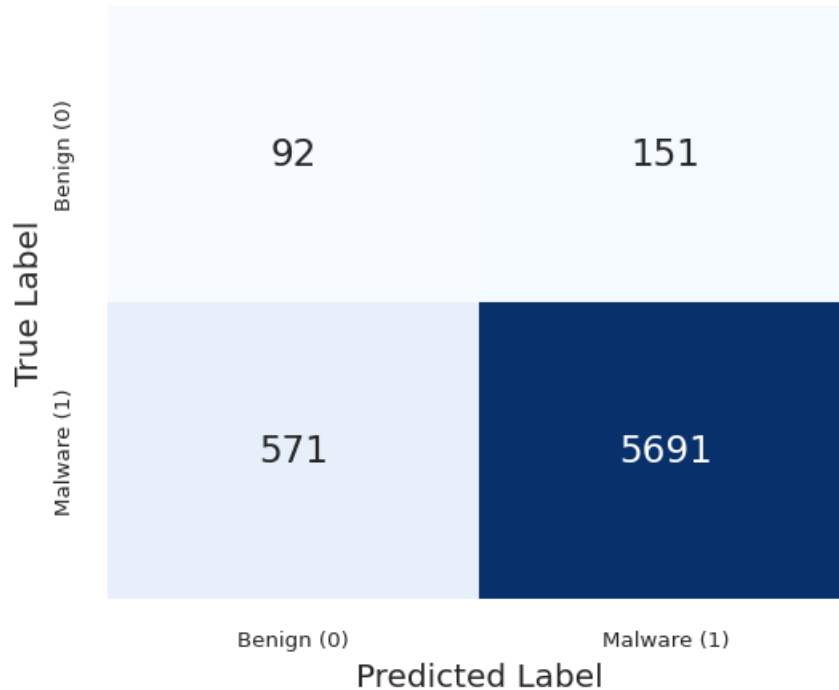
Generating confusion matrix for FFNN (Learnable Embeddings) with lr=1e-05...
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task2_plots/cm_ffnn_learnable_embeddings_(lr=1e_05).png

Confusion Matrix: FFNN Learnable Embeddings (lr=1e-05)



Generating confusion matrix for FFNN (Learnable Embeddings) with lr=5e-06...
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task2_plots/cm_ffnn_learnable_embeddings_(lr=5e_06).png

Confusion Matrix: FFNN Learnable Embeddings (lr=5e-06)



Embedding size sensitivity analysis To complete the learnable-embedding experiment, we keep the selected FFNN architecture and learning rate fixed, and vary only the embedding dimensionality. This isolates the effect of representation capacity from the effect of the optimizer or hidden-layer design.

```
[76]: # --- Embedding size sensitivity analysis ---

# The LR grid above selects the best learning rate for the learnable-embedding_
↳FFNN.
# Here we keep that LR and the rest of the architecture fixed, changing only_
↳embedding_dim.
embedding_sizes = [8, 16, 32, 64, 128]
if 'best_lr_emb' not in globals():
    raise RuntimeError('Run the learnable-embedding LR selection cell before_
↳the embedding-size sweep.')
selected_embedding_lr = best_lr_emb[0]

histories_emb_size = {}
models_emb_size = {}
elapsed_emb_size_times = {}
embedding_size_rows = []
```

```

def predict_ffnn(model, X_tensor):
    """Return binary predictions for an FFNN model."""
    model.eval()
    model_device = next(model.parameters()).device
    with torch.no_grad():
        logits = model(X_tensor.to(model_device))
        probs = torch.sigmoid(logits)
        preds = (probs > 0.5).long().cpu().numpy().flatten()
    return preds

print(f"\n{'='*80}")
print("Embedding size sensitivity analysis for FFNN Learnable Embeddings")
print(f"Fixed learning rate: {selected_embedding_lr:.0e}")
print(f"Embedding sizes: {embedding_sizes}")
print(f"\n{'='*80}")

for emb_dim in embedding_sizes:
    print(f"\n--- Training FFNN Learnable Embeddings with ↵
↵embedding_dim={emb_dim} ---")

    model_emb_size = MalwareFFNN(
        seq_len,
        hidden_dims,
        dropout_rates,
        batchnorm_flags,
        use_embedding=True,
        vocab_size=vocab_size,
        embedding_dim=emb_dim,
    ).to(device)

    criterion = nn.BCEWithLogitsLoss(pos_weight=pos_weight)
    optimizer = optim.AdamW(model_emb_size.parameters(), ↵
↵lr=selected_embedding_lr, weight_decay=1e-4)

    model_emb_size, history, elapsed_emb_size = train_model(
        model_emb_size,
        train_loader,
        val_loader,
        epochs,
        optimizer,
        criterion,
        min_delta,
        patience=patience_early_stop,
        scheduler=None,
    )

    histories_emb_size[emb_dim] = history

```

```

models_emb_size[emb_dim] = model_emb_size
elapsed_emb_size_times[emb_dim] = elapsed_emb_size

val_pred = predict_ffnn(model_emb_size, X_val_tensor)
test_pred = predict_ffnn(model_emb_size, X_test_tensor)

val_report = classification_report(y_val, val_pred, output_dict=True,
↪zero_division=0)
test_report = classification_report(y_test, test_pred, output_dict=True,
↪zero_division=0)

embedding_size_rows.append({
    'embedding_dim': emb_dim,
    'min_val_loss': min(history['val_loss']),
    'best_val_macro_f1_curve': max(history['val_f1_macro']),
    'val_macro_f1': val_report['macro avg']['f1-score'],
    'val_benign_recall': val_report['0']['recall'],
    'test_macro_f1': test_report['macro avg']['f1-score'],
    'test_benign_recall': test_report['0']['recall'],
    'training_time_s': elapsed_emb_size,
})

embedding_size_results = pd.DataFrame(embedding_size_rows)
embedding_size_results = embedding_size_results.sort_values('embedding_dim').
↪reset_index(drop=True)

print("\nEmbedding size comparison:")
display(embedding_size_results)

best_embedding_dim_row = embedding_size_results.
↪loc[embedding_size_results['min_val_loss'].idxmin()]
best_embedding_dim = int(best_embedding_dim_row['embedding_dim'])
print(
    f"\nBest embedding size by minimum validation loss: {best_embedding_dim} "
    f"(min val loss = {best_embedding_dim_row['min_val_loss']:.4f}, "
    f"test macro F1 = {best_embedding_dim_row['test_macro_f1']:.4f})"
)

```

```

=====
Embedding size sensitivity analysis for FFNN Learnable Embeddings
Fixed learning rate: 5e-05
Embedding sizes: [8, 16, 32, 64, 128]
=====

```

```

--- Training FFNN Learnable Embeddings with embedding_dim=8 ---
Epoch 1/100 - Train Loss: 0.1360, Val Loss: 0.1344, Val F1(W): 0.9386

```

Epoch 5/100 - Train Loss: 0.1153, Val Loss: 0.1203, Val F1(W): 0.9435
Epoch 10/100 - Train Loss: 0.1044, Val Loss: 0.1084, Val F1(W): 0.9475
Epoch 15/100 - Train Loss: 0.0960, Val Loss: 0.1007, Val F1(W): 0.9465
Epoch 20/100 - Train Loss: 0.0888, Val Loss: 0.0987, Val F1(W): 0.9420
Epoch 25/100 - Train Loss: 0.0820, Val Loss: 0.0937, Val F1(W): 0.9315
Epoch 30/100 - Train Loss: 0.0753, Val Loss: 0.0917, Val F1(W): 0.9208
Epoch 35/100 - Train Loss: 0.0698, Val Loss: 0.0903, Val F1(W): 0.9208
Epoch 40/100 - Train Loss: 0.0635, Val Loss: 0.0914, Val F1(W): 0.9148
Epoch 45/100 - Train Loss: 0.0582, Val Loss: 0.0907, Val F1(W): 0.9165
Epoch 50/100 - Train Loss: 0.0537, Val Loss: 0.0912, Val F1(W): 0.9173
Epoch 55/100 - Train Loss: 0.0485, Val Loss: 0.0924, Val F1(W): 0.9208
Early stopping at epoch 58 (best val loss: 0.088485)

Training completed in 00:00:51 (hh:mm:ss) - 51.09 seconds

--- Training FFNN Learnable Embeddings with embedding_dim=16 ---

Epoch 1/100 - Train Loss: 0.1671, Val Loss: 0.1491, Val F1(W): 0.8498
Epoch 5/100 - Train Loss: 0.1288, Val Loss: 0.1227, Val F1(W): 0.9393
Epoch 10/100 - Train Loss: 0.1045, Val Loss: 0.1198, Val F1(W): 0.9445
Epoch 15/100 - Train Loss: 0.0900, Val Loss: 0.0925, Val F1(W): 0.9407
Epoch 20/100 - Train Loss: 0.0791, Val Loss: 0.0861, Val F1(W): 0.9338
Epoch 25/100 - Train Loss: 0.0704, Val Loss: 0.0838, Val F1(W): 0.9270
Epoch 30/100 - Train Loss: 0.0609, Val Loss: 0.0823, Val F1(W): 0.9332
Epoch 35/100 - Train Loss: 0.0539, Val Loss: 0.0834, Val F1(W): 0.9343
Epoch 40/100 - Train Loss: 0.0487, Val Loss: 0.0877, Val F1(W): 0.9423
Early stopping at epoch 45 (best val loss: 0.082346)

Training completed in 00:00:39 (hh:mm:ss) - 39.54 seconds

--- Training FFNN Learnable Embeddings with embedding_dim=32 ---

Epoch 1/100 - Train Loss: 0.1700, Val Loss: 0.1551, Val F1(W): 0.7092
Epoch 5/100 - Train Loss: 0.1278, Val Loss: 0.1275, Val F1(W): 0.9315
Epoch 10/100 - Train Loss: 0.1000, Val Loss: 0.1076, Val F1(W): 0.9383
Epoch 15/100 - Train Loss: 0.0832, Val Loss: 0.0937, Val F1(W): 0.9369
Epoch 20/100 - Train Loss: 0.0679, Val Loss: 0.0866, Val F1(W): 0.9358
Epoch 25/100 - Train Loss: 0.0559, Val Loss: 0.0812, Val F1(W): 0.9383
Epoch 30/100 - Train Loss: 0.0445, Val Loss: 0.1025, Val F1(W): 0.9420
Epoch 35/100 - Train Loss: 0.0368, Val Loss: 0.0787, Val F1(W): 0.9432
Epoch 40/100 - Train Loss: 0.0289, Val Loss: 0.0833, Val F1(W): 0.9470
Epoch 45/100 - Train Loss: 0.0237, Val Loss: 0.0841, Val F1(W): 0.9507
Early stopping at epoch 50 (best val loss: 0.078678)

Training completed in 00:00:43 (hh:mm:ss) - 43.88 seconds

--- Training FFNN Learnable Embeddings with embedding_dim=64 ---

Epoch 1/100 - Train Loss: 0.2340, Val Loss: 0.1918, Val F1(W): 0.0191
Epoch 5/100 - Train Loss: 0.1685, Val Loss: 0.1612, Val F1(W): 0.6051
Epoch 10/100 - Train Loss: 0.1228, Val Loss: 0.1240, Val F1(W): 0.8958


```
Epoch 15/100 - Train Loss: 0.0934, Val Loss: 0.0996, Val F1(W): 0.9261
Epoch 20/100 - Train Loss: 0.0709, Val Loss: 0.0842, Val F1(W): 0.9372
Epoch 25/100 - Train Loss: 0.0531, Val Loss: 0.0760, Val F1(W): 0.9492
Epoch 30/100 - Train Loss: 0.0410, Val Loss: 0.0741, Val F1(W): 0.9523
Epoch 35/100 - Train Loss: 0.0348, Val Loss: 0.0805, Val F1(W): 0.9518
Epoch 40/100 - Train Loss: 0.0264, Val Loss: 0.0869, Val F1(W): 0.9551
Early stopping at epoch 45 (best val loss: 0.074088)
```

Training completed in 00:00:39 (hh:mm:ss) - 39.59 seconds

```
--- Training FFNN Learnable Embeddings with embedding_dim=128 ---
Epoch 1/100 - Train Loss: 0.1969, Val Loss: 0.1784, Val F1(W): 0.1645
Epoch 5/100 - Train Loss: 0.1405, Val Loss: 0.1408, Val F1(W): 0.8311
Epoch 10/100 - Train Loss: 0.1010, Val Loss: 0.1019, Val F1(W): 0.9275
Epoch 15/100 - Train Loss: 0.0720, Val Loss: 0.0854, Val F1(W): 0.9359
Epoch 20/100 - Train Loss: 0.0522, Val Loss: 0.0751, Val F1(W): 0.9422
Epoch 25/100 - Train Loss: 0.0400, Val Loss: 0.0749, Val F1(W): 0.9474
Epoch 30/100 - Train Loss: 0.0293, Val Loss: 0.0823, Val F1(W): 0.9500
Epoch 35/100 - Train Loss: 0.0223, Val Loss: 0.0862, Val F1(W): 0.9521
Early stopping at epoch 39 (best val loss: 0.073032)
```

Training completed in 00:00:35 (hh:mm:ss) - 35.30 seconds

Embedding size comparison:

	embedding_dim	min_val_loss	best_val_macro_f1_curve	val_macro_f1	\
0	8	0.088485	0.607215	0.584713	
1	16	0.082346	0.628196	0.603783	
2	32	0.078678	0.663323	0.636757	
3	64	0.074088	0.682119	0.675566	
4	128	0.073032	0.675566	0.644022	

	val_benign_recall	test_macro_f1	test_benign_recall	training_time_s
0	0.319672	0.538210	0.218107	51.091888
1	0.336066	0.556788	0.251029	39.538133
2	0.360656	0.570540	0.259259	43.876767
3	0.393443	0.590166	0.251029	39.585512
4	0.377049	0.573982	0.288066	35.296682

Best embedding size by minimum validation loss: 128 (min val loss = 0.0730, test macro F1 = 0.5740)

```
[77]: # --- Plot embedding size comparison ---

fig, axes = plt.subplots(1, 2, figsize=(12, 4))

axes[0].plot(
```

```

        embedding_size_results['embedding_dim'],
        embedding_size_results['min_val_loss'],
        marker='o',
        linewidth=2,
    )
axes[0].set_title('Minimum Validation Loss vs Embedding Size')
axes[0].set_xlabel('Embedding Size')
axes[0].set_ylabel('Minimum Validation Loss')
axes[0].grid(True, alpha=0.3)

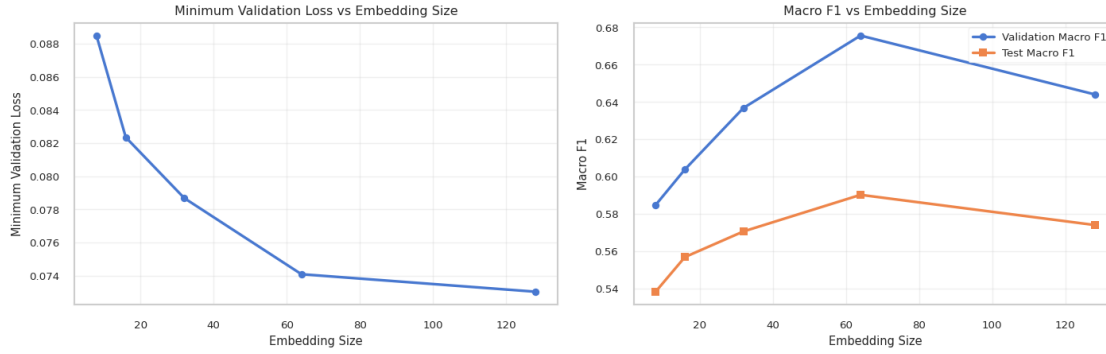
axes[1].plot(
    embedding_size_results['embedding_dim'],
    embedding_size_results['val_macro_f1'],
    marker='o',
    linewidth=2,
    label='Validation Macro F1',
)
axes[1].plot(
    embedding_size_results['embedding_dim'],
    embedding_size_results['test_macro_f1'],
    marker='s',
    linewidth=2,
    label='Test Macro F1',
)
axes[1].set_title('Macro F1 vs Embedding Size')
axes[1].set_xlabel('Embedding Size')
axes[1].set_ylabel('Macro F1')
axes[1].grid(True, alpha=0.3)
axes[1].legend()

plt.tight_layout()
if save_dir:
    save_plot(fig, 'ffnn_embedding_size_comparison', save_dir)
plt.show()

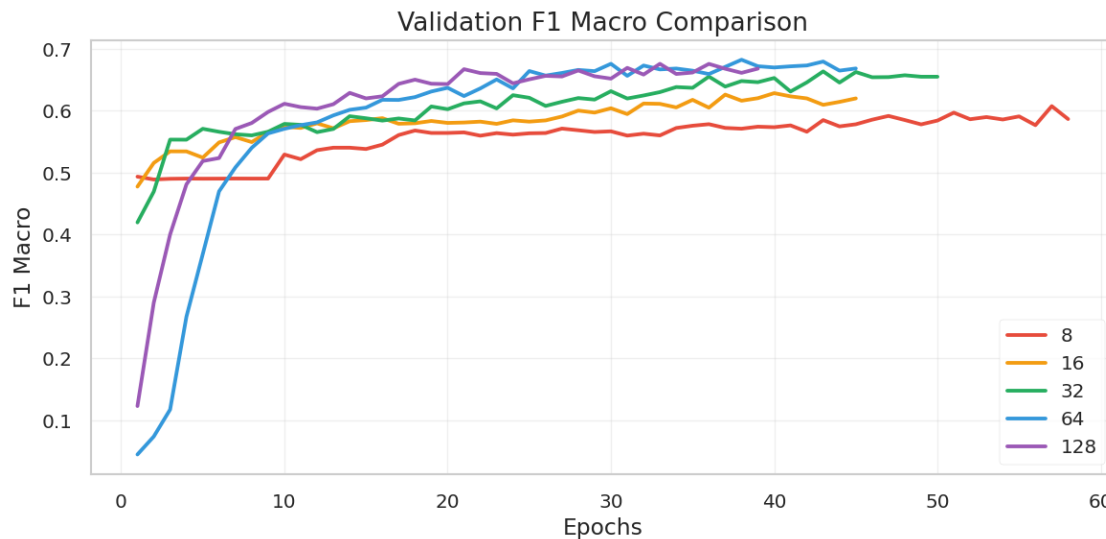
# Optional: compare the validation macro F1 curves across embedding sizes.
plot_training_metrics(
    histories_emb_size,
    save_dir=save_dir,
    plot_name='ffnn_embedding_size_f1_comparison'
)

```

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task2_plots/ffnn_embedding_size_comparison.png



Saved plot: `/content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task2_plots/ffnn_embedding_size_f1_comparison.png`



Q: Use a FFNN in both cases. Report how you selected the hyperparameters of your final model, and justify your choices. For Task 2, we implemented two FFNN configurations: one using raw sequential identifiers and one using learnable embeddings. In both cases, model selection used validation loss as the primary criterion, while validation macro F1 was monitored because the dataset is strongly imbalanced and accuracy alone would hide poor behavior on the benign class.

Sequential identifiers. The sequential-ID model receives a fixed-length vector of 90 API identifiers. We used a funnel-shaped architecture with hidden dimensions [256, 128, 64], Batch Normalization, Dropout, AdamW, and BCEWithLogitsLoss. The tested learning rates were $1e-4$, $5e-5$, and $1e-5$. The selected configuration is $lr=1e-4$, which reaches the lowest validation loss (0.0921) and validation macro F1 0.5400. On the test set, it obtains accuracy 93.33% and macro F1 0.5452.

Learnable embeddings. The embedding-based model first maps each API identifier into a continuous vector and then flattens the embedded sequence before the dense layers. We used hidden dimensions [128, 64, 32], Batch Normalization, Dropout, AdamW, and the same early-stopping strategy. The tested learning rates were 5e-5, 1e-5, and 5e-6; the selected learning rate is 5e-5, with validation loss 0.0712 and validation macro F1 0.6709. On the test set, this selected model reaches accuracy 92.79% and macro F1 0.5789.

To address the requirement of testing different embedding sizes, we then fixed the selected learning rate (5e-5) and repeated the experiment with embedding dimensions [8, 16, 32, 64, 128]. The validation macro F1 generally improves as the representation becomes richer: it goes from 0.5847 with dimension 8 to 0.6756 with dimension 64. The lowest validation loss is obtained with dimension 128 (0.0730), while the best validation macro F1 is obtained with dimension 64 (0.6756). On the test set, dimension 64 reaches macro F1 0.5902, slightly above the selected-loss dimension 128 (0.5740). This suggests that larger embeddings help, but the gain is modest and still limited by the FFNN fixed-length representation.

The loss uses `pos_weight=0.20` because label 1 corresponds to malware, which is the majority class. In `BCEWithLogitsLoss`, this down-weights malware errors and gives more relative influence to the minority benign samples during optimization.

Q: Can you obtain the same results for sequential identifiers and learnable embeddings? If not, why? No. The learnable-embedding representation performs better than raw sequential identifiers, especially when looking at validation macro F1 and benign-class behavior. The sequential-ID model treats API identifiers as ordinary numerical values, implicitly suggesting that nearby IDs have a meaningful numerical relationship. This is not true: the assignment of IDs is arbitrary, so the network may exploit accidental magnitudes rather than semantic similarity between API calls.

Learnable embeddings reduce this limitation by mapping each API call into a continuous vector space. During training, the model can learn representations that make API calls with similar roles or similar discriminative behavior easier to use jointly. This is reflected in the selected validation results: the best sequential-ID model reaches validation macro F1 0.5400, while the embedding-based model reaches 0.6709 with the selected learning rate.

The embedding-size experiment also shows that the embedding mechanism is useful, but not unlimited. Very small embeddings, such as dimension 8, do not provide enough representation capacity and obtain lower macro F1. Larger embeddings improve validation performance, with dimension 64 giving the best validation macro F1 (0.6756) and dimension 128 giving the lowest validation loss (0.0730). On the test set, dimension 64 obtains macro F1 0.5902, while dimension 128 obtains 0.5740, suggesting that extra capacity helps only up to a point and does not fully solve the FFNN limitations.

Overall, learnable embeddings are preferable to sequential identifiers, but the FFNN remains limited by the fixed-length representation. Since sequences longer than 90 calls must be truncated, and since the dense network cannot explicitly model temporal dependencies, both FFNN variants remain weaker than the later RNN and GNN models.

```
[78]: # =====
# FINAL SUMMARY
# =====
```

```

# Helper function to extract macro avg from the best FFNN model
def get_macro_avg_stats_ffnn(model, X_tensor):
    """
    Predict and get macro average statistics for an FFNN model.
    """
    # Get the device from the model itself
    device = next(model.parameters()).device

    model.eval()
    with torch.no_grad():
        outputs = model(X_tensor.to(device)) # Move input tensor to device
        probs = torch.sigmoid(outputs)
        y_pred = (probs > 0.5).long().cpu().numpy().flatten()

    y_true = y_val # Using validation set
    report_dict = classification_report(y_true, y_pred, output_dict=True,
    ↪ zero_division=0)
    return report_dict['macro avg']

# --- 1. Prepare Stats for FFNN Sequential IDs ---
best_model_seq_obj = models_seq[best_lr_seq[0]]
macro_seq = get_macro_avg_stats_ffnn(best_model_seq_obj, X_val_tensor)

# --- 2. Prepare Stats for FFNN Learnable Embeddings ---
best_model_emb_obj = models_emb[best_lr_emb[0]]
macro_emb = get_macro_avg_stats_ffnn(best_model_emb_obj, X_val_tensor)

# --- PRINTING THE REPORT ---

print("\n" + "=" * 100)
print("TASK 2 SUMMARY: Best Learning Rates & Macro Avg Performance")
print("=" * 100)
# Header for the manual table
print(f"{'Model':<30} | {'Best LR':<10} | {'Val Loss':<10} | {'Macro P':<8} | "
    ↪ {'Macro R':<8} {'Macro F1':<8} | {'Time':<10}")
print("-" * 100)

# 1. FFNN Sequential IDs
t_seq = time.strftime('%H:%M:%S', time.
    ↪ gmtime(elapsed_seq_times[best_lr_seq[0]]))
print(f"{'FFNN (Sequential IDs)':<30} | {best_lr_seq[0]:.0e} | "
    ↪ {min(best_lr_seq[1]['val_loss']):.4f} | "
        f"{macro_seq['precision']:.4f} {macro_seq['recall']:.4f} "
    ↪ {macro_seq['f1-score']:.4f} | {t_seq}")

# 2. FFNN Learnable Embeddings

```

```

t_emb = time.strftime('%H:%M:%S', time.
    ↳gmtime(elapsed_emb_times[best_lr_emb[0]]))
print(f"'FFNN (Learnable Embeddings)':<30> | {best_lr_emb[0]:.0e}      |␣
    ↳{min(best_lr_emb[1]['val_loss']):.4f}      | "
        f"{macro_emb['precision']:.4f}      {macro_emb['recall']:.4f}      ␣
    ↳{macro_emb['f1-score']:.4f}      | {t_emb}")

# --- 3. Optional Embedding Size Summary ---
if 'embedding_size_results' in globals():
    best_emb_loss_row = embedding_size_results.
    ↳loc[embedding_size_results['min_val_loss'].idxmin()]
    best_emb_f1_row = embedding_size_results.
    ↳loc[embedding_size_results['val_macro_f1'].idxmax()]

    print("\n" + "=" * 100)
    print("EMBEDDING SIZE SENSITIVITY SUMMARY")
    print("=" * 100)
    print(
        f"Best by min validation loss:␣
    ↳dim={int(best_emb_loss_row['embedding_dim'])} | "
        f"val loss={best_emb_loss_row['min_val_loss']:.4f} | "
        f"val macro F1={best_emb_loss_row['val_macro_f1']:.4f} | "
        f"test macro F1={best_emb_loss_row['test_macro_f1']:.4f} | "
        f"time={best_emb_loss_row['training_time_s']:.2f}s"
    )
    print(
        f"Best by validation macro F1:␣
    ↳dim={int(best_emb_f1_row['embedding_dim'])} | "
        f"val loss={best_emb_f1_row['min_val_loss']:.4f} | "
        f"val macro F1={best_emb_f1_row['val_macro_f1']:.4f} | "
        f"test macro F1={best_emb_f1_row['test_macro_f1']:.4f} | "
        f"time={best_emb_f1_row['training_time_s']:.2f}s"
    )

```

TASK 2 SUMMARY: Best Learning Rates & Macro Avg Performance

Model	Best LR	Val Loss	Macro P	Macro R
Macro F1 Time				
FFNN (Sequential IDs)	1e-04	0.0921	0.5349	0.5490
0.5400 00:00:32				
FFNN (Learnable Embeddings)	5e-05	0.0712	0.6542	0.6926

0.6709 | 00:00:33

EMBEDDING SIZE SENSITIVITY SUMMARY

Best by min validation loss: dim=128 | val loss=0.0730 | val macro F1=0.6440 |
test macro F1=0.5740 | time=35.30s
Best by validation macro F1: dim=64 | val loss=0.0741 | val macro F1=0.6756 |
test macro F1=0.5902 | time=39.59s

1.4 Task 3 - Recursive Neural Network (RNN)

```
[79]: # Create directory for plots
save_dir = results_path + 'images/' + 'task3_plots/'
os.makedirs(save_dir, exist_ok=True)

[80]: # --- Choose number of epochs depending on device (GPU vs CPU) ---

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

# Use fewer epochs when running on CPU to avoid extremely long runs
if torch.cuda.is_available():
    num_epochs = 150
else:
    num_epochs = 15

print(f"Number of epochs: {num_epochs}")
```

Number of epochs: 150

1.4.1 Sequence Modeling with RNNs

```
[81]: # --- Build vocabulary and API-to-ID mappings from training data ---

def build_vocab_from_train(df_train, seq_col="api_call_sequence"):
    """
    Extract the set of unique API calls from the training set only.
    Returns a sorted list of API call names (vocab).
    """
    vocab = sorted({api for seq in df_train[seq_col] for api in seq})
    return vocab

def build_api_mappings(vocab):
    """
    Create dictionaries mapping API names to integer IDs and vice versa.
    Reserves indices 0 and 1 for <PAD> and <UNK> special tokens.
    """
```

```

"""
api_to_id = {"<PAD>": PAD_IDX, "<UNK>": UNK_IDX}
for i, api in enumerate(vocab):
    api_to_id[api] = i + 2 # start after PAD/UNK
id_to_api = {i: api for api, i in api_to_id.items()}
return api_to_id, id_to_api

# Build vocabulary from training data only (no data leakage)
vocab_train = build_vocab_from_train(df_train)
api_to_id, id_to_api = build_api_mappings(vocab_train)

```

```

[82]: # --- Encode API call sequences to integer IDs ---

def encode_sequences(df, api_to_id, seq_col="api_call_sequence",
    ↪label_col="is_malware", max_len=None):
    """
    Convert API call sequences to lists of integer IDs using the mapping.
    Unknown API calls (not seen in training) are mapped to <UNK> (PAD_IDX).
    Optionally truncate sequences to max_len (but we preserve full length for
    ↪RNNs).
    """
    seqs, labels = [], []
    for seq, y in zip(df[seq_col], df[label_col]):
        ids = [api_to_id.get(api, PAD_IDX) for api in seq]
        if max_len is not None:
            ids = ids[:max_len] # only if you explicitly choose to truncate
        seqs.append(ids)
        labels.append(int(y))
    return seqs, labels

# Encode training and test sequences (no truncation - RNNs handle variable
    ↪lengths)
train_seqs, train_labels = encode_sequences(df_train, api_to_id)
test_seqs, test_labels = encode_sequences(df_test, api_to_id)

```

```

[83]: # --- Split training data into train/validation subsets ---

# Create indices for stratified train/val split (preserves class balance)
idx = list(range(len(train_seqs)))
train_idx, val_idx = train_test_split(
    idx, test_size=0.2, random_state=42, stratify=train_labels
)

def subset_by_idx(seqs, labels, indices):
    """Helper to extract sequences and labels at given indices."""
    return [seqs[i] for i in indices], [labels[i] for i in indices]

```



```
# Create training and validation subsets
train_seqs_sub, train_labels_sub = subset_by_idx(train_seqs, train_labels,
↳train_idx)
val_seqs_sub, val_labels_sub = subset_by_idx(train_seqs, train_labels, val_idx)
```

[84]: # --- Define PyTorch Dataset for variable-length sequences ---

```
class SeqDataset(torch.utils.data.Dataset):
    """
    A PyTorch Dataset that wraps sequences and their labels.
    Each item returns a tensor of API IDs and a scalar label.
    Note: Sequences have variable lengths at this stage.
    """
    def __init__(self, sequences, labels):
        self.sequences = sequences
        self.labels = labels

    def __len__(self):
        return len(self.sequences)

    def __getitem__(self, idx):
        x = torch.tensor(self.sequences[idx], dtype=torch.long)
        y = torch.tensor(self.labels[idx], dtype=torch.float32) # scalar
        return x, y
```

[85]: # --- Custom collate function for padding and packing ---

```
def collate_fn(batch):
    """
    Custom collate function for DataLoader:
    1. Sorts sequences by length (descending) - required for
↳pack_padded_sequence
    2. Pads all sequences to the max length in this batch
    3. Returns padded sequences, original lengths, and labels (CPU tensors)
    Note: Device transfer is handled by the training loop, not the collate
↳function.
    """
    sequences, labels = zip(*batch)

    # Compute original lengths before padding
    lengths = torch.tensor([len(s) for s in sequences], dtype=torch.long)
    # Sort by length (descending) for pack_padded_sequence compatibility
    lengths_sorted, perm_idx = lengths.sort(descending=True)

    # Reorder sequences and labels according to sorted indices
    sequences = [sequences[i] for i in perm_idx]
    labels = torch.stack([labels[i] for i in perm_idx]).unsqueeze(1) # (B,1)
```

```

# Pad sequences to max length in batch (dynamic padding)
padded = pad_sequence(sequences, batch_first=True, padding_value=PAD_IDX)

# Return CPU tensors - training loop handles device transfer
return padded, lengths_sorted, labels

```

```

[86]: # --- Create DataLoaders with custom collate function ---

# Wrap sequences in Dataset objects
train_ds = SeqDataset(train_seqs_sub, train_labels_sub)
val_ds = SeqDataset(val_seqs_sub, val_labels_sub)
test_ds = SeqDataset(test_seqs, test_labels)

# Create DataLoaders with dynamic padding via collate_fn
# Note: collate_fn handles variable-length sequences by padding per batch
rnn_train_generator = torch.Generator()
rnn_train_generator.manual_seed(42)
train_loader = DataLoader(
    train_ds,
    batch_size=64,
    shuffle=True,
    collate_fn=collate_fn,
    generator=rnn_train_generator,
)
val_loader = DataLoader(val_ds, batch_size=64, shuffle=False,
    ↪collate_fn=collate_fn)
test_loader = DataLoader(test_ds, batch_size=64, shuffle=False,
    ↪collate_fn=collate_fn)

```

Q: With RNNs, do you still have to pad your data? If yes, how? Yes. RNNs can process sequences of different lengths in principle, but mini-batch training still requires tensors with a consistent shape inside each batch. For this reason, we pad the sequences dynamically at batch level rather than padding every sequence to a global fixed length.

The custom `collate_fn` sorts the samples in each batch by decreasing sequence length, pads only up to the longest sequence in that batch, and returns both the padded tensor and the original lengths. The embedding layer uses `padding_idx=0`, so the padding token has a dedicated embedding. Then, before the recurrent layer, we use `torch.nn.utils.rnn.pack_padded_sequence`, which tells the RNN to ignore padded positions during the recurrent computation.

This approach keeps the batch representation compatible with PyTorch while avoiding hidden-state updates on dummy tokens.

Q: Do you have to truncate the testing sequences? Justify your answer with your understanding of why it is/it is not the case. No. Unlike an FFNN, an RNN does not have a fixed input layer tied to a specific sequence length. The same recurrent weights are reused at every time step, so a longer test sequence simply produces more recurrent updates.

In our implementation, padding and `pack_padded_sequence` are used only to manage batches efficiently. They do not impose the fixed length of 90 used in the FFNN. Therefore, test sequences longer than the maximum training length do not need to be truncated for architectural reasons.

Avoiding truncation is important because API calls near the end of a trace may contain useful behavioral information. Truncating them would remove part of the execution context, while the recurrent model can process the full sequence within normal memory limits.

```
[87]: # Reusable training loop for RNN-like modules
def train_rnn_common(model, train_loader, val_loader, optimizer, criterion,
    ↪ epochs, min_delta=None, patience=None, scheduler=None):
    """
    A generic training loop for RNN-based models that supports early stopping.
    Now also tracks validation F1 macro score.
    """
    device = next(model.parameters()).device

    best_val_f1 = float('-inf')
    best_val_loss = math.inf
    best_epoch = 0
    best_state = None
    trigger = 0
    history = {'train_loss': [], 'val_loss': [], 'val_f1_macro': []}
    start_time = time.time()

    for epoch in range(1, epochs+1):
        # --- Training Phase ---
        model.train()
        train_losses = []
        for X_batch, lengths, y_batch in train_loader:
            X_batch, y_batch = X_batch.to(device), y_batch.to(device)
            optimizer.zero_grad()
            outputs = model(X_batch, lengths)
            loss = criterion(outputs, y_batch)
            loss.backward()
            torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
            optimizer.step()
            train_losses.append(loss.item())
        train_loss = float(np.mean(train_losses)) if train_losses else 0.0

        # --- Validation Phase ---
        model.eval()
        val_losses = []
        all_preds = []
        all_targets = []
        with torch.no_grad():
            for X_batch, lengths, y_batch in val_loader:
                X_batch, y_batch = X_batch.to(device), y_batch.to(device)
```

```

        outputs = model(X_batch, lengths)
        loss = criterion(outputs, y_batch)
        val_losses.append(loss.item())

        # Collect predictions for F1 score
        probs = torch.sigmoid(outputs).view(-1)
        preds = (probs > 0.5).long()
        all_preds.extend(preds.cpu().tolist())
        all_targets.extend(y_batch.view(-1).cpu().tolist())

    val_loss = float(np.mean(val_losses)) if val_losses else 0.0
    val_f1_macro = f1_score(all_targets, all_preds, average='macro',
↪zero_division=0)

    history['train_loss'].append(train_loss)
    history['val_loss'].append(val_loss)
    history['val_f1_macro'].append(val_f1_macro)

    # --- Scheduler Step ---
    if scheduler is not None:
        last_lr = optimizer.param_groups[0]['lr']
        if isinstance(scheduler, torch.optim.lr_scheduler.
↪ReduceLROnPlateau):
            scheduler.step(val_loss)
        else:
            scheduler.step()
        curr_lr = optimizer.param_groups[0]['lr']
        if curr_lr != last_lr:
            print(f"Epoch {epoch}: LR reduced from {last_lr:.6f} to
↪{curr_lr:.6f}")

    if (epoch % 5 == 0) or (epoch == 1) or (epoch == epochs):
        print(f'Epoch {epoch}/{epochs} - Train Loss: {train_loss:.4f} -
↪Val Loss: {val_loss:.4f} - Val F1: {val_f1_macro:.4f}')

    # --- Checkpointing and early stopping on validation macro F1 ---
    f1_delta = min_delta if min_delta is not None else 0
    f1_improved = val_f1_macro > best_val_f1 + f1_delta
    f1_tied = abs(val_f1_macro - best_val_f1) <= f1_delta
    loss_improved_on_tie = f1_tied and val_loss < best_val_loss

    if f1_improved or loss_improved_on_tie:
        best_val_f1 = val_f1_macro
        best_val_loss = val_loss
        best_epoch = epoch
        best_state = {k: v.cpu().clone() for k, v in model.state_dict().
↪items()}

```

```

        trigger = 0
    else:
        trigger += 1
        if trigger >= patience:
            print(
                f'Early stopping at epoch {epoch} '
                f'(best val macro F1 {best_val_f1:.4f} at epoch_{
↪{best_epoch}}, '
                f'val loss {best_val_loss:.6f})'
            )
            break

    elapsed = time.time() - start_time

    if best_state is not None:
        model.load_state_dict(best_state)
        model.to(device)

    history['best_epoch'] = best_epoch
    history['best_val_f1_macro'] = best_val_f1
    history['best_val_loss_at_checkpoint'] = best_val_loss

    return model, history, elapsed

```

```

[88]: # Prediction helper (binary classification threshold)
def predict_loader(model, loader):
    """
    Run the model on a DataLoader and return integer arrays for
    ground-truth labels and predicted labels (0/1).
    """
    # Set model to evaluation mode (e.g., disables dropout)
    model.eval()

    # Detect device from model parameters
    device = next(model.parameters()).device

    ys_true = [] # To store all true labels
    ys_pred = [] # To store all predicted labels

    # Disable gradient calculations to save memory and computation
    with torch.no_grad():
        # Iterate over all batches in the data loader
        for X_batch, lengths, y_batch in loader:
            # Move batch to device
            X_batch = X_batch.to(device)

            # Get model outputs (logits)

```

```

out = model(X_batch, lengths)

# Convert logits to probabilities using sigmoid
probs = torch.sigmoid(out).cpu().numpy().ravel()

# Apply threshold for binary classification
preds = (probs >= 0.5).astype(int)

# Store the ground-truth and predicted labels from this batch
ys_true.extend(y_batch.cpu().numpy().ravel().astype(int).tolist())
ys_pred.extend(preds.tolist())

# Convert the lists of all labels/preds into final numpy arrays
return np.array(ys_true, dtype=int), np.array(ys_pred, dtype=int)

```

```

[89]: def plot_cm_task3(model, loader, model_name, save_dir=None):
        """
        Specific for RNNs. Uses your existing 'predict_loader' logic.
        """
        # Use the helper function you already defined in the notebook
        y_true, y_pred = predict_loader(model, loader)

        # Plot
        _internal_plot_cm(y_true, y_pred, model_name, save_dir)

```

```

[90]: # Function to train a model with a specific learning rate
def set_rnn_seed(seed=42):
    """Reset random state before every RNN experiment."""
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)

def train_with_lr(
    model_class, model_params, initial_lr, train_loader, val_loader,
    criterion, num_epochs, min_delta, patience, scheduler_params, device,
    weight_decay=1e-4, seed=42,
):
    """
    Train a model with a specific initial learning rate and return the history.
    """
    set_rnn_seed(seed)
    if getattr(train_loader, 'generator', None) is not None:
        train_loader.generator.manual_seed(seed)

```

```

# Create a fresh instance of the model
model = model_class(**model_params).to(device)

# Create optimizer with the specified learning rate
optimizer = optim.AdamW(model.parameters(), lr=initial_lr,
↪weight_decay=weight_decay)

# Create scheduler
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
    optimizer,
    **scheduler_params
)

# Train the model
model, history, elapsed = train_rnn_common(
    model,
    train_loader,
    val_loader,
    optimizer,
    criterion,
    epochs=num_epochs,
    min_delta=min_delta,
    patience=patience,
    scheduler=scheduler
)

return model, history, elapsed

def select_best_rnn_model(architecture_name, models, histories, val_loader):
    """Select a restored RNN model using validation metrics only."""
    rows = []

    for lr, model in models.items():
        y_val_true, y_val_pred = predict_loader(model, val_loader)
        report = classification_report(
            y_val_true, y_val_pred, output_dict=True, zero_division=0
        )
        rows.append({
            'learning_rate': lr,
            'val_macro_f1': report['macro avg']['f1-score'],
            'val_accuracy': report['accuracy'],
            'checkpoint_val_loss': histories[lr]['best_val_loss_at_checkpoint'],
            'checkpoint_epoch': histories[lr]['best_epoch'],
        })

    selection_table = pd.DataFrame(rows).sort_values(

```

```

        by=['val_macro_f1', 'val_accuracy', 'checkpoint_val_loss'],
        ascending=[False, False, True],
    ).reset_index(drop=True)

    best_lr = float(selection_table.loc[0, 'learning_rate'])
    print(f"\nAutomatic selection for {architecture_name}")
    print("Ranking criterion: validation macro F1, then validation accuracy,
    ↪then checkpoint validation loss")
    print(selection_table.to_string(index=False))
    print(
        f"Selected {architecture_name}: lr={best_lr:.0e}, "
        f"validation macro F1={selection_table.loc[0, 'val_macro_f1']:.4f}, "
        f"checkpoint epoch={int(selection_table.loc[0, 'checkpoint_epoch'])}"
    )

    return best_lr, selection_table

```

Q: Is the RNN padding more memory efficient compared to the FFNN's one? Why?

Yes, the RNN padding strategy is more efficient than the FFNN fixed-length strategy. In Task 2, every sequence must be converted into the same global length of 90 before entering the FFNN. This means that shorter sequences still occupy the full fixed-size input vector, and longer test sequences must be truncated.

In Task 3, padding is performed only inside each mini-batch, up to the longest sequence in that batch. Moreover, `pack_padded_sequence` prevents the recurrent layer from computing hidden-state transitions on padding tokens. The padded tensor still exists for batching, but the expensive recurrent computation is focused on real API calls.

This makes the RNN approach more flexible and usually more memory/computation efficient for variable-length sequences, especially when sequence lengths differ substantially across samples.

```

[91]: # --- Variation 1: Simple one-directional RNN (nn.RNN) ---

class SimpleRNClassifier(nn.Module):
    def __init__(self, vocab_size, embed_dim=32, hidden_size=128, num_layers=2,
    ↪num_classes=1, bidirectional=False, dropout=0.25, padding=0):
        super(SimpleRNClassifier, self).__init__()

        # Embedding layer: Maps vocabulary indices (integers) to dense vectors
    ↪(embeddings).
        # padding_idx=0 ensures the padding token doesn't contribute to
    ↪gradients.
        self.embedding = nn.Embedding(vocab_size, embed_dim,
    ↪padding_idx=padding)

        # Simple RNN layer: Processes the embedded sequences.
        self.rnn = nn.RNN(
            input_size=embed_dim,

```



```

        hidden_size=hidden_size,
        num_layers=num_layers,
        batch_first=True,
        nonlinearity='tanh',
        bidirectional=bidirectional,
        dropout=dropout if num_layers > 1 else 0.0,
    )

    self.bidirectional = bidirectional
    fc_in = hidden_size * (2 if bidirectional else 1)

    # Final fully connected layer for classification.
    self.fc = nn.Linear(fc_in, num_classes)
    self.dropout = nn.Dropout(dropout)

    def forward(self, x, lengths):
        # Embed the input indices
        emb = self.embedding(x) # Shape: (batch_size, max_seq_len, embed_dim)
        emb = self.dropout(emb)

        # Pack the sequence to let the RNN ignore padding
        packed = pack_padded_sequence(emb, lengths.cpu(), batch_first=True,
        ↪enforce_sorted=True)

        # Process the packed sequence through the RNN
        packed_out, hn = self.rnn(packed) # hn is the final hidden state(s)

        # hn shape: (num_layers * num_directions, batch_size, hidden_size)

        # Get the final hidden state to represent the whole sequence.
        if self.bidirectional:
            # Concatenate the final forward (hn[-2]) and backward (hn[-1])
            ↪states
            h_last = torch.cat([hn[-2], hn[-1]], dim=1)
        else:
            # Use the final hidden state from the last layer (hn[-1])
            h_last = hn[-1]

        # Pass the final hidden state through the classifier
        out = self.fc(h_last)
        return out

```

[92]: # --- Variation 1: Simple Unidirectional RNN ---

```

print("\n" + "=" * 80)
print("Training Simple RNN with different learning rates")
print("=" * 80)

```

```

# Define learning rates to test
learning_rates = [2e-04, 1e-04, 8e-05, 5e-05]

# Model parameters
vocab_size_rnn = len(api_to_id)
simple_rnn_params = {
    'vocab_size': vocab_size_rnn,
    'embed_dim': 32,
    'hidden_size': 128,
    'num_layers': 2,
    'num_classes': 1,
    'bidirectional': False,
    'dropout': 0.3
}

# Initialize early stopping parameters
min_delta = 0.00001
patience = 20

# Compute class balance from training subset and set pos_weight for
↳ BCEWithLogitsLoss
pos = sum(train_labels_sub) # number of positive (1) samples
neg = len(train_labels_sub) - pos

# Compute weights
pos_weight = compute_pos_weight(train_labels_sub).to(device)

criterion = nn.BCEWithLogitsLoss(pos_weight=pos_weight)

# Setup Scheduler
scheduler_params_simple = {
    'mode': 'min', # reduce when metric stops decreasing (val_loss)
    'factor': 0.5, # halve the LR when triggered
    'patience': 8, # wait 8 epochs with no improvement before cutting LR
    'min_lr': 1e-7
}

# Store histories and models for each LR
histories_simple = {}
models_simple = {}
elapsed_simple = {}

for lr in learning_rates:
    print(f"\n--- Training Simple RNN with lr={lr:.0e} ---")
    model, history, elapsed = train_with_lr(
        model_class=SimpleRNNClassifier,

```

```

        model_params=simple_rnn_params,
        initial_lr=lr,
        train_loader=train_loader,
        val_loader=val_loader,
        criterion=nn.BCEWithLogitsLoss(pos_weight=pos_weight),
        num_epochs=num_epochs,
        min_delta=min_delta,
        patience=patience,
        scheduler_params=scheduler_params_simple,
        device=device
    )
    histories_simple[lr] = history
    models_simple[lr] = model
    elapsed_simple[lr] = elapsed
    elapsed_str = time.strftime('%H:%M:%S', time.gmtime(elapsed))
    print(f"Completed. Training time: {elapsed_str} ({elapsed:.2f}s) - Final_
↪val loss: {history['val_loss'][-1]:.4f}")

```

=====

Training Simple RNN with different learning rates

=====

Negatives: 489, Positives: 12571
pos_weight (applied to y=1 samples): 0.2000

--- Training Simple RNN with lr=2e-04 ---

Epoch 1/150 - Train Loss: 0.0983 - Val Loss: 0.0817 - Val F1: 0.6291
Epoch 5/150 - Train Loss: 0.0765 - Val Loss: 0.0750 - Val F1: 0.6487
Epoch 10/150 - Train Loss: 0.0696 - Val Loss: 0.0729 - Val F1: 0.6878
Epoch 15/150 - Train Loss: 0.0611 - Val Loss: 0.0704 - Val F1: 0.7049
Epoch 20/150 - Train Loss: 0.0628 - Val Loss: 0.0699 - Val F1: 0.6749
Epoch 25/150 - Train Loss: 0.0498 - Val Loss: 0.0532 - Val F1: 0.7454
Epoch 30/150 - Train Loss: 0.0450 - Val Loss: 0.0552 - Val F1: 0.7538
Epoch 35/150 - Train Loss: 0.0405 - Val Loss: 0.0621 - Val F1: 0.7829
Epoch 40/150 - Train Loss: 0.0385 - Val Loss: 0.0530 - Val F1: 0.8088
Epoch 45/150 - Train Loss: 0.0354 - Val Loss: 0.0427 - Val F1: 0.8025
Epoch 50/150 - Train Loss: 0.0332 - Val Loss: 0.0512 - Val F1: 0.7921
Epoch 54: LR reduced from 0.000200 to 0.000100
Epoch 55/150 - Train Loss: 0.0260 - Val Loss: 0.0459 - Val F1: 0.8312
Epoch 60/150 - Train Loss: 0.0245 - Val Loss: 0.0426 - Val F1: 0.8377
Epoch 65/150 - Train Loss: 0.0226 - Val Loss: 0.0495 - Val F1: 0.8298
Epoch 69: LR reduced from 0.000100 to 0.000050
Epoch 70/150 - Train Loss: 0.0214 - Val Loss: 0.0491 - Val F1: 0.8444
Epoch 75/150 - Train Loss: 0.0193 - Val Loss: 0.0506 - Val F1: 0.8316
Epoch 78: LR reduced from 0.000050 to 0.000025
Epoch 80/150 - Train Loss: 0.0190 - Val Loss: 0.0523 - Val F1: 0.8350
Epoch 85/150 - Train Loss: 0.0199 - Val Loss: 0.0550 - Val F1: 0.8458
Epoch 87: LR reduced from 0.000025 to 0.000013

Epoch 90/150 - Train Loss: 0.0195 - Val Loss: 0.0527 - Val F1: 0.8418
Epoch 95/150 - Train Loss: 0.0192 - Val Loss: 0.0517 - Val F1: 0.8547
Epoch 96: LR reduced from 0.000013 to 0.000006
Epoch 100/150 - Train Loss: 0.0182 - Val Loss: 0.0522 - Val F1: 0.8503
Epoch 105: LR reduced from 0.000006 to 0.000003
Epoch 105/150 - Train Loss: 0.0212 - Val Loss: 0.0521 - Val F1: 0.8405
Epoch 110/150 - Train Loss: 0.0177 - Val Loss: 0.0529 - Val F1: 0.8533
Early stopping at epoch 111 (best val macro F1 0.8601 at epoch 91, val loss 0.056651)
Completed. Training time: 00:05:19 (319.36s) - Final val loss: 0.0527

--- Training Simple RNN with lr=1e-04 ---

Epoch 1/150 - Train Loss: 0.1058 - Val Loss: 0.0852 - Val F1: 0.5940
Epoch 5/150 - Train Loss: 0.0810 - Val Loss: 0.0753 - Val F1: 0.6653
Epoch 10/150 - Train Loss: 0.0726 - Val Loss: 0.0793 - Val F1: 0.6406
Epoch 15/150 - Train Loss: 0.0670 - Val Loss: 0.0765 - Val F1: 0.6694
Epoch 20/150 - Train Loss: 0.0677 - Val Loss: 0.0718 - Val F1: 0.6267
Epoch 25/150 - Train Loss: 0.0580 - Val Loss: 0.0589 - Val F1: 0.7173
Epoch 30/150 - Train Loss: 0.0555 - Val Loss: 0.0648 - Val F1: 0.6916
Epoch 35/150 - Train Loss: 0.0510 - Val Loss: 0.0695 - Val F1: 0.7384
Epoch 40/150 - Train Loss: 0.0469 - Val Loss: 0.0686 - Val F1: 0.7579
Epoch 42: LR reduced from 0.000100 to 0.000050
Epoch 45/150 - Train Loss: 0.0436 - Val Loss: 0.0471 - Val F1: 0.7810
Epoch 50/150 - Train Loss: 0.0412 - Val Loss: 0.0507 - Val F1: 0.7969
Epoch 55/150 - Train Loss: 0.0400 - Val Loss: 0.0502 - Val F1: 0.7896
Epoch 60/150 - Train Loss: 0.0392 - Val Loss: 0.0445 - Val F1: 0.7840
Epoch 65/150 - Train Loss: 0.0368 - Val Loss: 0.0491 - Val F1: 0.7775
Epoch 70/150 - Train Loss: 0.0370 - Val Loss: 0.0447 - Val F1: 0.8049
Epoch 75/150 - Train Loss: 0.0343 - Val Loss: 0.0531 - Val F1: 0.7647
Epoch 77: LR reduced from 0.000050 to 0.000025
Epoch 80/150 - Train Loss: 0.0345 - Val Loss: 0.0535 - Val F1: 0.7711
Epoch 85/150 - Train Loss: 0.0331 - Val Loss: 0.0478 - Val F1: 0.8210
Epoch 86: LR reduced from 0.000025 to 0.000013
Epoch 90/150 - Train Loss: 0.0326 - Val Loss: 0.0499 - Val F1: 0.8108
Epoch 95: LR reduced from 0.000013 to 0.000006
Epoch 95/150 - Train Loss: 0.0329 - Val Loss: 0.0444 - Val F1: 0.8147
Epoch 100/150 - Train Loss: 0.0313 - Val Loss: 0.0505 - Val F1: 0.8027
Epoch 104: LR reduced from 0.000006 to 0.000003
Epoch 105/150 - Train Loss: 0.0346 - Val Loss: 0.0492 - Val F1: 0.8030
Early stopping at epoch 105 (best val macro F1 0.8210 at epoch 85, val loss 0.047803)
Completed. Training time: 00:04:54 (294.19s) - Final val loss: 0.0492

--- Training Simple RNN with lr=8e-05 ---

Epoch 1/150 - Train Loss: 0.1089 - Val Loss: 0.0868 - Val F1: 0.5497
Epoch 5/150 - Train Loss: 0.0822 - Val Loss: 0.0760 - Val F1: 0.6538
Epoch 10/150 - Train Loss: 0.0742 - Val Loss: 0.0750 - Val F1: 0.6487
Epoch 15/150 - Train Loss: 0.0685 - Val Loss: 0.0792 - Val F1: 0.6668

```

Epoch 20/150 - Train Loss: 0.0704 - Val Loss: 0.0908 - Val F1: 0.5840
Epoch 25/150 - Train Loss: 0.0610 - Val Loss: 0.0656 - Val F1: 0.6910
Epoch 30/150 - Train Loss: 0.0591 - Val Loss: 0.0694 - Val F1: 0.7025
Epoch 35/150 - Train Loss: 0.0546 - Val Loss: 0.0908 - Val F1: 0.7069
Epoch 40/150 - Train Loss: 0.0513 - Val Loss: 0.0754 - Val F1: 0.7233
Epoch 45/150 - Train Loss: 0.0496 - Val Loss: 0.0701 - Val F1: 0.7319
Epoch 50/150 - Train Loss: 0.0462 - Val Loss: 0.0582 - Val F1: 0.7542
Epoch 55/150 - Train Loss: 0.0447 - Val Loss: 0.0555 - Val F1: 0.7665
Epoch 60/150 - Train Loss: 0.0431 - Val Loss: 0.0554 - Val F1: 0.7871
Epoch 65/150 - Train Loss: 0.0397 - Val Loss: 0.0480 - Val F1: 0.7765
Epoch 66: LR reduced from 0.000080 to 0.000040
Epoch 70/150 - Train Loss: 0.0373 - Val Loss: 0.0484 - Val F1: 0.7940
Epoch 75: LR reduced from 0.000040 to 0.000020
Epoch 75/150 - Train Loss: 0.0340 - Val Loss: 0.0497 - Val F1: 0.7856
Epoch 80/150 - Train Loss: 0.0352 - Val Loss: 0.0514 - Val F1: 0.7866
Epoch 84: LR reduced from 0.000020 to 0.000010
Epoch 85/150 - Train Loss: 0.0334 - Val Loss: 0.0506 - Val F1: 0.8042
Early stopping at epoch 88 (best val macro F1 0.8094 at epoch 68, val loss
0.043261)
Completed. Training time: 00:04:11 (251.20s) - Final val loss: 0.0499

```

```

--- Training Simple RNN with lr=5e-05 ---

```

```

Epoch 1/150 - Train Loss: 0.1171 - Val Loss: 0.0901 - Val F1: 0.5415
Epoch 5/150 - Train Loss: 0.0846 - Val Loss: 0.0786 - Val F1: 0.6314
Epoch 10/150 - Train Loss: 0.0786 - Val Loss: 0.0743 - Val F1: 0.6755
Epoch 15/150 - Train Loss: 0.0724 - Val Loss: 0.0799 - Val F1: 0.6374
Epoch 20/150 - Train Loss: 0.0745 - Val Loss: 0.0876 - Val F1: 0.5789
Epoch 21: LR reduced from 0.000050 to 0.000025
Epoch 25/150 - Train Loss: 0.0669 - Val Loss: 0.0770 - Val F1: 0.6426
Epoch 30: LR reduced from 0.000025 to 0.000013
Epoch 30/150 - Train Loss: 0.0664 - Val Loss: 0.0857 - Val F1: 0.6916
Epoch 35/150 - Train Loss: 0.0651 - Val Loss: 0.0849 - Val F1: 0.6638
Epoch 39: LR reduced from 0.000013 to 0.000006
Epoch 40/150 - Train Loss: 0.0637 - Val Loss: 0.0829 - Val F1: 0.6639
Epoch 45/150 - Train Loss: 0.0635 - Val Loss: 0.0789 - Val F1: 0.6370
Epoch 48: LR reduced from 0.000006 to 0.000003
Epoch 50/150 - Train Loss: 0.0621 - Val Loss: 0.0834 - Val F1: 0.6606
Early stopping at epoch 50 (best val macro F1 0.6916 at epoch 30, val loss
0.085665)
Completed. Training time: 00:02:21 (141.55s) - Final val loss: 0.0834

```

```

[93]: # --- Plot loss curves ---

```

```

# Plot comparison

```

```

plot_lr_comparison(histories_simple, "Simple RNN", save_dir)

```

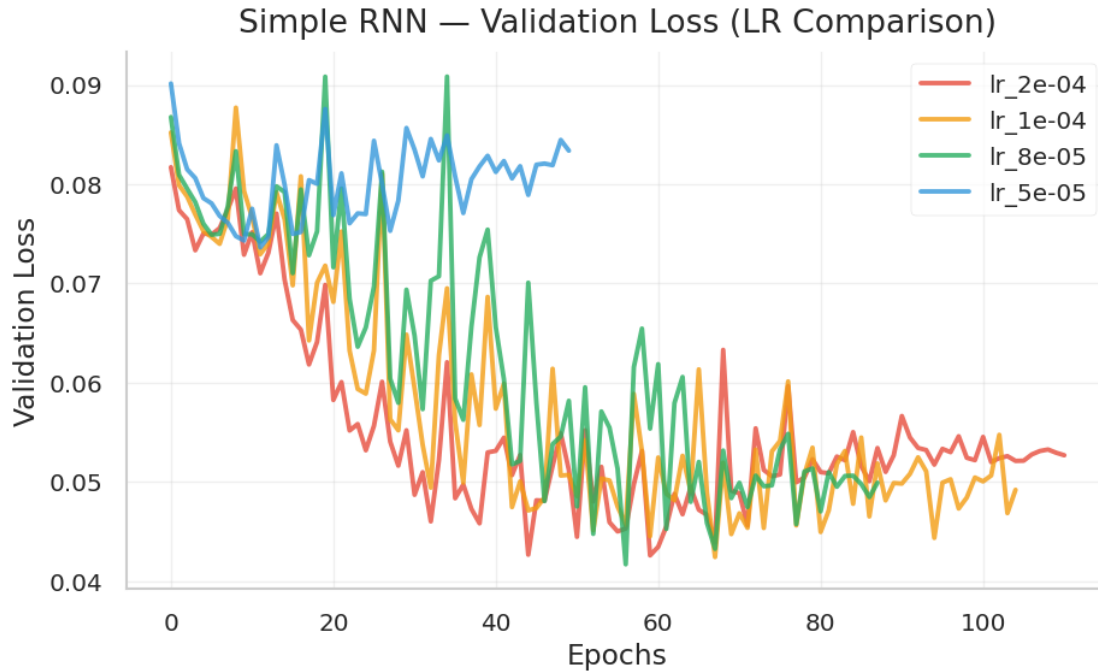
```

#Plot Train vs Validation

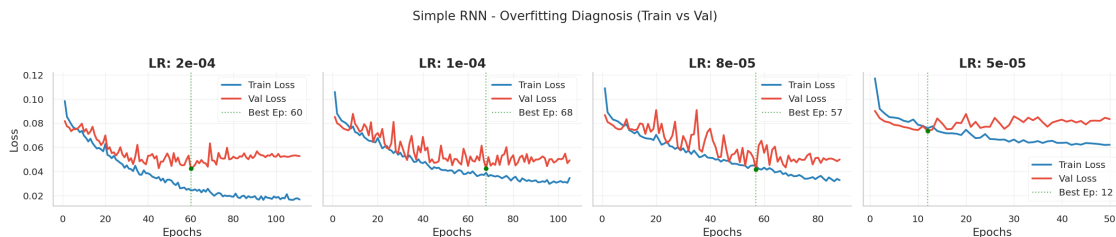
```

```
plot_train_val_grid(histories_simple, "Simple RNN", save_dir)
```

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/simple_rnn_lr_comparison.png

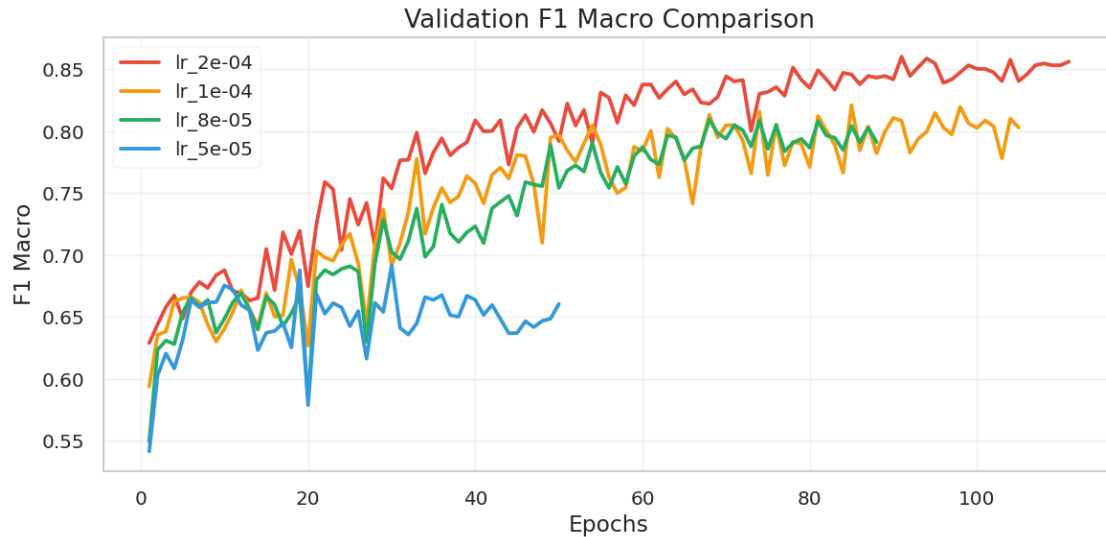


Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/simple_rnn_diagnosis.png



```
[94]: # --- Plot Validation F1 Macro for Simple RNN ---
plot_training_metrics(histories_simple, save_dir=save_dir,
    plot_name="simple_rnn_f1_comparison")
```

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/simple_rnn_f1_comparison.png



```
[95]: # --- Evaluate validation and test sets for ALL learning rates ---
print("\n" + "=" * 80)
print("Evaluation for ALL Simple RNN learning rates")
print("=" * 80)

for lr in learning_rates:
    model = models_simple[lr]

    print(f"\n{'='*60}")
    print(f"Simple RNN with lr={lr:.0e}")
    print(f"{'='*60}")

    # Validation evaluation
    y_val_true, y_val_pred = predict_loader(model, val_loader)
    print(f'\nValidation report (Simple RNN - lr={lr:.0e}):')
    print(classification_report(y_val_true, y_val_pred, digits=4,
    ↪zero_division=0))

    # Test evaluation
    y_test_true, y_test_pred = predict_loader(model, test_loader)
    print(f'Test report (Simple RNN - lr={lr:.0e}):')
    print(classification_report(y_test_true, y_test_pred, digits=4,
    ↪zero_division=0))

    # Select the restored model using validation metrics only
    best_simple_lr_value, simple_selection_table = select_best_rnn_model(
        'Simple RNN', models_simple, histories_simple, val_loader
    )
```

```

best_lr_simple = (best_simple_lr_value, histories_simple[best_simple_lr_value])
print(f"\n{'='*60}")
print(
    f"Best Simple RNN - LR: {best_lr_simple[0]:.0e} "
    f"(Val Macro F1: {simple_selection_table.loc[0, 'val_macro_f1']:.4f})"
)
print(f"{'='*60}")

```

```

=====
Evaluation for ALL Simple RNN learning rates
=====

```

```

=====
Simple RNN with lr=2e-04
=====

```

Validation report (Simple RNN - lr=2e-04):

	precision	recall	f1-score	support
0	0.7165	0.7459	0.7309	122
1	0.9901	0.9885	0.9893	3143
accuracy			0.9795	3265
macro avg	0.8533	0.8672	0.8601	3265
weighted avg	0.9799	0.9795	0.9797	3265

Test report (Simple RNN - lr=2e-04):

	precision	recall	f1-score	support
0	0.6626	0.6626	0.6626	243
1	0.9869	0.9869	0.9869	6262
accuracy			0.9748	6505
macro avg	0.8247	0.8247	0.8247	6505
weighted avg	0.9748	0.9748	0.9748	6505

```

=====
Simple RNN with lr=1e-04
=====

```

Validation report (Simple RNN - lr=1e-04):

	precision	recall	f1-score	support
0	0.6027	0.7213	0.6567	122
1	0.9891	0.9815	0.9853	3143

accuracy			0.9718	3265
macro avg	0.7959	0.8514	0.8210	3265
weighted avg	0.9747	0.9718	0.9730	3265

Test report (Simple RNN - lr=1e-04):

	precision	recall	f1-score	support
0	0.5693	0.6420	0.6035	243
1	0.9860	0.9812	0.9836	6262

accuracy			0.9685	6505
macro avg	0.7777	0.8116	0.7935	6505
weighted avg	0.9705	0.9685	0.9694	6505

=====

Simple RNN with lr=8e-05

=====

Validation report (Simple RNN - lr=8e-05):

	precision	recall	f1-score	support
0	0.5772	0.7049	0.6347	122
1	0.9884	0.9800	0.9842	3143

accuracy			0.9697	3265
macro avg	0.7828	0.8424	0.8094	3265
weighted avg	0.9731	0.9697	0.9711	3265

Test report (Simple RNN - lr=8e-05):

	precision	recall	f1-score	support
0	0.6270	0.6296	0.6283	243
1	0.9856	0.9855	0.9855	6262

accuracy			0.9722	6505
macro avg	0.8063	0.8075	0.8069	6505
weighted avg	0.9722	0.9722	0.9722	6505

=====

Simple RNN with lr=5e-05

=====

Validation report (Simple RNN - lr=5e-05):

	precision	recall	f1-score	support
0	0.4273	0.3852	0.4052	122

1	0.9762	0.9800	0.9781	3143
accuracy			0.9577	3265
macro avg	0.7018	0.6826	0.6916	3265
weighted avg	0.9557	0.9577	0.9567	3265

Test report (Simple RNN - lr=5e-05):

	precision	recall	f1-score	support
0	0.4906	0.3210	0.3881	243
1	0.9740	0.9871	0.9805	6262
accuracy			0.9622	6505
macro avg	0.7323	0.6540	0.6843	6505
weighted avg	0.9559	0.9622	0.9584	6505

Automatic selection for Simple RNN

Ranking criterion: validation macro F1, then validation accuracy, then
checkpoint validation loss

	learning_rate	val_macro_f1	val_accuracy	checkpoint_val_loss
checkpoint_epoch				
91	0.00020	0.860128	0.979479	0.056651
85	0.00010	0.821012	0.971822	0.047803
68	0.00008	0.809435	0.969678	0.043261
30	0.00005	0.691630	0.957734	0.085665

Selected Simple RNN: lr=2e-04, validation macro F1=0.8601, checkpoint epoch=91

=====
Best Simple RNN - LR: 2e-04 (Val Macro F1: 0.8601)
=====

```
[96]: # --- Plot confusion matrices for ALL learning rates - VALIDATION ---

print("\n" + "=" * 80)
print("Confusion matrices for ALL Simple RNN learning rates - VALIDATION")
print("=" * 80)

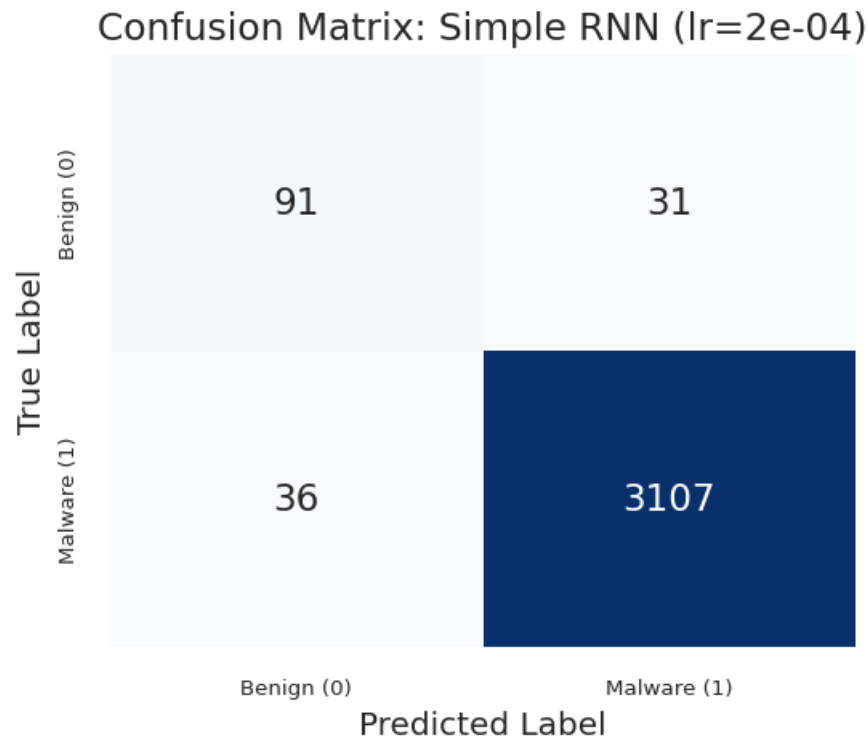
for lr in learning_rates:
    model = models_simple[lr]
    print(f"\nGenerating confusion matrix for Simple RNN with lr={lr:.0e}...")
```

```
plot_cm_task3(model=model, loader=val_loader, model_name=f"Simple RNN_
↳(lr={lr:.0e})", save_dir=save_dir)
```

=====
 Confusion matrices for ALL Simple RNN learning rates - VALIDATION
 =====

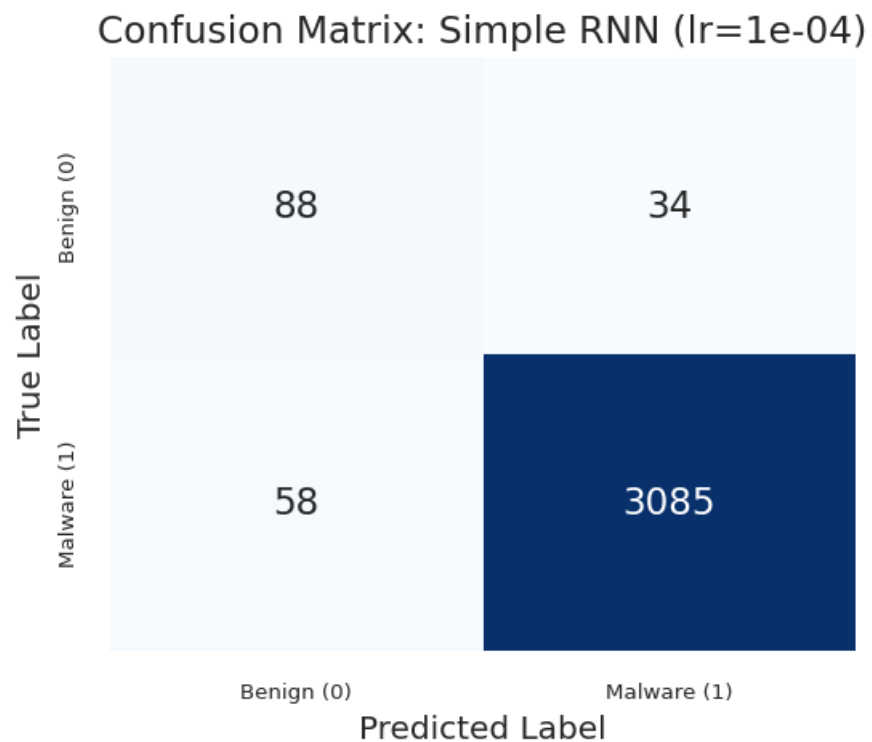
Generating confusion matrix for Simple RNN with lr=2e-04...

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
 /task3_plots/cm_simple_rnn_(lr=2e_04).png



Generating confusion matrix for Simple RNN with lr=1e-04...

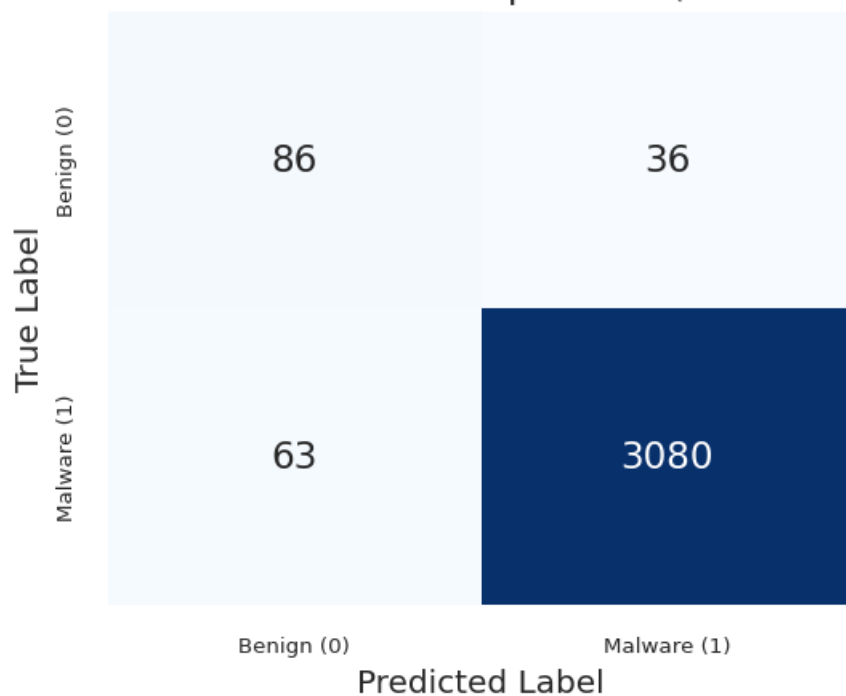
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
 /task3_plots/cm_simple_rnn_(lr=1e_04).png



Generating confusion matrix for Simple RNN with lr=8e-05...

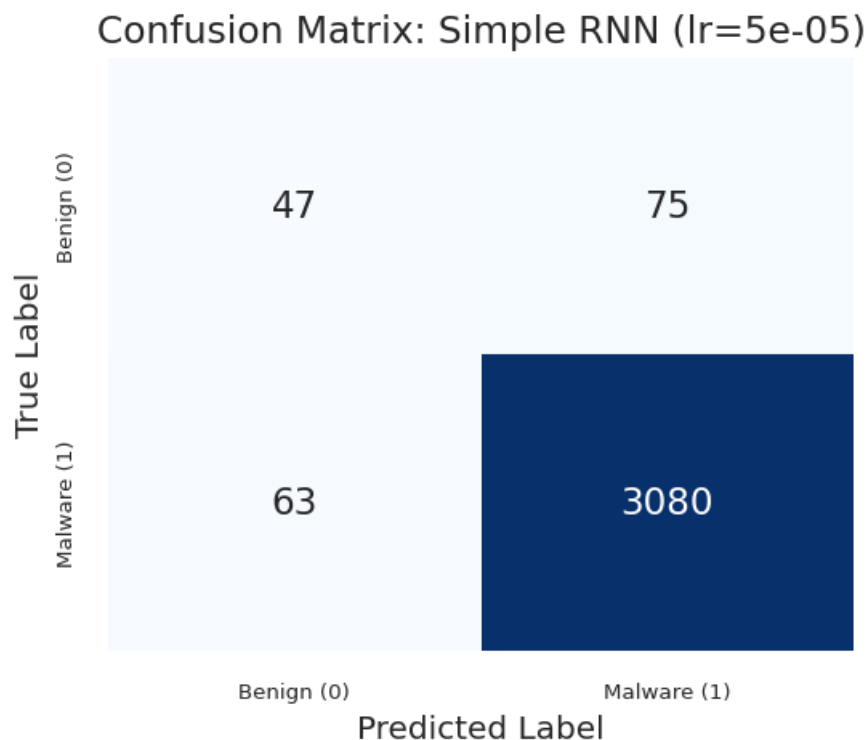
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/cm_simple_rnn_(lr=8e_05).png

Confusion Matrix: Simple RNN (lr=8e-05)



Generating confusion matrix for Simple RNN with lr=5e-05...

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/cm_simple_rnn_(lr=5e_05).png



```
[97]: # --- Plot confusion matrices for ALL learning rates - TEST ---

print("\n" + "=" * 80)
print("Confusion matrices for ALL Simple RNN learning rates - TEST")
print("=" * 80)

for lr in learning_rates:
    model = models_simple[lr]
    print(f"\nGenerating confusion matrix for Simple RNN with lr={lr:.0e}...")

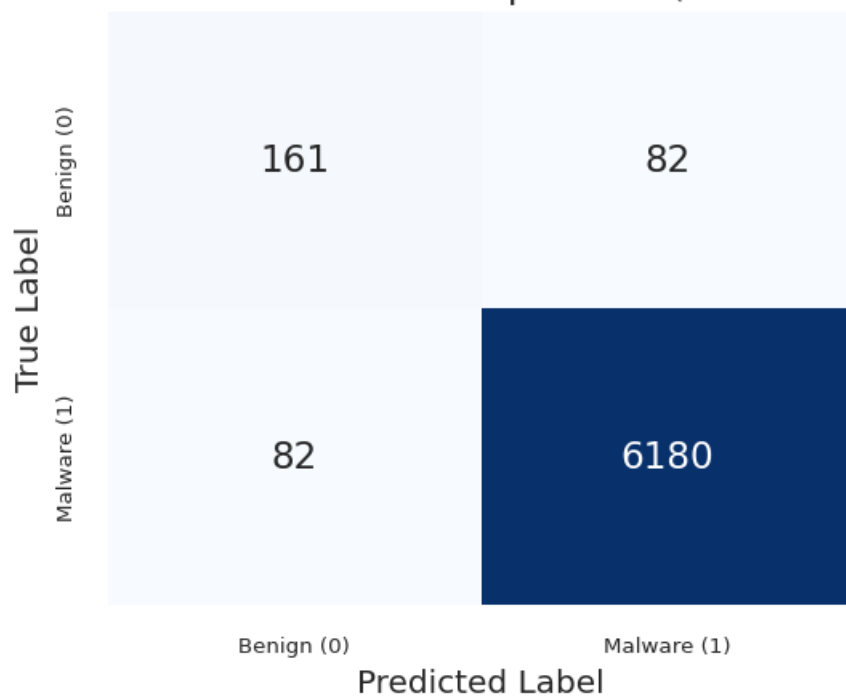
    plot_cm_task3(model=model, loader=test_loader, model_name=f"Simple RNN_
↵(lr={lr:.0e})", save_dir=save_dir)
```

```
=====
Confusion matrices for ALL Simple RNN learning rates - TEST
=====
```

```
Generating confusion matrix for Simple RNN with lr=2e-04...
```

```
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task3_plots/cm_simple_rnn_(lr=2e_04).png
```

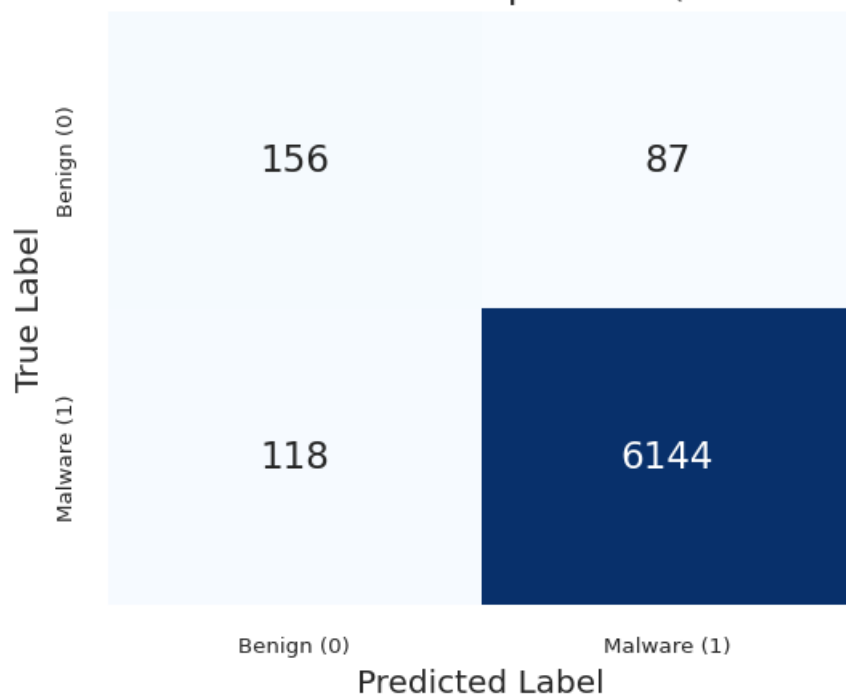
Confusion Matrix: Simple RNN (lr=2e-04)



Generating confusion matrix for Simple RNN with lr=1e-04...

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/cm_simple_rnn_(lr=1e_04).png

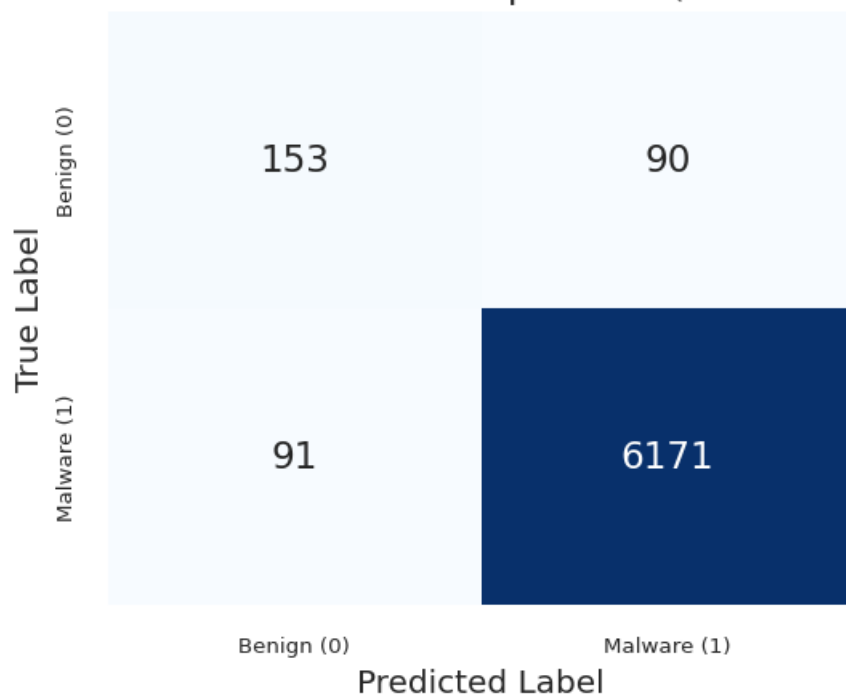
Confusion Matrix: Simple RNN (lr=1e-04)



Generating confusion matrix for Simple RNN with lr=8e-05...

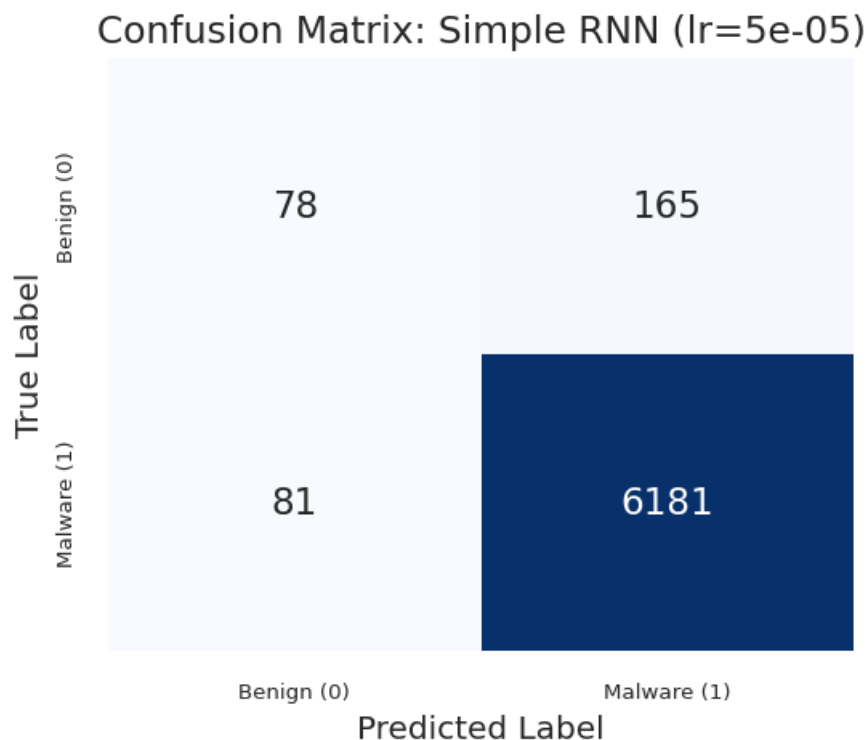
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/cm_simple_rnn_(lr=8e_05).png

Confusion Matrix: Simple RNN (lr=8e-05)



Generating confusion matrix for Simple RNN with lr=5e-05...

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/cm_simple_rnn_(lr=5e_05).png



Q: Start with a simple one-directional RNN. Is your network as fast as the FFNN? If not, why? No. The simple one-directional RNN is clearly slower than the FFNN models. In this run, the selected FFNN with sequential IDs trained in about 32 seconds, the selected FFNN with learnable embeddings in about 33 seconds, while the selected Simple RNN required about 5 min 19 s.

The main reason is the sequential dependency across time steps. An FFNN processes the whole fixed-length input vector through dense matrix operations, which are highly parallelizable. An RNN instead updates its hidden state step by step, and each step depends on the previous hidden state. This limits parallelization across the temporal dimension.

Training is also more expensive because backpropagation through time must propagate gradients through the entire sequence, and the data loader performs sorting, dynamic padding, and packed-sequence handling. Even though packing avoids wasted computation on padding tokens, the recurrent structure remains substantially slower than the FFNN.

1.4.2 Network Variations

```
[98]: # --- Variation 2: Bidirectional RNN ---

print("\n" + "=" * 80)
print("Training Bidirectional RNN with different learning rates")
print("=" * 80)
```

```

# Shared protocol for the bidirectional vanilla RNN and bidirectional LSTM
COMMON_RNN_LEARNING_RATES = [5e-5, 8e-5, 1e-4, 1.5e-4, 2e-4, 3e-4]
COMMON_RNN_DROPOUT = 0.3
COMMON_RNN_WEIGHT_DECAY = 1e-4
COMMON_RNN_PATIENCE = 25
COMMON_RNN_SEED = 42
COMMON_RNN_SCHEDULER = {
    'mode': 'min',
    'factor': 0.5,
    'patience': 7,
    'min_lr': 1e-7,
}

learning_rates_bi = COMMON_RNN_LEARNING_RATES

# Model parameters
bi_rnn_params = {
    'vocab_size': vocab_size_rnn,
    'embed_dim': 32,
    'hidden_size': 128,
    'num_layers': 2,
    'num_classes': 1,
    'bidirectional': True,
    'dropout': COMMON_RNN_DROPOUT
}

# Initialize early stopping parameters
min_delta = 0.00001
patience = COMMON_RNN_PATIENCE

# Scheduler parameters
scheduler_params_bi = COMMON_RNN_SCHEDULER.copy()

# Store histories and models for each LR
histories_bi = {}
models_bi = {}
elapsed_bi = {}

for lr in learning_rates_bi:
    print(f"\n--- Training Bi-RNN with lr={lr:.0e} ---")
    model, history, elapsed = train_with_lr(
        model_class=SimpleRNNClassifier,
        model_params=bi_rnn_params,
        initial_lr=lr,
        train_loader=train_loader,
        val_loader=val_loader,
        criterion=nn.BCEWithLogitsLoss(pos_weight=pos_weight),

```

```

        num_epochs=num_epochs,
        min_delta=min_delta,
        patience=patience,
        scheduler_params=scheduler_params_bi,
        device=device,
        weight_decay=COMMON_RNN_WEIGHT_DECAY,
        seed=COMMON_RNN_SEED,
    )
    histories_bi[lr] = history
    models_bi[lr] = model
    elapsed_bi[lr] = elapsed
    elapsed_str = time.strftime('%H:%M:%S', time.gmtime(elapsed))
    print(f"Completed. Training time: {elapsed_str} ({elapsed:.2f}s) - Final_
↪val loss: {history['val_loss'][-1]:.4f}")

```

=====
Training Bidirectional RNN with different learning rates
=====

--- Training Bi-RNN with lr=5e-05 ---

```

Epoch 1/150 - Train Loss: 0.1073 - Val Loss: 0.0906 - Val F1: 0.4904
Epoch 5/150 - Train Loss: 0.0820 - Val Loss: 0.0749 - Val F1: 0.6583
Epoch 10/150 - Train Loss: 0.0728 - Val Loss: 0.0817 - Val F1: 0.6138
Epoch 14: LR reduced from 0.000050 to 0.000025
Epoch 15/150 - Train Loss: 0.0687 - Val Loss: 0.0796 - Val F1: 0.6381
Epoch 20/150 - Train Loss: 0.0701 - Val Loss: 0.0814 - Val F1: 0.6108
Epoch 25/150 - Train Loss: 0.0652 - Val Loss: 0.0728 - Val F1: 0.6331
Epoch 30/150 - Train Loss: 0.0636 - Val Loss: 0.0749 - Val F1: 0.7016
Epoch 35/150 - Train Loss: 0.0625 - Val Loss: 0.0784 - Val F1: 0.6625
Epoch 36: LR reduced from 0.000025 to 0.000013
Epoch 40/150 - Train Loss: 0.0591 - Val Loss: 0.0749 - Val F1: 0.6624
Epoch 44: LR reduced from 0.000013 to 0.000006
Epoch 45/150 - Train Loss: 0.0596 - Val Loss: 0.0740 - Val F1: 0.6294
Epoch 50/150 - Train Loss: 0.0578 - Val Loss: 0.0769 - Val F1: 0.6537
Epoch 52: LR reduced from 0.000006 to 0.000003
Epoch 55/150 - Train Loss: 0.0563 - Val Loss: 0.0746 - Val F1: 0.6420
Early stopping at epoch 55 (best val macro F1 0.7016 at epoch 30, val loss
0.074918)
Completed. Training time: 00:03:49 (229.17s) - Final val loss: 0.0746

```

--- Training Bi-RNN with lr=8e-05 ---

```

Epoch 1/150 - Train Loss: 0.1020 - Val Loss: 0.0858 - Val F1: 0.5269
Epoch 5/150 - Train Loss: 0.0782 - Val Loss: 0.0775 - Val F1: 0.6205
Epoch 10/150 - Train Loss: 0.0704 - Val Loss: 0.0793 - Val F1: 0.6363
Epoch 15/150 - Train Loss: 0.0645 - Val Loss: 0.0786 - Val F1: 0.6517
Epoch 20/150 - Train Loss: 0.0647 - Val Loss: 0.0782 - Val F1: 0.6001
Epoch 25/150 - Train Loss: 0.0584 - Val Loss: 0.0616 - Val F1: 0.6969

```

Epoch 30/150 - Train Loss: 0.0540 - Val Loss: 0.0624 - Val F1: 0.7109
 Epoch 35/150 - Train Loss: 0.0525 - Val Loss: 0.0622 - Val F1: 0.7532
 Epoch 36: LR reduced from 0.000080 to 0.000040
 Epoch 40/150 - Train Loss: 0.0454 - Val Loss: 0.0599 - Val F1: 0.7453
 Epoch 45/150 - Train Loss: 0.0443 - Val Loss: 0.0612 - Val F1: 0.7506
 Epoch 50/150 - Train Loss: 0.0421 - Val Loss: 0.0534 - Val F1: 0.7660
 Epoch 55: LR reduced from 0.000040 to 0.000020
 Epoch 55/150 - Train Loss: 0.0404 - Val Loss: 0.0574 - Val F1: 0.7711
 Epoch 60/150 - Train Loss: 0.0361 - Val Loss: 0.0540 - Val F1: 0.7461
 Epoch 63: LR reduced from 0.000020 to 0.000010
 Epoch 65/150 - Train Loss: 0.0368 - Val Loss: 0.0551 - Val F1: 0.7491
 Early stopping at epoch 69 (best val macro F1 0.7817 at epoch 44, val loss 0.053858)
 Completed. Training time: 00:04:43 (283.64s) - Final val loss: 0.0540

--- Training Bi-RNN with lr=1e-04 ---

Epoch 1/150 - Train Loss: 0.0998 - Val Loss: 0.0836 - Val F1: 0.5870
 Epoch 5/150 - Train Loss: 0.0772 - Val Loss: 0.0780 - Val F1: 0.6198
 Epoch 10/150 - Train Loss: 0.0689 - Val Loss: 0.0744 - Val F1: 0.6475
 Epoch 15/150 - Train Loss: 0.0629 - Val Loss: 0.0743 - Val F1: 0.6661
 Epoch 20/150 - Train Loss: 0.0611 - Val Loss: 0.0750 - Val F1: 0.6191
 Epoch 25/150 - Train Loss: 0.0566 - Val Loss: 0.0582 - Val F1: 0.7078
 Epoch 30/150 - Train Loss: 0.0519 - Val Loss: 0.0566 - Val F1: 0.7546
 Epoch 35/150 - Train Loss: 0.0475 - Val Loss: 0.0604 - Val F1: 0.7561
 Epoch 36: LR reduced from 0.000100 to 0.000050
 Epoch 40/150 - Train Loss: 0.0410 - Val Loss: 0.0608 - Val F1: 0.7652
 Epoch 45/150 - Train Loss: 0.0405 - Val Loss: 0.0589 - Val F1: 0.7581
 Epoch 50/150 - Train Loss: 0.0389 - Val Loss: 0.0566 - Val F1: 0.7653
 Epoch 55/150 - Train Loss: 0.0358 - Val Loss: 0.0614 - Val F1: 0.7723
 Epoch 59: LR reduced from 0.000050 to 0.000025
 Epoch 60/150 - Train Loss: 0.0312 - Val Loss: 0.0523 - Val F1: 0.7659
 Epoch 65/150 - Train Loss: 0.0324 - Val Loss: 0.0548 - Val F1: 0.7654
 Epoch 67: LR reduced from 0.000025 to 0.000013
 Early stopping at epoch 69 (best val macro F1 0.7939 at epoch 44, val loss 0.052390)
 Completed. Training time: 00:04:45 (285.20s) - Final val loss: 0.0568

--- Training Bi-RNN with lr=1e-04 ---

Epoch 1/150 - Train Loss: 0.0964 - Val Loss: 0.0809 - Val F1: 0.6190
 Epoch 5/150 - Train Loss: 0.0756 - Val Loss: 0.0734 - Val F1: 0.6388
 Epoch 10/150 - Train Loss: 0.0661 - Val Loss: 0.0691 - Val F1: 0.6615
 Epoch 15/150 - Train Loss: 0.0592 - Val Loss: 0.0746 - Val F1: 0.6871
 Epoch 20/150 - Train Loss: 0.0561 - Val Loss: 0.0671 - Val F1: 0.6405
 Epoch 25/150 - Train Loss: 0.0507 - Val Loss: 0.0532 - Val F1: 0.7503
 Epoch 30/150 - Train Loss: 0.0457 - Val Loss: 0.0528 - Val F1: 0.7579
 Epoch 35/150 - Train Loss: 0.0395 - Val Loss: 0.0563 - Val F1: 0.7689
 Epoch 36: LR reduced from 0.000150 to 0.000075
 Epoch 40/150 - Train Loss: 0.0335 - Val Loss: 0.0633 - Val F1: 0.7745

Epoch 45/150 - Train Loss: 0.0321 - Val Loss: 0.0602 - Val F1: 0.7771
 Epoch 50/150 - Train Loss: 0.0301 - Val Loss: 0.0566 - Val F1: 0.7711
 Epoch 55/150 - Train Loss: 0.0266 - Val Loss: 0.0546 - Val F1: 0.7718
 Epoch 60/150 - Train Loss: 0.0245 - Val Loss: 0.0542 - Val F1: 0.7821
 Epoch 65/150 - Train Loss: 0.0235 - Val Loss: 0.0503 - Val F1: 0.7964
 Epoch 69: LR reduced from 0.000075 to 0.000037
 Epoch 70/150 - Train Loss: 0.0217 - Val Loss: 0.0484 - Val F1: 0.8110
 Epoch 75/150 - Train Loss: 0.0202 - Val Loss: 0.0519 - Val F1: 0.7959
 Epoch 77: LR reduced from 0.000037 to 0.000019
 Epoch 80/150 - Train Loss: 0.0195 - Val Loss: 0.0512 - Val F1: 0.8062
 Epoch 85/150 - Train Loss: 0.0193 - Val Loss: 0.0523 - Val F1: 0.8157
 Epoch 90: LR reduced from 0.000019 to 0.000009
 Epoch 90/150 - Train Loss: 0.0177 - Val Loss: 0.0613 - Val F1: 0.8077
 Epoch 95/150 - Train Loss: 0.0185 - Val Loss: 0.0498 - Val F1: 0.8222
 Epoch 98: LR reduced from 0.000009 to 0.000005
 Epoch 100/150 - Train Loss: 0.0166 - Val Loss: 0.0508 - Val F1: 0.8073
 Epoch 105/150 - Train Loss: 0.0196 - Val Loss: 0.0499 - Val F1: 0.8159
 Epoch 106: LR reduced from 0.000005 to 0.000002
 Epoch 110/150 - Train Loss: 0.0193 - Val Loss: 0.0511 - Val F1: 0.8109
 Epoch 114: LR reduced from 0.000002 to 0.000001
 Epoch 115/150 - Train Loss: 0.0171 - Val Loss: 0.0511 - Val F1: 0.8085
 Epoch 120/150 - Train Loss: 0.0171 - Val Loss: 0.0516 - Val F1: 0.8121
 Early stopping at epoch 120 (best val macro F1 0.8222 at epoch 95, val loss 0.049792)
 Completed. Training time: 00:08:09 (489.39s) - Final val loss: 0.0516

--- Training Bi-RNN with lr=2e-04 ---

Epoch 1/150 - Train Loss: 0.0947 - Val Loss: 0.0802 - Val F1: 0.6205
 Epoch 5/150 - Train Loss: 0.0743 - Val Loss: 0.0729 - Val F1: 0.6626
 Epoch 10/150 - Train Loss: 0.0641 - Val Loss: 0.0694 - Val F1: 0.6572
 Epoch 15/150 - Train Loss: 0.0572 - Val Loss: 0.0705 - Val F1: 0.6915
 Epoch 16: LR reduced from 0.000200 to 0.000100
 Epoch 20/150 - Train Loss: 0.0510 - Val Loss: 0.0688 - Val F1: 0.6473
 Epoch 25/150 - Train Loss: 0.0461 - Val Loss: 0.0563 - Val F1: 0.7383
 Epoch 30/150 - Train Loss: 0.0428 - Val Loss: 0.0515 - Val F1: 0.7711
 Epoch 35/150 - Train Loss: 0.0393 - Val Loss: 0.0609 - Val F1: 0.7784
 Epoch 36: LR reduced from 0.000100 to 0.000050
 Epoch 40/150 - Train Loss: 0.0345 - Val Loss: 0.0634 - Val F1: 0.7705
 Epoch 45/150 - Train Loss: 0.0329 - Val Loss: 0.0525 - Val F1: 0.7741
 Epoch 46: LR reduced from 0.000050 to 0.000025
 Epoch 50/150 - Train Loss: 0.0308 - Val Loss: 0.0507 - Val F1: 0.7832
 Epoch 55/150 - Train Loss: 0.0294 - Val Loss: 0.0539 - Val F1: 0.7719
 Early stopping at epoch 59 (best val macro F1 0.7893 at epoch 34, val loss 0.054008)
 Completed. Training time: 00:03:58 (238.96s) - Final val loss: 0.0525

--- Training Bi-RNN with lr=3e-04 ---

Epoch 1/150 - Train Loss: 0.0933 - Val Loss: 0.0792 - Val F1: 0.6372

Epoch 5/150 - Train Loss: 0.0731 - Val Loss: 0.0725 - Val F1: 0.6576
 Epoch 10/150 - Train Loss: 0.0643 - Val Loss: 0.0710 - Val F1: 0.6607
 Epoch 15/150 - Train Loss: 0.0545 - Val Loss: 0.0708 - Val F1: 0.7082
 Epoch 20/150 - Train Loss: 0.0481 - Val Loss: 0.0621 - Val F1: 0.6877
 Epoch 25/150 - Train Loss: 0.0425 - Val Loss: 0.0562 - Val F1: 0.7535
 Epoch 30/150 - Train Loss: 0.0343 - Val Loss: 0.0560 - Val F1: 0.7415
 Epoch 35/150 - Train Loss: 0.0283 - Val Loss: 0.0600 - Val F1: 0.7606
 Epoch 40/150 - Train Loss: 0.0261 - Val Loss: 0.0686 - Val F1: 0.7683
 Epoch 44: LR reduced from 0.000300 to 0.000150
 Epoch 45/150 - Train Loss: 0.0190 - Val Loss: 0.0581 - Val F1: 0.8017
 Epoch 50/150 - Train Loss: 0.0156 - Val Loss: 0.0734 - Val F1: 0.7829
 Epoch 54: LR reduced from 0.000150 to 0.000075
 Epoch 55/150 - Train Loss: 0.0125 - Val Loss: 0.0720 - Val F1: 0.7967
 Epoch 60/150 - Train Loss: 0.0113 - Val Loss: 0.0661 - Val F1: 0.8119
 Epoch 62: LR reduced from 0.000075 to 0.000037
 Epoch 65/150 - Train Loss: 0.0108 - Val Loss: 0.0760 - Val F1: 0.8012
 Epoch 70: LR reduced from 0.000037 to 0.000019
 Epoch 70/150 - Train Loss: 0.0104 - Val Loss: 0.0754 - Val F1: 0.8142
 Epoch 75/150 - Train Loss: 0.0094 - Val Loss: 0.0684 - Val F1: 0.8198
 Epoch 78: LR reduced from 0.000019 to 0.000009
 Epoch 80/150 - Train Loss: 0.0096 - Val Loss: 0.0721 - Val F1: 0.8170
 Epoch 85/150 - Train Loss: 0.0097 - Val Loss: 0.0687 - Val F1: 0.8170
 Epoch 86: LR reduced from 0.000009 to 0.000005
 Epoch 90/150 - Train Loss: 0.0089 - Val Loss: 0.0719 - Val F1: 0.8085
 Epoch 94: LR reduced from 0.000005 to 0.000002
 Epoch 95/150 - Train Loss: 0.0089 - Val Loss: 0.0735 - Val F1: 0.8114
 Epoch 100/150 - Train Loss: 0.0082 - Val Loss: 0.0730 - Val F1: 0.8114
 Early stopping at epoch 100 (best val macro F1 0.8198 at epoch 75, val loss 0.068449)
 Completed. Training time: 00:06:50 (410.33s) - Final val loss: 0.0730

[99]: `# --- Plot loss curves ---`

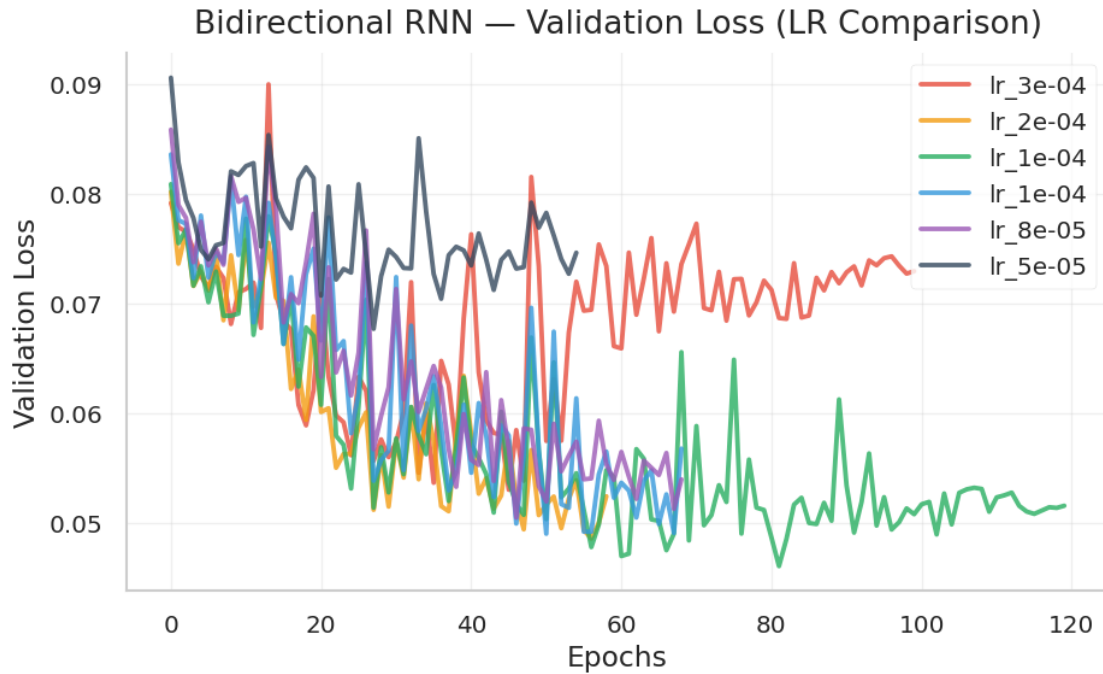
```

# Plot comparison
plot_lr_comparison(histories_bi, "Bidirectional RNN", save_dir)

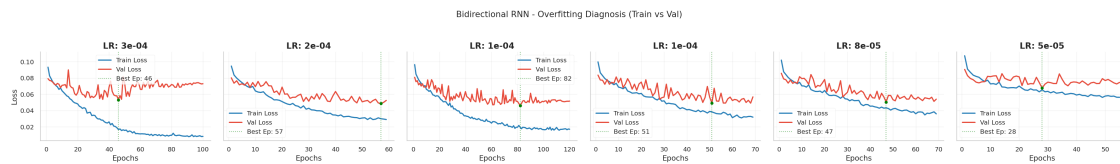
#Plot Train vs Validation
plot_train_val_grid(histories_bi, "Bidirectional RNN", save_dir)

```

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
 /task3_plots/bidirectional_rnn_lr_comparison.png

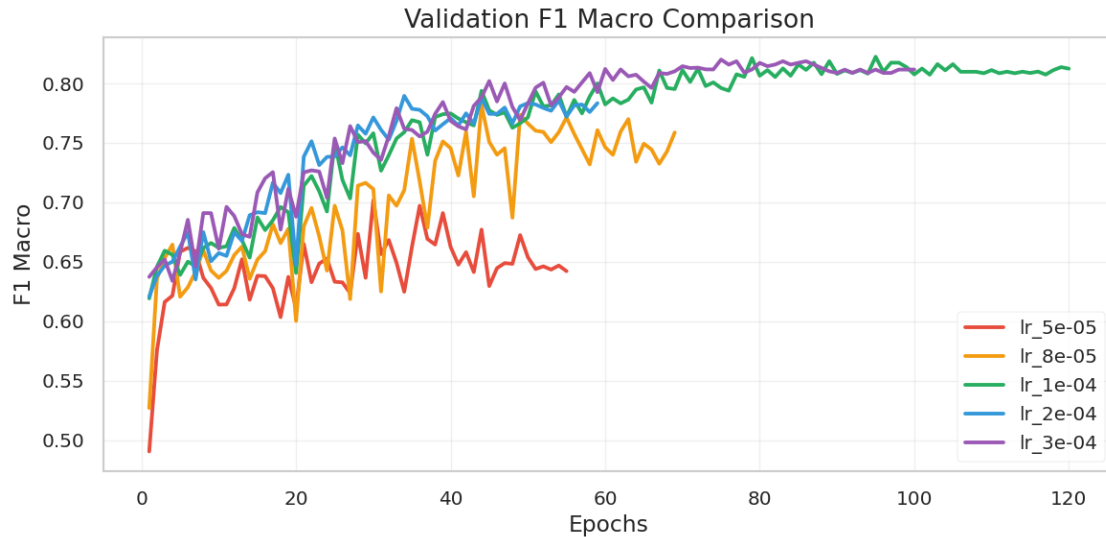


Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/bidirectional_rnn_diagnosis.png



```
[100]: # --- Plot Validation F1 Macro for Bidirectional RNN ---
plot_training_metrics(histories_bi, save_dir=save_dir,
plot_name="bidirectional_rnn_f1_comparison")
```

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/bidirectional_rnn_f1_comparison.png



```
[101]: # --- Evaluate validation and test sets for ALL learning rates ---

print("\n" + "=" * 80)
print("Evaluation for ALL Bi-RNN learning rates")
print("=" * 80)

for lr in learning_rates_bi:
    model = models_bi[lr]

    print(f"\n{'='*60}")
    print(f"Bi-RNN with lr={lr:.0e}")
    print(f"{'='*60}")

    # Validation evaluation
    y_val_true, y_val_pred = predict_loader(model, val_loader)
    print(f'\nValidation report (Bi-RNN - lr={lr:.0e}):')
    print(classification_report(y_val_true, y_val_pred, digits=4,
    ↪zero_division=0))

    # Test evaluation
    y_test_true, y_test_pred = predict_loader(model, test_loader)
    print(f'\nTest report (Bi-RNN - lr={lr:.0e}):')
    print(classification_report(y_test_true, y_test_pred, digits=4,
    ↪zero_division=0))

# Select the restored model using validation metrics only
best_bi_lr_value, bi_selection_table = select_best_rnn_model(
    'Bidirectional RNN', models_bi, histories_bi, val_loader
```

```

)
best_lr_bi = (best_bi_lr_value, histories_bi[best_bi_lr_value])
print(f"\n{'='*60}")
print(
    f"Best Bi-RNN - LR: {best_lr_bi[0]:.0e} "
    f"(Val Macro F1: {bi_selection_table.loc[0, 'val_macro_f1']:.4f})"
)
print(f"{'='*60}")

```

```

=====
Evaluation for ALL Bi-RNN learning rates
=====

```

```

=====
Bi-RNN with lr=5e-05
=====

```

Validation report (Bi-RNN - lr=5e-05):

	precision	recall	f1-score	support
0	0.4700	0.3852	0.4234	122
1	0.9763	0.9831	0.9797	3143
accuracy			0.9608	3265
macro avg	0.7232	0.6842	0.7016	3265
weighted avg	0.9574	0.9608	0.9589	3265

Test report (Bi-RNN - lr=5e-05):

	precision	recall	f1-score	support
0	0.4467	0.3621	0.4000	243
1	0.9754	0.9826	0.9790	6262
accuracy			0.9594	6505
macro avg	0.7111	0.6724	0.6895	6505
weighted avg	0.9557	0.9594	0.9574	6505

```

=====
Bi-RNN with lr=8e-05
=====

```

Validation report (Bi-RNN - lr=8e-05):

	precision	recall	f1-score	support
0	0.5564	0.6066	0.5804	122
1	0.9847	0.9812	0.9829	3143

accuracy			0.9672	3265
macro avg	0.7705	0.7939	0.7817	3265
weighted avg	0.9687	0.9672	0.9679	3265

Test report (Bi-RNN - lr=8e-05):

	precision	recall	f1-score	support
0	0.5378	0.5556	0.5466	243
1	0.9827	0.9815	0.9821	6262

accuracy			0.9656	6505
macro avg	0.7603	0.7685	0.7643	6505
weighted avg	0.9661	0.9656	0.9658	6505

=====

Bi-RNN with lr=1e-04

=====

Validation report (Bi-RNN - lr=1e-04):

	precision	recall	f1-score	support
0	0.5789	0.6311	0.6039	122
1	0.9856	0.9822	0.9839	3143

accuracy			0.9691	3265
macro avg	0.7823	0.8067	0.7939	3265
weighted avg	0.9704	0.9691	0.9697	3265

Test report (Bi-RNN - lr=1e-04):

	precision	recall	f1-score	support
0	0.5500	0.5885	0.5686	243
1	0.9840	0.9813	0.9826	6262

accuracy			0.9666	6505
macro avg	0.7670	0.7849	0.7756	6505
weighted avg	0.9678	0.9666	0.9672	6505

=====

Bi-RNN with lr=1e-04

=====

Validation report (Bi-RNN - lr=1e-04):

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

0	0.5860	0.7541	0.6595	122
1	0.9903	0.9793	0.9848	3143
accuracy			0.9709	3265
macro avg	0.7882	0.8667	0.8222	3265
weighted avg	0.9752	0.9709	0.9726	3265

Test report (Bi-RNN - lr=1e-04):

	precision	recall	f1-score	support
0	0.5734	0.6749	0.6200	243
1	0.9873	0.9805	0.9839	6262
accuracy			0.9691	6505
macro avg	0.7804	0.8277	0.8020	6505
weighted avg	0.9718	0.9691	0.9703	6505

=====

Bi-RNN with lr=2e-04

=====

Validation report (Bi-RNN - lr=2e-04):

	precision	recall	f1-score	support
0	0.5571	0.6393	0.5954	122
1	0.9859	0.9803	0.9831	3143
accuracy			0.9675	3265
macro avg	0.7715	0.8098	0.7893	3265
weighted avg	0.9699	0.9675	0.9686	3265

Test report (Bi-RNN - lr=2e-04):

	precision	recall	f1-score	support
0	0.4856	0.5556	0.5182	243
1	0.9827	0.9772	0.9799	6262
accuracy			0.9614	6505
macro avg	0.7341	0.7664	0.7491	6505
weighted avg	0.9641	0.9614	0.9627	6505

=====

Bi-RNN with lr=3e-04

=====

Validation report (Bi-RNN - lr=3e-04):

	precision	recall	f1-score	support
0	0.6357	0.6721	0.6534	122
1	0.9872	0.9850	0.9861	3143
accuracy			0.9734	3265
macro avg	0.8115	0.8286	0.8198	3265
weighted avg	0.9741	0.9734	0.9737	3265

Test report (Bi-RNN - lr=3e-04):

	precision	recall	f1-score	support
0	0.6078	0.5802	0.5937	243
1	0.9837	0.9855	0.9846	6262
accuracy			0.9703	6505
macro avg	0.7957	0.7829	0.7891	6505
weighted avg	0.9697	0.9703	0.9700	6505

Automatic selection for Bidirectional RNN

Ranking criterion: validation macro F1, then validation accuracy, then
checkpoint validation loss

	learning_rate	val_macro_f1	val_accuracy	checkpoint_val_loss	checkpoint_epoch
95	0.00015	0.822150	0.970904	0.049792	
75	0.00030	0.819765	0.973354	0.068449	
44	0.00010	0.793913	0.969066	0.052390	
34	0.00020	0.789254	0.967534	0.054008	
44	0.00008	0.781670	0.967228	0.053858	
30	0.00005	0.701566	0.960796	0.074918	

Selected Bidirectional RNN: lr=1e-04, validation macro F1=0.8222, checkpoint
epoch=95

=====
Best Bi-RNN - LR: 1e-04 (Val Macro F1: 0.8222)
=====

[106]: # --- Plot confusion matrices for ALL learning rates - VALIDATION ---

```
print("\n" + "=" * 80)
```

```

print("Confusion matrices for ALL Bi-RNN learning rates - VALIDATION")
print("=" * 80)

for lr in learning_rates_bi:
    model = models_bi[lr]
    print(f"\nGenerating confusion matrix for Bi-RNN with lr={lr:.0e}...")

    plot_cm_task3(model=model, loader=val_loader, model_name=f"Bi-RNN (lr={lr:.0e})", save_dir=save_dir)

```

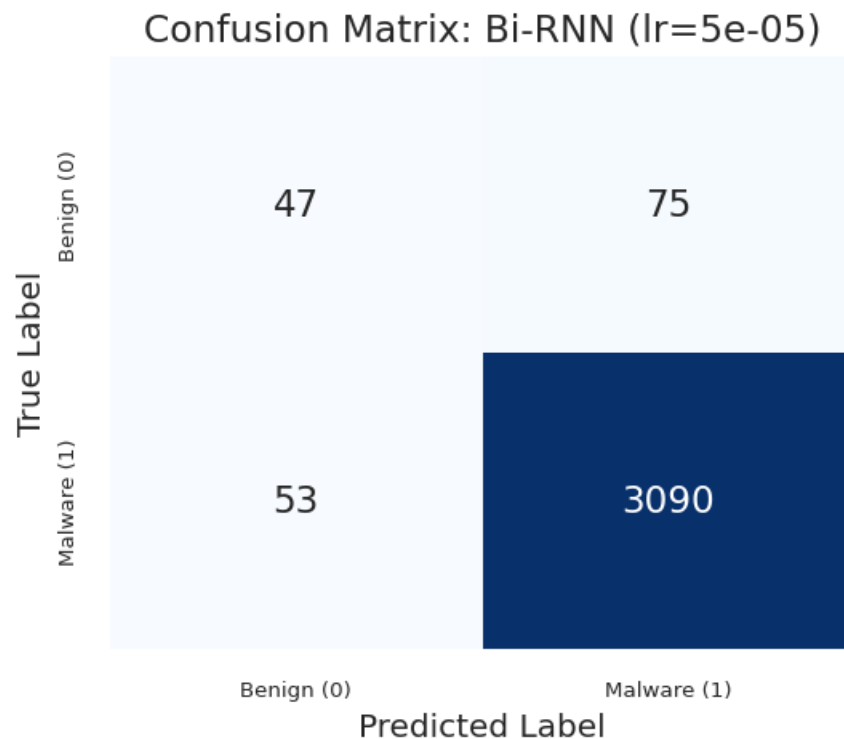
```

=====
Confusion matrices for ALL Bi-RNN learning rates - VALIDATION
=====

```

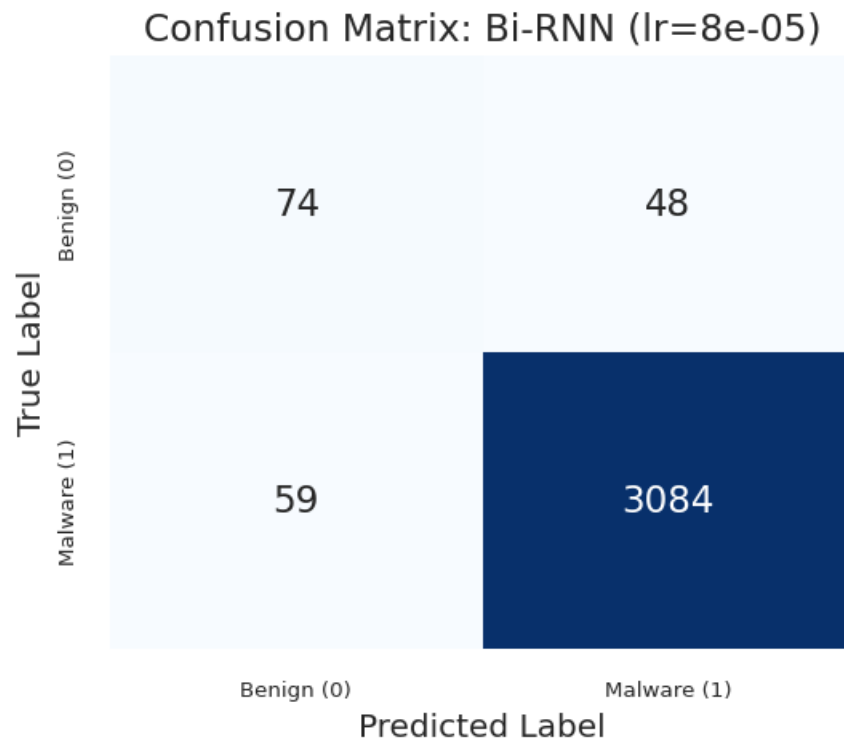
Generating confusion matrix for Bi-RNN with lr=5e-05...

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/cm_bi_rnn_(lr=5e_05).png



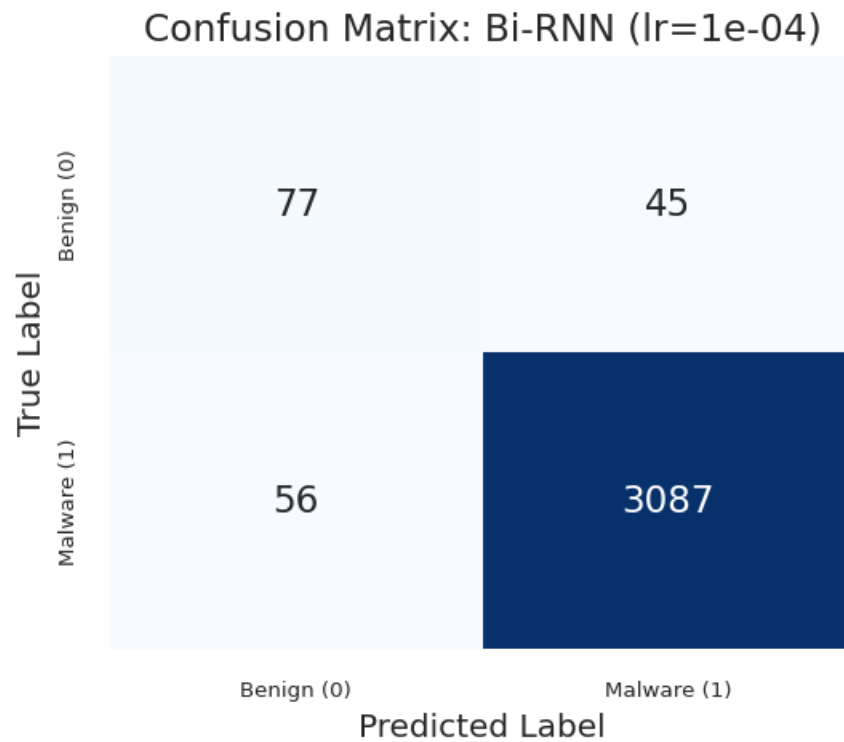
Generating confusion matrix for Bi-RNN with lr=8e-05...

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/cm_bi_rnn_(lr=8e_05).png



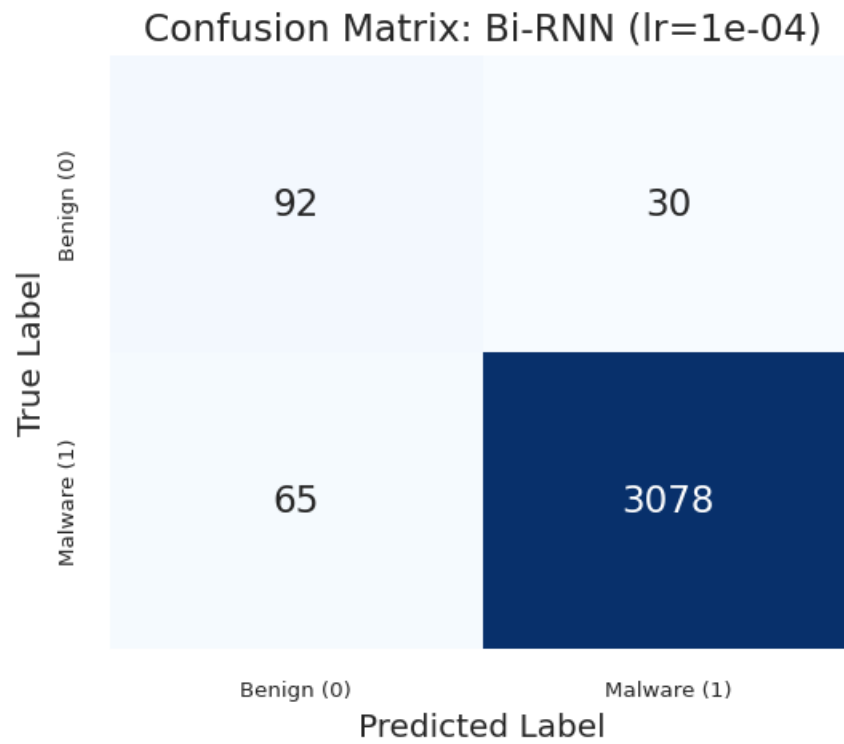
Generating confusion matrix for Bi-RNN with lr=1e-04...

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/cm_bi_rnn_(lr=1e_04).png



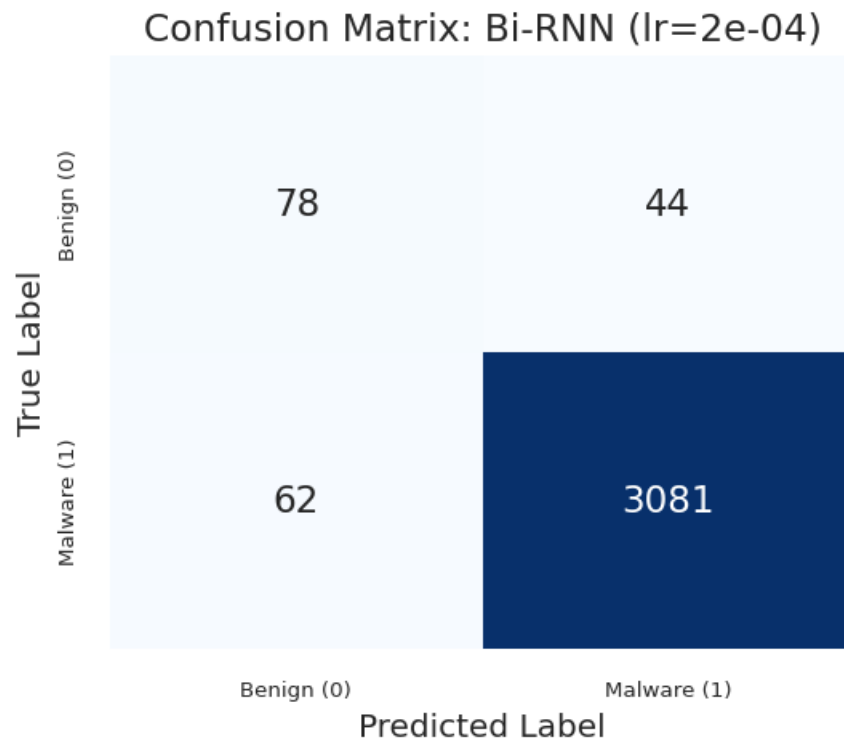
Generating confusion matrix for Bi-RNN with lr=1e-04...

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/cm_bi_rnn_(lr=1e_04).png



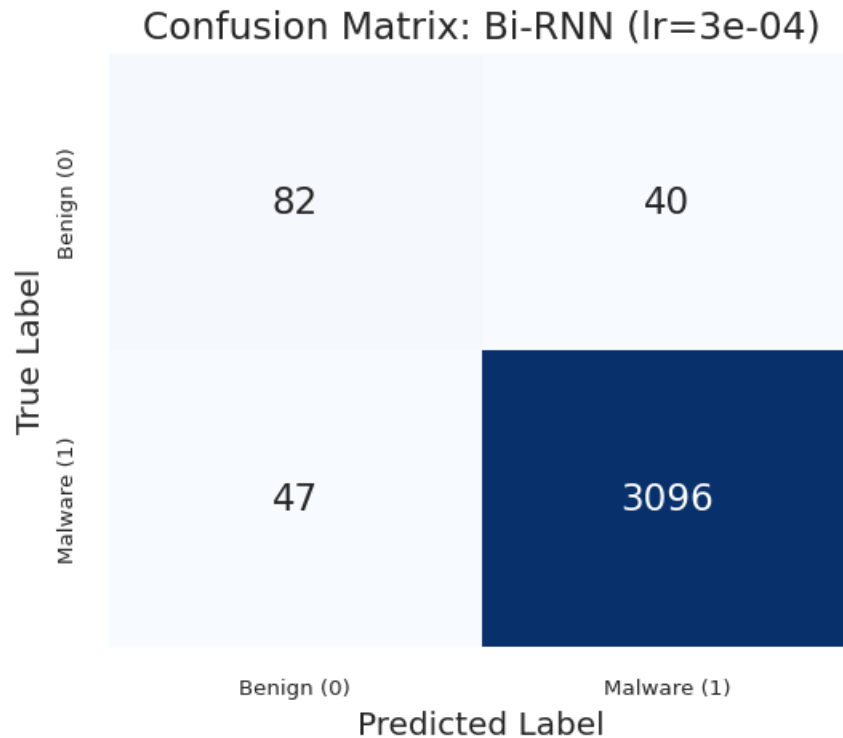
Generating confusion matrix for Bi-RNN with lr=2e-04...

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/cm_bi_rnn_(lr=2e_04).png



Generating confusion matrix for Bi-RNN with lr=3e-04...

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/cm_bi_rnn_(lr=3e_04).png



```
[107]: # --- Plot confusion matrices for ALL learning rates - TEST ---

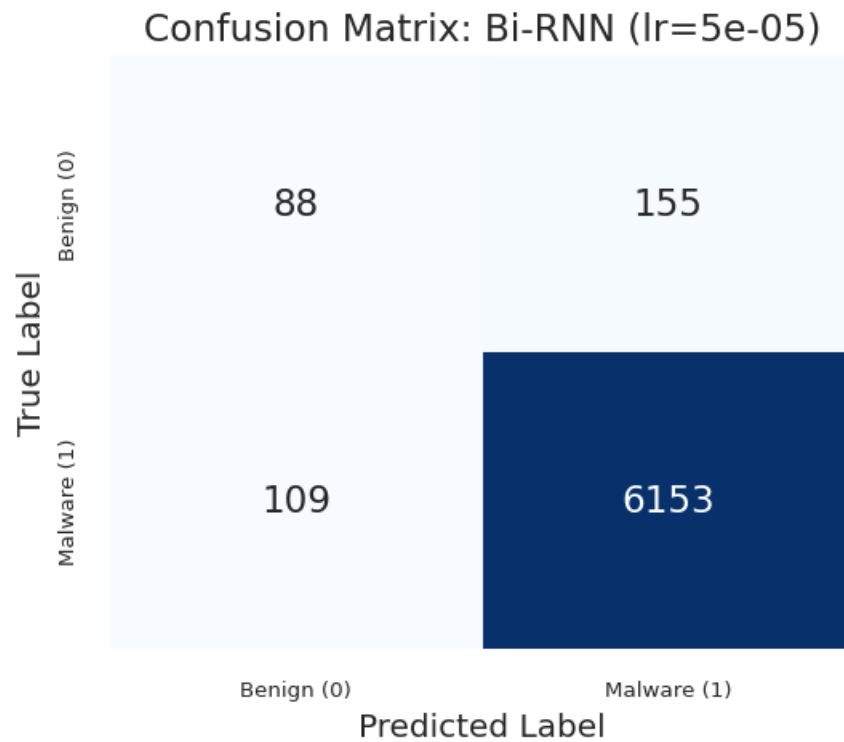
print("\n" + "=" * 80)
print("Confusion matrices for ALL Bi-RNN learning rates - TEST")
print("=" * 80)

for lr in learning_rates_bi:
    model = models_bi[lr]
    print(f"\nGenerating confusion matrix for Bi-RNN with lr={lr:.0e}...")

    plot_cm_task3(model=model, loader=test_loader, model_name=f"Bi-RNN (lr={lr:.0e})", save_dir=save_dir)
```

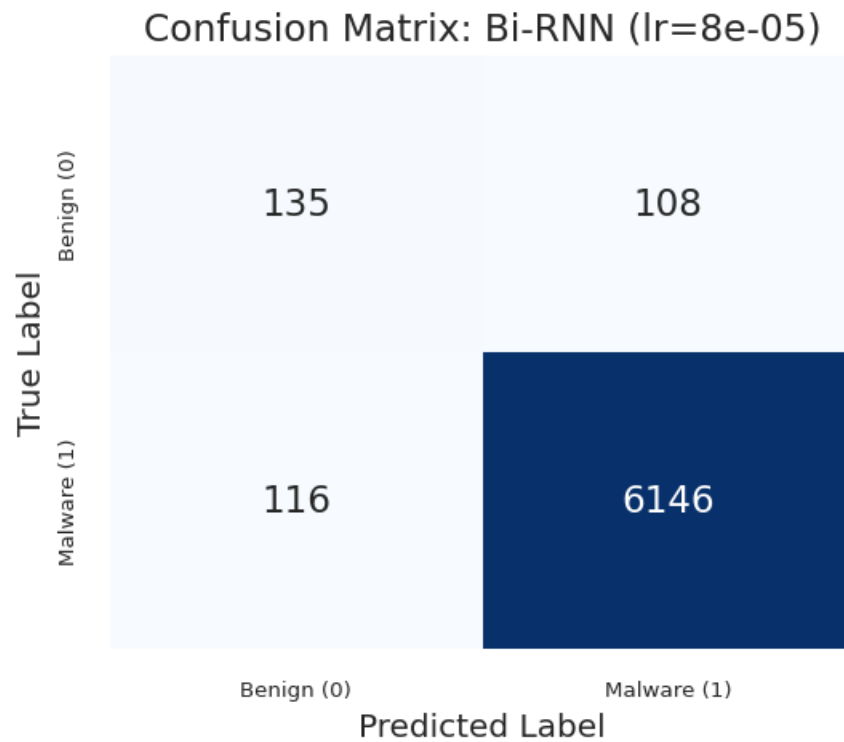
```
=====
Confusion matrices for ALL Bi-RNN learning rates - TEST
=====
```

```
Generating confusion matrix for Bi-RNN with lr=5e-05...
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task3_plots/cm_bi_rnn_(lr=5e_05).png
```



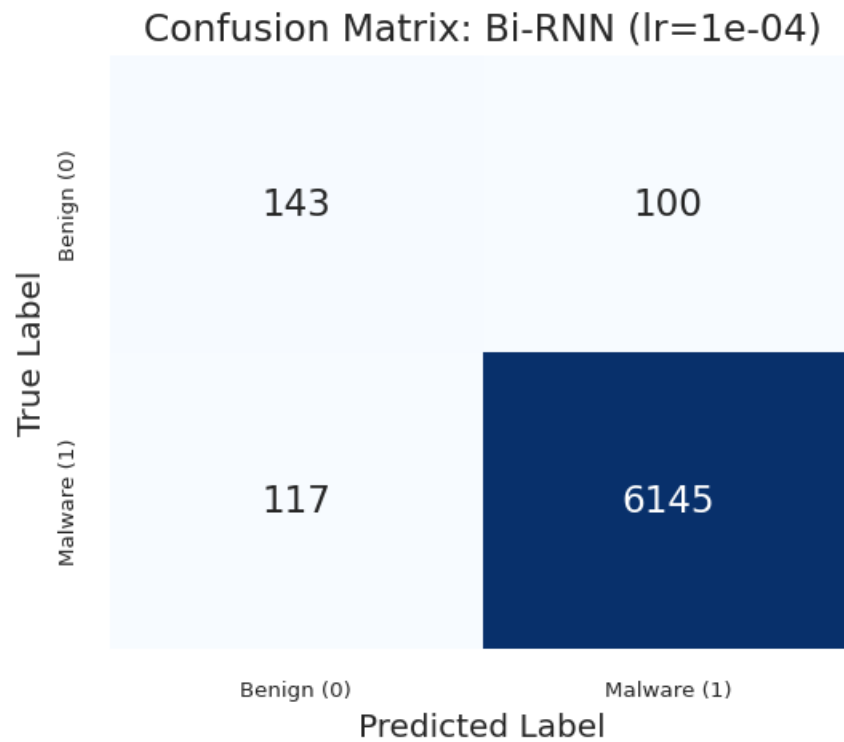
Generating confusion matrix for Bi-RNN with lr=8e-05...

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/cm_bi_rnn_(lr=8e_05).png



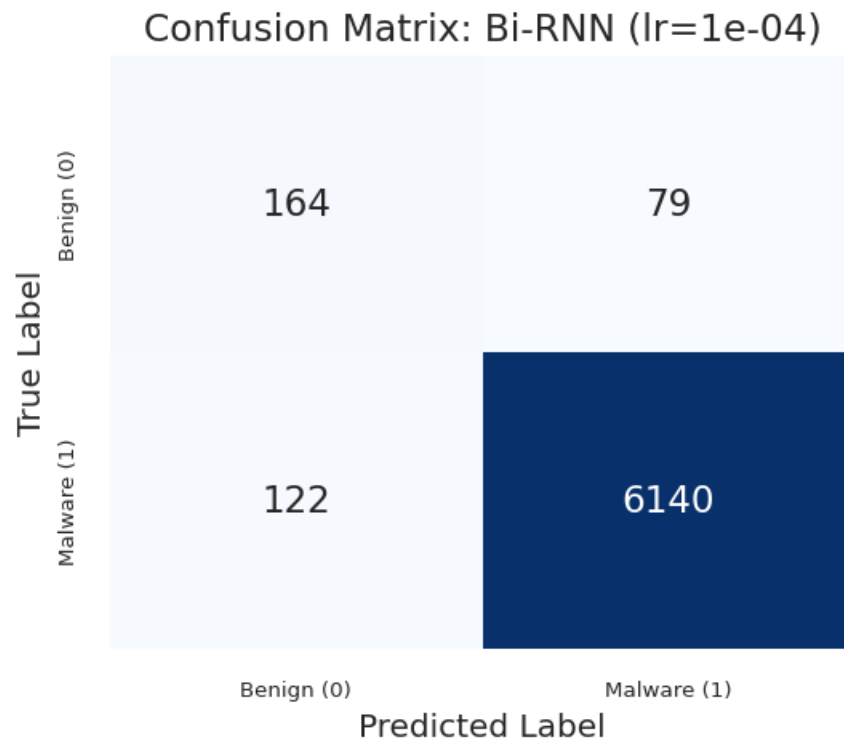
Generating confusion matrix for Bi-RNN with lr=1e-04...

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/cm_bi_rnn_(lr=1e_04).png

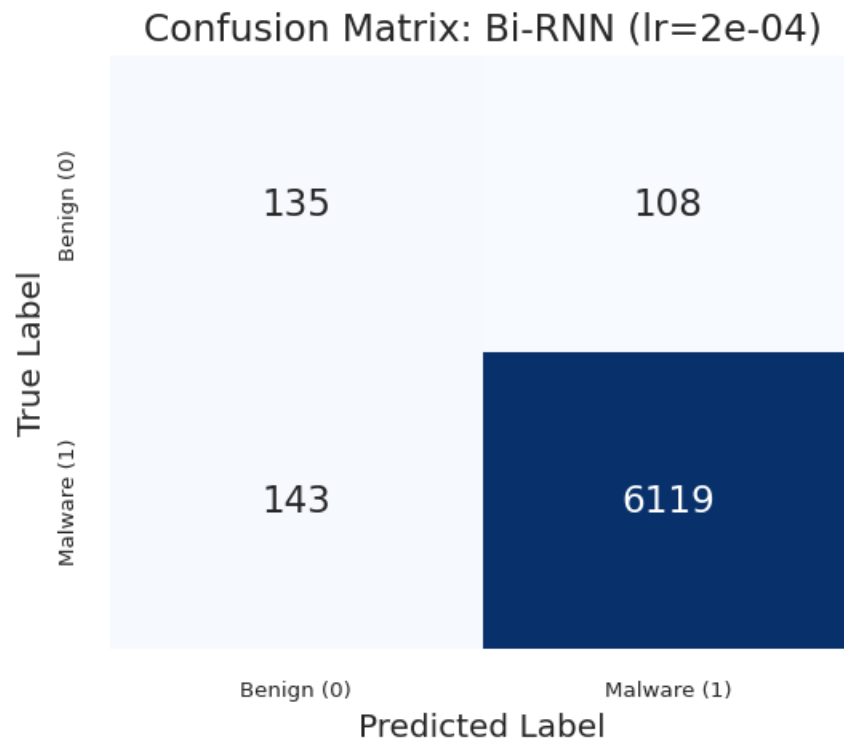


Generating confusion matrix for Bi-RNN with lr=1e-04...

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/cm_bi_rnn_(lr=1e_04).png

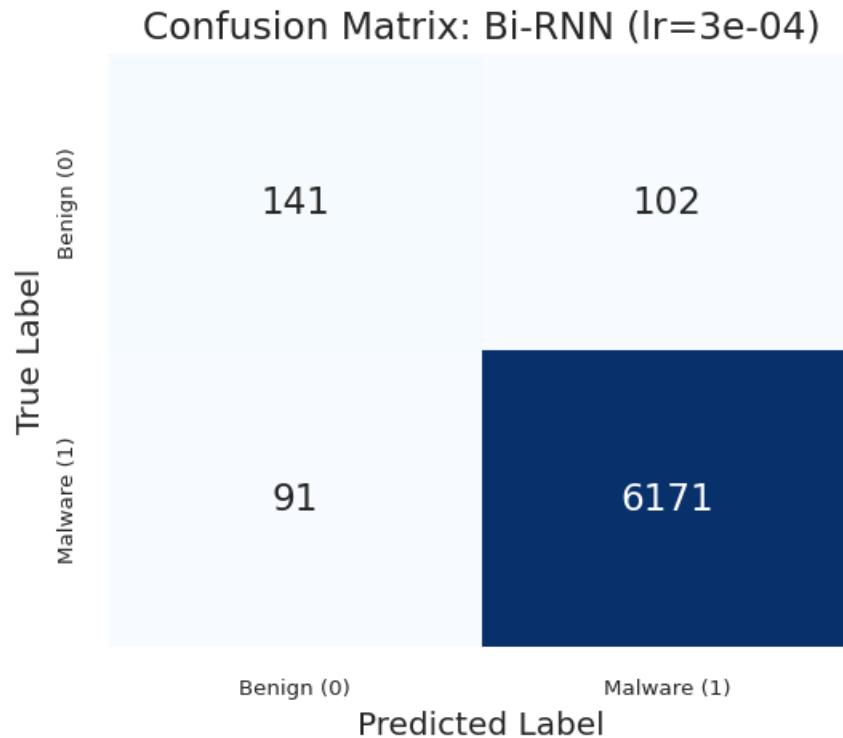


Generating confusion matrix for Bi-RNN with lr=2e-04...
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task3_plots/cm_bi_rnn_(lr=2e_04).png



Generating confusion matrix for Bi-RNN with lr=3e-04...

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/cm_bi_rnn_(lr=3e_04).png



```
[108]: # --- Variation 3: LSTM (bidirectional) ---

class LSTMClassifier(nn.Module):
    def __init__(self, vocab_size, embed_dim=32, hidden_size=128, num_layers=2,
        ↪ num_classes=1, bidirectional=True, dropout=0.3, padding=0):
        super(LSTMClassifier, self).__init__()

        # Embedding layer: Maps API call indices to dense vectors. padding_idx
        ↪ is set.
        self.embedding = nn.Embedding(vocab_size, embedding_dim=embed_dim,
        ↪ padding_idx=padding)

        # LSTM layer: A more complex RNN unit that helps prevent vanishing
        ↪ gradients.
        # batch_first=True expects input shape (batch_size, seq_len, features).
        self.lstm = nn.LSTM(
            input_size=embed_dim,
            hidden_size=hidden_size,
            num_layers=num_layers,
            batch_first=True,
            bidirectional=bidirectional,
            dropout=dropout if num_layers > 1 else 0.0,
```

```

    )

    self.bidirectional = bidirectional
    fc_in = hidden_size * (2 if bidirectional else 1)

    self.fc = nn.Linear(fc_in, num_classes)
    self.dropout = nn.Dropout(dropout)

    def forward(self, x, lengths):
        # Embed the input indices
        emb = self.embedding(x) # (batch_size, max_seq_len, embed_dim)
        emb = self.dropout(emb)

        # Pack the padded sequence to efficiently process variable lengths
        packed = pack_padded_sequence(emb, lengths.cpu(), batch_first=True,
        ↪ enforce_sorted=True)

        # LSTM returns packed_output, (final_hidden_state, final_cell_state)
        packed_out, (hn, cn) = self.lstm(packed)

        # hn shape: (num_layers * num_directions, batch_size, hidden_size)

        # Get the final hidden state from the last layer
        if self.bidirectional:
            # Concatenate the final forward state (hn[-2]) and backward state
            ↪ (hn[-1])
            h_last = torch.cat([hn[-2], hn[-1]], dim=1)
        else:
            # Use the final hidden state of the last layer (hn[-1])
            h_last = hn[-1]

        # Pass the final hidden state through the classifier
        out = self.fc(h_last)
        return out

```

```

[109]: # --- Variation 3: LSTM (bidirectional) ---

print("\n" + "=" * 80)
print("Training LSTM with different learning rates")
print("=" * 80)

# Model parameters
lstm_params = {
    'vocab_size': vocab_size_rnn,
    'embed_dim': 32,
    'hidden_size': 128,
    'num_layers': 2,

```

```

    'num_classes': 1,
    'bidirectional': True,
    'dropout': COMMON_RNN_DROPOUT
}

# Initialize early stopping parameters
min_delta = 0.00001
patience = COMMON_RNN_PATIENCE

# Scheduler parameters
scheduler_params_lstm = COMMON_RNN_SCHEDULER.copy()

# Use the same learning-rate grid as the bidirectional vanilla RNN
learning_rates_lstm = COMMON_RNN_LEARNING_RATES

# Store histories and models for each LR
histories_lstm = {}
models_lstm = {}
elapsed_lstm = {}

for lr in learning_rates_lstm:
    print(f"\n--- Training LSTM with lr={lr:.0e} ---")
    model, history, elapsed = train_with_lr(
        model_class=LSTMClassifier,
        model_params=lstm_params,
        initial_lr=lr,
        train_loader=train_loader,
        val_loader=val_loader,
        criterion=nn.BCEWithLogitsLoss(pos_weight=pos_weight),
        num_epochs=num_epochs,
        min_delta=min_delta,
        patience=patience,
        scheduler_params=scheduler_params_lstm,
        device=device,
        weight_decay=COMMON_RNN_WEIGHT_DECAY,
        seed=COMMON_RNN_SEED,
    )
    histories_lstm[lr] = history
    models_lstm[lr] = model
    elapsed_lstm[lr] = elapsed
    elapsed_str = time.strftime('%H:%M:%S', time.gmtime(elapsed))
    print(f"Completed. Training time: {elapsed_str} ({elapsed:.2f}s) - Final_
    ↪val loss: {history['val_loss'][-1]:.4f}")

```

```

=====
Training LSTM with different learning rates
=====

```

--- Training LSTM with lr=5e-05 ---

Epoch 1/150 - Train Loss: 0.1180 - Val Loss: 0.0892 - Val F1: 0.4905
Epoch 5/150 - Train Loss: 0.0811 - Val Loss: 0.0755 - Val F1: 0.5549
Epoch 10/150 - Train Loss: 0.0701 - Val Loss: 0.0695 - Val F1: 0.6199
Epoch 15/150 - Train Loss: 0.0660 - Val Loss: 0.0667 - Val F1: 0.6481
Epoch 20/150 - Train Loss: 0.0625 - Val Loss: 0.0633 - Val F1: 0.6596
Epoch 25/150 - Train Loss: 0.0579 - Val Loss: 0.0623 - Val F1: 0.6705
Epoch 30/150 - Train Loss: 0.0561 - Val Loss: 0.0646 - Val F1: 0.6850
Epoch 35/150 - Train Loss: 0.0516 - Val Loss: 0.0604 - Val F1: 0.7056
Epoch 40/150 - Train Loss: 0.0483 - Val Loss: 0.0640 - Val F1: 0.7187
Epoch 45/150 - Train Loss: 0.0468 - Val Loss: 0.0586 - Val F1: 0.7215
Epoch 50/150 - Train Loss: 0.0411 - Val Loss: 0.0572 - Val F1: 0.7423
Epoch 55: LR reduced from 0.000050 to 0.000025
Epoch 55/150 - Train Loss: 0.0383 - Val Loss: 0.0567 - Val F1: 0.7632
Epoch 60/150 - Train Loss: 0.0384 - Val Loss: 0.0506 - Val F1: 0.7792
Epoch 65/150 - Train Loss: 0.0359 - Val Loss: 0.0476 - Val F1: 0.7696
Epoch 70/150 - Train Loss: 0.0353 - Val Loss: 0.0592 - Val F1: 0.7771
Epoch 74: LR reduced from 0.000025 to 0.000013
Epoch 75/150 - Train Loss: 0.0346 - Val Loss: 0.0445 - Val F1: 0.7912
Epoch 80/150 - Train Loss: 0.0311 - Val Loss: 0.0475 - Val F1: 0.8007
Epoch 83: LR reduced from 0.000013 to 0.000006
Epoch 85/150 - Train Loss: 0.0332 - Val Loss: 0.0519 - Val F1: 0.8064
Epoch 90/150 - Train Loss: 0.0316 - Val Loss: 0.0528 - Val F1: 0.7964
Epoch 91: LR reduced from 0.000006 to 0.000003
Epoch 95/150 - Train Loss: 0.0321 - Val Loss: 0.0485 - Val F1: 0.8073
Epoch 99: LR reduced from 0.000003 to 0.000002
Epoch 100/150 - Train Loss: 0.0295 - Val Loss: 0.0490 - Val F1: 0.8159
Epoch 105/150 - Train Loss: 0.0339 - Val Loss: 0.0495 - Val F1: 0.8094
Epoch 107: LR reduced from 0.000002 to 0.000001
Epoch 110/150 - Train Loss: 0.0316 - Val Loss: 0.0485 - Val F1: 0.8146
Epoch 115: LR reduced from 0.000001 to 0.000000
Epoch 115/150 - Train Loss: 0.0309 - Val Loss: 0.0511 - Val F1: 0.8146
Epoch 120/150 - Train Loss: 0.0309 - Val Loss: 0.0489 - Val F1: 0.8134
Epoch 123: LR reduced from 0.000000 to 0.000000
Epoch 125/150 - Train Loss: 0.0323 - Val Loss: 0.0494 - Val F1: 0.8134
Epoch 130/150 - Train Loss: 0.0308 - Val Loss: 0.0495 - Val F1: 0.8134
Epoch 131: LR reduced from 0.000000 to 0.000000
Epoch 135/150 - Train Loss: 0.0294 - Val Loss: 0.0496 - Val F1: 0.8146
Early stopping at epoch 138 (best val macro F1 0.8159 at epoch 113, val loss 0.049484)
Completed. Training time: 00:10:51 (651.70s) - Final val loss: 0.0498

--- Training LSTM with lr=8e-05 ---

Epoch 1/150 - Train Loss: 0.1098 - Val Loss: 0.0869 - Val F1: 0.4904
Epoch 5/150 - Train Loss: 0.0757 - Val Loss: 0.0719 - Val F1: 0.6063
Epoch 10/150 - Train Loss: 0.0660 - Val Loss: 0.0671 - Val F1: 0.6141
Epoch 15/150 - Train Loss: 0.0600 - Val Loss: 0.0652 - Val F1: 0.6832

Epoch 20/150 - Train Loss: 0.0607 - Val Loss: 0.0584 - Val F1: 0.6672
 Epoch 25/150 - Train Loss: 0.0541 - Val Loss: 0.0578 - Val F1: 0.6907
 Epoch 30/150 - Train Loss: 0.0498 - Val Loss: 0.0616 - Val F1: 0.6987
 Epoch 35/150 - Train Loss: 0.0462 - Val Loss: 0.0569 - Val F1: 0.7249
 Epoch 40/150 - Train Loss: 0.0414 - Val Loss: 0.0630 - Val F1: 0.7479
 Epoch 45/150 - Train Loss: 0.0383 - Val Loss: 0.0472 - Val F1: 0.7406
 Epoch 50/150 - Train Loss: 0.0352 - Val Loss: 0.0470 - Val F1: 0.7624
 Epoch 55: LR reduced from 0.000080 to 0.000040
 Epoch 55/150 - Train Loss: 0.0306 - Val Loss: 0.0487 - Val F1: 0.7917
 Epoch 60/150 - Train Loss: 0.0297 - Val Loss: 0.0393 - Val F1: 0.8028
 Epoch 65: LR reduced from 0.000040 to 0.000020
 Epoch 65/150 - Train Loss: 0.0269 - Val Loss: 0.0393 - Val F1: 0.8078
 Epoch 70/150 - Train Loss: 0.0248 - Val Loss: 0.0478 - Val F1: 0.8044
 Epoch 73: LR reduced from 0.000020 to 0.000010
 Epoch 75/150 - Train Loss: 0.0246 - Val Loss: 0.0408 - Val F1: 0.8043
 Epoch 80/150 - Train Loss: 0.0255 - Val Loss: 0.0414 - Val F1: 0.8036
 Epoch 81: LR reduced from 0.000010 to 0.000005
 Epoch 85/150 - Train Loss: 0.0242 - Val Loss: 0.0426 - Val F1: 0.8086
 Epoch 89: LR reduced from 0.000005 to 0.000003
 Epoch 90/150 - Train Loss: 0.0240 - Val Loss: 0.0423 - Val F1: 0.8075
 Epoch 95/150 - Train Loss: 0.0234 - Val Loss: 0.0426 - Val F1: 0.8111
 Epoch 97: LR reduced from 0.000003 to 0.000001
 Epoch 100/150 - Train Loss: 0.0218 - Val Loss: 0.0423 - Val F1: 0.8123
 Epoch 105: LR reduced from 0.000001 to 0.000001
 Epoch 105/150 - Train Loss: 0.0258 - Val Loss: 0.0420 - Val F1: 0.8099
 Epoch 110/150 - Train Loss: 0.0247 - Val Loss: 0.0422 - Val F1: 0.8123
 Epoch 113: LR reduced from 0.000001 to 0.000000
 Early stopping at epoch 114 (best val macro F1 0.8147 at epoch 89, val loss 0.042487)
 Completed. Training time: 00:08:46 (526.97s) - Final val loss: 0.0424

--- Training LSTM with lr=1e-04 ---

Epoch 1/150 - Train Loss: 0.1066 - Val Loss: 0.0856 - Val F1: 0.4904
 Epoch 5/150 - Train Loss: 0.0724 - Val Loss: 0.0697 - Val F1: 0.6208
 Epoch 10/150 - Train Loss: 0.0645 - Val Loss: 0.0649 - Val F1: 0.6502
 Epoch 15/150 - Train Loss: 0.0587 - Val Loss: 0.0703 - Val F1: 0.6879
 Epoch 20/150 - Train Loss: 0.0558 - Val Loss: 0.0572 - Val F1: 0.6767
 Epoch 25/150 - Train Loss: 0.0527 - Val Loss: 0.0584 - Val F1: 0.7040
 Epoch 30/150 - Train Loss: 0.0476 - Val Loss: 0.0578 - Val F1: 0.7314
 Epoch 35/150 - Train Loss: 0.0406 - Val Loss: 0.0516 - Val F1: 0.7410
 Epoch 40/150 - Train Loss: 0.0366 - Val Loss: 0.0613 - Val F1: 0.7884
 Epoch 45/150 - Train Loss: 0.0339 - Val Loss: 0.0411 - Val F1: 0.7850
 Epoch 50/150 - Train Loss: 0.0306 - Val Loss: 0.0418 - Val F1: 0.8067
 Epoch 55/150 - Train Loss: 0.0266 - Val Loss: 0.0460 - Val F1: 0.8133
 Epoch 60/150 - Train Loss: 0.0265 - Val Loss: 0.0361 - Val F1: 0.8208
 Epoch 65/150 - Train Loss: 0.0240 - Val Loss: 0.0319 - Val F1: 0.8231
 Epoch 70/150 - Train Loss: 0.0226 - Val Loss: 0.0423 - Val F1: 0.8378
 Epoch 73: LR reduced from 0.000100 to 0.000050

Epoch 75/150 - Train Loss: 0.0189 - Val Loss: 0.0378 - Val F1: 0.8172
 Epoch 80/150 - Train Loss: 0.0179 - Val Loss: 0.0404 - Val F1: 0.8458
 Epoch 81: LR reduced from 0.000050 to 0.000025
 Epoch 85/150 - Train Loss: 0.0166 - Val Loss: 0.0396 - Val F1: 0.8496
 Epoch 89: LR reduced from 0.000025 to 0.000013
 Epoch 90/150 - Train Loss: 0.0150 - Val Loss: 0.0443 - Val F1: 0.8470
 Epoch 95/150 - Train Loss: 0.0150 - Val Loss: 0.0415 - Val F1: 0.8511
 Epoch 97: LR reduced from 0.000013 to 0.000006
 Epoch 100/150 - Train Loss: 0.0135 - Val Loss: 0.0413 - Val F1: 0.8598
 Epoch 105: LR reduced from 0.000006 to 0.000003
 Epoch 105/150 - Train Loss: 0.0159 - Val Loss: 0.0402 - Val F1: 0.8633
 Epoch 110/150 - Train Loss: 0.0141 - Val Loss: 0.0408 - Val F1: 0.8598
 Epoch 113: LR reduced from 0.000003 to 0.000002
 Epoch 115/150 - Train Loss: 0.0135 - Val Loss: 0.0422 - Val F1: 0.8588
 Epoch 120/150 - Train Loss: 0.0133 - Val Loss: 0.0415 - Val F1: 0.8583
 Epoch 121: LR reduced from 0.000002 to 0.000001
 Epoch 125/150 - Train Loss: 0.0145 - Val Loss: 0.0418 - Val F1: 0.8613
 Epoch 129: LR reduced from 0.000001 to 0.000000
 Epoch 130/150 - Train Loss: 0.0142 - Val Loss: 0.0414 - Val F1: 0.8583
 Early stopping at epoch 130 (best val macro F1 0.8633 at epoch 105, val loss 0.040234)
 Completed. Training time: 00:10:01 (601.71s) - Final val loss: 0.0414

--- Training LSTM with lr=1e-04 ---

Epoch 1/150 - Train Loss: 0.1018 - Val Loss: 0.0831 - Val F1: 0.4903
 Epoch 5/150 - Train Loss: 0.0702 - Val Loss: 0.0667 - Val F1: 0.6631
 Epoch 10/150 - Train Loss: 0.0644 - Val Loss: 0.0640 - Val F1: 0.6752
 Epoch 15/150 - Train Loss: 0.0551 - Val Loss: 0.0615 - Val F1: 0.6892
 Epoch 20/150 - Train Loss: 0.0473 - Val Loss: 0.0561 - Val F1: 0.6985
 Epoch 25/150 - Train Loss: 0.0407 - Val Loss: 0.0459 - Val F1: 0.7415
 Epoch 30/150 - Train Loss: 0.0353 - Val Loss: 0.0438 - Val F1: 0.7574
 Epoch 35/150 - Train Loss: 0.0295 - Val Loss: 0.0447 - Val F1: 0.7885
 Epoch 40/150 - Train Loss: 0.0244 - Val Loss: 0.0336 - Val F1: 0.8352
 Epoch 45/150 - Train Loss: 0.0243 - Val Loss: 0.0356 - Val F1: 0.8046
 Epoch 50/150 - Train Loss: 0.0188 - Val Loss: 0.0408 - Val F1: 0.8342
 Epoch 54: LR reduced from 0.000150 to 0.000075
 Epoch 55/150 - Train Loss: 0.0145 - Val Loss: 0.0371 - Val F1: 0.8315
 Epoch 60/150 - Train Loss: 0.0145 - Val Loss: 0.0319 - Val F1: 0.8494
 Epoch 64: LR reduced from 0.000075 to 0.000037
 Epoch 65/150 - Train Loss: 0.0115 - Val Loss: 0.0364 - Val F1: 0.8328
 Epoch 70/150 - Train Loss: 0.0112 - Val Loss: 0.0347 - Val F1: 0.8590
 Epoch 72: LR reduced from 0.000037 to 0.000019
 Epoch 75/150 - Train Loss: 0.0096 - Val Loss: 0.0375 - Val F1: 0.8387
 Epoch 80: LR reduced from 0.000019 to 0.000009
 Epoch 80/150 - Train Loss: 0.0101 - Val Loss: 0.0381 - Val F1: 0.8556
 Epoch 85/150 - Train Loss: 0.0101 - Val Loss: 0.0371 - Val F1: 0.8581
 Epoch 88: LR reduced from 0.000009 to 0.000005
 Epoch 90/150 - Train Loss: 0.0089 - Val Loss: 0.0371 - Val F1: 0.8572

Epoch 95/150 - Train Loss: 0.0102 - Val Loss: 0.0367 - Val F1: 0.8586
Epoch 96: LR reduced from 0.000005 to 0.000002
Epoch 100/150 - Train Loss: 0.0102 - Val Loss: 0.0374 - Val F1: 0.8581
Epoch 104: LR reduced from 0.000002 to 0.000001
Epoch 105/150 - Train Loss: 0.0102 - Val Loss: 0.0379 - Val F1: 0.8590
Early stopping at epoch 107 (best val macro F1 0.8647 at epoch 82, val loss 0.036980)
Completed. Training time: 00:08:17 (497.24s) - Final val loss: 0.0379

--- Training LSTM with lr=2e-04 ---

Epoch 1/150 - Train Loss: 0.0987 - Val Loss: 0.0814 - Val F1: 0.5283
Epoch 5/150 - Train Loss: 0.0687 - Val Loss: 0.0654 - Val F1: 0.6537
Epoch 10/150 - Train Loss: 0.0556 - Val Loss: 0.0613 - Val F1: 0.6509
Epoch 15/150 - Train Loss: 0.0483 - Val Loss: 0.0604 - Val F1: 0.7098
Epoch 20/150 - Train Loss: 0.0421 - Val Loss: 0.0577 - Val F1: 0.6915
Epoch 25/150 - Train Loss: 0.0323 - Val Loss: 0.0424 - Val F1: 0.7743
Epoch 30/150 - Train Loss: 0.0272 - Val Loss: 0.0395 - Val F1: 0.8067
Epoch 35/150 - Train Loss: 0.0213 - Val Loss: 0.0391 - Val F1: 0.8276
Epoch 40/150 - Train Loss: 0.0172 - Val Loss: 0.0446 - Val F1: 0.8135
Epoch 45: LR reduced from 0.000200 to 0.000100
Epoch 45/150 - Train Loss: 0.0172 - Val Loss: 0.0348 - Val F1: 0.8039
Epoch 50/150 - Train Loss: 0.0127 - Val Loss: 0.0379 - Val F1: 0.8375
Epoch 53: LR reduced from 0.000100 to 0.000050
Epoch 55/150 - Train Loss: 0.0106 - Val Loss: 0.0400 - Val F1: 0.8569
Epoch 60/150 - Train Loss: 0.0102 - Val Loss: 0.0402 - Val F1: 0.8533
Epoch 61: LR reduced from 0.000050 to 0.000025
Epoch 65/150 - Train Loss: 0.0083 - Val Loss: 0.0378 - Val F1: 0.8564
Epoch 69: LR reduced from 0.000025 to 0.000013
Epoch 70/150 - Train Loss: 0.0086 - Val Loss: 0.0403 - Val F1: 0.8613
Epoch 75/150 - Train Loss: 0.0082 - Val Loss: 0.0391 - Val F1: 0.8518
Epoch 77: LR reduced from 0.000013 to 0.000006
Epoch 80/150 - Train Loss: 0.0091 - Val Loss: 0.0395 - Val F1: 0.8575
Epoch 85: LR reduced from 0.000006 to 0.000003
Epoch 85/150 - Train Loss: 0.0081 - Val Loss: 0.0414 - Val F1: 0.8594
Early stopping at epoch 87 (best val macro F1 0.8623 at epoch 62, val loss 0.040708)
Completed. Training time: 00:06:44 (404.84s) - Final val loss: 0.0415

--- Training LSTM with lr=3e-04 ---

Epoch 1/150 - Train Loss: 0.0948 - Val Loss: 0.0789 - Val F1: 0.5747
Epoch 5/150 - Train Loss: 0.0712 - Val Loss: 0.0694 - Val F1: 0.6700
Epoch 10/150 - Train Loss: 0.0525 - Val Loss: 0.0651 - Val F1: 0.7046
Epoch 15/150 - Train Loss: 0.0429 - Val Loss: 0.0493 - Val F1: 0.7468
Epoch 20/150 - Train Loss: 0.0324 - Val Loss: 0.0472 - Val F1: 0.7643
Epoch 25/150 - Train Loss: 0.0259 - Val Loss: 0.0315 - Val F1: 0.7989
Epoch 30/150 - Train Loss: 0.0212 - Val Loss: 0.0374 - Val F1: 0.8285
Epoch 35/150 - Train Loss: 0.0171 - Val Loss: 0.0390 - Val F1: 0.8087
Epoch 37: LR reduced from 0.000300 to 0.000150

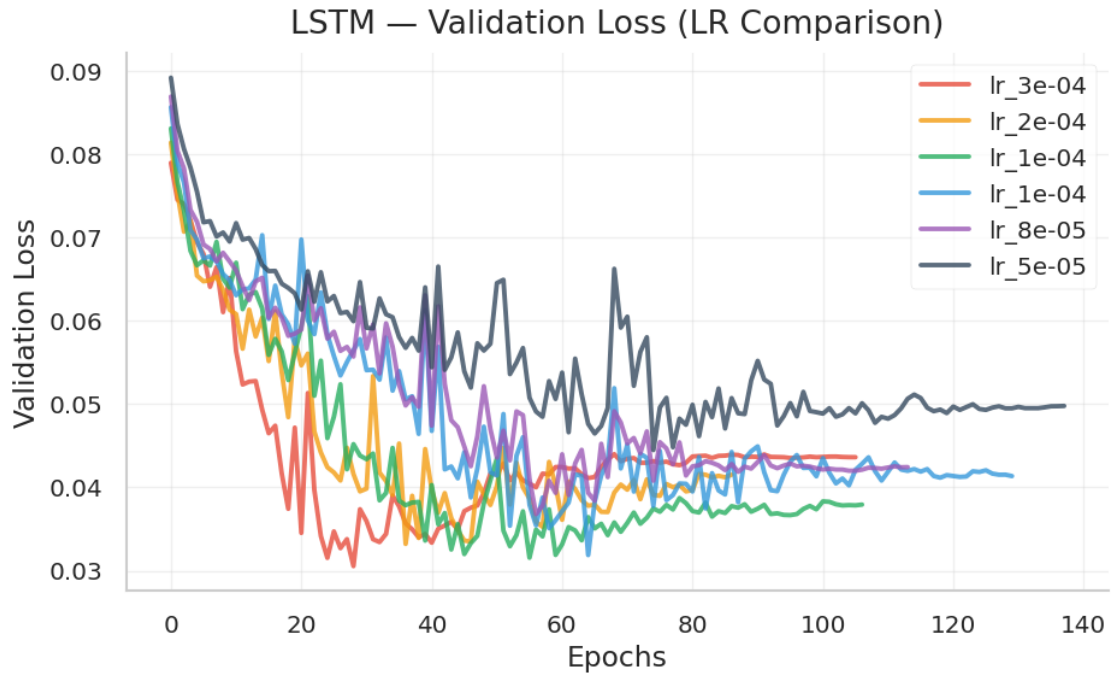
Epoch 40/150 - Train Loss: 0.0112 - Val Loss: 0.0344 - Val F1: 0.8409
Epoch 45: LR reduced from 0.000150 to 0.000075
Epoch 45/150 - Train Loss: 0.0123 - Val Loss: 0.0350 - Val F1: 0.8393
Epoch 50/150 - Train Loss: 0.0086 - Val Loss: 0.0417 - Val F1: 0.8657
Epoch 53: LR reduced from 0.000075 to 0.000037
Epoch 55/150 - Train Loss: 0.0074 - Val Loss: 0.0409 - Val F1: 0.8562
Epoch 60/150 - Train Loss: 0.0066 - Val Loss: 0.0424 - Val F1: 0.8575
Epoch 61: LR reduced from 0.000037 to 0.000019
Epoch 65/150 - Train Loss: 0.0070 - Val Loss: 0.0411 - Val F1: 0.8576
Epoch 69: LR reduced from 0.000019 to 0.000009
Epoch 70/150 - Train Loss: 0.0066 - Val Loss: 0.0431 - Val F1: 0.8633
Epoch 75/150 - Train Loss: 0.0056 - Val Loss: 0.0432 - Val F1: 0.8686
Epoch 77: LR reduced from 0.000009 to 0.000005
Epoch 80/150 - Train Loss: 0.0063 - Val Loss: 0.0430 - Val F1: 0.8662
Epoch 85: LR reduced from 0.000005 to 0.000002
Epoch 85/150 - Train Loss: 0.0059 - Val Loss: 0.0438 - Val F1: 0.8692
Epoch 90/150 - Train Loss: 0.0061 - Val Loss: 0.0437 - Val F1: 0.8692
Epoch 93: LR reduced from 0.000002 to 0.000001
Epoch 95/150 - Train Loss: 0.0070 - Val Loss: 0.0436 - Val F1: 0.8692
Epoch 100/150 - Train Loss: 0.0067 - Val Loss: 0.0436 - Val F1: 0.8686
Epoch 101: LR reduced from 0.000001 to 0.000001
Epoch 105/150 - Train Loss: 0.0063 - Val Loss: 0.0436 - Val F1: 0.8710
Early stopping at epoch 106 (best val macro F1 0.8722 at epoch 81, val loss 0.043708)
Completed. Training time: 00:08:05 (485.53s) - Final val loss: 0.0436

[110]: `# --- Plot loss curves ---`

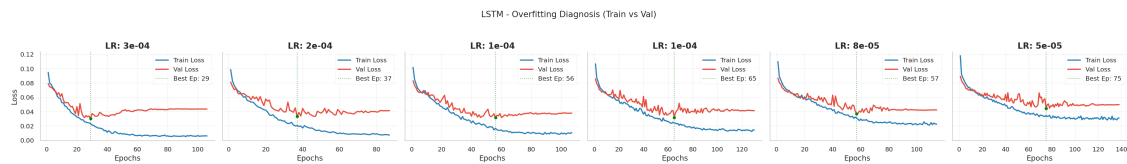
```
# Plot comparison
plot_lr_comparison(histories_lstm, "LSTM", save_dir)

#Plot Train vs Validation
plot_train_val_grid(histories_lstm, "LSTM", save_dir)
```

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/lstm_lr_comparison.png

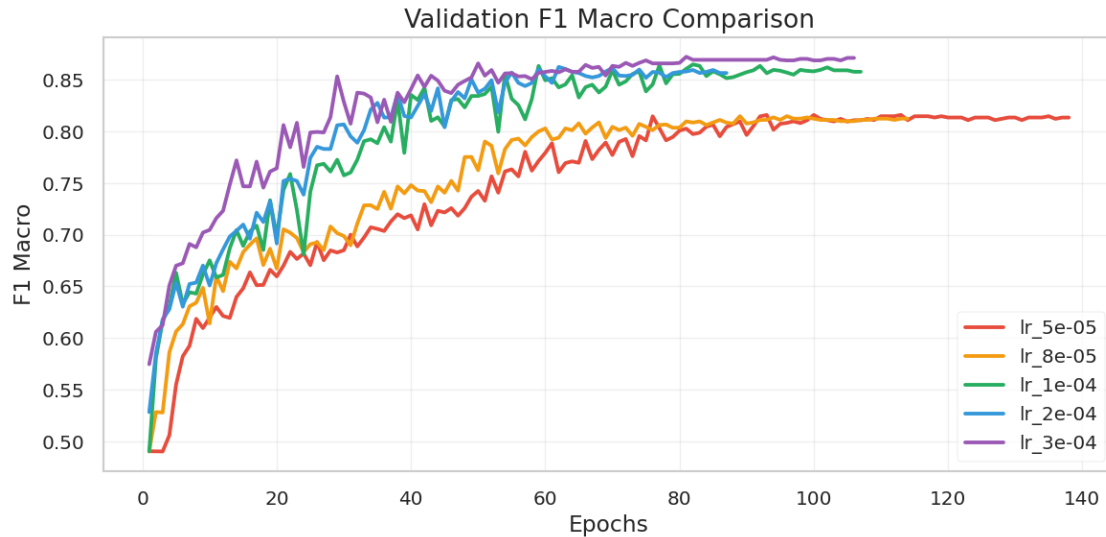


Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/lstm_diagnosis.png



```
[111]: # --- Plot Validation F1 Macro for LSTM ---
plot_training_metrics(histories_lstm, save_dir=save_dir,
    ↪plot_name="lstm_f1_comparison")
```

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/lstm_f1_comparison.png



```
[112]: # --- Evaluate validation and test sets for ALL learning rates ---

print("\n" + "=" * 80)
print("Evaluation for ALL LSTM RNN learning rates")
print("=" * 80)

for lr in learning_rates_lstm: # Changed 'learning_rates' to
    ↪ 'learning_rates_lstm'
    model = models_lstm[lr]

    print(f"\n{'='*60}")
    print(f"LSTM RNN with lr={lr:.0e}")
    print(f"{'='*60}")

    # Validation evaluation
    y_val_true, y_val_pred = predict_loader(model, val_loader)
    print(f'\nValidation report (LSTM RNN - lr={lr:.0e}):')
    print(classification_report(y_val_true, y_val_pred, digits=4,
    ↪ zero_division=0))

    # Test evaluation
    y_test_true, y_test_pred = predict_loader(model, test_loader)
    print(f'Test report (LSTM RNN - lr={lr:.0e}):')
    print(classification_report(y_test_true, y_test_pred, digits=4,
    ↪ zero_division=0))

    # Select the restored model using validation metrics only
    best_lstm_lr_value, lstm_selection_table = select_best_rnn_model(
```

```

        'Bidirectional LSTM', models_lstm, histories_lstm, val_loader
    )
    best_lr_lstm = (best_lstm_lr_value, histories_lstm[best_lstm_lr_value])
    print(f"\n{'='*60}")
    print(
        f"Best Bidirectional LSTM - LR: {best_lr_lstm[0]:.0e} "
        f"(Val Macro F1: {lstm_selection_table.loc[0, 'val_macro_f1']:.4f})"
    )
    print(f"{'='*60}")

```

```

=====
Evaluation for ALL LSTM RNN learning rates
=====

```

```

=====
LSTM RNN with lr=5e-05
=====

```

Validation report (LSTM RNN - lr=5e-05):

	precision	recall	f1-score	support
0	0.5817	0.7295	0.6473	122
1	0.9894	0.9796	0.9845	3143
accuracy			0.9703	3265
macro avg	0.7855	0.8546	0.8159	3265
weighted avg	0.9742	0.9703	0.9719	3265

Test report (LSTM RNN - lr=5e-05):

	precision	recall	f1-score	support
0	0.6066	0.6091	0.6078	243
1	0.9848	0.9847	0.9847	6262
accuracy			0.9706	6505
macro avg	0.7957	0.7969	0.7963	6505
weighted avg	0.9707	0.9706	0.9707	6505

```

=====
LSTM RNN with lr=8e-05
=====

```

Validation report (LSTM RNN - lr=8e-05):

	precision	recall	f1-score	support
0	0.5687	0.7459	0.6454	122

1	0.9900	0.9780	0.9840	3143
accuracy			0.9694	3265
macro avg	0.7794	0.8620	0.8147	3265
weighted avg	0.9743	0.9694	0.9713	3265

Test report (LSTM RNN - lr=8e-05):

	precision	recall	f1-score	support
0	0.5524	0.6502	0.5974	243
1	0.9863	0.9796	0.9829	6262
accuracy			0.9673	6505
macro avg	0.7694	0.8149	0.7901	6505
weighted avg	0.9701	0.9673	0.9685	6505

=====

LSTM RNN with lr=1e-04

=====

Validation report (LSTM RNN - lr=1e-04):

	precision	recall	f1-score	support
0	0.6879	0.7951	0.7376	122
1	0.9920	0.9860	0.9890	3143
accuracy			0.9789	3265
macro avg	0.8400	0.8905	0.8633	3265
weighted avg	0.9806	0.9789	0.9796	3265

Test report (LSTM RNN - lr=1e-04):

	precision	recall	f1-score	support
0	0.6236	0.6955	0.6576	243
1	0.9881	0.9837	0.9859	6262
accuracy			0.9729	6505
macro avg	0.8059	0.8396	0.8218	6505
weighted avg	0.9745	0.9729	0.9737	6505

=====

LSTM RNN with lr=1e-04

=====

Validation report (LSTM RNN - lr=1e-04):

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

0	0.6603	0.8443	0.7410	122
1	0.9939	0.9831	0.9885	3143
accuracy			0.9779	3265
macro avg	0.8271	0.9137	0.8647	3265
weighted avg	0.9814	0.9779	0.9792	3265

Test report (LSTM RNN - lr=1e-04):

	precision	recall	f1-score	support
0	0.6605	0.7366	0.6965	243
1	0.9897	0.9853	0.9875	6262
accuracy			0.9760	6505
macro avg	0.8251	0.8610	0.8420	6505
weighted avg	0.9774	0.9760	0.9766	6505

=====

LSTM RNN with lr=2e-04

=====

Validation report (LSTM RNN - lr=2e-04):

	precision	recall	f1-score	support
0	0.6906	0.7869	0.7356	122
1	0.9917	0.9863	0.9890	3143
accuracy			0.9789	3265
macro avg	0.8412	0.8866	0.8623	3265
weighted avg	0.9804	0.9789	0.9795	3265

Test report (LSTM RNN - lr=2e-04):

	precision	recall	f1-score	support
0	0.6273	0.6996	0.6615	243
1	0.9883	0.9839	0.9861	6262
accuracy			0.9733	6505
macro avg	0.8078	0.8417	0.8238	6505
weighted avg	0.9748	0.9733	0.9739	6505

=====

LSTM RNN with lr=3e-04

=====

Validation report (LSTM RNN - lr=3e-04):

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

0	0.6993	0.8197	0.7547	122
1	0.9930	0.9863	0.9896	3143

accuracy			0.9801	3265
macro avg	0.8461	0.9030	0.8722	3265
weighted avg	0.9820	0.9801	0.9808	3265

Test report (LSTM RNN - lr=3e-04):

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

0	0.6763	0.6708	0.6736	243
1	0.9872	0.9875	0.9874	6262

accuracy			0.9757	6505
macro avg	0.8318	0.8292	0.8305	6505
weighted avg	0.9756	0.9757	0.9757	6505

Automatic selection for Bidirectional LSTM

Ranking criterion: validation macro F1, then validation accuracy, then checkpoint validation loss

	learning_rate	val_macro_f1	val_accuracy	checkpoint_val_loss
checkpoint_epoch				
81	0.00030	0.872171	0.980092	0.043708
82	0.00015	0.864745	0.977948	0.036980
105	0.00010	0.863316	0.978867	0.040234
62	0.00020	0.862313	0.978867	0.040708
113	0.00005	0.815883	0.970291	0.049484
89	0.00008	0.814692	0.969372	0.042487

Selected Bidirectional LSTM: lr=3e-04, validation macro F1=0.8722, checkpoint epoch=81

=====
 Best Bidirectional LSTM - LR: 3e-04 (Val Macro F1: 0.8722)
 =====

[113]: # --- Plot confusion matrices for ALL learning rates - VALIDATION ---

```

print("\n" + "=" * 80)
print("Confusion matrices for ALL LSTM RNN learning rates - VALIDATION")
print("=" * 80)

for lr in learning_rates_lstm:
    model = models_lstm[lr]
    print(f"\nGenerating confusion matrix for LSTM RNN with lr={lr:.0e}...")

    plot_cm_task3(model=model, loader=val_loader, model_name=f"LSTM RNN (lr={lr:.0e})", save_dir=save_dir)

```

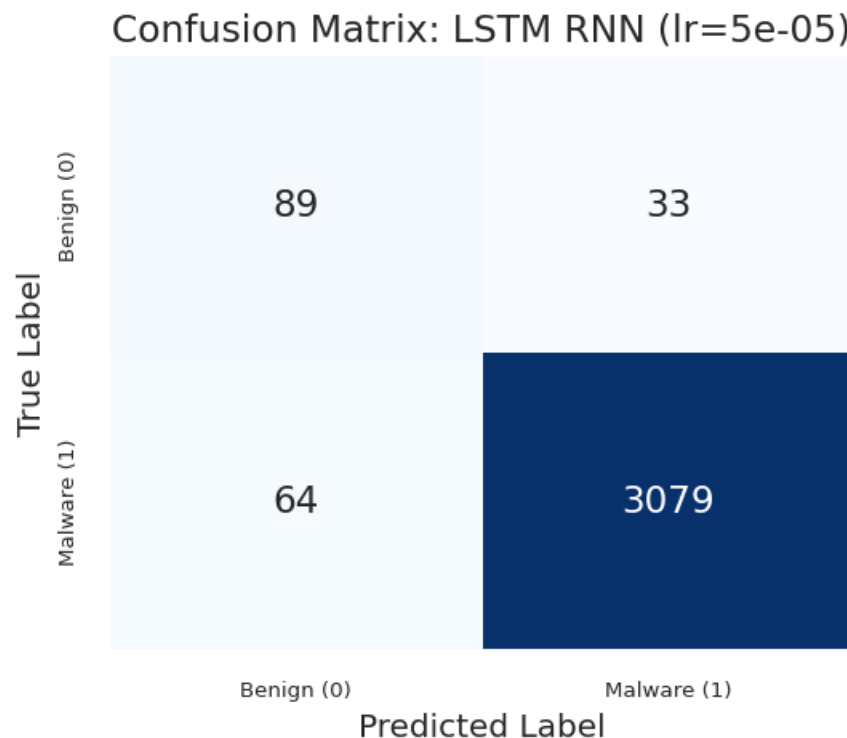
```

=====
Confusion matrices for ALL LSTM RNN learning rates - VALIDATION
=====

```

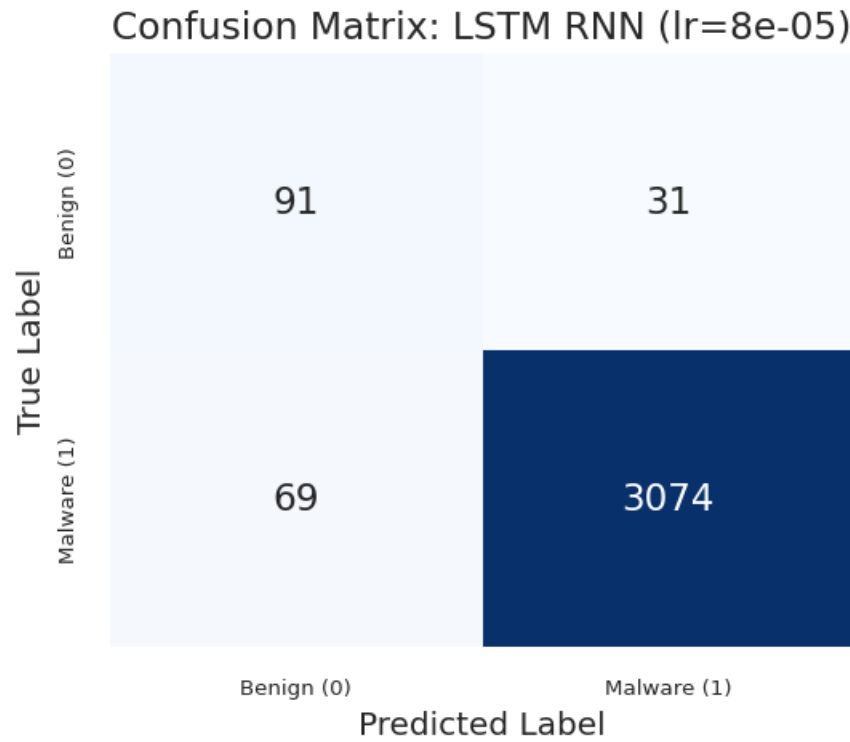
Generating confusion matrix for LSTM RNN with lr=5e-05...

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/cm_lstm_rnn_(lr=5e_05).png



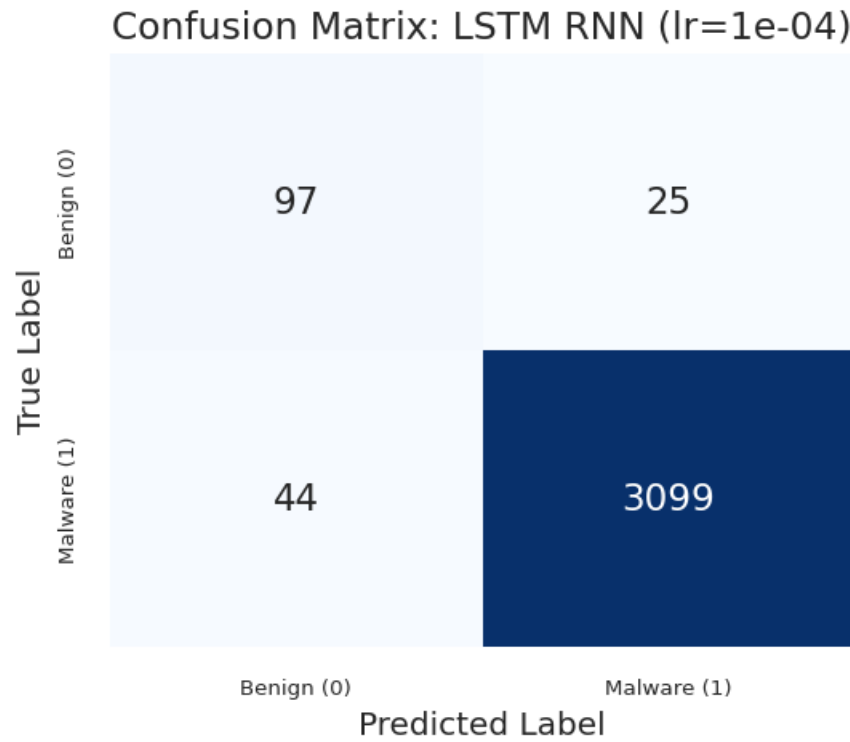
Generating confusion matrix for LSTM RNN with lr=8e-05...

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/cm_lstm_rnn_(lr=8e_05).png



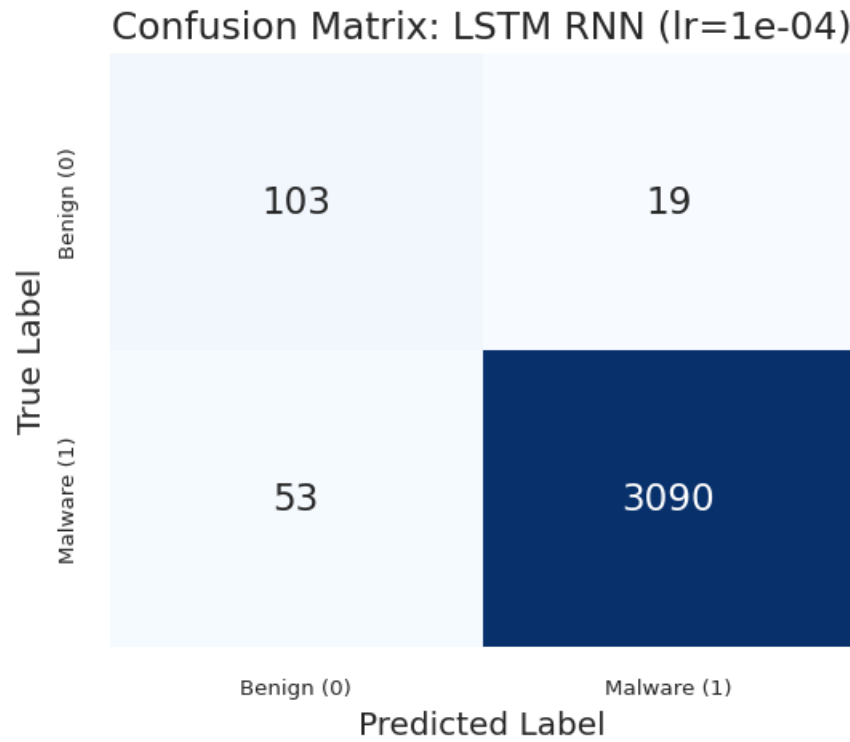
Generating confusion matrix for LSTM RNN with lr=1e-04...

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/cm_lstm_rnn_(lr=1e_04).png



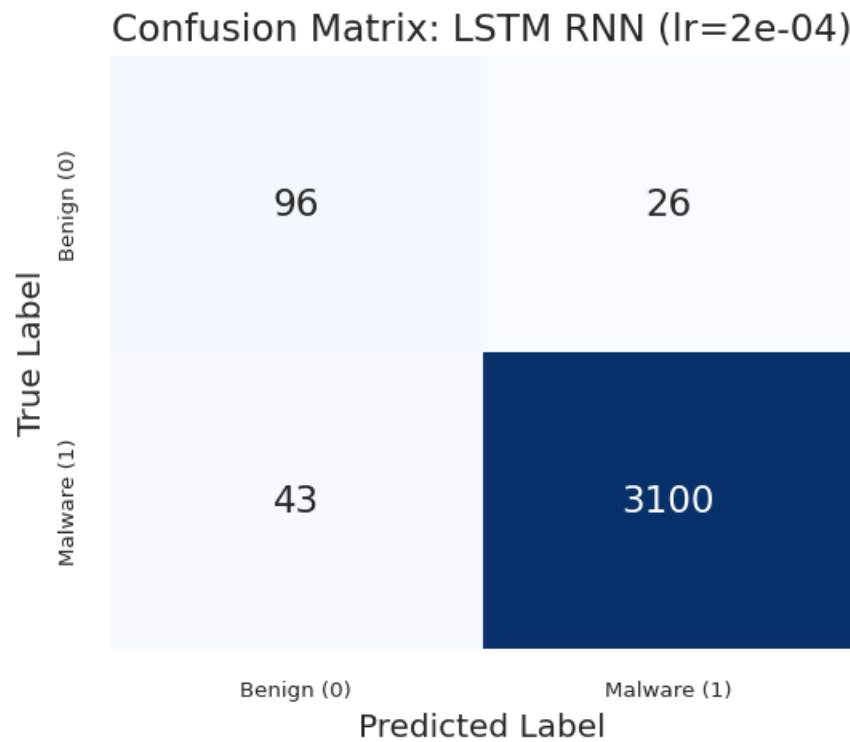
Generating confusion matrix for LSTM RNN with lr=1e-04...

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/cm_lstm_rnn(lr=1e_04).png



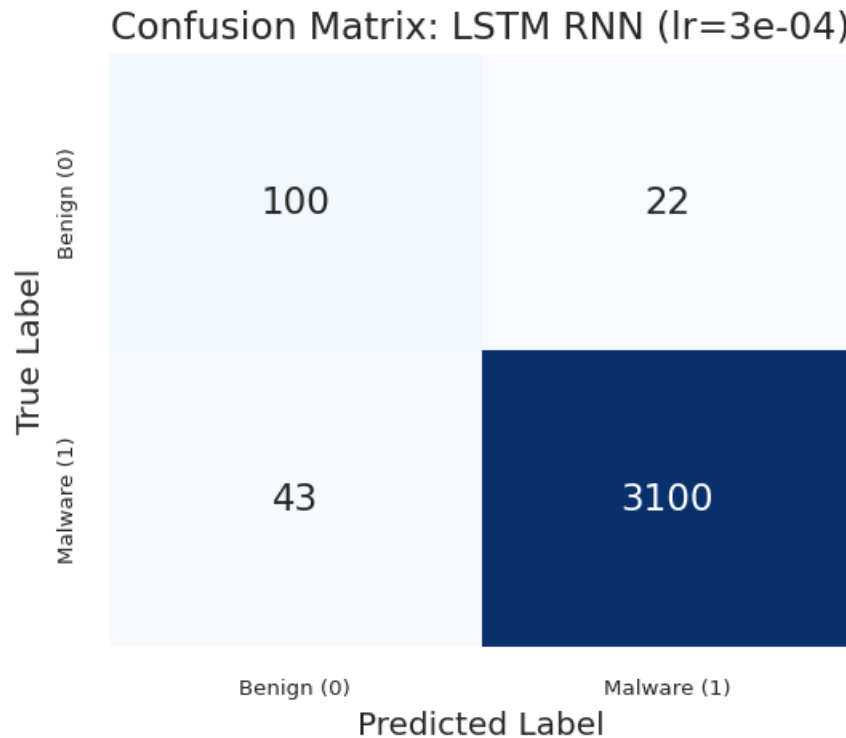
Generating confusion matrix for LSTM RNN with lr=2e-04...

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/cm_lstm_rnn(lr=2e_04).png



Generating confusion matrix for LSTM RNN with lr=3e-04...

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/cm_lstm_rnn_(lr=3e_04).png



```
[114]: # --- Plot confusion matrices for ALL learning rates - TEST ---

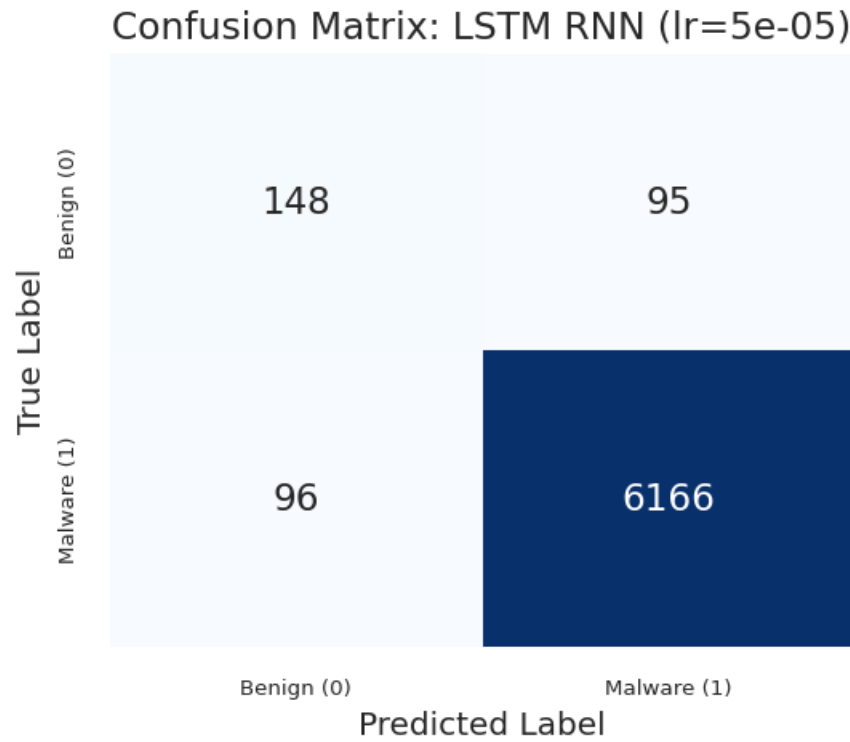
print("\n" + "=" * 80)
print("Confusion matrices for ALL LSTM RNN learning rates - TEST")
print("=" * 80)

for lr in learning_rates_lstm:
    model = models_lstm[lr]
    print(f"\nGenerating confusion matrix for LSTM RNN with lr={lr:.0e}...")

    plot_cm_task3(model=model, loader=test_loader, model_name=f"LSTM RNN_
↳(lr={lr:.0e})", save_dir=save_dir)
```

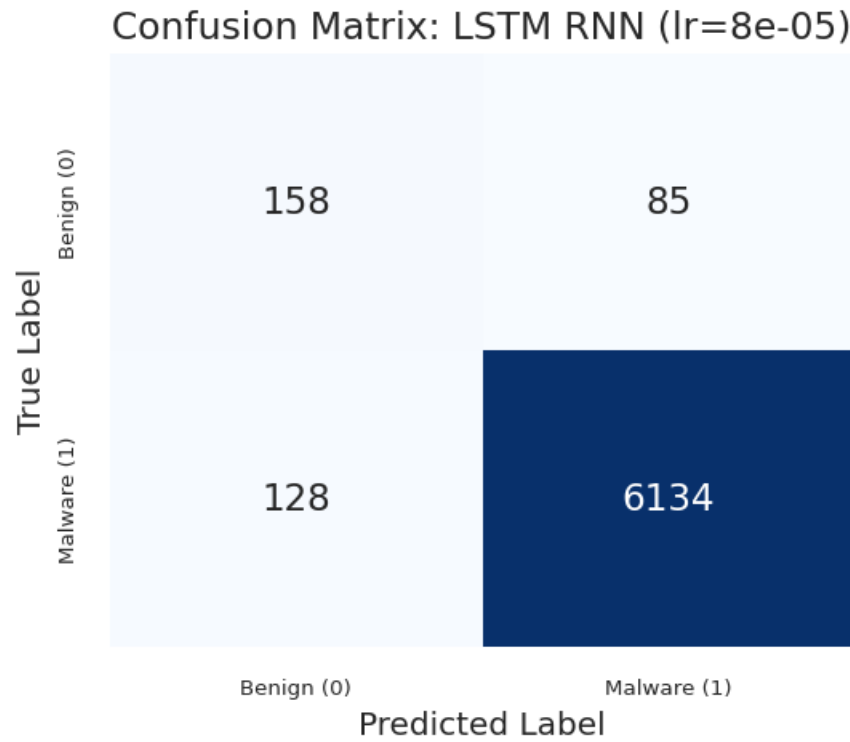
```
=====
Confusion matrices for ALL LSTM RNN learning rates - TEST
=====
```

```
Generating confusion matrix for LSTM RNN with lr=5e-05...
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task3_plots/cm_lstm_rnn_(lr=5e_05).png
```



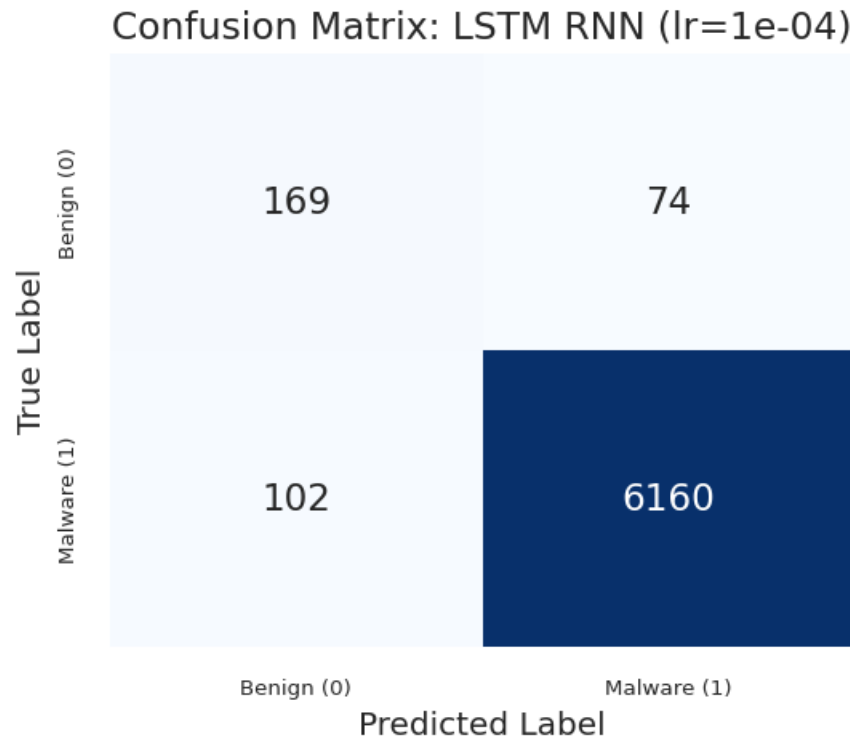
Generating confusion matrix for LSTM RNN with lr=8e-05...

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/cm_lstm_rnn_(lr=8e_05).png



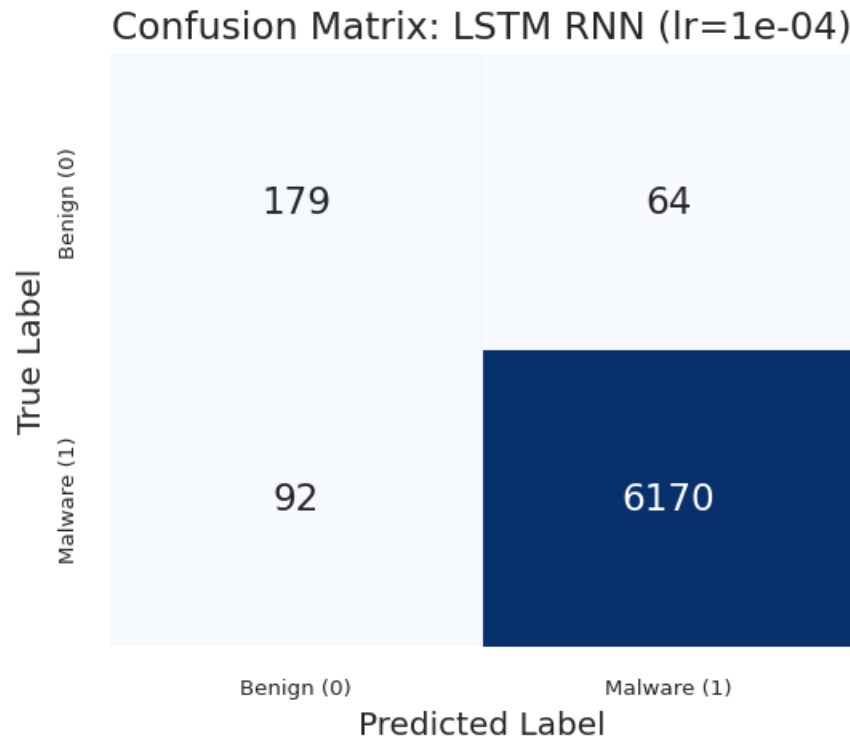
Generating confusion matrix for LSTM RNN with lr=1e-04...

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/cm_lstm_rnn_(lr=1e_04).png



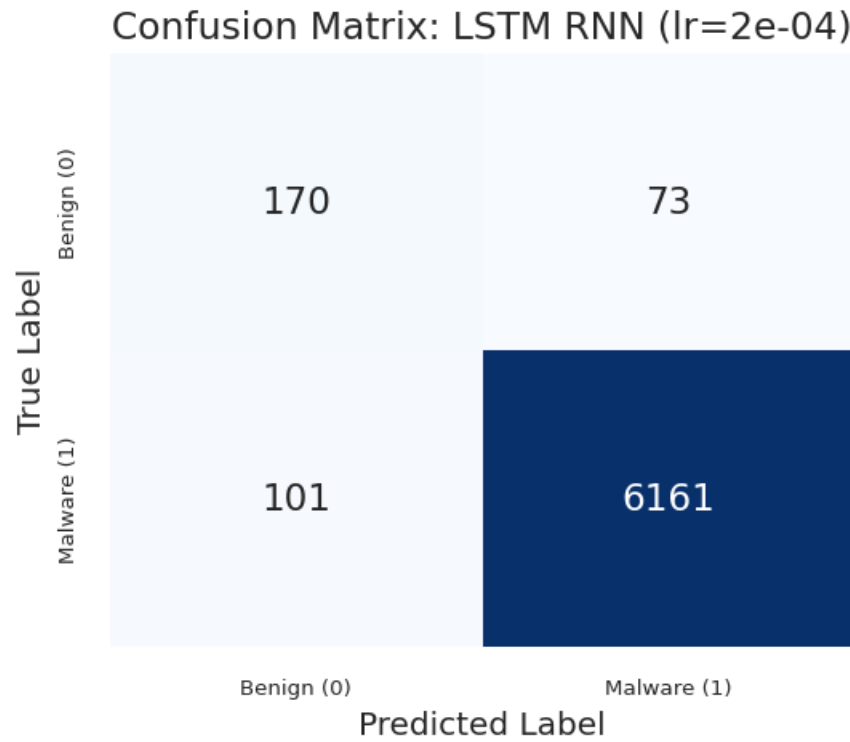
Generating confusion matrix for LSTM RNN with lr=1e-04...

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/cm_lstm_rnn_(lr=1e_04).png



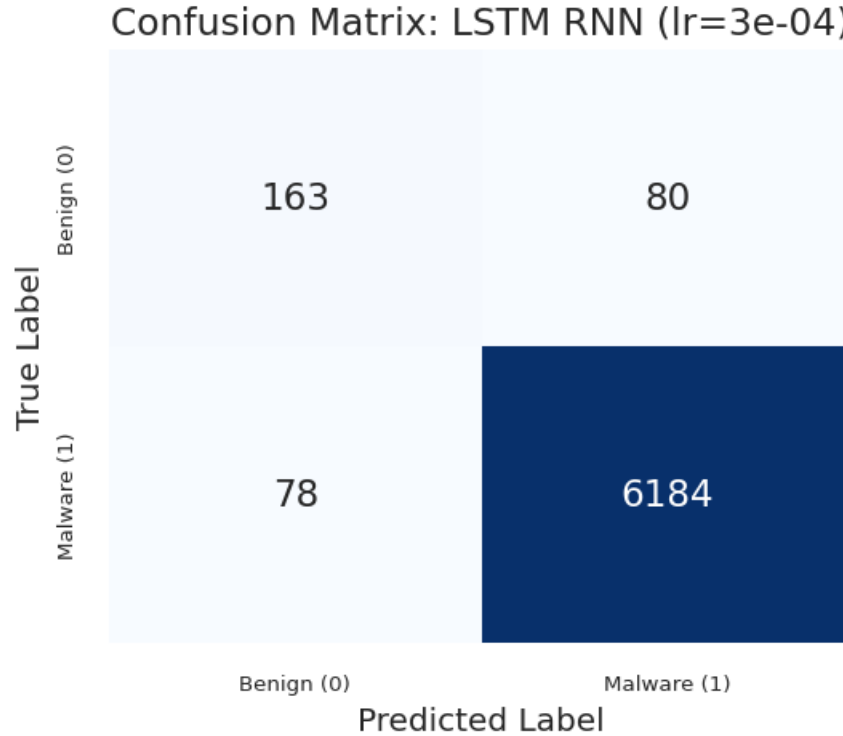
Generating confusion matrix for LSTM RNN with lr=2e-04...

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/cm_lstm_rnn_(lr=2e_04).png



Generating confusion matrix for LSTM RNN with lr=3e-04...

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/cm_lstm_rnn(lr=3e_04).png



Q: Is the RNN training as stable as the FFNN's one? No. RNN training is less stable than FFNN training. The FFNN validation-loss curves are comparatively smoother, while the recurrent models show more oscillations and stronger sensitivity to the learning rate.

This happens because recurrent models backpropagate through time, producing a deeper computational graph whose effective depth depends on the sequence length. This increases the risk of vanishing or unstable gradients, especially for vanilla RNNs.

Among the recurrent architectures, the bidirectional LSTM is the most stable and effective in our experiments. With the common selection protocol based on validation macro F1, it selects $lr=3e-4$ and reaches validation macro F1 0.8722, higher than the selected Simple RNN (0.8601) and Bi-RNN (0.8222). Its checkpoint validation loss is also the lowest among the selected recurrent models (0.0437). The gating mechanism of the LSTM helps regulate what information is kept or forgotten, making optimization more reliable than in vanilla recurrent variants.

Q: How does your model's performance compare to the simple frequency baseline? The frequency baseline remains very competitive in accuracy, reaching 96.97% on the test set, but its macro F1 is 0.7223 and its benign recall is only 0.3457. This means that it is excellent at recognizing malware (0.9939 recall), but it misclassifies many benign samples as malware.

The recurrent models improve the balance between the two classes. The selected Simple RNN reaches test accuracy 97.48%, macro F1 0.8247, and benign recall 0.6626, already improving strongly over the frequency baseline in macro F1. The selected Bi-RNN reaches test macro F1 0.8020, accuracy 96.91%, and benign recall 0.6749.

The selected bidirectional LSTM is the strongest recurrent model in this run. With $lr=3e-4$, it reaches test accuracy 97.57%, macro F1 0.8305, and benign recall 0.6708. It therefore beats the frequency baseline both in macro F1 and in accuracy, while providing a much better balance between benign and malware recognition. Overall, the recurrent results show that temporal order contains discriminative information that the frequency baseline cannot capture.

```
[115]: # =====
# FINAL SUMMARY
# =====

# Helper function to extract macro avg from the best model
def get_macro_avg_stats(model, loader):
    # Predict using your existing helper function
    y_true, y_pred = predict_loader(model, loader)
    # Get report as a DICTIONARY, not a string
    report_dict = classification_report(y_true, y_pred, output_dict=True,
    ↪ zero_division=0)
    return report_dict['macro avg']

# --- 1. Prepare Stats for Simple RNN ---
# Retrieve the actual model object using the best LR key
best_model_simple_obj = models_simple[best_lr_simple[0]]
macro_simple = get_macro_avg_stats(best_model_simple_obj, val_loader)

# --- 2. Prepare Stats for Bi-RNN ---
best_model_bi_obj = models_bi[best_lr_bi[0]]
macro_bi = get_macro_avg_stats(best_model_bi_obj, val_loader)

# --- 3. Prepare Stats for LSTM ---
best_model_lstm_obj = models_lstm[best_lr_lstm[0]]
macro_lstm = get_macro_avg_stats(best_model_lstm_obj, val_loader)

# --- PRINTING THE REPORT ---

print("\n" + "=" * 90)
print("TASK 3 SUMMARY: Models selected by validation macro F1")
print("=" * 90)
# Header for the manual table we are creating
print(f"{'Model':<20} | {'Best LR':<10} | {'Ckpt Loss':<10} | {'Macro P':<8} | "
    ↪ {'Macro R':<8} {'Macro F1':<8} | {'Time':<10}")
print("-" * 90)

# 1. Simple RNN
t_simple = time.strftime('%H:%M:%S', time.
    ↪ gmtime(elapsed_simple[best_lr_simple[0]]))
print(f"{'Simple RNN':<20} | {best_lr_simple[0]:.0e} | "
    ↪ {best_lr_simple[1]['best_val_loss_at_checkpoint']:.4f} | "
```

```

        f"{macro_simple['precision']:.4f}    {macro_simple['recall']:.4f}    ␣
↪{macro_simple['f1-score']:.4f}    | {t_simple}")

# 2. Bi-RNN
t_bi = time.strftime('%H:%M:%S', time.gmtime(elapsed_bi[best_lr_bi[0]]))
print(f"'Bidirectional RNN':<20} | {best_lr_bi[0]:.0e}    |␣
↪{best_lr_bi[1]['best_val_loss_at_checkpoint']:.4f}    | "
        f"{macro_bi['precision']:.4f}    {macro_bi['recall']:.4f}    ␣
↪{macro_bi['f1-score']:.4f}    | {t_bi}")

# 3. LSTM
t_lstm = time.strftime('%H:%M:%S', time.gmtime(elapsed_lstm[best_lr_lstm[0]]))
print(f"'Bi-LSTM':<20} | {best_lr_lstm[0]:.0e}    |␣
↪{best_lr_lstm[1]['best_val_loss_at_checkpoint']:.4f}    | "
        f"{macro_lstm['precision']:.4f}    {macro_lstm['recall']:.4f}    ␣
↪{macro_lstm['f1-score']:.4f}    | {t_lstm}")

```

```

=====
=====
TASK 3 SUMMARY: Models selected by validation macro F1
=====
=====
Model                | Best LR    | Ckpt Loss  | Macro P   Macro R   Macro F1 |
Time
-----
Simple RNN           | 2e-04      | 0.0567     | 0.8533    0.8672    0.8601    |
00:05:19
Bidirectional RNN    | 1e-04      | 0.0498     | 0.7882    0.8667    0.8222    |
00:08:09
Bi-LSTM              | 3e-04      | 0.0437     | 0.8461    0.9030    0.8722    |
00:08:05

```

1.5 Task 4 - Graph Neural Network (GNN)

In this task we model API-call sequences as graphs and train GNN architectures (GCN, GraphSAGE, GAT) to classify samples. We reuse the lab preprocessing and reporting utilities for consistency.

```

[116]: # Create directory for plots
save_dir = results_path + 'images/' + 'task4_plots/'
os.makedirs(save_dir, exist_ok=True)

```

1.5.1 Modeling API Sequences as Graphs

```
[117]: df_train
```

```
[117]:
```

		api_call_sequence	is_malware	\
0		[LdrGetDllHandle, LdrGetProcedureAddress, LdrL...	1	
1		[NtAllocateVirtualMemory, LdrLoadDll, LdrGetPr...	1	
2		[FindResourceExW, LoadResource, FindResourceEx...	1	
3		[FindResourceExW, LoadResource, FindResourceEx...	1	
4		[LdrGetProcedureAddress, SetErrorMode, LdrLoad...	1	
...		...	...	
16320		[LdrGetProcedureAddress, LdrLoadDll, LdrGetPro...	1	
16321		[NtClose, LdrGetProcedureAddress, CryptCreateH...	1	
16322		[LdrGetProcedureAddress, LdrGetDllHandle, LdrG...	1	
16323		[LdrGetProcedureAddress, LdrGetDllHandle, LdrG...	1	
16324		[SetFilePointer, NtReadFile, SetFilePointer, N...	1	

	seq_length		seq_fixed	\
0	73	[LdrGetDllHandle, LdrGetProcedureAddress, LdrL...		
1	88	[NtAllocateVirtualMemory, LdrLoadDll, LdrGetPr...		
2	79	[FindResourceExW, LoadResource, FindResourceEx...		
3	71	[FindResourceExW, LoadResource, FindResourceEx...		
4	63	[LdrGetProcedureAddress, SetErrorMode, LdrLoad...		
...	...	...	...	
16320	64	[LdrGetProcedureAddress, LdrLoadDll, LdrGetPro...		
16321	78	[NtClose, LdrGetProcedureAddress, CryptCreateH...		
16322	84	[LdrGetProcedureAddress, LdrGetDllHandle, LdrG...		
16323	62	[LdrGetProcedureAddress, LdrGetDllHandle, LdrG...		
16324	77	[SetFilePointer, NtReadFile, SetFilePointer, N...		

	seq_sequential_ids
0	[117, 118, 119, 118, 117, 118, 86, 123, 201, 2...
1	[133, 119, 118, 82, 80, 137, 82, 152, 153, 163...
2	[46, 121, 46, 121, 46, 121, 46, 121, 46, 121, ...
3	[46, 121, 46, 121, 46, 121, 46, 121, 46, 121, ...
4	[118, 219, 119, 219, 81, 46, 121, 46, 121, 46,...
...	...
16320	[118, 119, 118, 119, 118, 119, 118, 119, 118, ...
16321	[134, 118, 24, 118, 30, 118, 135, 66, 221, 166...
16322	[118, 117, 118, 81, 216, 119, 118, 46, 121, 46...
16323	[118, 117, 118, 117, 118, 117, 118, 117, 118, ...
16324	[221, 166, 221, 166, 221, 45, 121, 133, 146, 1...

[16325 rows x 5 columns]

```
[118]: df_train_split, df_val = train_test_split(
        df_train,
        test_size=0.2,
```

```

    random_state=42,
    stratify=df_train['is_malware'], # Preserve class balance
    shuffle=True
)

```

```

[119]: # Use original api_call_sequence (not truncated seq_sequential_ids)
# GNNs can handle variable-length graphs, so we use full sequences
X_train_split = df_train_split['api_call_sequence'].values
y_train_split = df_train_split['is_malware'].values

X_val = df_val['api_call_sequence'].values
y_val = df_val['is_malware'].values

X_test = df_test['api_call_sequence'].values
y_test = df_test['is_malware'].values

print(f"Train length: {len(X_train_split)}")
print(f"Val length: {len(X_val)}")
print(f"Test length: {len(X_test)}")

```

```

Train length: 13060
Val length: 3265
Test length: 6505

```

```

[120]: # --- Graph Construction ---

def create_graph_from_sequence(sequence, api_to_id, label, window_size=1):
    """
    Create a sparse graph containing ONLY the nodes present in the sequence.
    Accepts API name strings and converts them to integer IDs using api_to_id_
    ↪mapping.
    Unknown APIs are mapped to UNK_IDX (index 1).
    """
    UNK_IDX = 1 # Same as defined in Task 3

    # 1. Convert API names to integer IDs (handle unknown APIs with UNK_IDX)
    sequence_ids = [api_to_id.get(api, UNK_IDX) for api in sequence]

    # 2. Identify unique node IDs present in this sequence
    # IMPORTANT: Filter out padding (0) before creating node set
    unique_api_ids = sorted(list(set([api_id for api_id in sequence_ids if
    ↪api_id != 0])))
    num_nodes = len(unique_api_ids)

    # 3. Create a mapping from Global API ID -> Local Graph Node Index
    global_to_local = {global_id: local_id for local_id, global_id in
    ↪enumerate(unique_api_ids)}

```

```

    # 4. Create Node Features (The Global IDs to be fed into the Embedding
    ↪Layer)
    x = torch.tensor(unique_api_ids, dtype=torch.long).unsqueeze(1)

    # 5. Create Edges (Transition Graph)
    src_list = []
    dst_list = []

    # Sliding window to find transitions
    for i in range(len(sequence_ids) - window_size):
        global_src = sequence_ids[i]
        global_dst = sequence_ids[i + window_size]

        # Skip padding (0) if present
        if global_src == 0 or global_dst == 0:
            continue

        # Map to local indices
        src_list.append(global_to_local[global_src])
        dst_list.append(global_to_local[global_dst])

    edge_index = torch.tensor([src_list, dst_list], dtype=torch.long)

    data = Data(
        x=x,
        edge_index=edge_index,
        y=torch.tensor([label], dtype=torch.float32),
        num_nodes=num_nodes
    )

    return data

def create_graphs_from_sequences(sequences, api_to_id, labels, window_size=1):
    graph_data_list = []
    for sequence, label in zip(sequences, labels):
        # Ensure sequence is a list (handles numpy arrays)
        seq_list = sequence if isinstance(sequence, list) else list(sequence)
        data = create_graph_from_sequence(seq_list, api_to_id, label,
        ↪window_size)
        graph_data_list.append(data)

    # Sanity check for each graph
    if data.edge_index.numel() > 0 and data.edge_index.max().item() >= data.
    ↪num_nodes:
        print(f"Warning: Graph {idx} has invalid edge index!")

```

```

    return graph_data_list

# --- Execute Creation ---
print("Creating sparse graphs...")
train_graphs = create_graphs_from_sequences(X_train_split, api_to_id,
    ↪y_train_split)
val_graphs = create_graphs_from_sequences(X_val, api_to_id, y_val)
test_graphs = create_graphs_from_sequences(X_test, api_to_id, y_test)

print(f"Training graphs: {len(train_graphs)}")
print(f"First graph nodes: {train_graphs[0].num_nodes} (Subset of total vocab)")
print(f"First graph edges: {train_graphs[0].num_edges}")

```

Creating sparse graphs...

Training graphs: 13060

First graph nodes: 21 (Subset of total vocab)

First graph edges: 80

```

[121]: import networkx as nx
import matplotlib.pyplot as plt
from torch_geometric.utils import to_networkx

def visualize_graph(data, id_to_api_mapping=None):
    # Convert PyG data to NetworkX
    G = to_networkx(data, to_undirected=False) # Keep it directed to see flow

    plt.figure(figsize=(12, 8))

    # Layout (Spring layout tends to show clusters and loops well)
    pos = nx.spring_layout(G, seed=42)

    # Draw nodes
    nx.draw_networkx_nodes(G, pos, node_size=800, node_color='lightblue',
    ↪alpha=0.8)

    # Draw edges (with arrows)
    nx.draw_networkx_edges(G, pos, width=1.0, alpha=0.5, arrowstyle='->',
    ↪arrowsize=20)

    # Labels: If you have the mapping, show API names. Otherwise show IDs.
    if id_to_api_mapping:
        # Get the Global API ID from data.x for each local node index
        labels = {i: id_to_api_mapping.get(data.x[i].item(), str(i)) for i in
    ↪range(data.num_nodes)}
        nx.draw_networkx_labels(G, pos, labels, font_size=10,
    ↪font_weight="bold")
    else:

```



```

nx.draw_networkx_labels(G, pos)

plt.title(f"Malware Execution Graph\nLabel: {data.y.item()} (1=Malware, 0=Benign)")
plt.axis('off')
plt.show()

# --- RUN THE CHECK ---
# Plot example graphs from each dataset
print("Plotting example graphs...\n")
print("Training set example:")
visualize_graph(train_graphs[0], id_to_api)

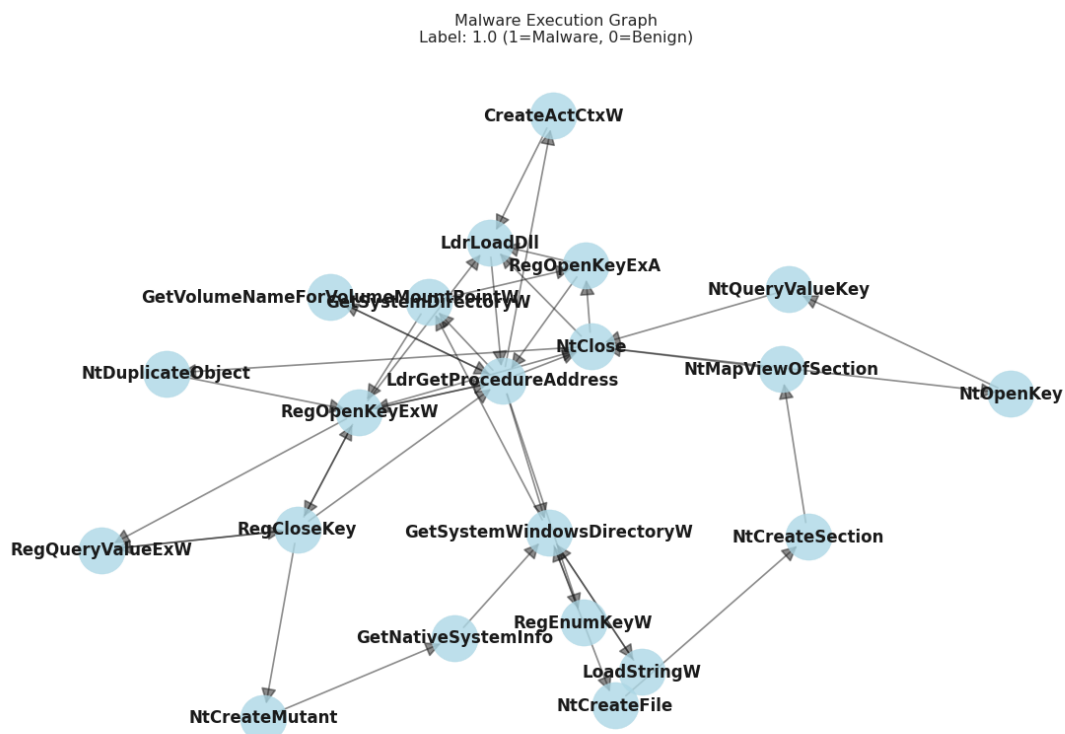
print("\nValidation set example:")
visualize_graph(val_graphs[0], id_to_api)

print("\nTest set example:")
visualize_graph(test_graphs[0], id_to_api)

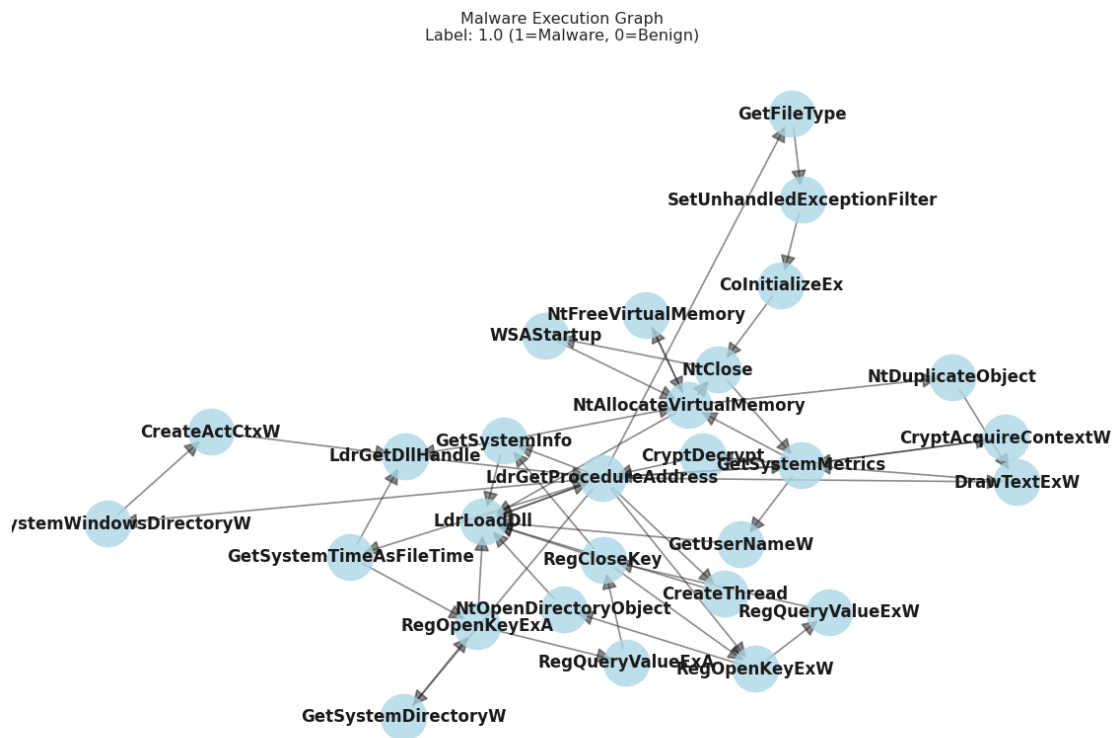
```

Plotting example graphs...

Training set example:

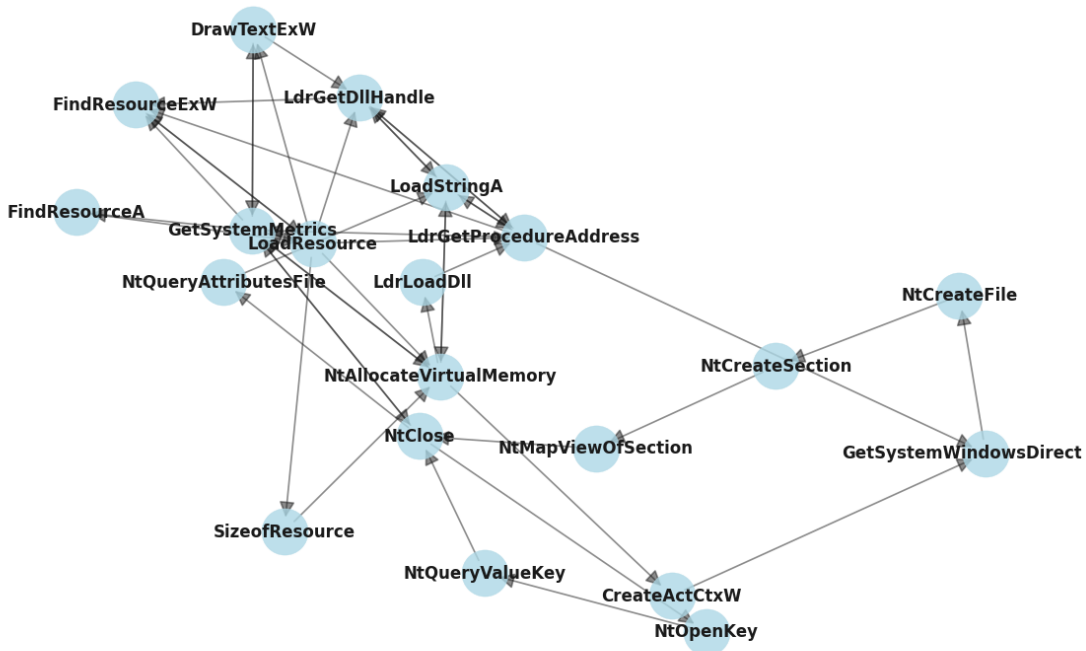


Validation set example:



Test set example:

Malware Execution Graph
Label: 1.0 (1=Malware, 0=Benign)



```

[122]: data = train_graphs[0]

print(f"1. Node Features Shape: {data.x.shape}")
print(f"    → {data.num_nodes} unique APIs in this sequence")

print(f"\n2. Edge Index Shape: {data.edge_index.shape}")
print(f"    → {data.num_edges} transitions (edges) in the sequence")

print(f"\n3. Max Node Index: {data.edge_index.max().item()}")
print(f"    → Valid range: [0, {data.num_nodes - 1}]")
if data.edge_index.numel() > 0:
    is_valid = data.edge_index.max().item() < data.num_nodes
    print(f"    → {' VALID' if is_valid else ' INVALID - edges point to_"}
    ↪non-existent nodes'})")
else:
    print(f"    → No edges in this graph")

print(f"\n4. Label Shape: {data.y.shape}")
print(f"    → Label value: {data.y.item()} ({'Malware' if data.y.item() == 1_"}
    ↪else 'Benign'})")

print(f"\n5. Summary:")

```

```
print(f"    → Graph has {data.num_nodes} nodes and {data.num_edges} edges")
print(f"    → Sparsity: {(data.num_edges / (data.num_nodes * (data.num_nodes - 1))) * 100:.2f}% (if fully connected)")
```

1. Node Features Shape: torch.Size([21, 1])
→ 21 unique APIs in this sequence
2. Edge Index Shape: torch.Size([2, 80])
→ 80 transitions (edges) in the sequence
3. Max Node Index: 20
→ Valid range: [0, 20]
→ VALID
4. Label Shape: torch.Size([1])
→ Label value: 1.0 (Malware)
5. Summary:
→ Graph has 21 nodes and 80 edges
→ Sparsity: 19.05% (if fully connected)

```
[123]: # Get the raw sequence and the graph
raw_seq_ids = X_train_split[0] # The list of IDs: [117, 118, 119...]
graph = train_graphs[0]

# Check 1: Unique counts
unique_apis_raw = len(set(raw_seq_ids))
# Padding (0) is usually removed in your graph logic, so:
if 0 in raw_seq_ids: unique_apis_raw -= 1

print(f"Unique APIs in raw sequence: {unique_apis_raw}")
print(f"Nodes in Graph: {graph.num_nodes}")
# THESE SHOULD MATCH EXACTLY.

# Check 2: Transitions vs Edges
# Raw edges = length of sequence - 1 (minus padding transitions)
# Note: Your graph combines duplicate edges.
# If raw sequence is A->B->A->B, graph has edges (A,B) and (B,A).
# This check is qualitative: Does graph.num_edges roughly track with sequence
# complexity?
print(f"Raw sequence length: {len([x for x in raw_seq_ids if x != 0])}")
print(f"Graph edges: {graph.num_edges}")
```

```
Unique APIs in raw sequence: 21
Nodes in Graph: 21
Raw sequence length: 81
Graph edges: 80
```

Q: Do you still have to pad your data? If yes, how? No. The graph-based representation eliminates the need for padding. Each API sequence is converted into its own graph, where nodes correspond to the unique API calls appearing in the sequence and edges represent transitions between consecutive calls.

Longer sequences simply create more transitions and possibly more nodes. Shorter sequences create smaller graphs. PyTorch Geometric can batch these variable-sized graphs by merging them into a larger disconnected graph and using a `batch` vector to keep track of graph membership.

As a result, no dummy padding nodes or padding edges are required. The model works directly on the sparse graph structure of each execution trace.

Q: Do you have to truncate the testing sequences? Justify your answer. No. GNNs do not require test-time truncation because there is no fixed-size input vector that would overflow when a sequence is longer than expected. A longer sequence simply generates additional edges, and possibly additional nodes, in its graph representation.

Truncating a test sequence would remove transitions from the graph and would therefore discard part of the behavioral structure we are trying to model. Since the GNN can process variable-sized graphs through PyTorch Geometric batching and graph-level pooling, keeping the full test sequence is the correct choice.

Q: What is the advantage of modeling your problem with a GNN compared to an RNN? What do you lose? The advantage of a GNN is that it models the API trace as a transition structure rather than only as a left-to-right chain. If an API call repeatedly transitions to several other calls, or if a small cycle appears multiple times, the GNN can aggregate information from those local neighborhoods and learn structural motifs that may be discriminative for malware detection.

This representation also handles variable-length sequences naturally: graphs can have different numbers of nodes and edges, and batching does not require padding or truncation. Message passing can summarize repeated transition patterns and relationships between API calls in a way that is different from the hidden-state memory of an RNN.

What we lose is strict chronological information. After converting the sequence into a graph and applying graph-level pooling, the model knows that certain API calls are connected, but it does not know exactly whether a transition happened early or late in the trace. RNNs preserve this temporal evolution explicitly through their hidden states, while GNNs emphasize transition structure.

1.5.2 GNN Training and Helper Functions

We need new training and prediction functions, as the `PyGDataLoader` returns batches differently from the standard `DataLoader`.

```
[124]: # Create DataLoader for batching
train_loader = PyGDataLoader(train_graphs, batch_size=32, shuffle=True)
val_loader = PyGDataLoader(val_graphs, batch_size=32, shuffle=False)
test_loader = PyGDataLoader(test_graphs, batch_size=32, shuffle=False)
```

```
print(f"Train loader: {len(train_loader)}, Validation loader: {len(val_loader)}, Test loader: {len(test_loader)}")
```

Train loader: 409, Validation loader: 103, Test loader: 204

```
[125]: # =====
# GNN TRAINING FUNCTION (with Macro F1 tracking)
# =====

def train_gnn_common(model, train_loader, val_loader, optimizer, criterion,
    epochs=50, min_delta=0.00001, patience=20):
    """
    Train a GNN model with early stopping and track macro F1 score.
    """
    device = next(model.parameters()).device

    best_val = math.inf
    best_state = None
    trigger = 0
    history = {'train_loss': [], 'val_loss': [], 'val_f1_macro': []}
    start_time = time.time()

    for epoch in range(1, epochs + 1):
        model.train()
        train_losses = []
        for data in train_loader:
            data = data.to(device)
            optimizer.zero_grad()
            outputs = model(data)
            loss = criterion(outputs, data.y.unsqueeze(1).float())
            loss.backward()
            optimizer.step()
            train_losses.append(loss.item())
        train_loss = float(np.mean(train_losses)) if train_losses else 0.0

        # Validation
        model.eval()
        val_losses = []
        all_preds = []
        all_targets = []
        with torch.no_grad():
            for data in val_loader:
                data = data.to(device)
                outputs = model(data)
                loss = criterion(outputs, data.y.unsqueeze(1).float())
                val_losses.append(loss.item())
```

```

        # Collect predictions for F1 score
        # FIX: use view(-1) instead of squeeze() to handle
↪single-element batches
        probs = torch.sigmoid(outputs).view(-1)
        preds = (probs > 0.5).long()

        # FIX: convert to list properly to handle both single and
↪multi-element tensors
        all_preds.extend(preds.cpu().tolist())
        all_targets.extend(data.y.cpu().tolist())

    val_loss = float(np.mean(val_losses)) if val_losses else 0.0
    val_f1_macro = f1_score(all_targets, all_preds, average='macro',
↪zero_division=0)

    history['train_loss'].append(train_loss)
    history['val_loss'].append(val_loss)
    history['val_f1_macro'].append(val_f1_macro)

    if (epoch % 10 == 0) or (epoch == 1) or (epoch == epochs):
        print(f'Epoch {epoch}/{epochs} - Train Loss: {train_loss:.4f} - Val
↪Loss: {val_loss:.4f} - Val F1: {val_f1_macro:.4f}')

    # Early stopping logic
    if val_loss < best_val - min_delta:
        best_val = val_loss
        best_state = {k: v.cpu().clone() for k, v in model.state_dict().
↪items()}
        trigger = 0
    else:
        trigger += 1
        if trigger >= patience:
            print(f'Early stopping at epoch {epoch} (best val loss
↪{best_val:.6f})')
            break

    elapsed = time.time() - start_time

    if best_state is not None:
        model.load_state_dict(best_state)
        model.to(device)

    return model, history, elapsed

```

[126]: # --- GNN Prediction Helper ---

```
def predict_loader_gnn(model, loader, threshold=0.5):
```

```

model.eval()

# Detect device from model parameters
device = next(model.parameters()).device

ys_true = []
ys_pred = []
with torch.no_grad():
    for data in loader:
        # Move batch to device
        data = data.to(device)

        out = model(data)
        probs = torch.sigmoid(out).cpu().numpy().ravel()
        preds = (probs >= threshold).astype(int)

        # Get true labels from the DataBatch object
        ys_true.extend(data.y.cpu().numpy().ravel().astype(int).tolist())
        ys_pred.extend(preds.tolist())

return np.array(ys_true, dtype=int), np.array(ys_pred, dtype=int)

```

```

[127]: def plot_cm_task4(model, loader, model_name, save_dir=None):
        """
        Specific for GNNs. Uses your existing 'predict_loader_gnn' logic.
        """

        # Use the GNN specific helper
        y_true, y_pred = predict_loader_gnn(model, loader)

        # Plot
        _internal_plot_cm(y_true, y_pred, model_name, save_dir)

```

1.5.3 GNN Variations

Now we define and train the three GNN architectures as requested by the lab PDF. We will use the same hyperparameters (vocab_size, embed_dim, hidden_dim) for a fair comparison. We also re-use the pos_weight from Task 3 to handle the imbalanced dataset.

```

[128]: # Hyperparameters for all GNNs
VOCAB_SIZE_GNN = len(api_to_id)
EMBED_DIM_GNN = 32
HIDDEN_DIM_GNN = 64
OUTPUT_DIM_GNN = 1
EPOCHS_GNN = 100
PATIENCE_GNN = 15
MIN_DELTA_GNN = 0.0001
LR_GNN = 5e-4 # Learning rate

```



```

# Re-compute pos_weight for the GNN training subset
pos_weight_gnn = compute_pos_weight(y_train_split).to(device)

criterion_gnn = nn.BCEWithLogitsLoss(pos_weight=pos_weight_gnn)

def select_best_gnn_model(architecture_name, models, histories, elapsed_times,
    val_loader):
    """
    Select the best Task 4 model automatically.

    Primary criterion: validation macro F1 evaluated on the restored model.
    Tie-breakers: validation accuracy of the restored model, then minimum
    validation loss.

    This avoids manually hardcoding the selected learning rate.
    """
    selection_rows = []

    for lr_label, model in models.items():
        history = histories[lr_label]
        y_val_true, y_val_pred = predict_loader_gnn(model, val_loader)

        last_history_val_f1 = history['val_f1_macro'][-1] if history.
        get('val_f1_macro') else float('-inf')
        val_accuracy = accuracy_score(y_val_true, y_val_pred)
        eval_val_f1 = f1_score(y_val_true, y_val_pred, average='macro',
        zero_division=0)
        min_val_loss = min(history['val_loss']) if history.get('val_loss') else
        float('inf')

        selection_rows.append({
            'lr_label': lr_label,
            'last_history_val_f1_macro': last_history_val_f1,
            'val_accuracy': val_accuracy,
            'eval_val_f1_macro': eval_val_f1,
            'min_val_loss': min_val_loss,
            'elapsed_s': elapsed_times[lr_label],
        })

    selection_table = pd.DataFrame(selection_rows).sort_values(
        by=['eval_val_f1_macro', 'val_accuracy', 'min_val_loss'],
        ascending=[False, False, True]
    ).reset_index(drop=True)

    best_lr = selection_table.loc[0, 'lr_label']
    best_model = models[best_lr]

```

```

best_history = histories[best_lr]
best_elapsed = elapsed_times[best_lr]

print(f"Automatic selection for {architecture_name}")
print("Ranking criterion: restored-model validation macro F1, then
↪ validation accuracy, then minimum validation loss")
print(selection_table.to_string(index=False))
print(f"\nSelected best {architecture_name} model: {best_lr}")
print(f"Restored-model Val F1 Macro: {selection_table.loc[0,
↪ 'eval_val_f1_macro']:.4f}")
print(f"Validation Accuracy: {selection_table.loc[0, 'val_accuracy']:.4f}")

return best_lr, best_model, best_history, best_elapsed, selection_table

print(f"GNN Vocab Size: {VOCAB_SIZE_GNN}")
print(f"GNN Positive Weight: {pos_weight_gnn.item():.4f}") # Use .item() for
↪ scalar tensor print

```

Negatives: 489, Positives: 12571
 pos_weight (applied to y=1 samples): 0.2000
 GNN Vocab Size: 260
 GNN Positive Weight: 0.2000

1. Simple GCN This model uses Graph Convolutional Networks (GCN).

```

[129]: # --- 1. GCN Model Definition ---

class GCNmodel(torch.nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim, output_dim):
        super(GCNmodel, self).__init__()

        # Embedding: Maps Global API IDs -> Dense Vector
        self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=0)

        # GCN Layers
        self.conv1 = GCNConv(embed_dim, hidden_dim)
        self.conv2 = GCNConv(hidden_dim, hidden_dim)
        self.conv3 = GCNConv(hidden_dim, hidden_dim)

        # Pooling
        self.pool = global_mean_pool

        # Classifier
        self.classifier = nn.Sequential(
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),

```

```

        nn.Dropout(0.5),
        nn.Linear(hidden_dim, output_dim)
    )

    def forward(self, data):
        x, edge_index, batch = data.x, data.edge_index, data.batch

        # x comes in as [num_nodes_in_batch, 1]
        x = x.squeeze(1) # [num_nodes_in_batch]

        # 1. Embedding
        x = self.embedding(x) # [num_nodes, embed_dim]

        # 2. GCN
        x = self.conv1(x, edge_index)
        x = F.relu(x)
        x = F.dropout(x, p=0.2, training=self.training)

        x = self.conv2(x, edge_index)
        x = F.relu(x)
        x = F.dropout(x, p=0.2, training=self.training)

        x = self.conv3(x, edge_index)
        x = F.relu(x)
        x = F.dropout(x, p=0.2, training=self.training)

        # 3. Global Pooling
        # Averages node vectors belonging to the same graph
        x = self.pool(x, batch)

        # 4. Classification
        out = self.classifier(x)
        return out

```

```

[130]: # =====
# GCN: Training with Multiple Learning Rates
# =====

gcn_learning_rates = [1e-4, 5e-4, 1e-3]
gcn_lr_labels = ['LR=1e-4', 'LR=5e-4', 'LR=1e-3']

# Storage for results
gcn_histories = {}
gcn_models = {}
gcn_elapsed_times = {}

for lr, lr_label in zip(gcn_learning_rates, gcn_lr_labels):

```

```

print(f"\n{'='*60}")
print(f"Training GCN with {lr_label}")
print('='*60)

# Set seed for reproducibility
torch.manual_seed(42)
np.random.seed(42)

# Create model
model_gcn_temp = GCNmodel(
    vocab_size=VOCAB_SIZE_GNN,
    embed_dim=EMBED_DIM_GNN,
    hidden_dim=HIDDEN_DIM_GNN,
    output_dim=OUTPUT_DIM_GNN
).to(device)

optimizer_temp = optim.AdamW(model_gcn_temp.parameters(), lr=lr)

model_gcn_temp, history_temp, elapsed_temp = train_gnn_common(
    model_gcn_temp,
    train_loader,
    val_loader,
    optimizer_temp,
    criterion_gnn,
    epochs=EPOCHS_GNN,
    min_delta=MIN_DELTA_GNN,
    patience=PATIENCE_GNN
)

gcn_models[lr_label] = model_gcn_temp
gcn_histories[lr_label] = history_temp
gcn_elapsed_times[lr_label] = elapsed_temp

elapsed_str = time.strftime('%H:%M:%S', time.gmtime(elapsed_temp))
print(f'\n{lr_label} - Training time: {elapsed_str}')

```

```

=====
Training GCN with LR=1e-4
=====
Epoch 1/100 - Train Loss: 0.1204 - Val Loss: 0.1005 - Val F1: 0.4905
Epoch 10/100 - Train Loss: 0.0745 - Val Loss: 0.0731 - Val F1: 0.6072
Epoch 20/100 - Train Loss: 0.0622 - Val Loss: 0.0666 - Val F1: 0.6562
Epoch 30/100 - Train Loss: 0.0579 - Val Loss: 0.0632 - Val F1: 0.6853
Epoch 40/100 - Train Loss: 0.0539 - Val Loss: 0.0607 - Val F1: 0.6803
Epoch 50/100 - Train Loss: 0.0499 - Val Loss: 0.0581 - Val F1: 0.7115
Epoch 60/100 - Train Loss: 0.0476 - Val Loss: 0.0566 - Val F1: 0.7114
Epoch 70/100 - Train Loss: 0.0447 - Val Loss: 0.0553 - Val F1: 0.7363

```

Epoch 80/100 - Train Loss: 0.0417 - Val Loss: 0.0529 - Val F1: 0.7317
 Epoch 90/100 - Train Loss: 0.0405 - Val Loss: 0.0514 - Val F1: 0.7377
 Epoch 100/100 - Train Loss: 0.0385 - Val Loss: 0.0518 - Val F1: 0.7461

LR=1e-4 - Training time: 00:05:38

=====
 Training GCN with LR=5e-4
 =====

Epoch 1/100 - Train Loss: 0.1031 - Val Loss: 0.0874 - Val F1: 0.4905
 Epoch 10/100 - Train Loss: 0.0530 - Val Loss: 0.0579 - Val F1: 0.7034
 Epoch 20/100 - Train Loss: 0.0387 - Val Loss: 0.0513 - Val F1: 0.7426
 Epoch 30/100 - Train Loss: 0.0324 - Val Loss: 0.0511 - Val F1: 0.7985
 Epoch 40/100 - Train Loss: 0.0266 - Val Loss: 0.0466 - Val F1: 0.7715
 Early stopping at epoch 49 (best val loss 0.044381)

LR=5e-4 - Training time: 00:02:45

=====
 Training GCN with LR=1e-3
 =====

Epoch 1/100 - Train Loss: 0.0957 - Val Loss: 0.0754 - Val F1: 0.5865
 Epoch 10/100 - Train Loss: 0.0466 - Val Loss: 0.0548 - Val F1: 0.7390
 Epoch 20/100 - Train Loss: 0.0302 - Val Loss: 0.0562 - Val F1: 0.8099
 Epoch 30/100 - Train Loss: 0.0251 - Val Loss: 0.0593 - Val F1: 0.7954
 Epoch 40/100 - Train Loss: 0.0208 - Val Loss: 0.0643 - Val F1: 0.8281
 Early stopping at epoch 43 (best val loss 0.047833)

LR=1e-3 - Training time: 00:02:26

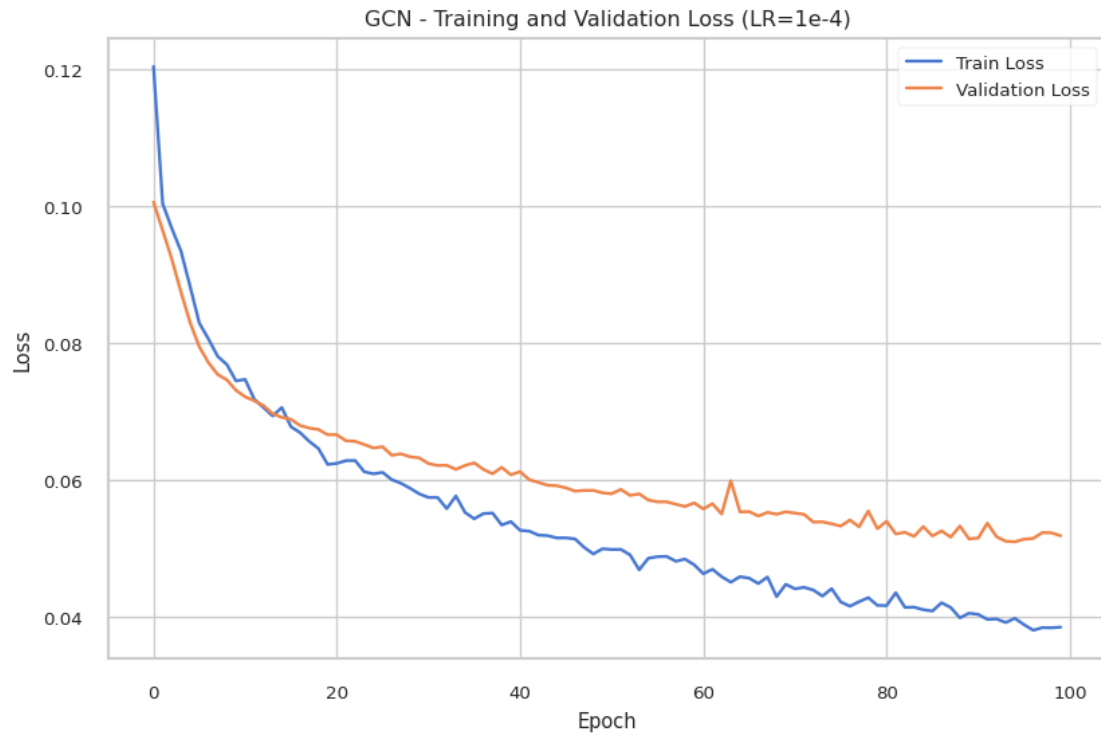
```
[131]: # =====
# Plot Training/Validation Loss per OGNI learning rate (GCN)
# =====

for lr_label, history in gcn_histories.items():
    plt.figure(figsize=(8, 5))
    plt.plot(history['train_loss'], label='Train Loss')
    plt.plot(history['val_loss'], label='Validation Loss')
    plt.title(f'GCN - Training and Validation Loss ({lr_label})')
    plt.xlabel('Epoch')
    plt.ylabel('Loss')
    plt.legend()
    plt.grid(True)

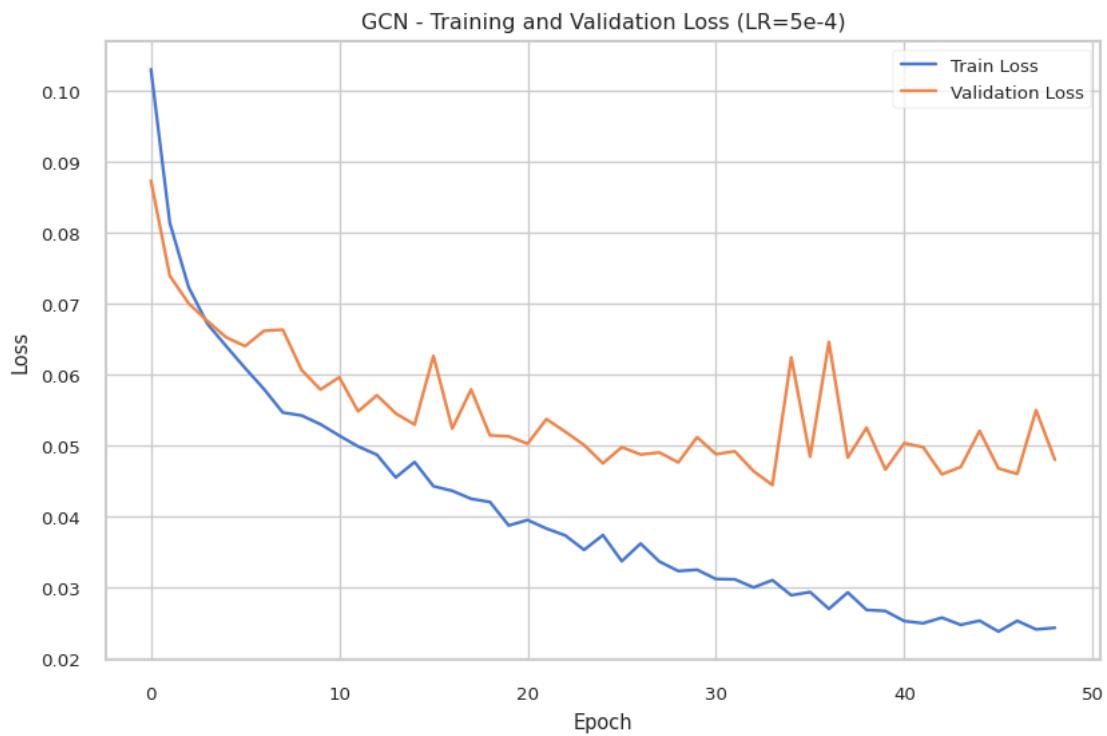
    # Nome file unico per ogni lr
    save_plot(plt.gcf(), f'task4_gcn_loss_curves_{lr_label.replace("=", "")}',
    ↪save_dir)
```

```
plt.show()
```

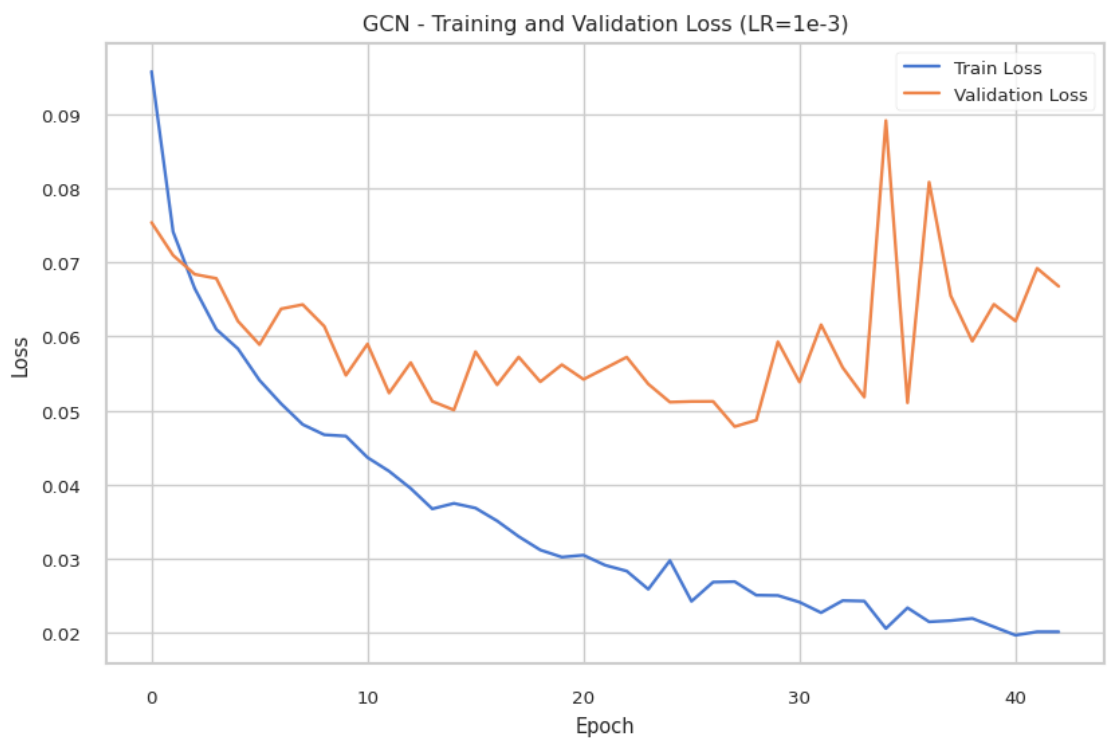
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task4_plots/task4_gcn_loss_curves_LR1e-4.png



Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task4_plots/task4_gcn_loss_curves_LR5e-4.png



Saved plot: `/content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task4_plots/task4_gcn_loss_curves_LR1e-3.png`



```
[132]: # =====
# GCN: Plot Learning Rate Comparison (Validation Loss + Macro F1)
# =====

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

colors = ['#1b9e77', '#d95f02', '#7570b3']

# Plot Validation Loss
for (lr_label, history), color in zip(gcn_histories.items(), colors):
    epochs_range = range(1, len(history['val_loss']) + 1)
    ax1.plot(epochs_range, history['val_loss'], label=lr_label,
             linewidth=REPORT_PLOT_LINEWIDTH, color=color)

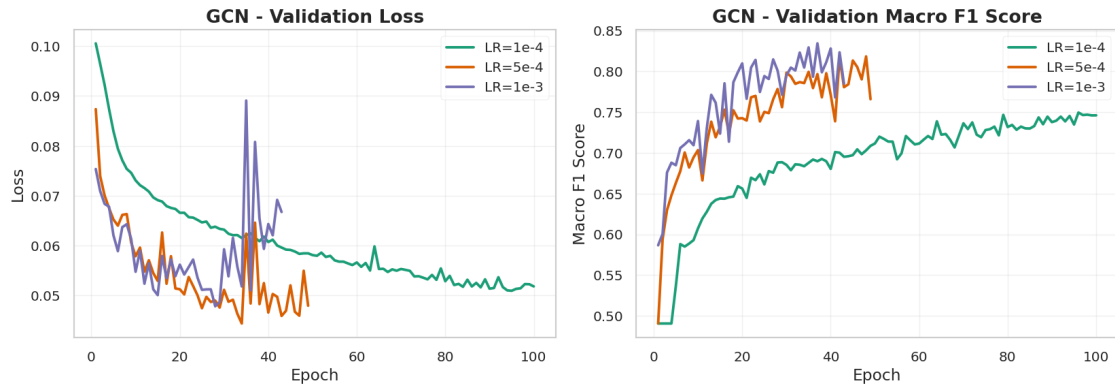
ax1.set_title('GCN - Validation Loss', fontsize=REPORT_PLOT_TITLE_SIZE,
             fontweight='bold')
ax1.set_xlabel('Epoch', fontsize=REPORT_PLOT_LABEL_SIZE)
ax1.set_ylabel('Loss', fontsize=REPORT_PLOT_LABEL_SIZE)
format_report_axes(ax1)
ax1.grid(alpha=0.3)

# Plot Validation Macro F1
for (lr_label, history), color in zip(gcn_histories.items(), colors):
    epochs_range = range(1, len(history['val_f1_macro']) + 1)
    ax2.plot(epochs_range, history['val_f1_macro'], label=lr_label,
             linewidth=REPORT_PLOT_LINEWIDTH, color=color)

ax2.set_title('GCN - Validation Macro F1 Score',
             fontsize=REPORT_PLOT_TITLE_SIZE, fontweight='bold')
ax2.set_xlabel('Epoch', fontsize=REPORT_PLOT_LABEL_SIZE)
ax2.set_ylabel('Macro F1 Score', fontsize=REPORT_PLOT_LABEL_SIZE)
format_report_axes(ax2)
ax2.grid(alpha=0.3)

plt.tight_layout()
save_plot(plt.gcf(), f"task4_gcn_lr_comparison", save_dir)
plt.show()
```

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task4_plots/task4_gcn_lr_comparison.png



```
[133]: # --- Evaluate GCN ---
for lr_label, model_gcn in gcn_models.items():
    print(f"\n{'='*60}")
    print(f"Evaluation for GCN - {lr_label}")
    print(f"{'='*60}")

    y_val_true_gcn, y_val_pred_gcn = predict_loader_gnn(model_gcn, val_loader)
    print('\nValidation report (GCN):')
    print(classification_report(y_val_true_gcn, y_val_pred_gcn, digits=4,
↪zero_division=0))

    y_test_true_gcn, y_test_pred_gcn = predict_loader_gnn(model_gcn,
↪test_loader)
    print('Test report (GCN):')
    print(classification_report(y_test_true_gcn, y_test_pred_gcn, digits=4,
↪zero_division=0))
```

```
=====
Evaluation for GCN - LR=1e-4
=====
```

Validation report (GCN):

	precision	recall	f1-score	support
0	0.4009	0.7295	0.5174	122
1	0.9892	0.9577	0.9732	3143
accuracy			0.9492	3265
macro avg	0.6950	0.8436	0.7453	3265
weighted avg	0.9672	0.9492	0.9561	3265

Test report (GCN):

	precision	recall	f1-score	support
0	0.3892	0.6502	0.4869	243
1	0.9861	0.9604	0.9731	6262
accuracy			0.9488	6505
macro avg	0.6876	0.8053	0.7300	6505
weighted avg	0.9638	0.9488	0.9549	6505

=====

Evaluation for GCN - LR=5e-4

=====

Validation report (GCN):

	precision	recall	f1-score	support
0	0.4752	0.7869	0.5926	122
1	0.9915	0.9663	0.9787	3143
accuracy			0.9596	3265
macro avg	0.7334	0.8766	0.7857	3265
weighted avg	0.9722	0.9596	0.9643	3265

Test report (GCN):

	precision	recall	f1-score	support
0	0.4724	0.7037	0.5653	243
1	0.9883	0.9695	0.9788	6262
accuracy			0.9596	6505
macro avg	0.7303	0.8366	0.7720	6505
weighted avg	0.9690	0.9596	0.9634	6505

=====

Evaluation for GCN - LR=1e-3

=====

Validation report (GCN):

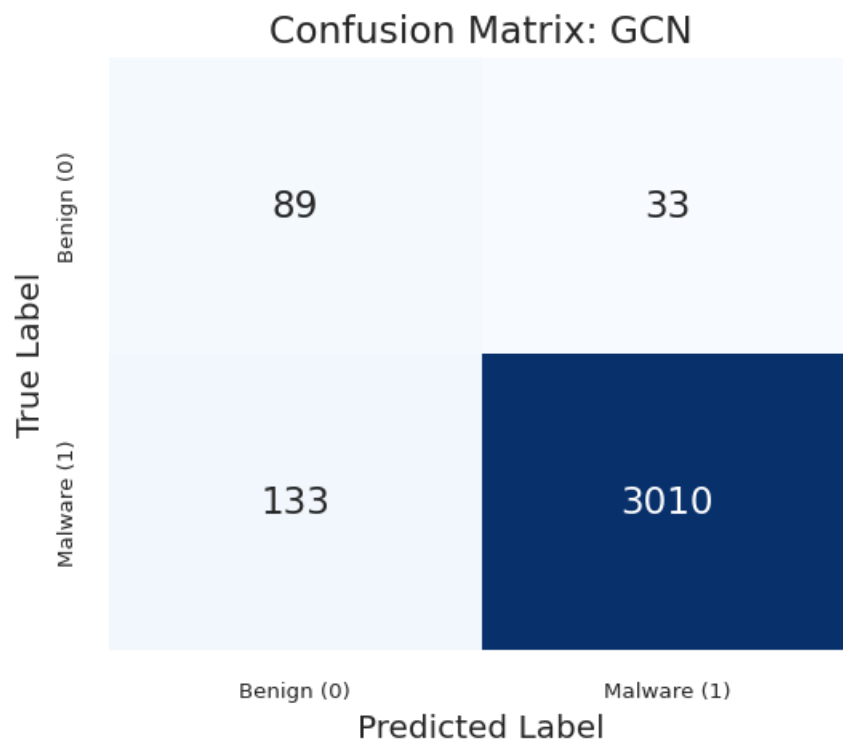
	precision	recall	f1-score	support
0	0.5193	0.7705	0.6205	122
1	0.9909	0.9723	0.9815	3143
accuracy			0.9648	3265
macro avg	0.7551	0.8714	0.8010	3265
weighted avg	0.9733	0.9648	0.9680	3265

Test report (GCN):

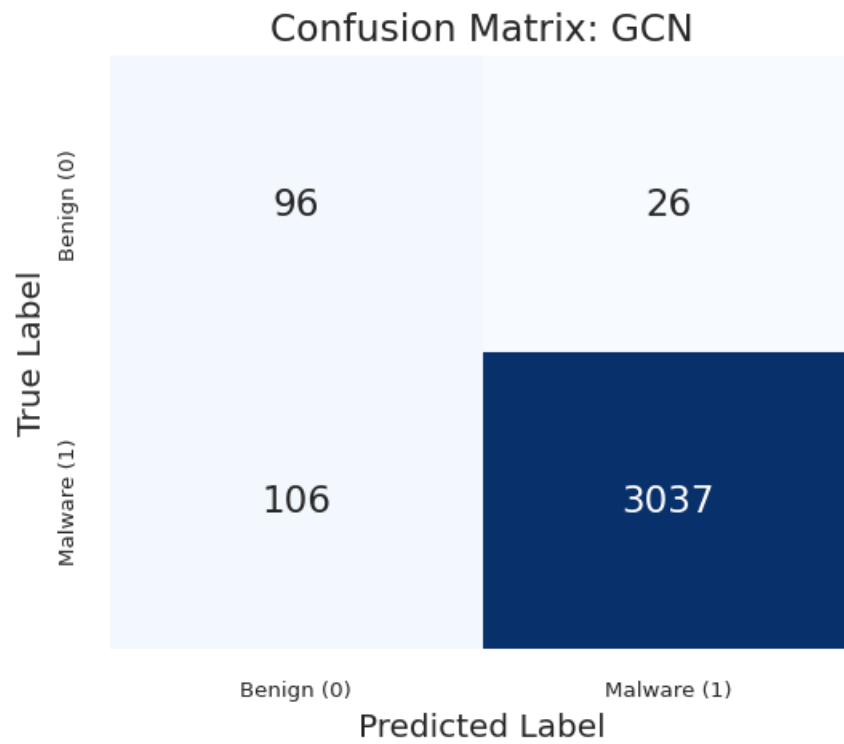
	precision	recall	f1-score	support
0	0.4788	0.6502	0.5515	243
1	0.9862	0.9725	0.9793	6262
accuracy			0.9605	6505
macro avg	0.7325	0.8114	0.7654	6505
weighted avg	0.9673	0.9605	0.9634	6505

```
[134]: for lr_label, model_gcn in gcn_models.items():  
        plot_cm_task4(model=model_gcn,  
                      loader=val_loader,  
                      model_name="GCN",  
                      save_dir=results_path + 'images/task4_plots/')
```

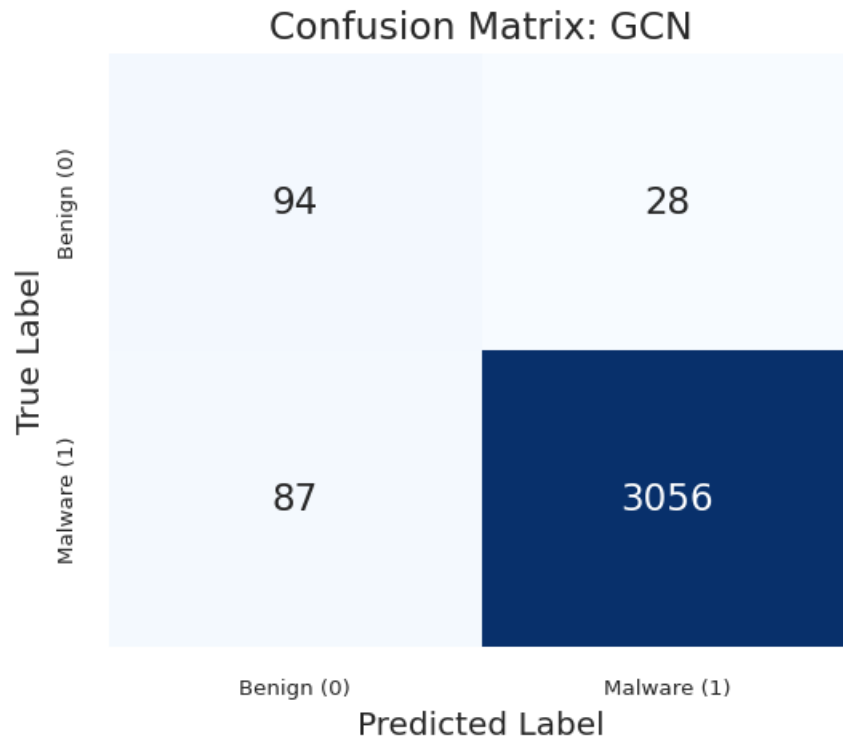
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task4_plots/cm_gcn.png



Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task4_plots/cm_gcn.png



Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task4_plots/cm_gcn.png



```
[135]: # =====
# GCN: Select Best Model and Evaluate
# =====

best_gcn_lr, model_gcn, history_gcn, elapsed_gcn, gcn_selection_table = \
    select_best_gnn_model(
        architecture_name='GCN',
        models=gcn_models,
        histories=gcn_histories,
        elapsed_times=gcn_elapsed_times,
        val_loader=val_loader,
    )

# Evaluate selected model
y_val_true_gcn, y_val_pred_gcn = predict_loader_gnn(model_gcn, val_loader)
print('\nValidation report (GCN):')
print(classification_report(y_val_true_gcn, y_val_pred_gcn, digits=4, \
    zero_division=0))

y_test_true_gcn, y_test_pred_gcn = predict_loader_gnn(model_gcn, test_loader)
print('Test report (GCN):')
```

```
print(classification_report(y_test_true_gcn, y_test_pred_gcn, digits=4,
↪zero_division=0))
```

Automatic selection for GCN

Ranking criterion: restored-model validation macro F1, then validation accuracy, then minimum validation loss

lr_label	last_history_val_f1_macro	val_accuracy	eval_val_f1_macro
min_val_loss	elapsed_s		
LR=1e-3	0.784958	0.964778	0.800997
0.047833	146.011812		
LR=5e-4	0.765904	0.959571	0.785661
0.044381	165.100300		
LR=1e-4	0.746085	0.949158	0.745304
0.050943	338.551973		

Selected best GCN model: LR=1e-3

Restored-model Val F1 Macro: 0.8010

Validation Accuracy: 0.9648

Validation report (GCN):

	precision	recall	f1-score	support
0	0.5193	0.7705	0.6205	122
1	0.9909	0.9723	0.9815	3143
accuracy			0.9648	3265
macro avg	0.7551	0.8714	0.8010	3265
weighted avg	0.9733	0.9648	0.9680	3265

Test report (GCN):

	precision	recall	f1-score	support
0	0.4788	0.6502	0.5515	243
1	0.9862	0.9725	0.9793	6262
accuracy			0.9605	6505
macro avg	0.7325	0.8114	0.7654	6505
weighted avg	0.9673	0.9605	0.9634	6505

2. GraphSAGE This model uses the GraphSAGE architecture

```
[136]: # --- 2. GraphSAGE Model Definition ---

class GraphSAGEmodel(torch.nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim, output_dim):
        super(GraphSAGEmodel, self).__init__()
```

```

self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=0)

# SAGEConv layers
self.conv1 = SAGEConv(embed_dim, hidden_dim)
self.conv2 = SAGEConv(hidden_dim, hidden_dim)
self.conv3 = SAGEConv(hidden_dim, hidden_dim)

self.pool = global_mean_pool # or global_max_pool

self.classifier = nn.Sequential(
    nn.Linear(hidden_dim, hidden_dim),
    nn.ReLU(),
    nn.Dropout(0.5),
    nn.Linear(hidden_dim, output_dim)
)

def forward(self, data):
    x, edge_index, batch = data.x, data.edge_index, data.batch

    x = self.embedding(x.squeeze(1))

    x = self.conv1(x, edge_index)
    x = F.relu(x)
    x = F.dropout(x, p=0.2, training=self.training)

    x = self.conv2(x, edge_index)
    x = F.relu(x)
    x = F.dropout(x, p=0.2, training=self.training)

    x = self.conv3(x, edge_index)
    x = F.relu(x)
    x = F.dropout(x, p=0.2, training=self.training)

    x = self.pool(x, batch)
    out = self.classifier(x)
    return out

```

```

[137]: # =====
# GraphSAGE: Training with Multiple Learning Rates
# =====

sage_learning_rates = [1e-4, 5e-4, 1e-3]
sage_lr_labels = ['LR=1e-4', 'LR=5e-4', 'LR=1e-3']

sage_histories = {}
sage_models = {}

```

```

sage_elapsed_times = {}

for lr, lr_label in zip(sage_learning_rates, sage_lr_labels):
    print(f"\n{'='*60}")
    print(f"Training GraphSAGE with {lr_label}")
    print('='*60)

    torch.manual_seed(42)
    np.random.seed(42)

    model_sage_temp = GraphSAGEmodel(
        vocab_size=VOCAB_SIZE_GNN,
        embed_dim=EMBED_DIM_GNN,
        hidden_dim=HIDDEN_DIM_GNN,
        output_dim=OUTPUT_DIM_GNN
    ).to(device)

    optimizer_temp = optim.AdamW(model_sage_temp.parameters(), lr=lr)

    model_sage_temp, history_temp, elapsed_temp = train_gnn_common(
        model_sage_temp,
        train_loader,
        val_loader,
        optimizer_temp,
        criterion_gnn,
        epochs=EPOCHS_GNN,
        min_delta=MIN_DELTA_GNN,
        patience=PATIENCE_GNN
    )

    sage_models[lr_label] = model_sage_temp
    sage_histories[lr_label] = history_temp
    sage_elapsed_times[lr_label] = elapsed_temp

    elapsed_str = time.strftime('%H:%M:%S', time.gmtime(elapsed_temp))
    print(f'\n{lr_label} - Training time: {elapsed_str}')

```

```

=====
Training GraphSAGE with LR=1e-4
=====

```

```

Epoch 1/100 - Train Loss: 0.1102 - Val Loss: 0.0954 - Val F1: 0.4905
Epoch 10/100 - Train Loss: 0.0657 - Val Loss: 0.0667 - Val F1: 0.6451
Epoch 20/100 - Train Loss: 0.0535 - Val Loss: 0.0608 - Val F1: 0.6635
Epoch 30/100 - Train Loss: 0.0474 - Val Loss: 0.0558 - Val F1: 0.7122
Epoch 40/100 - Train Loss: 0.0430 - Val Loss: 0.0543 - Val F1: 0.7385
Epoch 50/100 - Train Loss: 0.0386 - Val Loss: 0.0555 - Val F1: 0.6847
Epoch 60/100 - Train Loss: 0.0356 - Val Loss: 0.0525 - Val F1: 0.7131

```


Epoch 70/100 - Train Loss: 0.0322 - Val Loss: 0.0508 - Val F1: 0.7429
 Epoch 80/100 - Train Loss: 0.0297 - Val Loss: 0.0539 - Val F1: 0.7036
 Epoch 90/100 - Train Loss: 0.0267 - Val Loss: 0.0498 - Val F1: 0.7527
 Epoch 100/100 - Train Loss: 0.0268 - Val Loss: 0.0516 - Val F1: 0.7885

LR=1e-4 - Training time: 00:04:56

=====

Training GraphSAGE with LR=5e-4

=====

Epoch 1/100 - Train Loss: 0.0974 - Val Loss: 0.0829 - Val F1: 0.4905
 Epoch 10/100 - Train Loss: 0.0415 - Val Loss: 0.0532 - Val F1: 0.7214
 Epoch 20/100 - Train Loss: 0.0288 - Val Loss: 0.0516 - Val F1: 0.7899
 Epoch 30/100 - Train Loss: 0.0232 - Val Loss: 0.0531 - Val F1: 0.8222
 Epoch 40/100 - Train Loss: 0.0198 - Val Loss: 0.0604 - Val F1: 0.8294
 Early stopping at epoch 42 (best val loss 0.044889)

LR=5e-4 - Training time: 00:02:04

=====

Training GraphSAGE with LR=1e-3

=====

Epoch 1/100 - Train Loss: 0.0913 - Val Loss: 0.0793 - Val F1: 0.5415
 Epoch 10/100 - Train Loss: 0.0329 - Val Loss: 0.0463 - Val F1: 0.7581
 Epoch 20/100 - Train Loss: 0.0229 - Val Loss: 0.0540 - Val F1: 0.8315
 Early stopping at epoch 29 (best val loss 0.042609)

LR=1e-3 - Training time: 00:01:24

```
[138]: # =====
# GraphSAGE: Plot Learning Rate Comparison
# =====

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

colors = ['#377eb8', '#e41a1c', '#4daf4a']

for (lr_label, history), color in zip(sage_histories.items(), colors):
    epochs_range = range(1, len(history['val_loss']) + 1)
    ax1.plot(epochs_range, history['val_loss'], label=lr_label,
             linewidth=REPORT_PLOT_LINEWIDTH, color=color)

ax1.set_title('GraphSAGE - Validation Loss', fontsize=REPORT_PLOT_TITLE_SIZE,
             fontweight='bold')
ax1.set_xlabel('Epoch', fontsize=REPORT_PLOT_LABEL_SIZE)
ax1.set_ylabel('Loss', fontsize=REPORT_PLOT_LABEL_SIZE)
format_report_axes(ax1)
```

```

ax1.grid(alpha=0.3)

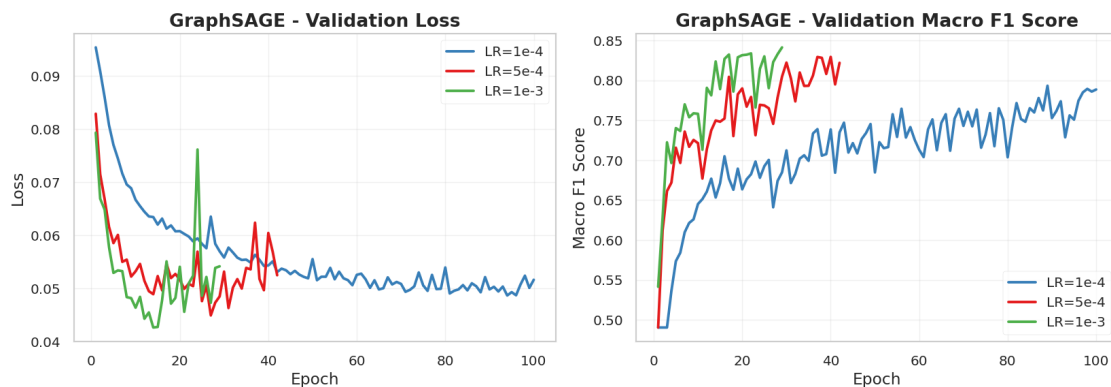
for (lr_label, history), color in zip(sage_histories.items(), colors):
    epochs_range = range(1, len(history['val_f1_macro']) + 1)
    ax2.plot(epochs_range, history['val_f1_macro'], label=lr_label,
             linewidth=REPORT_PLOT_LINEWIDTH, color=color)

ax2.set_title('GraphSAGE - Validation Macro F1 Score',
             linewidth=REPORT_PLOT_TITLE_SIZE, fontweight='bold')
ax2.set_xlabel('Epoch', fontsize=REPORT_PLOT_LABEL_SIZE)
ax2.set_ylabel('Macro F1 Score', fontsize=REPORT_PLOT_LABEL_SIZE)
format_report_axes(ax2)
ax2.grid(alpha=0.3)

plt.tight_layout()
save_plot(plt.gcf(), f"task4_sage_lr_comparison.png", save_dir)
plt.show()

```

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task4_plots/task4_sage_lr_comparison.png



```

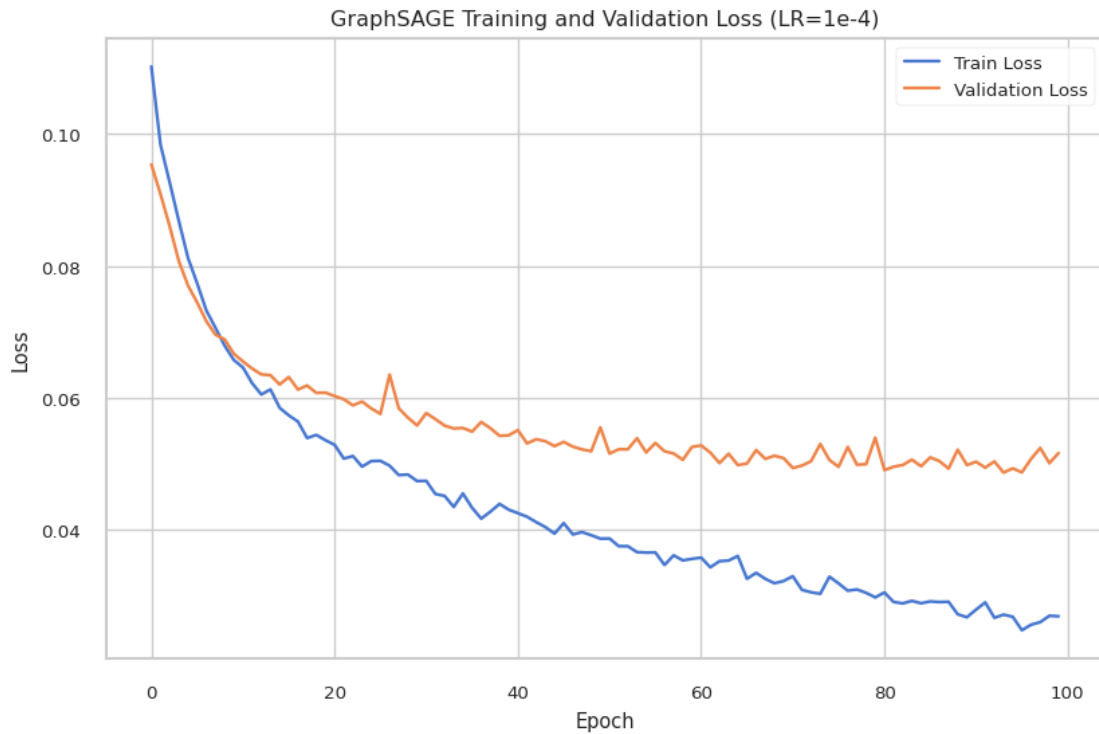
[139]: for lr_label, model_sage in sage_models.items():
        history = sage_histories[lr_label] # <-- Prendi la history per questo LR

        plt.figure(figsize=(8, 5))
        plt.plot(history['train_loss'], label='Train Loss')
        plt.plot(history['val_loss'], label='Validation Loss')
        plt.title(f'GraphSAGE Training and Validation Loss ({lr_label})')
        plt.xlabel('Epoch')
        plt.ylabel('Loss')
        plt.legend()
        plt.grid(True)

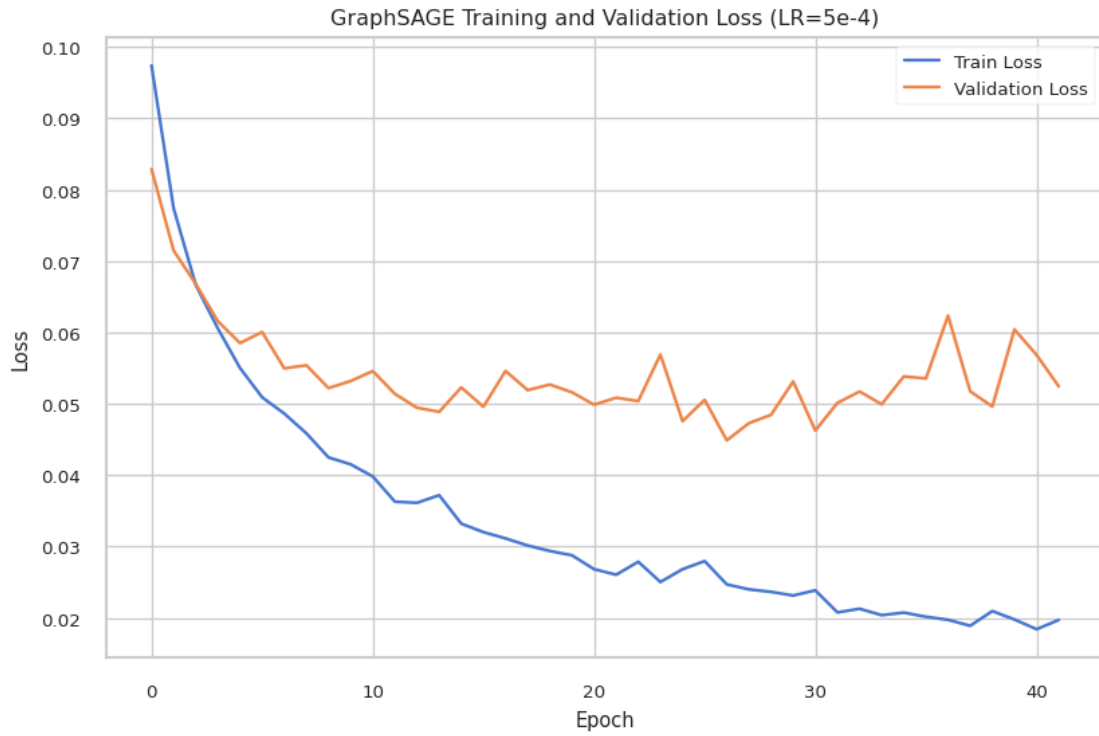
```

```
save_plot(plt.gcf(), f'graphsage_loss_curves_{lr_label.replace("=", "")}',  
↪save_dir)  
plt.show()
```

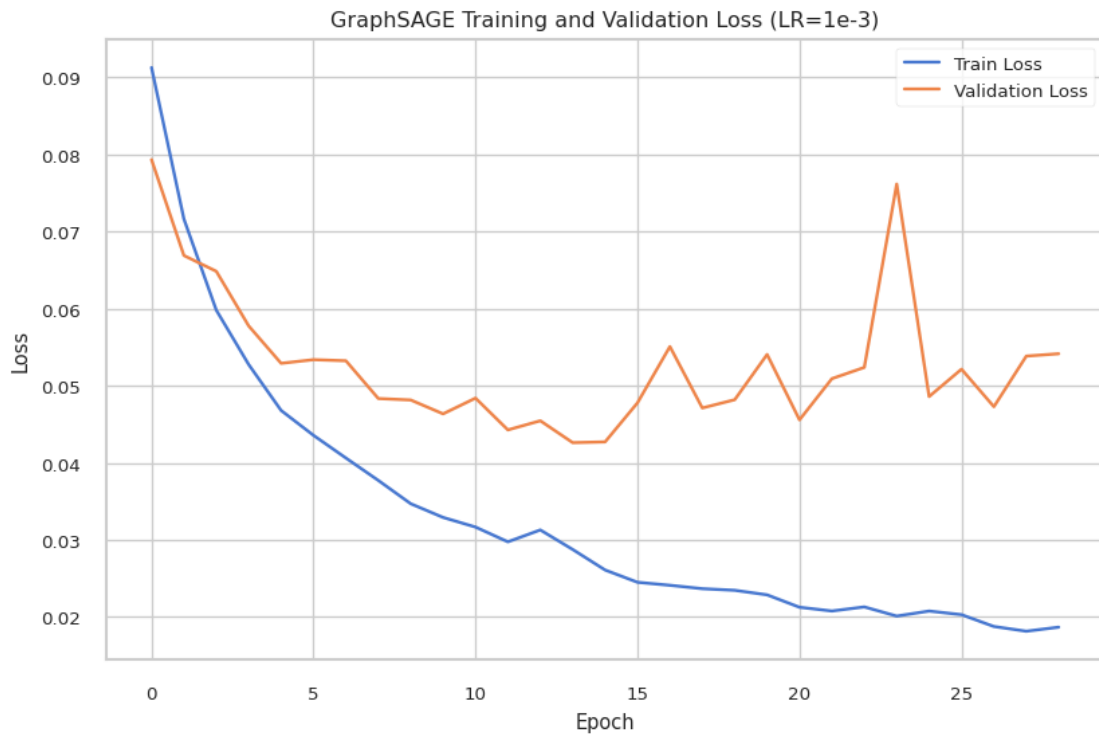
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task4_plots/graphsage_loss_curves_LR1e-4.png



Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task4_plots/graphsage_loss_curves_LR5e-4.png



Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task4_plots/graphsage_loss_curves_LR1e-3.png



```
[140]: # --- Evaluate GraphSAGE ---
for lr_label, model_sage in sage_models.items():
    y_val_true_sage, y_val_pred_sage = predict_loader_gnn(model_sage, val_loader)
    print('\nValidation report (GraphSAGE):')
    print(classification_report(y_val_true_sage, y_val_pred_sage, digits=4,
    ↪zero_division=0))

    y_test_true_sage, y_test_pred_sage = predict_loader_gnn(model_sage,
    ↪test_loader)
    print('Test report (GraphSAGE):')
    print(classification_report(y_test_true_sage, y_test_pred_sage, digits=4,
    ↪zero_division=0))
```

Validation report (GraphSAGE):

	precision	recall	f1-score	support
0	0.4041	0.8115	0.5395	122
1	0.9924	0.9535	0.9726	3143
accuracy			0.9482	3265
macro avg	0.6982	0.8825	0.7560	3265
weighted avg	0.9704	0.9482	0.9564	3265

Test report (GraphSAGE):

	precision	recall	f1-score	support
0	0.4311	0.7984	0.5599	243
1	0.9919	0.9591	0.9752	6262
accuracy			0.9531	6505
macro avg	0.7115	0.8787	0.7676	6505
weighted avg	0.9710	0.9531	0.9597	6505

Validation report (GraphSAGE):

	precision	recall	f1-score	support
0	0.3817	0.8197	0.5208	122
1	0.9927	0.9485	0.9701	3143
accuracy			0.9436	3265
macro avg	0.6872	0.8841	0.7454	3265
weighted avg	0.9698	0.9436	0.9533	3265

Test report (GraphSAGE):

	precision	recall	f1-score	support
0	0.4056	0.8395	0.5469	243
1	0.9935	0.9523	0.9724	6262
accuracy			0.9480	6505
macro avg	0.6995	0.8959	0.7597	6505
weighted avg	0.9715	0.9480	0.9565	6505

Validation report (GraphSAGE):

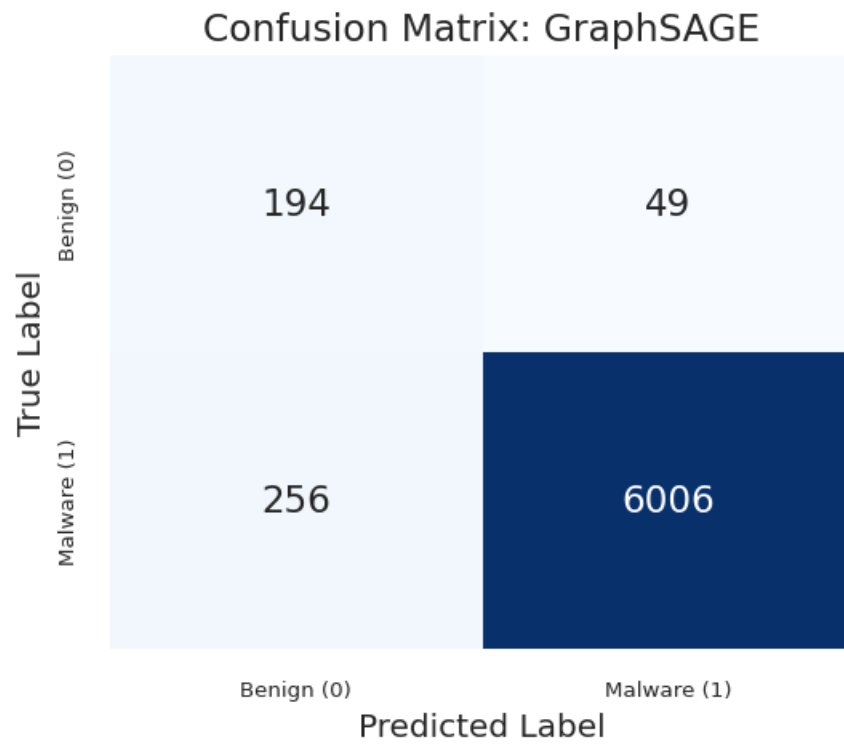
	precision	recall	f1-score	support
0	0.6054	0.7295	0.6617	122
1	0.9894	0.9815	0.9855	3143
accuracy			0.9721	3265
macro avg	0.7974	0.8555	0.8236	3265
weighted avg	0.9751	0.9721	0.9734	3265

Test report (GraphSAGE):

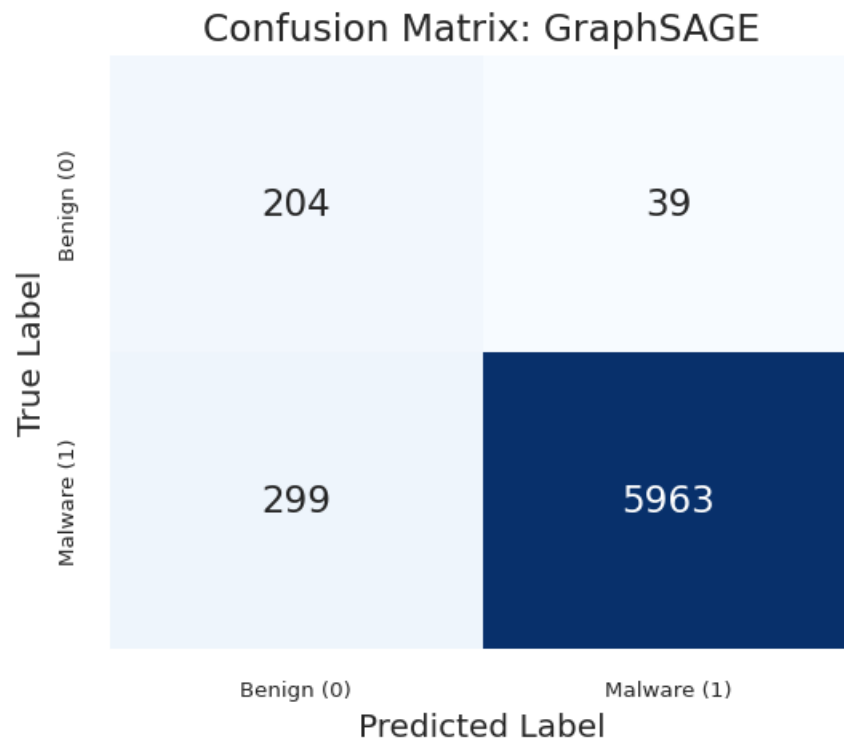
	precision	recall	f1-score	support
0	0.5813	0.6914	0.6316	243
1	0.9879	0.9807	0.9843	6262
accuracy			0.9699	6505
macro avg	0.7846	0.8360	0.8079	6505
weighted avg	0.9727	0.9699	0.9711	6505

```
[141]: for lr_label, model_sage in sage_models.items():
        plot_cm_task4(model=model_sage,
                      loader=test_loader,
                      model_name="GraphSAGE",
                      save_dir=results_path + 'images/task4_plots/')
```

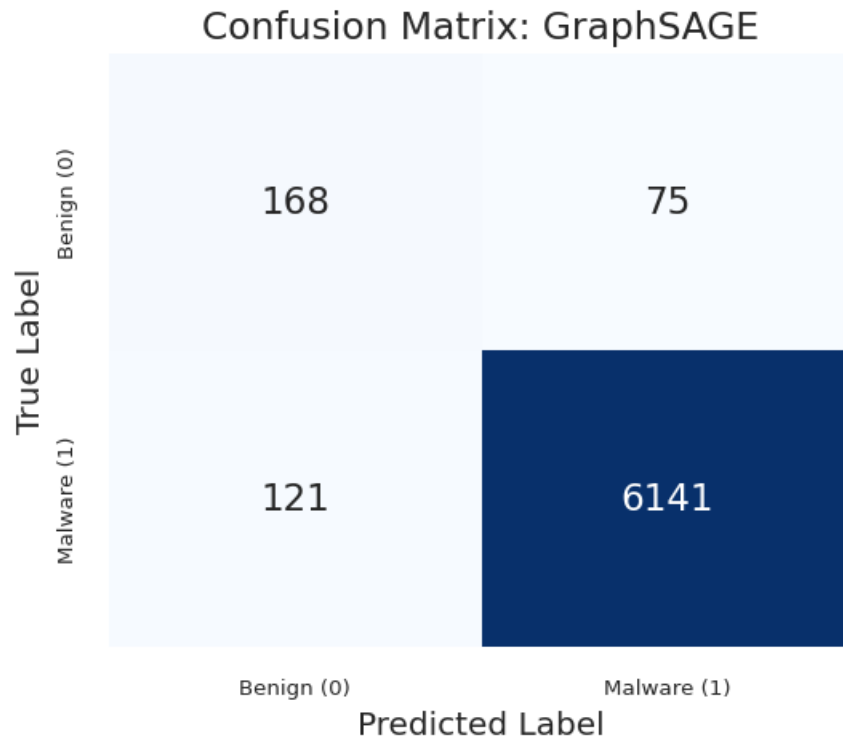
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task4_plots/cm_graphsage.png



Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task4_plots/cm_graphsage.png



Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task4_plots/cm_graphsage.png



```
[142]: # =====
# GraphSAGE: Select Best Model and Evaluate
# =====

best_sage_lr, model_sage, history_sage, elapsed_sage, sage_selection_table = \
    ↪select_best_gnn_model(
        architecture_name='GraphSAGE',
        models=sage_models,
        histories=sage_histories,
        elapsed_times=sage_elapsed_times,
        val_loader=val_loader,
    )

y_val_true_sage, y_val_pred_sage = predict_loader_gnn(model_sage, val_loader)
print('\nValidation report (GraphSAGE):')
print(classification_report(y_val_true_sage, y_val_pred_sage, digits=4, \
    ↪zero_division=0))

y_test_true_sage, y_test_pred_sage = predict_loader_gnn(model_sage, test_loader)
print('Test report (GraphSAGE):')
print(classification_report(y_test_true_sage, y_test_pred_sage, digits=4, \
    ↪zero_division=0))
```

Automatic selection for GraphSAGE

Ranking criterion: restored-model validation macro F1, then validation accuracy, then minimum validation loss

lr_label	last_history_val_f1_macro	val_accuracy	eval_val_f1_macro
min_val_loss	elapsed_s		
LR=1e-3	0.841225	0.972129	0.823588
0.042609	84.880655		
LR=1e-4	0.788487	0.948239	0.756044
0.048644	296.050101		
LR=5e-4	0.821914	0.943645	0.745448
0.044889	124.018934		

Selected best GraphSAGE model: LR=1e-3

Restored-model Val F1 Macro: 0.8236

Validation Accuracy: 0.9721

Validation report (GraphSAGE):

	precision	recall	f1-score	support
0	0.6054	0.7295	0.6617	122
1	0.9894	0.9815	0.9855	3143
accuracy			0.9721	3265
macro avg	0.7974	0.8555	0.8236	3265
weighted avg	0.9751	0.9721	0.9734	3265

Test report (GraphSAGE):

	precision	recall	f1-score	support
0	0.5813	0.6914	0.6316	243
1	0.9879	0.9807	0.9843	6262
accuracy			0.9699	6505
macro avg	0.7846	0.8360	0.8079	6505
weighted avg	0.9727	0.9699	0.9711	6505

3. GAT This model uses the Graph Attention Network (GAT) architecture.

```
[143]: class GATmodel(torch.nn.Module):
        def __init__(self, vocab_size, embed_dim, hidden_dim, output_dim, heads=4):
            super(GATmodel, self).__init__()

            self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=0)

            # GAT Layers
            # Layer 1: Output is hidden_dim * heads
```

```

        self.conv1 = GATConv(embed_dim, hidden_dim, heads=heads, dropout=0.2)

        # Layer 2: Input is hidden_dim * heads. We set heads=1 for the final
        ↪graph embedding before pooling.
        # concat=False means we average the heads or just output one head.
        self.conv2 = GATConv(hidden_dim * heads, hidden_dim, heads=1,
        ↪concat=False, dropout=0.2)

        # Layer 3: Input should now be hidden_dim (output of conv2)
        self.conv3 = GATConv(hidden_dim, hidden_dim, heads=1, concat=False,
        ↪dropout=0.2)

        self.pool = global_mean_pool

        self.classifier = nn.Sequential(
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, output_dim)
        )

    def forward(self, data):
        x, edge_index, batch = data.x, data.edge_index, data.batch

        x = self.embedding(x.squeeze(1))

        # Conv 1
        x = self.conv1(x, edge_index)
        x = F.elu(x)

        # Conv 2
        x = self.conv2(x, edge_index)

        # Conv 3
        x = self.conv3(x, edge_index)

        # Pooling
        x = self.pool(x, batch)

        # Classifier
        out = self.classifier(x)
        return out

```

```

[144]: # =====
# GAT: Training with Multiple Learning Rates
# =====

gat_learning_rates = [5e-5, 1e-4, 5e-4]

```

```

gat_lr_labels = ['LR=5e-5', 'LR=1e-4', 'LR=5e-4']

gat_histories = {}
gat_models = {}
gat_elapsed_times = {}

for lr, lr_label in zip(gat_learning_rates, gat_lr_labels):
    print(f"\n{'='*60}")
    print(f"Training GAT with {lr_label}")
    print('='*60)

    torch.manual_seed(42)
    np.random.seed(42)

    model_gat_temp = GATmodel(
        vocab_size=VOCAB_SIZE_GNN,
        embed_dim=EMBED_DIM_GNN,
        hidden_dim=HIDDEN_DIM_GNN,
        output_dim=OUTPUT_DIM_GNN,
        heads=4
    ).to(device)

    optimizer_temp = optim.AdamW(model_gat_temp.parameters(), lr=lr)

    model_gat_temp, history_temp, elapsed_temp = train_gnn_common(
        model_gat_temp,
        train_loader,
        val_loader,
        optimizer_temp,
        criterion_gnn,
        epochs=EPOCHS_GNN,
        min_delta=MIN_DELTA_GNN,
        patience=PATIENCE_GNN
    )

    gat_models[lr_label] = model_gat_temp
    gat_histories[lr_label] = history_temp
    gat_elapsed_times[lr_label] = elapsed_temp

    elapsed_str = time.strftime('%H:%M:%S', time.gmtime(elapsed_temp))
    print(f'\n{lr_label} - Training time: {elapsed_str}')

```

```

=====
Training GAT with LR=5e-5
=====
Epoch 1/100 - Train Loss: 0.1187 - Val Loss: 0.0990 - Val F1: 0.4902
Epoch 10/100 - Train Loss: 0.0725 - Val Loss: 0.0689 - Val F1: 0.6139

```

```
Epoch 20/100 - Train Loss: 0.0610 - Val Loss: 0.0597 - Val F1: 0.6937
Epoch 30/100 - Train Loss: 0.0557 - Val Loss: 0.0562 - Val F1: 0.7069
Epoch 40/100 - Train Loss: 0.0509 - Val Loss: 0.0541 - Val F1: 0.7257
Epoch 50/100 - Train Loss: 0.0485 - Val Loss: 0.0531 - Val F1: 0.7267
Epoch 60/100 - Train Loss: 0.0436 - Val Loss: 0.0490 - Val F1: 0.7633
Epoch 70/100 - Train Loss: 0.0432 - Val Loss: 0.0479 - Val F1: 0.7604
Epoch 80/100 - Train Loss: 0.0420 - Val Loss: 0.0484 - Val F1: 0.7569
Epoch 90/100 - Train Loss: 0.0413 - Val Loss: 0.0468 - Val F1: 0.7664
Epoch 100/100 - Train Loss: 0.0392 - Val Loss: 0.0452 - Val F1: 0.7802
```

LR=5e-5 - Training time: 00:07:29

```
=====
Training GAT with LR=1e-4
=====
```

```
Epoch 1/100 - Train Loss: 0.1088 - Val Loss: 0.0926 - Val F1: 0.5266
Epoch 10/100 - Train Loss: 0.0629 - Val Loss: 0.0613 - Val F1: 0.6957
Epoch 20/100 - Train Loss: 0.0524 - Val Loss: 0.0528 - Val F1: 0.7311
Epoch 30/100 - Train Loss: 0.0465 - Val Loss: 0.0504 - Val F1: 0.7483
Epoch 40/100 - Train Loss: 0.0424 - Val Loss: 0.0464 - Val F1: 0.7664
Epoch 50/100 - Train Loss: 0.0380 - Val Loss: 0.0458 - Val F1: 0.7479
Epoch 60/100 - Train Loss: 0.0343 - Val Loss: 0.0433 - Val F1: 0.8057
Epoch 70/100 - Train Loss: 0.0364 - Val Loss: 0.0440 - Val F1: 0.7490
Epoch 80/100 - Train Loss: 0.0326 - Val Loss: 0.0420 - Val F1: 0.7802
Early stopping at epoch 87 (best val loss 0.041429)
```

LR=1e-4 - Training time: 00:06:33

```
=====
Training GAT with LR=5e-4
=====
```

```
Epoch 1/100 - Train Loss: 0.0914 - Val Loss: 0.0724 - Val F1: 0.6441
Epoch 10/100 - Train Loss: 0.0500 - Val Loss: 0.0526 - Val F1: 0.7306
Epoch 20/100 - Train Loss: 0.0430 - Val Loss: 0.0438 - Val F1: 0.7764
Epoch 30/100 - Train Loss: 0.0344 - Val Loss: 0.0445 - Val F1: 0.7743
Epoch 40/100 - Train Loss: 0.0300 - Val Loss: 0.0380 - Val F1: 0.7947
Epoch 50/100 - Train Loss: 0.0286 - Val Loss: 0.0409 - Val F1: 0.7844
Epoch 60/100 - Train Loss: 0.0236 - Val Loss: 0.0432 - Val F1: 0.8209
Epoch 70/100 - Train Loss: 0.0274 - Val Loss: 0.0383 - Val F1: 0.8172
Early stopping at epoch 71 (best val loss 0.035316)
```

LR=5e-4 - Training time: 00:05:22

```
[145]: # =====
# GAT: Plot Learning Rate Comparison
# =====
```

```

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

colors = ['#984ea3', '#17becf', '#ff7f00']

for (lr_label, history), color in zip(gat_histories.items(), colors):
    epochs_range = range(1, len(history['val_loss']) + 1)
    ax1.plot(epochs_range, history['val_loss'], label=lr_label,
            linewidth=REPORT_PLOT_LINEWIDTH, color=color)

ax1.set_title('GAT - Validation Loss', fontsize=REPORT_PLOT_TITLE_SIZE,
            fontweight='bold')
ax1.set_xlabel('Epoch', fontsize=REPORT_PLOT_LABEL_SIZE)
ax1.set_ylabel('Loss', fontsize=REPORT_PLOT_LABEL_SIZE)
format_report_axes(ax1)
ax1.grid(alpha=0.3)

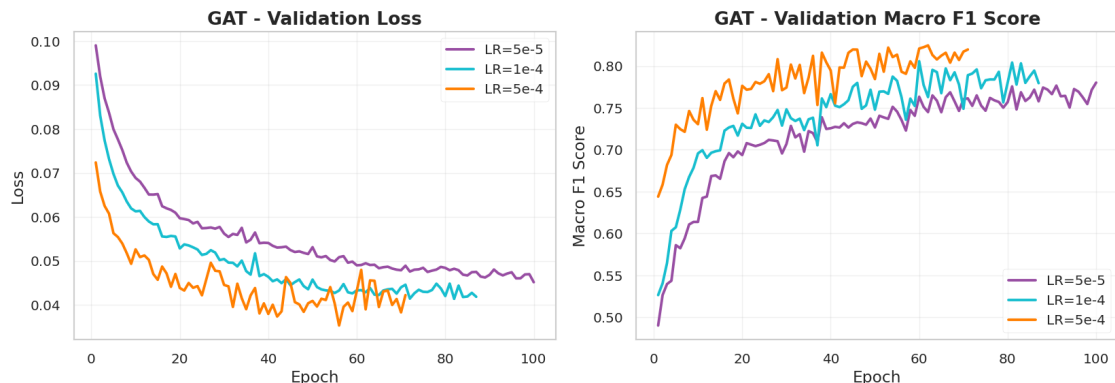
for (lr_label, history), color in zip(gat_histories.items(), colors):
    epochs_range = range(1, len(history['val_f1_macro']) + 1)
    ax2.plot(epochs_range, history['val_f1_macro'], label=lr_label,
            linewidth=REPORT_PLOT_LINEWIDTH, color=color)

ax2.set_title('GAT - Validation Macro F1 Score',
            fontsize=REPORT_PLOT_TITLE_SIZE, fontweight='bold')
ax2.set_xlabel('Epoch', fontsize=REPORT_PLOT_LABEL_SIZE)
ax2.set_ylabel('Macro F1 Score', fontsize=REPORT_PLOT_LABEL_SIZE)
format_report_axes(ax2)
ax2.grid(alpha=0.3)

plt.tight_layout()
save_plot(plt.gcf(), f"task4_gat_lr_comparison.png", save_dir)
plt.show()

```

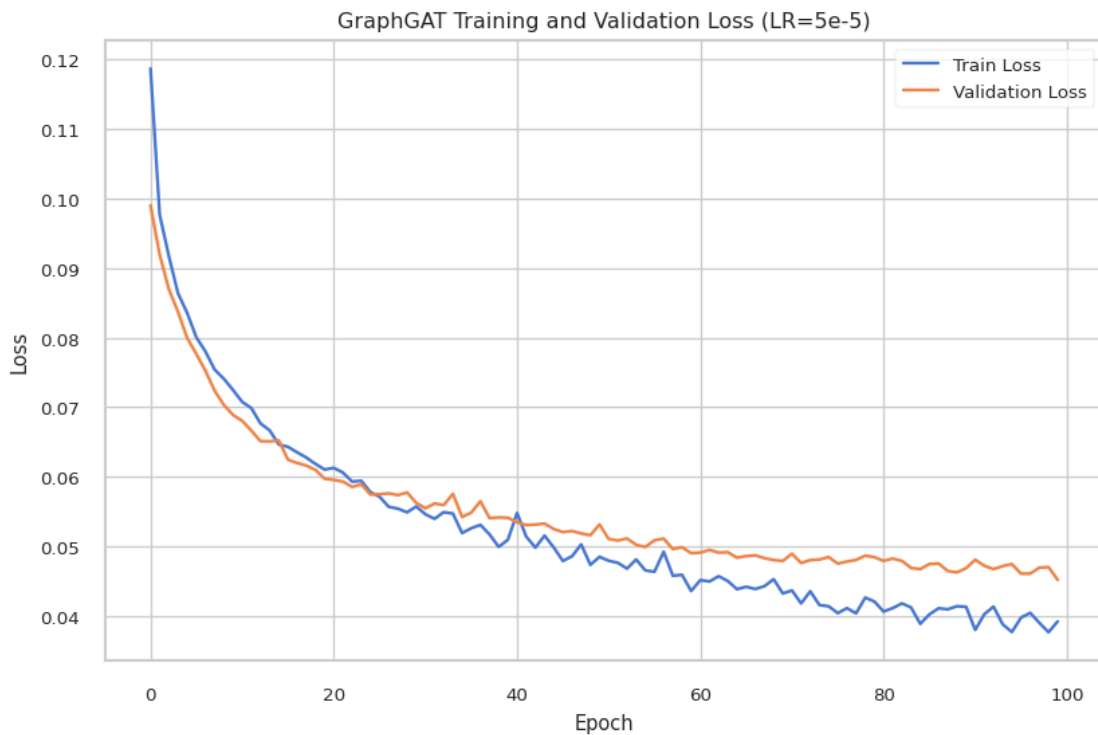
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task4_plots/task4_gat_lr_comparison.png.png



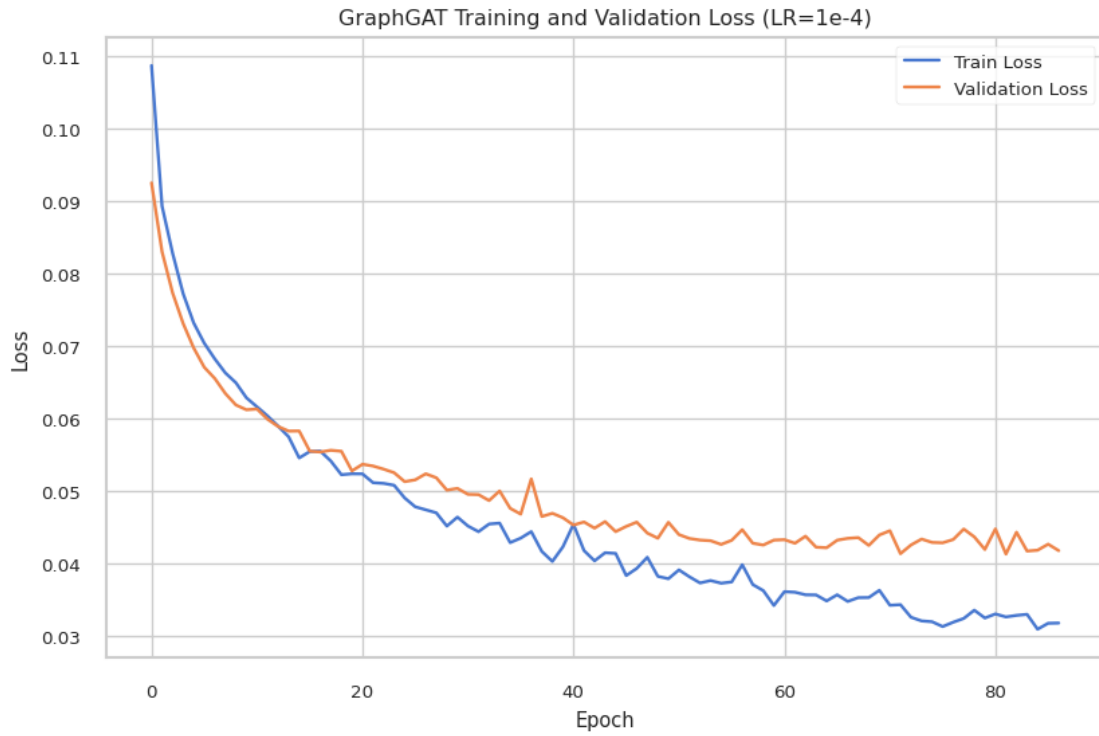
```
[146]: for lr_label, model_gat in gat_models.items():
        history = gat_histories[lr_label] # <-- Prendi la history per questo LR

        plt.figure(figsize=(8, 5))
        plt.plot(history['train_loss'], label='Train Loss')
        plt.plot(history['val_loss'], label='Validation Loss')
        plt.title(f'GraphGAT Training and Validation Loss ({lr_label})')
        plt.xlabel('Epoch')
        plt.ylabel('Loss')
        plt.legend()
        plt.grid(True)
        save_plot(plt.gcf(), f'graphgat_loss_curves_{lr_label.replace("=", "")}',
        ↪save_dir)
        plt.show()
```

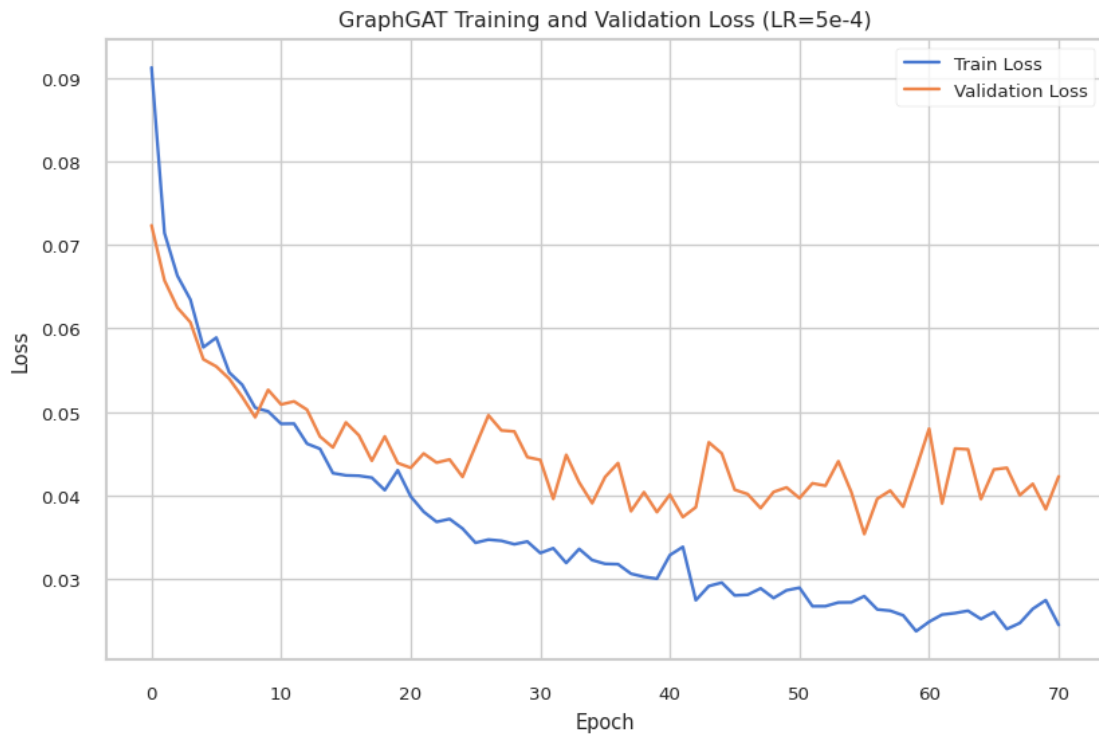
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task4_plots/graphgat_loss_curves_LR5e-5.png



Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task4_plots/graphgat_loss_curves_LR1e-4.png



Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task4_plots/graphgat_loss_curves_LR5e-4.png




```
[147]: # --- Evaluate GAT ---
for lr_label, model_gat in gat_models.items():
    y_val_true_gat, y_val_pred_gat = predict_loader_gnn(model_gat, val_loader)
    print('\nValidation report (GAT):')
    print(classification_report(y_val_true_gat, y_val_pred_gat, digits=4,
    ↪zero_division=0))

    y_test_true_gat, y_test_pred_gat = predict_loader_gnn(model_gat, test_loader)
    print('Test report (GAT):')
    print(classification_report(y_test_true_gat, y_test_pred_gat, digits=4,
    ↪zero_division=0))
```

Validation report (GAT):

	precision	recall	f1-score	support
0	0.5093	0.6721	0.5795	122
1	0.9871	0.9749	0.9810	3143
accuracy			0.9636	3265
macro avg	0.7482	0.8235	0.7802	3265
weighted avg	0.9693	0.9636	0.9660	3265

Test report (GAT):

	precision	recall	f1-score	support
0	0.5079	0.6626	0.5750	243
1	0.9867	0.9751	0.9809	6262
accuracy			0.9634	6505
macro avg	0.7473	0.8188	0.7779	6505
weighted avg	0.9689	0.9634	0.9657	6505

Validation report (GAT):

	precision	recall	f1-score	support
0	0.5316	0.6885	0.6000	122
1	0.9878	0.9765	0.9821	3143
accuracy			0.9657	3265
macro avg	0.7597	0.8325	0.7910	3265
weighted avg	0.9707	0.9657	0.9678	3265

Test report (GAT):

	precision	recall	f1-score	support
0	0.5503	0.7202	0.6239	243
1	0.9890	0.9772	0.9831	6262
accuracy			0.9676	6505
macro avg	0.7697	0.8487	0.8035	6505
weighted avg	0.9726	0.9676	0.9696	6505

Validation report (GAT):

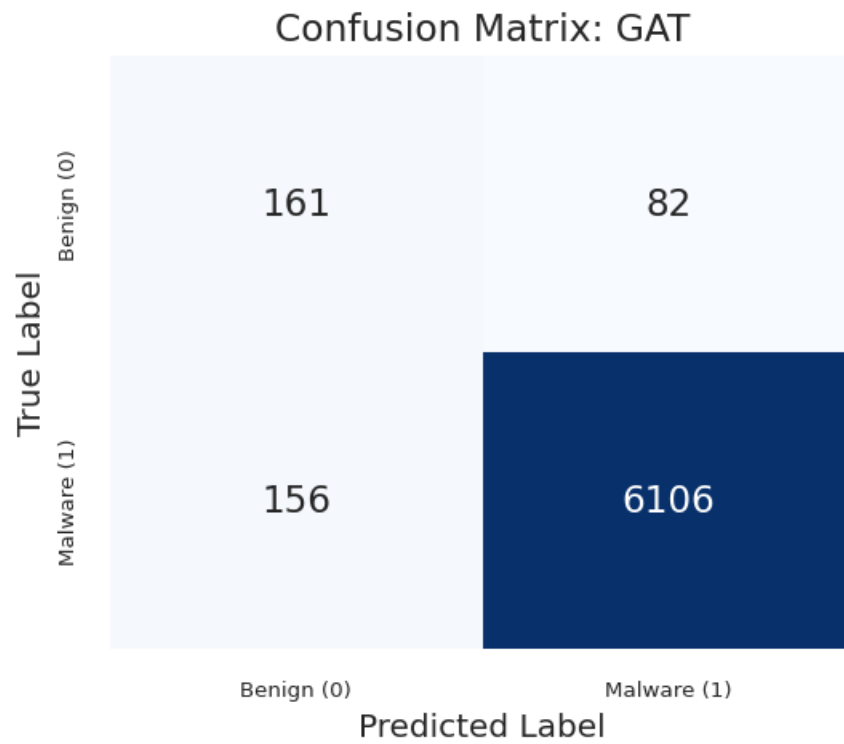
	precision	recall	f1-score	support
0	0.4876	0.8033	0.6068	122
1	0.9922	0.9672	0.9795	3143
accuracy			0.9611	3265
macro avg	0.7399	0.8853	0.7932	3265
weighted avg	0.9733	0.9611	0.9656	3265

Test report (GAT):

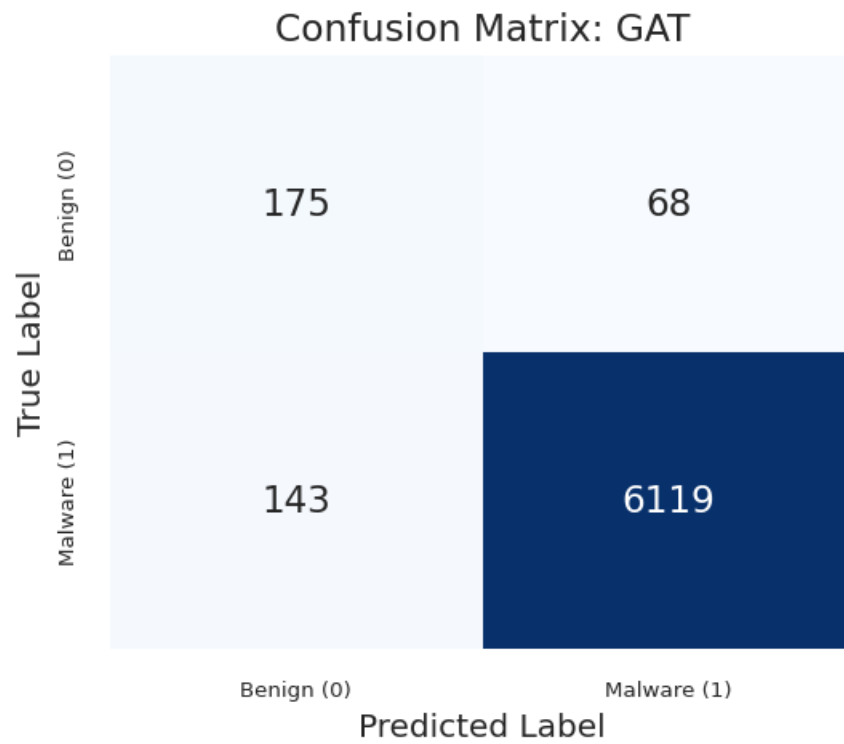
	precision	recall	f1-score	support
0	0.5125	0.7572	0.6113	243
1	0.9904	0.9721	0.9811	6262
accuracy			0.9640	6505
macro avg	0.7515	0.8646	0.7962	6505
weighted avg	0.9725	0.9640	0.9673	6505

```
[148]: for lr_label, model_gat in gat_models.items():
        plot_cm_task4(model=model_gat,
                      loader=test_loader,
                      model_name="GAT",
                      save_dir=results_path + 'images/task4_plots/')
```

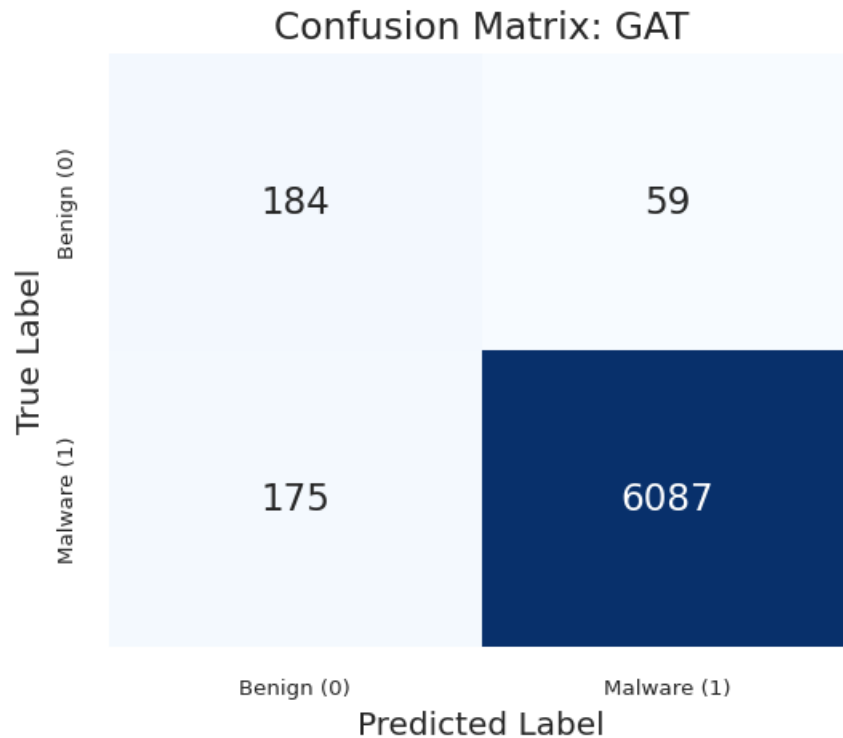
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task4_plots/cm_gat.png



Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task4_plots/cm_gat.png



Saved plot: `/content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task4_plots/cm_gat.png`



```
[149]: # =====
# GAT: Select Best Model and Evaluate
# =====

best_gat_lr, model_gat, history_gat, elapsed_gat, gat_selection_table =
    ↪select_best_gnn_model(
        architecture_name='GAT',
        models=gat_models,
        histories=gat_histories,
        elapsed_times=gat_elapsed_times,
        val_loader=val_loader,
    )

y_val_true_gat, y_val_pred_gat = predict_loader_gnn(model_gat, val_loader)
print('\nValidation report (GAT):')
print(classification_report(y_val_true_gat, y_val_pred_gat, digits=4,
    ↪zero_division=0))

y_test_true_gat, y_test_pred_gat = predict_loader_gnn(model_gat, test_loader)
print('Test report (GAT):')
print(classification_report(y_test_true_gat, y_test_pred_gat, digits=4,
    ↪zero_division=0))
```

Automatic selection for GAT

Ranking criterion: restored-model validation macro F1, then validation accuracy, then minimum validation loss

lr_label	last_history_val_f1_macro	val_accuracy	eval_val_f1_macro
min_val_loss	elapsed_s		
LR=5e-4	0.819724	0.961103	0.793175
0.035316	322.178674		
LR=1e-4	0.779538	0.965697	0.791040
0.041377	393.894694		
LR=5e-5	0.780228	0.963553	0.780228
0.045157	449.578457		

Selected best GAT model: LR=5e-4

Restored-model Val F1 Macro: 0.7932

Validation Accuracy: 0.9611

Validation report (GAT):

	precision	recall	f1-score	support
0	0.4876	0.8033	0.6068	122
1	0.9922	0.9672	0.9795	3143
accuracy			0.9611	3265
macro avg	0.7399	0.8853	0.7932	3265
weighted avg	0.9733	0.9611	0.9656	3265

Test report (GAT):

	precision	recall	f1-score	support
0	0.5125	0.7572	0.6113	243
1	0.9904	0.9721	0.9811	6262
accuracy			0.9640	6505
macro avg	0.7515	0.8646	0.7962	6505
weighted avg	0.9725	0.9640	0.9673	6505

```
[150]: # =====
# GNN Comparison: Validation Macro F1 Score (Best LR for each model)
# =====

fig, ax = plt.subplots(figsize=(10, 6))

# Plot best model for each architecture
best_models_gnn = {
    'GCN': history_gcn,
    'GraphSAGE': history_sage,
```

```

    'GAT': history_gat
}

colors_gnn = ['#1b9e77', '#377eb8', '#984ea3']

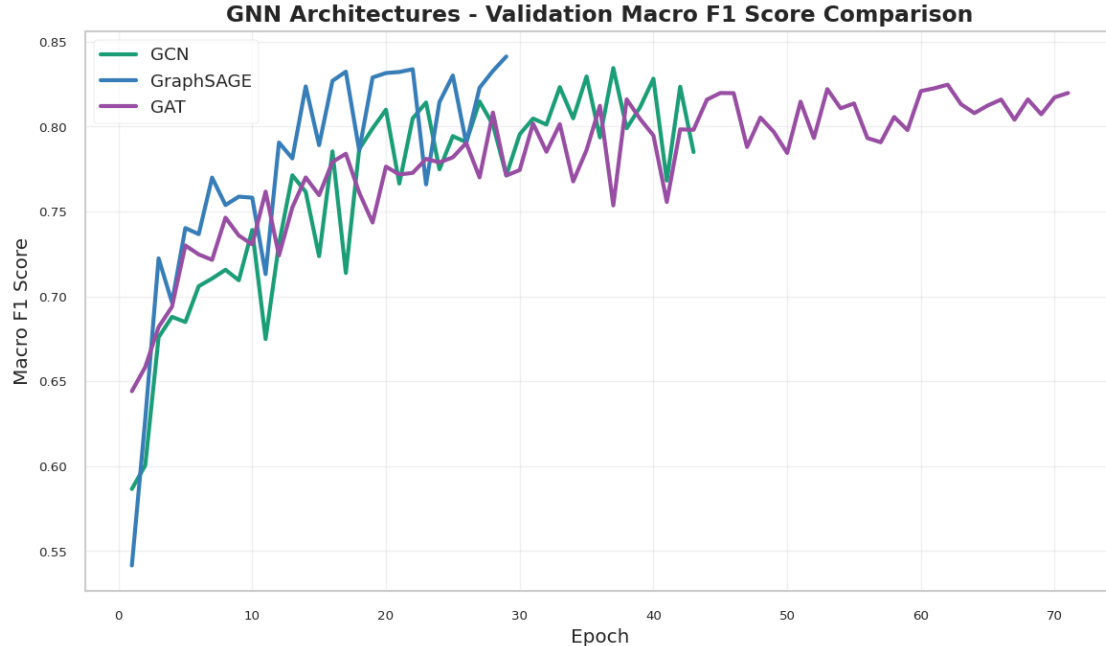
for (name, history), color in zip(best_models_gnn.items(), colors_gnn):
    epochs_range = range(1, len(history['val_f1_macro']) + 1)
    ax.plot(epochs_range, history['val_f1_macro'], label=name, linewidth=2.5,
            color=color)

ax.set_title('GNN Architectures - Validation Macro F1 Score Comparison',
            fontweight='bold')
ax.set_xlabel('Epoch', fontsize=12)
ax.set_ylabel('Macro F1 Score', fontsize=12)
ax.legend(fontsize=11)
ax.grid(alpha=0.3)

plt.tight_layout()
save_plot(plt.gcf(), f"task4_gnn_comparison_f1_macro", save_dir)
plt.show()

```

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task4_plots/task4_gnn_comparison_f1_macro.png



```

[151]: # =====
# FINAL REPORT: GNN Architecture Comparison
# =====

def get_test_metrics(y_true, y_pred):
    """Extract test metrics from predictions."""
    report = classification_report(y_true, y_pred, output_dict=True,
    ↪zero_division=0)
    return {
        'precision': report['macro avg']['precision'],
        'recall': report['macro avg']['recall'],
        'f1_score': report['macro avg']['f1-score'],
        'accuracy': report['accuracy'],
        'benign_recall': report['0']['recall'],
        'malware_recall': report['1']['recall'],
    }

metrics_gcn = get_test_metrics(y_test_true_gcn, y_test_pred_gcn)
metrics_sage = get_test_metrics(y_test_true_sage, y_test_pred_sage)
metrics_gat = get_test_metrics(y_test_true_gat, y_test_pred_gat)

gnn_summary = pd.DataFrame([
    {
        'architecture': 'GCN',
        'selected_lr': best_gcn_lr,
        'val_macro_f1': gcn_selection_table.loc[0, 'eval_val_f1_macro'],
        'test_macro_f1': metrics_gcn['f1_score'],
        'test_accuracy': metrics_gcn['accuracy'],
        'test_benign_recall': metrics_gcn['benign_recall'],
        'test_malware_recall': metrics_gcn['malware_recall'],
        'training_time_s': elapsed_gcn,
    },
    {
        'architecture': 'GraphSAGE',
        'selected_lr': best_sage_lr,
        'val_macro_f1': sage_selection_table.loc[0, 'eval_val_f1_macro'],
        'test_macro_f1': metrics_sage['f1_score'],
        'test_accuracy': metrics_sage['accuracy'],
        'test_benign_recall': metrics_sage['benign_recall'],
        'test_malware_recall': metrics_sage['malware_recall'],
        'training_time_s': elapsed_sage,
    },
    {
        'architecture': 'GAT',
        'selected_lr': best_gat_lr,
        'val_macro_f1': gat_selection_table.loc[0, 'eval_val_f1_macro'],
        'test_macro_f1': metrics_gat['f1_score'],
    }
])

```



```

        'test_accuracy': metrics_gat['accuracy'],
        'test_benign_recall': metrics_gat['benign_recall'],
        'test_malware_recall': metrics_gat['malware_recall'],
        'training_time_s': elapsed_gat,
    },
])

gnn_summary = gnn_summary.sort_values(
    by=['test_macro_f1', 'test_accuracy', 'test_benign_recall'],
    ascending=[False, False, False]
).reset_index(drop=True)

best_gnn_row = gnn_summary.iloc[0]

print("\n" + "="*120)
print("FINAL REPORT: GNN Architecture Comparison (Test Set)")
print("Ranking criterion: test macro F1, then test accuracy, then benign_
    ↪recall")
print("="*120)
display(gnn_summary)
print(
    f"Best-performing GNN on the test set: {best_gnn_row['architecture']} "
    f"({best_gnn_row['selected_lr']}) | "
    f"test macro F1={best_gnn_row['test_macro_f1']:.4f}, "
    f"accuracy={best_gnn_row['test_accuracy']:.4f}, "
    f"benign recall={best_gnn_row['test_benign_recall']:.4f}"
)

```

```

=====
=====
FINAL REPORT: GNN Architecture Comparison (Test Set)
Ranking criterion: test macro F1, then test accuracy, then benign recall
=====
=====

```

	architecture	selected_lr	val_macro_f1	test_macro_f1	test_accuracy \
0	GraphSAGE	LR=1e-3	0.823588	0.807936	0.969869
1	GAT	LR=5e-4	0.793175	0.796218	0.964028
2	GCN	LR=1e-3	0.800997	0.765410	0.960492

	test_benign_recall	test_malware_recall	training_time_s
0	0.691358	0.980677	84.880655
1	0.757202	0.972054	322.178674
2	0.650206	0.972533	146.011812

Best-performing GNN on the test set: GraphSAGE (LR=1e-3) | test macro F1=0.8079, accuracy=0.9699, benign recall=0.6914

Q: How does each model perform compared to the previous architectures? Can you beat the baseline? Yes, the graph models beat the frequency baseline in macro F1, although the baseline remains very competitive in accuracy. The baseline reaches test accuracy 96.97% and macro F1 0.7223, but its benign recall is only 0.3457. This confirms that accuracy alone is misleading on this imbalanced dataset.

Among the GNNs, GraphSAGE is the strongest selected architecture in this run. It selects $\text{LR}=1\text{e-}3$ and reaches test macro F1 0.8079, accuracy 96.99%, and benign recall 0.6914. GAT selects $\text{LR}=5\text{e-}4$ and obtains test macro F1 0.7962, while GCN selects $\text{LR}=1\text{e-}3$ and obtains test macro F1 0.7654. All three GNNs improve benign recall with respect to the frequency baseline, while keeping high malware recall.

Compared with the previous architectures, FFNNs remain the weakest family: even with learnable embeddings and embedding-size tuning, the best FFNN test macro F1 is around 0.59, mainly because the model relies on fixed-length truncation and cannot explicitly model temporal or structural dependencies. RNNs improve substantially: the selected bidirectional LSTM reaches test macro F1 0.8305, making it the best overall model in this run. GNNs are slightly below the Bi-LSTM, but clearly stronger than FFNNs and the frequency baseline in terms of balanced classification.

Overall, the experiments show that both sequential information and transition-graph structure are useful. The frequency baseline is strong because API-call counts are informative, but RNNs and GNNs provide a better balance between malware detection and benign recognition.

[]: